Understanding Machine Learning

Lectures 1 to 5: the experiments, the mistakes, the differences

In the format of the course textbook, for the midterm

Fundamentals of Machine Learning

2026-11-03

Contents

How to read this book **3**

0 The Experiments **4**
0.1 One experiment behind everything . . . 4
0.2 Train, validation, test . . . 6
0.3 Why the test set must not be touched . . . 11
0.4 Cross-validation . . . 13
0.5 Hyperparameter tuning . . . 15
0.6 Learning curves . . . 18
0.7 Bias–variance experiments . . . 20
0.8 The complete protocol . . . 23

3 From Fitting to Generalization **25**
How to read the formulas in this chapter . . . 25
3.1 Generalization and Inductive Bias . . . 26
3.2 Model Capacity . . . 30
3.3 Bias and Variance . . . 32
3.4 Controlling Complexity . . . 35
3.5 Feature and Model Selection . . . 39
3.6 Reliable Evaluation and Data Splitting . . . 43
3.7 Diagnosis and Lab Preparation . . . 46
Chapter takeaways . . . 48

4 Discriminative and Generative Classification **49**
4.1 Two Routes to the Posterior . . . 49
4.2 The Discriminative Route . . . 51
4.3 The Generative Route . . . 57
4.4 Gaussian Class-Conditional Models . . . 59
4.5 Naive Bayes . . . 63
4.6 Comparing the Two Routes . . . 68
4.7 Class Imbalance and Prior Shift . . . 69
4.8 Unfamiliar Inputs and Combined Systems . . . 72
4.9 Model Choice and Lab 2 . . . 74

5 Learning by Combining Models **76**
5.1 Why Combine Models? . . . 76
5.2 Averaging and Voting . . . 78
5.3 Decision Trees as Base Learners . . . 81
5.4 Bagging and Out-of-Bag Evaluation . . . 84
5.5 Random Forests . . . 86
5.6 Feature Importance . . . 88
5.7 Boosting for Regression . . . 90

5.8 Boosting for Binary Classification . . . 93
5.9 XGBoost: One Objective for Structure and Leaf Values . . . 96
5.10 Multi-Class Boosting and Other Libraries . . . 99
5.11 Bagging versus Boosting, and When Ensembles Do Not Help . . . 100
5.12 Stacking . . . 102
5.13 Mixture of Experts . . . 103
Chapter takeaways . . . 105

1 What Does It Mean to Learn? 106
1.1 Learning from Experience . . . 106
1.2 Two Routes to Machine Ability . . . 108
1.3 What Counts as Experience? . . . 112
1.4 Supervised Learning: Learning from Examples . . . 114
1.5 Representations . . . 118
1.6 Four Ingredients of a Learning System . . . 121
1.7 The Challenge of Generalization . . . 123
1.8 A Connection to Information Theory . . . 127
Chapter takeaways . . . 130

2 From Representations to Decisions 131
How to read the formulas in this chapter . . . 131
2.1 From Objects to Numerical Representations . . . 132
2.2 Decision Making When the Distribution Is Known . . . 135
2.3 From Known Distributions to Finite Data . . . 143
2.4 Parameterized Models . . . 148
2.5 Learning the Parameters from Data . . . 151
2.6 Evaluating a Learned Model . . . 158
2.7 The Learning Problem in One Picture . . . 162
Chapter takeaways . . . 163

How to read this book

This is the textbook treatment of Lectures 1 to 5 rewritten for one purpose: to understand, before tomorrow, how each method works, what each experiment manipulates and measures, what people do wrong, and how to tell neighbouring methods apart. It keeps the pace and the boxes of the course textbook. A blue *What are we trying to get?* box opens each section and says why the section exists. Margin notes carry the intuition in one line. A green *Definition* box is followed by a cyan *Reading the formula* box that says, symbol by symbol, what the formula means and gives a number. A *Theorem* or *Proposition* is followed by its proof, written so that each step says why it is allowed. New in this edition are four boxes: a violet *Experiment* box narrates the experiment behind the concept as if it were run in front of you (what is held fixed, what is changed, what is measured, what the plot shows, how to read it); a red *What people do wrong* box says what you will be tempted to do, why it feels right, which number it corrupts and how to catch it; a grey *Telling them apart* box clears up the confusion between two neighbouring methods; and a teal *How the professor asks it* box ends each section with his questions, answered the way an expert answers at the board. Formulas marked [on your sheet] are on your two-page sheet, so do not memorise them; learn what each symbol means and when the formula applies. The few marked [not on your sheet: know it] you must carry in your head. Read the Experiments chapter first, then chapters 3, 4 and 5, then 1 and 2.

Chapter 0

The Experiments

Every method of Lectures 1 to 5 is judged by the same experiment. We fit a model on some rows, we measure it on other rows, and we change one thing at a time. The thing we change is called the manipulated variable, and it can be the amount of data, the capacity of the family, a regularisation strength, the number of training epochs, the number of trees, or the rule by which we split the data. The thing we measure is always a risk, which is an average loss on a set of rows, and the whole art of the subject lies in choosing *which* rows, because the rows that chose a model cannot also grade it. This chapter walks through each standard experiment slowly: what is held fixed, what is varied, what is measured, what the curve looks like, how each shape is read, and what people do wrong. It comes first because the professor's "what is the issue here, what would you do" questions nearly all come from it. Formulas marked [on your sheet] are on your sheet; what you need to carry in your head is why they are true and when they apply.

0.1 One experiment behind everything

What are we trying to get?

We want a precise vocabulary for "how good is this model on the next case," a clear statement of why that question cannot be answered on the rows that produced the model, and a single skeleton that every later experiment fills in.

Two populations of rows: the ones you have, and the ones you will meet.

Definition 0.1.1 (Empirical risk, population risk)**.** For a model f, a loss ℓ and a set S of rows, the *empirical risk* on S and the *population risk* are

$$\widehat{R}_S(f) = \frac{1}{|S|} \sum_{i \in S} \ell(y^{(i)}, f(x^{(i)})), \qquad R(f) = \mathbb{E}[\ell(Y, f(X))]. \quad \text{[on your sheet]}$$

Reading the formula

The population risk is the number we actually care about: it is the average loss the model will make on cases drawn from the world where it will be used. It is also the one number we can never compute, because we do not have the world, only a sample of it. The empirical risk is the same average taken over the rows we happen to hold. Every experiment in this chapter is a way of estimating R with some $\widehat{R}_S$, and the choice of S decides whether the estimate

can be believed. If a spam filter makes 2% errors on its training emails and 9% on next month's emails, both numbers are empirical risks; the second is the one that resembles R.

Imagine rerunning the whole training on many fresh samples.

Every quantity in this course that wears a hat is random in a specific sense. Draw a training set $\mathcal{D}$ from the world, run the learning procedure, obtain a fitted model $\widehat{f}_{\mathcal{D}}$. Throw the sample away, draw a fresh one of the same size, run the procedure again: you obtain a different model $\widehat{f}_{\mathcal{D}'}$. When we say that something is "biased," "unbiased," "optimistic," or that a procedure has "variance," we are always making a statement about what happens on average over these imaginary repetitions, which we write $\mathbb{E}_{\mathcal{D}}[\cdot]$. You never see this experiment with real data, because you only have one sample; but you can run it exactly in simulation, where you know the truth, and you can approximate it on real data by refitting with different seeds, different folds or different bootstrap samples. Each refit is one draw, and when the predictions scatter from refit to refit you are looking at variance with your own eyes.

Experiment: the generic learning experiment

Setup. A dataset, a learning procedure with one knob h (the polynomial degree, the tree depth, the regularisation strength λ, the number of neighbours k, the number of epochs, the number of trees, or the amount of training data N), a loss and a metric.

What we hold fixed. Everything except the knob: the rows in each subset, the preprocessing, the random seed wherever randomness enters, the metric.

What we change. The knob, one value at a time, over a grid.

What we measure. Two numbers at each value: the risk on the rows that fitted the model, which we call the training risk, and the risk on rows the model never saw, which we call the held-out risk.

What the plot shows. Two curves against the knob. The training curve, and above it the held-out curve. Where the two sit, how far apart they are, and whether they are still moving is the entire diagnosis.

How to read it. Ask three questions in this order. Are both curves high? Then the family is too restricted or the features are poor, and we call that underfitting. Is the gap between them large? Then the fit depends on the particular sample, and we call that overfitting, or high variance. Are the curves still moving as the knob increases? Then you have not reached the setting where the knob stops mattering, and the grid should be extended.

Key insight

Every experiment has the same skeleton: hold everything fixed, vary one knob, measure the risk twice (on fitting rows and on unseen rows), and read the two curves. The method you are testing only decides what the knob is.

Telling them apart: the four curves that look alike

Students mix up four plots, because all four show a curve that falls. The way to keep them apart is to look at the horizontal axis and ask what was manipulated.

Curve	Manipulated	Typical shape	Diagnoses
validation (complexity) curve	capacity: degree, depth, $1/k$, $1/\lambda$	training falls monotonically; held-out falls then rises (a U)	the best capacity; underfitting on the left, overfitting on the right
training curve	epochs or boosting rounds	the same U, over time	when to stop (early stopping)
learning curve	amount of data N	training rises, held-out falls, both toward one plateau	bias (plateau height) versus variance (gap) versus data-limited
ensemble-size curve	number of models M	held-out falls then plateaus	when more models stop helping; the floor $\rho\sigma^2$

What people do wrong: "the error goes down when we make the model bigger"

A student will tell you this as if it were a result. It is not, until they say which error. The training error *always* goes down when capacity increases, because a bigger family contains the smaller one and can at least reuse its best fit; so a falling training error carries no information about new cases. Ask for the held-out error at each capacity, and for the gap. If the held-out error also falls, capacity was the limiting factor; if it rises while the training error falls, the extra capacity is fitting the sample.

How the professor asks it

Q. *A colleague reports that "the error decreases when we increase the model". Explain why this sentence is not yet a result, and what two numbers you would ask for.*
A. It is not a result because it does not say which error. Training error decreases, weakly, whenever capacity increases, because a larger nested family contains the smaller one's best fit, so a falling training error says nothing about new cases. Ask for the held-out error at each capacity and for the gap between the two curves. If the held-out error also falls, capacity was the limiting factor and the model was underfitting; if it rises while the training error falls, the extra capacity is fitting the particular sample, which is overfitting.

0.2 Train, validation, test

What are we trying to get?

We want to see exactly why the rows that chose a model grade it too kindly, which question each of the three subsets answers, how the rule for cutting the data follows from the question the model will face at deployment, and the list of ways information leaks from the future into the model.

Why the training rows cannot judge

The same rows choose the model and then grade it.

Proposition 0.2.1 (A fixed model is graded honestly; a fitted model is graded kindly)**.** *Let f be any model chosen before the rows of S were seen, with S an i.i.d. sample of size n. Then $\mathbb{E}[\widehat{R}_S(f)] = R(f)$. Let instead $\widehat{f}_{\mathcal{D}}$ minimise $\widehat{R}_{\mathcal{D}}$ over a family $\mathcal{H}$, and let $f^*_{\mathcal{H}}$ minimise R over $\mathcal{H}$. Then*

$$\mathbb{E}_{\mathcal{D}}[\widehat{R}_{\mathcal{D}}(\widehat{f}_{\mathcal{D}})] \;\le\; R(f^*_{\mathcal{H}}) \;\le\; \mathbb{E}_{\mathcal{D}}[R(\widehat{f}_{\mathcal{D}})]. \quad \textit{[on your sheet]}$$

Proof. The first claim is the fact that an average of n independent copies of the random variable $\ell(Y, f(X))$ has expectation $R(f)$; nothing more is needed because f does not depend on the sample. For the second claim, three facts are chained. First, since $\widehat{f}_{\mathcal{D}}$ minimises the empirical risk, $\widehat{R}_{\mathcal{D}}(\widehat{f}_{\mathcal{D}}) \le \widehat{R}_{\mathcal{D}}(f^*_{\mathcal{H}})$ on every sample; this is the only place minimisation is used. Second, $f^*_{\mathcal{H}}$ was fixed before the sample was drawn, so by the first claim $\mathbb{E}[\widehat{R}_{\mathcal{D}}(f^*_{\mathcal{H}})] = R(f^*_{\mathcal{H}})$. Third, $f^*_{\mathcal{H}}$ is the best member of the family, so $R(\widehat{f}_{\mathcal{D}}) \ge R(f^*_{\mathcal{H}})$ for every sample. Taking expectations in the first inequality and combining with the third gives the sandwich. □

Reading the formula

Read the sandwich from the middle outward. In the middle sits the best risk the family can achieve. The training score of the fitted model sits *below* it, and the true risk of the fitted model sits *above* it; so the training score is optimistic by at least the whole gap, which is the expected generalisation gap $\mathbb{E}[G] \ge 0$ [on your sheet]. The one word that made the difference is *fixed*. A validation set grades a model that was fixed before the validation rows were looked at, so the first claim applies to it and its number is honest. The training rows played two roles, $\mathcal{D} \to \widehat{f}_{\mathcal{D}} \to \widehat{R}_{\mathcal{D}}(\widehat{f}_{\mathcal{D}})$: they chose the model and then graded it, and a procedure that is allowed to do both will exploit the accidents of the sample.

Experiment: honest versus optimistic, seen directly

Setup. A simulated world where the truth is known, say $y = \sin(2\pi x) + \varepsilon$; a model family, say polynomials of degree 5; a large pool of fresh test points.

What we hold fixed. The family, the noise level, the test pool.

What we change. Nothing but the random training sample: repeat 200 times, each time drawing a fresh training set of 20 points.

What we measure. For each repetition, (a) the training risk of the fitted polynomial on its own 20 points, (b) its risk on the fresh test pool, and (c) the training risk of one polynomial that was chosen before any data were drawn.

What the plot shows. Three histograms. Histogram (c) is centred on that fixed model's true risk. Histogram (a) is centred well below the centre of (b).

How to read it. The distance between the centres of (a) and (b) is the expected generalisation gap; it is positive, it grows with capacity, and it shrinks with the sample size. Histogram (c) is the point of the experiment: the problem is not "training data", it is "the data that chose the model".

Three subsets, three questions

Definition 0.2.2 (Training, validation and test sets)**.** The labelled rows are cut into three disjoint sets. The *training set* fits the parameters θ. The *validation set* chooses between alternatives that training cannot choose for itself: the regularisation strength, the number of neighbours, the depth, the

feature subset, the decision threshold, the stopping epoch; together these are the hyperparameters h. The *test set* is used once, at the end, to report the performance of the single chosen model. [on your sheet]

Reading the formula

Why three and not two? Because each question would be answered dishonestly by the rows used for the previous question. The training rows grade the fitted model too kindly, by Proposition 0.2.1. The validation rows grade each candidate honestly, since each candidate was fitted without them; but the moment you pick the candidate with the best validation score, that best score is flattering, because you selected the luckiest of several noisy numbers (Section 0.3). Only the test set took no part in any choice, and that is the only reason its number can be believed.

Hold out exactly the kind of new case you will meet.

Definition 0.2.3 (Random, stratified, grouped and temporal splits)**.** *Random*: each row is assigned to a subset approximately independently; credible only when new deployment cases are reasonably approximated by independent draws from the same population. *Stratified*: random, but keeping the class proportions approximately equal in every subset, so that a rare class is not accidentally absent from the test rows. *Grouped*: every group (user, device, patient, hospital, session) lies entirely in one subset, so the learner cannot use identity-specific patterns and is evaluated on groups it has never seen. *Temporal*: train on the past, validate and test on later rows, which preserves the direction of prediction and reveals deterioration as behaviour, sensors or attacks change. [on your sheet]

Experiment: the split protocol

Setup. N labelled rows and a deployment question: "will this flow next month be an attack?", "does the model work on a hospital it has never seen?", "will this customer churn next quarter?".

What we hold fixed. The rows of each subset, written down before any model is fitted, and never moved.

What we change. The split rule, chosen from the question: random when new cases are independent draws from the same population; stratified when a class is rare; grouped when deployment meets new entities; temporal when the model predicts the future.

What we measure. Metrics on validation during development; metrics on test once.

What the plot shows. Not a curve but a table, one row per protocol for the same model. The random-split number is the largest, the grouped number smaller, the temporal number smallest.

How to read it. The protocol that mirrors deployment gives the only number that predicts deployment. The other numbers are not wrong arithmetic; they answer questions nobody asked.

Example 0.2.4 (The same intrusion dataset, three verdicts)**.** Flows are recorded from 50 devices over 6 months. A random split puts flows from every device and every week into both train and test, so the model can learn "device 17 is the compromised laptop" and scores 99.5%. A grouped split holds out 10 devices; the model must recognise attack behaviour and scores 96%. A temporal split trains on months 1 to 4 and tests on months 5 and 6, during which a new attack tool appeared; the model scores 90%. Only the third number describes next month. A bank predicting churn "next quarter" is in the same situation: the features must come from before the quarter and the outcome from after it, so the split is temporal.

Example 0.2.5 (How large must the validation set be?)**.** A validation score is an estimate and estimates have noise. The standard error of an accuracy near 0.9 measured on n rows is $\sqrt{0.9 \cdot 0.1/n}$: about 0.03 for $n = 100$, 0.0095 for $n = 1{,}000$, and 0.003 for $n = 10{,}000$. Comparing two models that differ by half a point on 100 validation rows is reading noise, because the difference is six times smaller than the standard error. With few rows, cross-validation (Section 0.4) uses every row for validation once.

Leakage

Definition 0.2.6 (Leakage)**.** Leakage is any use, during development, of information that will not be available when a new deployment decision is made. [on your sheet] Its common forms: preprocessing fitted on all rows before the split; duplicates or near-duplicates in several subsets; records of the same device, session or patient in training and test; predictors created after the outcome or after a human investigation (a "ticket closed as intrusion" field, "number of analyst comments"); a time series shuffled before splitting; hyperparameters repeatedly chosen on test results.

Technique: Preprocessing belongs inside the training pipeline

The training rows estimate $\hat{\mu}_j = \frac{1}{N_{\text{tr}}} \sum_{i \in \text{tr}} x_j^{(i)}$ and $\hat{\sigma}_j^2$, and the *same* $\hat{\mu}_j, \hat{\sigma}_j$ are reused unchanged on validation, test and deployment rows [on your sheet]. The same rule applies to imputation, feature selection, dimensionality reduction and resampling; in cross-validation each fold fits these operations on its own training portion only. The leaking pipeline is "all data $\rightarrow$ estimate μ_j, σ_j $\rightarrow$ split"; the valid pipeline is "split $\rightarrow$ estimate on training $\rightarrow$ reuse". The question that catches leaking features is: at what moment does this value become known, relative to the moment of the decision?

What people do wrong: standardising before splitting

A student standardises all 10,000 rows, then splits 80/20, trains logistic regression and reports 94% on the 20%. It feels harmless because the scaler "only subtracts a mean". But the means and standard deviations were computed with the test rows included, so the test rows helped shape the representation on which they are then graded. For a scaler on 10,000 rows the effect is small, since two sample means differ little; the same habit with feature selection, where the 20 most correlated features are chosen on all rows, or with impu-

tation by a fitted model, inflates the estimate by several points, and with few rows and many features it can produce near-perfect scores on labels that are pure noise. The fix costs nothing: split first, fit the scaler on the 8,000, apply it unchanged to the 2,000.

What people do wrong: a random split when deployment is in the future

The random split is the default in every library and it gives the nicest number, which is exactly why it is tempting. If the model will meet new devices, the random split lets it recognise devices; if it predicts the future, the random split lets it see the future of the series it is tested on. The number is inflated, sometimes by many points, and the way to catch it is to ask the deployment question first and then check that the held-out rows are novel in that sense. A second trap is stratifying across groups: scattering one device's flows across train and test to balance the classes destroys the grouping and the model can again recognise the device.

Telling them apart: training set, validation set, test set

	Training	Validation	Test
used for	fitting θ	choosing h: λ, k, depth, features, threshold, epoch	reporting the final number
looked at	every iteration	every candidate	once
honest for	nothing after fitting	each candidate, not the winner	the winner
becomes dishonest when	always	you pick the minimum over many candidates	you use its result to change anything

How the professor asks it

Q. *You standardise all 10,000 rows, then split 80/20, train logistic regression and report 94% on the 20%. Is the 94% trustworthy? Explain exactly what leaked and how large you expect the effect to be.*

A. Not fully. The means and standard deviations used to scale the training rows were computed with the test rows included, so the test rows influenced the representation they are then graded on. For standardisation on 10,000 rows the effect is usually small, because two sample means differ little; but the same mistake with feature selection or with model-based imputation can be large, and in cross-validation with few rows it is routinely several points. The fix: split first, fit the scaler on the 8,000, apply it unchanged to the 2,000.

Q. *Explain why a model that scores 99.5% on a random split of device-level intrusion data may score 90% in deployment, and name the split that would have predicted the 90%.*

A. The random split places flows of the same device, and the same weeks, in both subsets, so the learner can exploit device identity and period-specific patterns that recur in the test rows: a shortcut that holds inside the collection and not outside it. In deployment the model meets new devices and later

weeks, where the shortcut is absent. A temporal split (and, for new devices, a grouped split) removes the shortcut from the evaluation and would have shown the lower number before deployment.

0.3 Why the test set must not be touched

What are we trying to get?

We want to prove that the best of many honest validation scores is an optimistic number, to put a size on the effect, and to see why using the test set to make any decision turns it into another validation set.

The minimum of noisy numbers is pulled below the truth.

Proposition 0.3.1 (The best of many validation scores is optimistic)**.** *Let $\widehat{R}_{\text{val}}(\widehat{f}_h)$, $h \in \mathcal{C}$, be validation risks of models fitted without the validation rows, so that each is an unbiased estimate of $R(\widehat{f}_h)$. Then*

$$\mathbb{E}\Big[\min_{h\in\mathcal{C}} \widehat{R}_{\text{val}}(\widehat{f}_h)\Big] \;\le\; \min_{h\in\mathcal{C}} R(\widehat{f}_h), \qquad \textit{[on your sheet]}$$

with strict inequality as soon as the estimates are noisy and no single h is best in every sample.

Proof. Take any one candidate h_0. The minimum over all candidates is at most the value at h_0, so $\min_h \widehat{R}_{\text{val}}(\widehat{f}_h) \le \widehat{R}_{\text{val}}(\widehat{f}_{h_0})$ on every sample. Take expectations; the right side equals $R(\widehat{f}_{h_0})$ because each candidate's score is honest. This holds for every h_0, so it holds for the best one, which gives the display. When the scores have noise and the winner changes from sample to sample, the minimum is sometimes achieved by a candidate whose score fell below its own true risk, and the inequality is strict. □

Reading the formula

Each score, taken alone, was honest. The best score is the minimum of several noisy numbers, and part of what made it the minimum was luck; so the winner's reported score is pulled below the winner's true risk. How large is the pull? For two candidates with the same true risk R and independent Gaussian noise of standard deviation σ, the expected minimum is $R - \sigma/\sqrt{\pi} \approx R - 0.56\sigma$ [on your sheet]. For n candidates it grows like $\sigma\sqrt{2\ln n}$ [on your sheet]: 1.18σ for $n = 2$, 2.15σ for $n = 10$, 3.0σ for $n = 100$, 3.7σ for $n = 1{,}000$. Notice the asymmetry: the noise enters linearly, the number of candidates only through a square root of a logarithm. A small validation set therefore does more damage than a long search.

Experiment: adaptive overfitting of a validation set

Setup. Labels that are pure coin flips, so there is nothing to learn; 1,000 rows split 500/500.

What we hold fixed. The data; every "model" is a random rule that ignores the input entirely.

What we change. The number n of random rules tried: 1, 10, 100, 1,000,

10,000. For each n, keep the rule with the best validation accuracy.

What we measure. The best validation accuracy, and the accuracy of that same winning rule on the 500 test rows.

What the plot shows. The best validation accuracy climbs with n: about 50%, 53%, 56%, 57%, 59%. The test accuracy of the winner stays at 50% for every n.

How to read it. Nothing was learned, yet the validation number "improved" steadily, because you selected the luckiest of n coin-flip scores. Every real development loop does a weaker version of this: each look at the validation score steers the next choice, so development gradually adapts to the validation set. The flat test line is what an untouched test set buys. If you now start choosing among test scores, the test line starts climbing the same way and you have no honest number left anywhere.

Example 0.3.2 (Public leaderboards)**.** In data-science competitions the public leaderboard is scored on a fixed subset. Teams that submit hundreds of times and keep whatever scores best on it routinely drop many places on the private leaderboard, which is scored on rows they never saw. The public subset had become their validation set; the private subset is the untouched test set. The same happens inside a company when "the test set" is re-run after every change: the reported number drifts upward while deployment performance does not.

What people do wrong: touching the test set without noticing

The ways the test set gets touched are rarely dramatic. Someone runs the test set "just to see" after each change and keeps the change when the number improved: that is selection on the test set. Someone chooses the decision threshold, or which metric to report, after looking at test results. Someone stops training at the epoch where the test loss is lowest. Someone tunes by cross-validation over all rows and then reports the cross-validation score of the winner as if it were a test score; those folds chose the hyperparameters, so that score is a validation score and it is optimistic by Proposition 0.3.1. The fix: freeze the test rows before development; use them once; if you must keep developing after seeing the test result, obtain new test data or use nested cross-validation and report its outer estimate.

How the professor asks it

Q. *You tried 50 hyperparameter configurations on a validation set of 200 rows and the best scored 91%. Why should you expect less than 91% in deployment, and what two things would reduce the discrepancy?*

A. Because 91% is the minimum validation error among 50 noisy estimates, and the expected minimum of honest estimates lies below the best true risk. With 200 rows the estimation noise is about $\sqrt{0.09 \cdot 0.91/200} \approx 0.02$, and the optimism for 50 candidates is of the order of $\sigma\sqrt{2\ln 50} \approx 2.8\sigma \approx 0.06$, so several points are plausible. Reduce it by a larger validation set or cross-validation, which shrinks the noise (the linear factor), and by a coarser or smaller search, which shrinks $\sqrt{2\ln n}$ (the logarithmic factor). To *know* the

deployment number, evaluate the winner once on a test set that took no part in the 50 comparisons.

0.4 Cross-validation

What are we trying to get?

We want a way to validate on every row when the rows are few, to be precise about what the resulting number estimates, and to see the leaks that hide inside the folds.

Definition 0.4.1 (K-fold cross-validation)**.** Partition the development rows (everything except the frozen test set) into K folds of equal size. For a configuration h and a fold j, fit $\widehat{f}_h^{(-j)}$ on the other $K-1$ folds and evaluate it on fold j; then

$$\widehat{R}_{\mathrm{CV}}(h) = \frac{1}{K}\sum_{j=1}^{K} \widehat{R}_{\mathrm{val}}^{(j)}(\widehat{f}_h^{(-j)}), \qquad \hat{h} \in \arg\min_{h\in\mathcal{C}} \widehat{R}_{\mathrm{CV}}(h). \quad \text{[on your sheet]}$$

Each row validates exactly once and trains $K-1$ times.

Reading the formula

Fold j holds N/K rows that $\widehat{f}_h^{(-j)}$ never saw, so its score is an honest estimate of the risk of a model trained on $N(K-1)/K$ rows. Averaging the K scores estimates the *expected* risk of the procedure "run configuration h on $N(K-1)/K$ rows". Be precise about this: the number describes the procedure, not any one of the K models fitted along the way. With $K = N$ (leave-one-out) the training sets are almost the full data, so the estimate is almost unbiased for the full-data procedure, but the N models are nearly identical, their scores are highly correlated, the average has high variance, and the cost is N fits. With $K = 5$ or 10 the training sets are 80 to 90% of the data (slightly pessimistic, since the final model sees more), the folds differ more (lower variance of the average), and the cost is 5 or 10 fits per configuration: 20 configurations at $K = 5$ is 100 fits.

Experiment: the cross-validation protocol

Setup. Development rows, a candidate set $\mathcal{C}$, and K (5 or 10 are standard).

What we hold fixed. One fold assignment, drawn once and reused for every configuration, so that all candidates are judged on the same slices; grouped or forward-chaining folds if deployment demands it.

What we change. Which fold is held out (the inner loop over j) and which configuration is run (the outer loop over h).

What we measure. $\widehat{R}_{\mathrm{val}}^{(j)}$ for every pair (h, j); their mean $\widehat{R}_{\mathrm{CV}}(h)$ and their spread across folds.

What the plot shows. $\widehat{R}_{\mathrm{CV}}(h)$ against h with error bars from the fold-to-fold spread: the validation curve of Section 0.5, now with its noise made visible.

How to read it. Pick $\hat{h}$ at the minimum, but treat configurations whose means lie within the error bars as tied, and among ties prefer the simpler. Then do the step students forget: refit the chosen configuration on all development rows, because the K fold models were only instruments; and score the test set once. The score at $\hat{h}$ is still a validation score of a winner, hence optimistic; the test set gives the honest number.

Example 0.4.2 (Spam filter with 600 labelled emails)**.** A single 60/20/20 split leaves 120 validation emails, on which a difference of 2% is below the noise. Five-fold cross-validation on the 480 development emails validates every email once and gives five numbers per configuration; their average is far less noisy and their spread tells you whether "$\lambda = 0.1$ beats $\lambda = 1$" is real. After choosing λ, refit on all 480, then score the 120 frozen test emails once.

What people do wrong: selecting features before cutting the folds

Take 300 rows, 5,000 features, and labels generated independently of the features, so there is no signal. Rank the features by correlation with the label on all 300 rows, keep the best 20, and then run 10-fold cross-validation of logistic regression on those 20: you obtain about 97%. It feels right because "the classifier was cross-validated". But with 5,000 features and 300 rows some features are strongly correlated with the labels by chance, the ranking on all rows picks exactly those, and every validation fold then scores rows that already took part in choosing the features: the validation labels leaked into the representation. Run the selection inside each fold on the 270 training rows only and the folds score near 50%. Everything data-dependent goes inside the fold.

What people do wrong: the other cross-validation leaks

Random folds on grouped or temporal data put rows of the same patient, device or session on both sides, or shuffle a time series; use grouped folds or forward-chaining folds. Deploying one of the K fold models, or averaging them, instead of refitting on all development rows (averaging them is a bagging-like ensemble, legitimate but a different method). Different fold assignments for different candidates, so that the differences between candidates include the differences between slices. And reporting the cross-validation score of the winner as the final estimate: it chose $\hat{h}$, so it is a validation score. When you need an honest number without a separate test set, use *nested* cross-validation: an outer loop whose held-out folds are never used for any selection, and an inner cross-validation inside each outer training portion that chooses h; the outer average is honest for the whole procedure "select, then fit".

Telling them apart: validation split, cross-validation, out-of-bag, nested cross-validation

	What is held out	Cost	Honest for
single validation set	one fixed slice	1 fit per candidate	each candidate; noisy if small
K-fold CV	each fold in turn	K fits per candidate	each candidate; less noisy; optimistic for the winner
out-of-bag (bagging)	the $\approx 36.8\%$ rows missing from each bootstrap sample	free	the fixed ensemble; same caveat if used to select
nested CV	outer folds for estimation, inner folds for selection	$K_{\text{out}} \times K_{\text{in}}$ fits per candidate	the whole select-then-fit procedure

How the professor asks it

Q. *Why must feature selection be repeated inside every cross-validation fold rather than done once before cross-validation?*
A. Because selection reads the labels. If it reads the labels of the rows that will later be a validation fold, that fold's labels have influenced the representation on which the fold is then scored, so the fold is no longer unseen; on label-free data this produces high cross-validation accuracy from nothing. Repeating the selection inside each fold, on the training portion only, restores the condition that the validation rows played no part in anything that was chosen.

Q. *Why does leave-one-out cross-validation have high variance although each of its N fitted models uses almost all the data?*
A. Its N training sets differ by one row, so the N fitted models are nearly identical and their N held-out losses are strongly correlated; the average of correlated quantities does not shrink its variance like $1/N$, which is the ensemble variance formula $\sigma^2(\rho + (1-\rho)/M)$ [on your sheet] with ρ close to one. Five- or ten-fold folds are more different from each other, so their average is more stable, at the price of a slight pessimism because each model sees 80 to 90% of the rows.

0.5 Hyperparameter tuning

What are we trying to get?

We want a definition of a hyperparameter by what it does, a proof that the training set cannot choose one, the validation-curve experiment that does, and the two special cases that trip people up: early stopping, and knobs that are coupled.

A knob that training error would turn all the way up.

Definition 0.5.1 (Parameters, hyperparameters, and the selection loop)**.** A *parameter* θ is set by the training algorithm from the training rows: weights, coefficients, split thresholds, leaf values. A *hyperparameter* h is any knob that the training error, left to itself, would turn to maximal capacity: λ, k,

the degree, the depth, the number of boosting rounds, the learning rate, the number of epochs, the number of candidate features per split, the feature subset, the decision threshold. The selection loop is

$$\widehat{\theta}_h \in \arg\min_{\theta} \widehat{R}_{\text{train}}(\theta; h), \qquad \hat{h} \in \arg\min_{h \in \mathcal{C}} \widehat{R}_{\text{val}}(\widehat{f}_h), \quad \text{[on your sheet]}$$

an inner problem on training rows and an outer problem on validation rows or folds.

Proposition 0.5.2 (Training risk is minimised at $\lambda = 0$)**.** *Let* $\widehat{\theta}_\lambda \in \arg\min_\theta[\widehat{R}_{\text{train}}(\theta) + \lambda\Omega(\theta)]$ *and put* $c = \Omega(\widehat{\theta}_\lambda)$*. Then* $\widehat{\theta}_\lambda$ *also minimises* $\widehat{R}_{\text{train}}$ *subject to* $\Omega(\theta) \le c$*, the constrained families* $\mathcal{H}_c = \{f_\theta : \Omega(\theta) \le c\}$ *are nested in* c*, and consequently* $\widehat{R}_{\text{train}}(\widehat{\theta}_\lambda)$ *is nondecreasing in* λ*.* *[on your sheet]*

Proof. Take any θ with $\Omega(\theta) \le c$. Optimality of $\widehat{\theta}_\lambda$ for the penalised problem gives $\widehat{R}(\widehat{\theta}_\lambda) + \lambda c \le \widehat{R}(\theta) + \lambda\Omega(\theta) \le \widehat{R}(\theta) + \lambda c$, hence $\widehat{R}(\widehat{\theta}_\lambda) \le \widehat{R}(\theta)$: no feasible point beats $\widehat{\theta}_\lambda$, and $\widehat{\theta}_\lambda$ is itself feasible. A larger λ makes large Ω more expensive and so gives a smaller c and a smaller family; the families are nested because $\Omega(\theta) \le c_1$ implies $\Omega(\theta) \le c_2$ when $c_1 < c_2$; and a minimum over a smaller family is never smaller. □

Reading the formula

The training set, allowed to choose λ, always answers "no regularisation", because every increase of λ removes functions from the contest and can only raise the best achievable training risk. The same argument, with nested families indexed by degree, depth, epochs or $1/k$, sends every capacity knob to its maximum. That is the whole reason validation exists: a hyperparameter is a knob the training rows would abuse. On four points with $\sum x^2 = 14$ and $\sum xy = 19$ the ridge slope is $19/(14 + 4\lambda)$ [on your sheet], which is 1.357 at $\lambda = 0$, 1.056 at $\lambda = 1$ and 0.352 at $\lambda = 10$; the training error points to $\lambda = 0$ every time, and only validation rows can say whether $\lambda = 1$ predicts better.

Experiment: the validation curve: sweep one hyperparameter

Setup. Training rows and validation rows (or folds); a model with one knob, say ridge's λ, kNN's k, or the tree depth.

What we hold fixed. All other knobs, the preprocessing fitted on the training rows, the rows, the metric. Standardise before penalising, otherwise λ penalises a feature in millimetres a thousand times more than the same feature in metres.

What we change. The knob on a grid: a logarithmic grid $10^{-3}, 10^{-2}, \ldots, 10^{2}$ for λ, because the useful range spans orders of magnitude; the odd integers for k; the integers $1, \ldots, 12$ for depth.

What we measure. Training and validation risk at every grid point.

What the plot shows. Training risk decreasing monotonically in capacity, which means increasing in λ and in k; validation risk U-shaped.

How to read it. The minimum of the validation curve is $\hat{h}$. A flat valley means the choice does not matter much; a sharp valley means it does; a minimum on the edge of the grid means you have not seen the far side, so

extend the grid. Keep the direction of each knob straight: capacity grows with degree, depth, epochs and boosting rounds, and it grows when λ *decreases* and when k *decreases*.

Definition 0.5.3 (Early stopping)**.** Iterative optimisation produces $\widehat{\theta}_0, \widehat{\theta}_1, \ldots$; early iterations learn strong recurring patterns, later ones fit exceptions and noise. Early stopping keeps the checkpoint with the smallest validation risk, $t^* = \arg\min_t \widehat{R}_{\text{val}}(\widehat{\theta}_t)$ [on your sheet], and a *patience* p lets training continue p epochs without improvement before stopping, keeping the best checkpoint seen. Training time thereby controls effective capacity while the parameter count stays fixed.

Experiment: early stopping, and coupled knobs in boosting

Setup. An iterative learner (a network trained by gradient descent, or a boosted ensemble), training and validation rows.

What we hold fixed. Everything except the number of epochs or rounds.

What we change. The epoch t, simply by continuing to train and saving a checkpoint each epoch.

What we measure. Training and validation loss after each epoch.

What the plot shows. Training loss falls throughout; validation loss falls, reaches its minimum at t^*, and rises.

How to read it. Roll back to the checkpoint at t^*; never keep the last epoch. In gradient boosting three knobs are coupled: halving the learning rate η roughly doubles the best number of rounds M, because each tree applies a smaller correction, and deeper trees reach the minimum in fewer rounds. So grid over (η, depth), let early stopping choose M for each pair, and compare the pairs at their own selected rounds. Stopping on the test loss is tuning on the test set.

What people do wrong: changing the learning rate without re-selecting the rounds

Your boosting model is best at 400 rounds with learning rate 0.1. A colleague switches to 0.01, keeps 400 rounds, and reports a worse validation loss. With a ten times smaller step, 400 rounds apply about one tenth of the total correction, so the model is underfitted: its validation curve over rounds has not reached its minimum. The rounds must be re-selected by early stopping for the new rate, which will land near 4,000, and the two configurations are compared at their own selected rounds; the smaller rate may end slightly lower, at ten times the cost.

Telling them apart: the knobs, their direction, and what they trade

Knob	Capacity grows when	What is traded	Chosen on
degree, depth, width	it increases	approximation error down, estimation error up	validation curve
k in kNN	k decreases	smoother average versus local sensitivity	validation curve
λ (ridge, lasso)	λ decreases	shrinkage bias versus variance	log grid on validation
epochs, boosting rounds	they increase	recurring patterns first, then noise	early stopping
bagged trees M	never: stability only	computation	OOB plateau
m_{try} in a forest	larger values make trees stronger and more alike	tree strength versus correlation	OOB or validation

How the professor asks it

Q. *Prove that the training error of ridge regression is minimised at $\lambda = 0$, and state what this implies for how λ must be chosen.*
A. The penalised solution $\widehat{\theta}_\lambda$ also solves the constrained problem $\min \widehat{R}_{\text{train}}$ subject to $\Omega(\theta) \le c_\lambda$ with $c_\lambda = \Omega(\widehat{\theta}_\lambda)$: if some feasible θ had smaller training risk, then $\widehat{R}(\theta) + \lambda\Omega(\theta) \le \widehat{R}(\theta) + \lambda c_\lambda < \widehat{R}(\widehat{\theta}_\lambda) + \lambda c_\lambda$ would contradict the optimality of $\widehat{\theta}_\lambda$. The feasible sets shrink as λ grows, and a minimum over a smaller set is at least as large, so $\widehat{R}_{\text{train}}(\widehat{\theta}_\lambda)$ is nondecreasing in λ and smallest at $\lambda = 0$. The training set would therefore always choose no regularisation; λ is chosen on validation rows or by cross-validation, and the test set is kept for the final number.

0.6 Learning curves

What are we trying to get?

We want the one experiment that distinguishes "the family cannot express the relation" from "the family could, but the sample is too small", and a proof of why its two curves rise and fall toward the same limit.

Definition 0.6.1 (Learning curve)**.** A learning curve plots training and validation performance against the number of training rows N used, with everything else fixed. [on your sheet]

Proposition 0.6.2 (The two curves bracket the same limit)**.** *Fix the family $\mathcal{H}$ with best member $f^*_{\mathcal{H}}$ and let $\widehat{f}_N$ be the fitted model on N rows. At every N,*

$$\mathbb{E}[\widehat{R}_{\text{train}}(\widehat{f}_N)] \;\le\; R(f^*_{\mathcal{H}}) \;\le\; \mathbb{E}[R(\widehat{f}_N)],$$

and as $N \to \infty$ both outer terms approach $R(f^*_{\mathcal{H}})$.

Proof. The sandwich is Proposition 0.2.1 applied at sample size N. As N grows, the empirical risk of every member of a reasonable family approaches its population risk uniformly, so the empirical minimiser $\widehat{f}_N$ approaches the population minimiser $f^*_{\mathcal{H}}$; then the training risk of $\widehat{f}_N$, which is close to its population risk, and the population risk of $\widehat{f}_N$ both approach $R(f^*_{\mathcal{H}})$. □

Reading the formula

The lower term rises to the limit: that is the training curve, because more rows are harder to fit perfectly. The upper term falls to it: that is the validation curve. The common limit is the *approximation error* of the family plus the irreducible noise, and no amount of data changes it; the gap between the curves at finite N is the *estimation error*, which shrinks with N and grows with capacity. In the bias–variance language at a fixed x, where $\mathbb{E}(Y - \widehat{f}(x))^2 = \sigma^2 + \text{Bias}^2 + \text{Var}$ [on your sheet], the plateau is $\sigma^2 + \text{Bias}^2$ and the gap is mostly Var.

Experiment: the learning-curve protocol

Setup. Development rows and a fixed configuration h (already tuned, or tuned at each size).

What we hold fixed. The configuration, the validation rows or folds, the metric, the preprocessing rule.

What we change. The number of training rows N: nested subsamples of $10\%, 20\%, \ldots, 100\%$ of the training set, so that each larger set contains the smaller, with several random subsamples at each size, averaged.

What we measure. At each N: the training risk of the fitted model on its own N rows, and the validation risk on the fixed validation rows.

What the plot shows. Training risk rising with N; validation risk falling; the two approaching each other.

How to read it. Three shapes. Both curves high and already flat, close together: bias; more data will not help; change the representation or increase capacity. Training low, validation high, a wide gap that narrows only slowly: variance; regularise, reduce capacity, or collect more representative data, and the gap will close with N. Both still falling and converging: data-limited; more rows are likely to help. A fourth situation does not show on a learning curve: validation performance that degrades over calendar time, which is distribution shift and calls for a time-aware evaluation.

Example 0.6.3 (Should the security team label 20,000 more flows?)**.** The team has 10,000 labelled flows. They train a random forest on 1,000, 2,000, ..., 8,000 rows and validate on the same 2,000. Validation recall goes $0.71, 0.78, 0.83, 0.86, 0.87, 0.875, 0.878, 0.88$ while training recall stays near 1.0: a large gap that is closing slowly, and a validation curve that has almost flattened. Doubling the data would move recall from 0.88 to perhaps 0.89; the remaining gap is variance that larger leaves or fewer features per split address more cheaply, and the plateau says the features carry no more information. Had the validation curve still been rising steeply at 8,000, the answer would be yes.

What people do wrong: a gap that does not narrow

A learning curve shows training error at 2% and validation error at 15% at every size from 1,000 to 50,000 rows. The diagnostic table says "variance" because the gap is large, and the student reaches for regularisation. But variance shrinks with N, and a gap that does not move across a fiftyfold increase in data is not sampling variance: it is a systematic difference between the training rows and the validation rows, such as a shortcut present in the training collection and absent from a grouped or temporal validation set, duplicates in the training rows, or label noise concentrated on one side. The response is an audit of the split and the features, not more data.

Telling them apart: the three curves and the diagnostic table

Observed pattern	Likely issue	Response
training and validation both high, flat in N	underfitting (bias)	richer representation, more capacity, less regularisation
training low, validation high, gap narrows slowly with N	high variance	regularise, reduce capacity, more representative data, ensembles
both decreasing and converging	data-limited	more representative examples
gap constant across a fiftyfold increase in N	systematic train/ validation difference	audit the split, duplicates, shortcuts, label noise
fine on the split, degrades over calendar time	distribution shift	temporal evaluation, retraining, monitoring

[on your sheet]

How the professor asks it

Q. *Explain, using the sandwich inequality, why the training curve of a learning curve rises with N while the validation curve falls.*
A. At every N, $\mathbb{E}[\widehat{R}_{\text{train}}(\widehat{f}_N)] \le R(f^*_{\mathcal{H}}) \le \mathbb{E}[R(\widehat{f}_N)]$. With few rows the minimiser can fit the sample's accidents, so the training risk is far below $R(f^*_{\mathcal{H}})$ and the true risk far above it; with many rows the empirical risk of every member of the family is close to its population risk, so the fitted model is close to $f^*_{\mathcal{H}}$ and both terms approach $R(f^*_{\mathcal{H}})$. The lower term therefore rises toward the limit and the upper term falls toward it. The limit is the approximation error plus noise, which N cannot change.

0.7 Bias–variance experiments

What are we trying to get?

We want to measure bias and variance with an experiment, since both are averages over training sets that no single fit can show, and to see how each lever of the course moves them.

Definition 0.7.1 (Bias and variance of a procedure)**.** At a fixed input x, with $f^*(x) = \mathbb{E}[Y \mid X = x]$ and $\bar{f}(x) = \mathbb{E}_{\mathcal{D}}[\widehat{f}_{\mathcal{D}}(x)]$,

$$\mathrm{Bias}(x) = \bar{f}(x) - f^*(x), \qquad \mathrm{Var}(x) = \mathbb{E}_{\mathcal{D}}\big[(\widehat{f}_{\mathcal{D}}(x) - \bar{f}(x))^2\big],$$
$$\mathbb{E}\big[(Y - \widehat{f}_{\mathcal{D}}(x))^2\big] = \sigma^2(x) + \mathrm{Bias}(x)^2 + \mathrm{Var}(x). \quad \text{[on your sheet]}$$

Reading the formula

Fix one house. Train a thousand times on a thousand different training sets and record the thousand predicted prices. Their average is $\bar{f}(x)$. Bias asks whether that average is off from the true mean price in a consistent direction; variance asks how widely the thousand predictions scatter around their own average. These are properties of the *procedure*, meaning the family together with the fitting rule and the sample size, never of one fitted model, which is one draw and has no variance. The decomposition itself is proved in Chapter 3; it is the move "add and subtract a mean, square, and let the cross term die" applied twice.

Experiment: simulating bias and variance

Setup. A world you control: $f^*(x) = \sin(2\pi x)$, noise $\sigma = 0.3$, $N = 25$ training points per set, a grid of 50 test inputs.

What we hold fixed. The truth, the noise, N, the test grid.

What we change. Two things in two separate runs. First, the training set, drawn 500 times; this is the repetition that defines $\mathbb{E}_{\mathcal{D}}$. Second, the capacity: polynomial degree $0, 1, 3, 5, 9$, or k in kNN.

What we measure. For each capacity and each test input: the 500 predictions; their mean $\bar{f}(x)$; $\mathrm{Bias}^2(x)$ as the squared distance of that mean from $f^*(x)$; $\mathrm{Var}(x)$ as their spread; both averaged over the grid; and the measured test error for comparison.

What the plot shows. Bias^2 falling with capacity, variance rising, and their sum plus σ^2 U-shaped and matching the measured test error.

How to read it. The U of the validation curve is the sum of a falling term and a rising term. Degree 0 is all bias; degree 9 on 25 points is all variance. Now repeat the capacity sweep with $N = 250$: the variance curve drops everywhere, the bias curve does not move, and the minimum of the U shifts to a higher degree. That is the whole story of "more data allows a bigger model".

Example 0.7.2 (Bias and variance computed exactly)**.** No input: $Y = \mu + \varepsilon$ with $\mathrm{Var}(\varepsilon) = \sigma^2 = 1$, and $N = 4$ draws. The *sample mean* is unbiased with variance $\sigma^2/N = 0.25$. *Ignoring the data and predicting* 0 has bias $-\mu$ and variance 0. The *shrunk mean*, half the sample mean, has bias $-\mu/2$ and variance $0.25/4 = 0.0625$. Bias^2 plus variance: for $\mu = 1$ the three totals are 0.25, 1 and 0.3125, so the unbiased procedure wins; for $\mu = 0.2$ they are 0.25, 0.04 and 0.0725, so both biased procedures win, because they traded a little systematic error for a large reduction in sample sensitivity. Ridge regression is this trade with a dial.

Experiment: variance reduction by averaging: the ensemble-size curve

Setup. Bootstrap samples of one training set, a deep tree fitted on each, a validation set.

What we hold fixed. The data and the tree settings.

What we change. The number of trees $M = 1, 5, 10, 25, 50, 100, 200$; in a second run, the number of candidate features per split, all d versus $\sqrt{d}$.

What we measure. Validation error of the average of the first M trees.

What the plot shows. Error falling steeply then flattening; the curve with $\sqrt{d}$ candidates flattens lower.

How to read it. The floor is $\rho\sigma^2$ in $\mathrm{Var}(\bar{e}) = \sigma^2(\rho + \frac{1-\rho}{M})$ [on your sheet]: with $\rho = 0.2$ and $M = 25$ the variance is $0.232\sigma^2$, not $\sigma^2/25$, and no number of trees takes it below $0.2\sigma^2$. Feature randomisation lowers ρ and therefore the floor, which is why the forest beats bagging. Averaging does nothing for bias: if every tree lacks the same feature, every error points the same way and $\rho \approx 1$.

What people do wrong: expecting regularisation, or data, to be free

Students expect more data to reduce bias; it reduces variance, and the plateau is set by the family and the features. They expect regularisation to reduce variance for free; it adds bias, and whether the total falls depends on where the truth is, as the $\mu = 1$ versus $\mu = 0.2$ table shows. They speak of the variance of one trained model; variance is over training sets, and the experiment that measures it must refit many times. And they take the classical U for a law; very large models can show double descent, with test error peaking near the interpolation threshold and falling again beyond it [on your sheet], so validation behaviour must be examined, not assumed.

Telling them apart: what each lever does to bias and variance

Lever	Bias	Variance	Note
more capacity	down	up	U-shaped sum
more data N	unchanged	down	plateau unchanged
λ, shrinkage	up	down	worth it when the truth is near the target
early stopping	up slightly	down	capacity through training time
bagging, forests	unchanged	down to the floor $\rho\sigma^2$	shared errors survive
boosting	down	up with rounds	early stopping on rounds
better features	down	often down	the only lever on the plateau

How the professor asks it

Q. *A 1-nearest-neighbour regressor and a 50-nearest-neighbour regressor are both fitted on the same 200 points. Describe the bias–variance experiment that*

compares them and predict its outcome.
A. Draw many training sets of 200 points from the same world, fit both regressors on each, record both predictions at a fixed x, and compute the spread of each cloud (variance) and the distance of each cloud's centre from $f^*(x)$ (bias). The 1-NN prediction is one noisy neighbour's y, so its cloud is as wide as the noise, Var $\approx \sigma^2$, and its centre is close to $f^*(x)$: low bias, high variance. The 50-NN prediction averages 50 neighbours, so its spread is about $\sigma^2/50$, but the neighbours reach far from x and their average follows a smoothed version of f^*: higher bias, low variance. On the sheet: Bias $= \frac{1}{k}\sum_{j\le k} f(x^{(j)}) - f(x)$, Var $= \sigma^2/k$ [on your sheet]. Which total is smaller depends on how curved f^* is relative to the noise; the validation curve over k finds the balance.

Q. *Give a counterexample to "averaging many models always reduces error".*
A. Take M copies of the same fitted model, or M models that all lack the same essential feature. Their errors at any x are identical, so $\rho = 1$ and $\mathrm{Var}(\bar{e}) = \sigma^2(1 + 0) = \sigma^2$: the average has the error of one model. More generally the floor $\rho\sigma^2$ is untouched by M; averaging removes only the part of the error that differs between models, never shared bias.

0.8 The complete protocol

Technique: the experiment everyone runs, as twelve steps

1. State the deployment question (new cases of what kind: future? new devices? same population?); it decides the split rule.
2. Audit the data: duplicates, conflicting labels, missing values, class imbalance, features known only after the outcome.
3. Freeze the test set according to the split rule and write down its row identifiers; do not look at it again.
4. Fix the metric from the costs: accuracy hides minority failure, so use recall, precision, specificity, balanced accuracy or F_1 [on your sheet] as the costs dictate.
5. Fit baselines (majority class, a shallow tree, logistic regression, kNN) under the same protocol; anything you build must beat them.
6. Build the pipeline so that every data-dependent step (scaler, imputer, selector, model) is fitted on training rows only.
7. Tune each hyperparameter on validation rows or by cross-validation on a log or integer grid; read the U; treat near-ties as ties; choose epochs or rounds by early stopping; choose the decision threshold on validation.
8. Diagnose with learning curves (data-limited? bias? variance?) and, where instability is suspected, with repeated fits over seeds or bootstrap samples.
9. Probe shortcuts: shuffle the labels (performance must collapse to chance), drop metadata columns (performance must not collapse), evaluate under the deployment-like protocol.
10. Refit the chosen configuration on all development rows.

11. Evaluate once on the test set; report the number with its uncertainty; do not go back.
12. Monitor in deployment: a degrading metric over calendar time is distribution shift; schedule retraining and keep a time-aware evaluation.

Summary: The experiments

- Every experiment: hold all fixed, vary one knob, measure risk on fitting rows and on unseen rows, read two curves; four curves, four horizontal axes.
- A fixed model is graded honestly; a fitted model is graded kindly: $\mathbb{E}\widehat{R}_{\text{train}}(\widehat{f}) \le R(f^*_{\mathcal{H}}) \le \mathbb{E}R(\widehat{f})$.
- Three sets, three jobs; the split rule comes from the deployment question; leakage is tomorrow's information today.
- The best of many honest scores is optimistic, linearly in the noise and logarithmically in the number of candidates; the test set chose nothing, so it is used once.
- Cross-validation holds out each fold in turn, refits the whole pipeline inside each fold, then refits on all development rows; nested cross-validation when an honest number is needed without a test set.
- A hyperparameter is a knob training error would turn to maximal capacity; it is chosen at the bottom of the validation U; epochs by early stopping; boosting's knobs are coupled.
- Learning curves: training rises, validation falls, both toward approximation error plus noise; the gap is variance, the plateau is bias.
- Bias and variance are measured by refitting on many samples; capacity trades them, data lowers variance, averaging lowers variance to the floor $\rho\sigma^2$.

Chapter 3

From Fitting to Generalization

In Chapter 2 you learned how to fit: take the training rows, choose the parameters that make those rows look as good as possible, and call the result a model. This chapter asks the question that fitting cannot answer: will the model also be good on the next case, the one it has never seen? We give that worry a name, the generalization gap, and we prove that the score a model gets on its own training rows is always too flattering, because those rows chose the model and then graded it. We then see that the culprit is capacity: a flexible family can bend itself around the accidents of one sample. We split the error at a single input into three pieces, noise, bias and variance, and you will see that each piece answers to a different lever. The second half of the chapter is about the dials that control how flexible a model is allowed to be (penalties, early stopping, feature selection), and the rules that keep an evaluation honest: a split that looks like deployment, metrics that cannot hide a failure on the rare class, and no leakage. Keep one question in mind all the way through: *which of the regularities I see in my sample will still be there on the next case?* Formulas marked [on your sheet] are on your sheet.

How to read the formulas in this chapter

The chapter reuses the symbols of Chapter 2 (R, $\widehat{R}$, $\mathcal{H}$, λ, the three subsets) and adds a few. $\widehat{f}_{\mathcal{D}}$ is the model a fixed learning procedure produces when fed the dataset $\mathcal{D}$, and $G(\widehat{f}_{\mathcal{D}})$ is its generalization gap. $\mathbb{E}_{\mathcal{D}}[\cdot]$ is an average over *many possible training sets* drawn from the same world. $\bar{f}(x) = \mathbb{E}_{\mathcal{D}}[\widehat{f}_{\mathcal{D}}(x)]$ is the average prediction at x across those training sets. $\mathrm{Bias}(x)$ and $\mathrm{Var}(x)$ are the systematic error and the sample-to-sample spread of the prediction at x. ε and $\sigma^2(x)$ are the noise in Y around the best possible prediction and its variance. $\Omega(\theta)$ is a penalty saying how "complicated" θ is; the two standard ones are $\|\theta\|_2^2 = \sum_j \theta_j^2$ and $\|\theta\|_1 = \sum_j |\theta_j|$. $h \in \mathcal{C}$ is one full configuration of hyperparameters from a candidate set, and $\widehat{f}_h^{(-j)}$ is the model with configuration h fitted on all folds except fold j. TP, FP, TN, FN count true and false positives and negatives on a test set.

3.1 Generalization and Inductive Bias

What are we trying to get?

Chapter 2 taught us to *fit*: choose the parameters that make the training examples look good. But nobody deploys a model to re-predict its own training set. We want a precise name for "does the fit carry over to new cases," a clear reason why training performance cannot answer that question, and an honest account of what *does* decide the answer: the assumptions built into the learner.

Two averages of the same loss

Same loss, two populations of examples: the ones you have, and the ones you will meet.

Given a dataset $\mathcal{D}$ of N pairs $(x^{(i)}, y^{(i)})$, empirical risk minimisation picks the parameters $\widehat{\theta}$ that minimise $\widehat{R}_{\text{train}}(\theta)$. This solves a *fitting* problem. The purpose of learning is to perform well on new cases drawn from the environment in which the model will be used.

Definition 3.1.1 (Empirical risk, population risk, generalization gap)**.** For a model f, the *empirical risk* on a dataset $\mathcal{D}$ and the *population risk* are

$$\widehat{R}_{\mathcal{D}}(f) = \frac{1}{N}\sum_{i=1}^{N} \ell(y^{(i)}, f(x^{(i)})), \quad \text{[on your sheet]}$$

$$R(f) = \mathbb{E}_{(X,Y)\sim\mathbb{P}}[\ell(Y, f(X))]. \quad \text{[on your sheet]}$$

For the model $\widehat{f}_{\mathcal{D}}$ fitted on $\mathcal{D}$, the *generalization gap* is $G(\widehat{f}_{\mathcal{D}}) = R(\widehat{f}_{\mathcal{D}}) - \widehat{R}_{\mathcal{D}}(\widehat{f}_{\mathcal{D}})$ [on your sheet].

Reading the formula

Both terms of the gap refer to the *same* fitted model; only the source of the examples differs. $\widehat{R}_{\mathcal{D}}(\widehat{f}_{\mathcal{D}})$ is the score on the very examples that chose the model. $R(\widehat{f}_{\mathcal{D}})$ is the score on fresh cases from the world. A gap near 0 means training performance transfers; a large positive gap means the training score was flattering. If a spam filter makes 2% errors on its training emails and 9% on the emails of the next month, the gap is 0.07. Nothing about the filter changed between the two measurements; only the rows did.

Why training performance is special

Training data play two roles when training performance is reported: they first influence *which* model is selected, and are then used again to *evaluate* that model,

$$\mathcal{D} \longrightarrow \widehat{f}_{\mathcal{D}} \longrightarrow \widehat{R}_{\mathcal{D}}(\widehat{f}_{\mathcal{D}}).$$

Any accidental regularity in the sample is rewarded twice: once when it helps the model win, once when it is counted as a success. A second sample $\mathcal{D}'$ from the same population would generally produce another fitted model $\widehat{f}_{\mathcal{D}'}$; this sampling variability is part of the learning problem, not a defect of the notation.

Proposition 3.1.2 (Training risk of a fitted model is optimistic)**.** *Let the learning procedure return $\widehat{f}_{\mathcal{D}}$ minimising $\widehat{R}_{\mathcal{D}}$ over a class $\mathcal{H}$, with $\mathcal{D}$ an i.i.d.*

sample. Then

$$\mathbb{E}_{\mathcal{D}}[\widehat{R}_{\mathcal{D}}(\widehat{f}_{\mathcal{D}})] \;\le\; \min_{f\in\mathcal{H}} R(f) \;\le\; \mathbb{E}_{\mathcal{D}}[R(\widehat{f}_{\mathcal{D}})], \quad \textit{[on your sheet]}$$

so the expected generalization gap is nonnegative: $\mathbb{E}_{\mathcal{D}}[G(\widehat{f}_{\mathcal{D}})] \ge 0$.

Proof. Let $f^*_{\mathcal{H}}$ minimise R over $\mathcal{H}$. (1) Because $\widehat{f}_{\mathcal{D}}$ minimises the empirical risk, $\widehat{R}_{\mathcal{D}}(\widehat{f}_{\mathcal{D}}) \le \widehat{R}_{\mathcal{D}}(f^*_{\mathcal{H}})$ on every sample; this is the only place where minimisation is used. (2) Because $f^*_{\mathcal{H}}$ is a fixed function that did not look at the sample, $\mathbb{E}_{\mathcal{D}}[\widehat{R}_{\mathcal{D}}(f^*_{\mathcal{H}})] = R(f^*_{\mathcal{H}})$: an average of i.i.d. losses is an unbiased estimate of the expected loss. (3) Because $f^*_{\mathcal{H}}$ is the best member of the family, $R(\widehat{f}_{\mathcal{D}}) \ge R(f^*_{\mathcal{H}})$ for every sample. Chain the three inequalities and subtract to get the sign of the expected gap. □

Interpretation & discussion

Low training loss is *necessary* evidence that optimisation has fitted the data, but *incomplete* evidence that learning has succeeded. The training score sits below the best achievable risk, and the true risk of the fitted model sits above it; the rows that chose the model cannot grade it. The one word that makes a validation set different is *fixed*: it grades a model that was fixed before the validation rows were seen, so step (2) of the proof applies to it and its number is honest. That is the entire justification for validation and test sets.

Experiment: Seeing the gap

Setup. Any supervised dataset and a flexible family, say a deep tree or a high-degree polynomial.

What we hold fixed. The family and the training rows; we fit once.

What we change. Nothing yet. The point of this first experiment is to measure twice.

What we measure. The risk on the training rows, and the risk on rows that were never used, either held out before fitting or collected next month.

What the plot shows. Two bars. The held-out bar is higher, and the distance between them is the gap.

How to read it. Now repeat the whole thing with a more flexible family: the gap grows. Repeat it with more training rows: the gap shrinks. Those two movements are the whole story of this chapter, and every later section explains one of them.

What people do wrong: Reading a low training loss as proof that the model works

It feels right because the loss is a real number computed on real data. But a low training loss only proves that the optimiser did its job; it says nothing about whether learning carried over. It overstates performance in one direction only, upward, by at least the gap. You catch it by insisting on a held-out set: if the held-out number is far above the training number, the training number was flattering.

Stable and spurious regularities

Example 3.1.3 (Cows on grass, camels on sand)**.** Suppose cows in the training collection are photographed on grass and camels on sand. *Both* animal shape and background predict every training label. Shape is closely related to the intended task; background is a shortcut created by the way the images were collected. Training performance cannot tell the two cues apart, because both work on every observed image. A cow photographed on sand separates them: the shape cue says "cow," the background cue says "camel." This is the wolves-and-snow story of Chapter 1 seen from the fitting side: a spurious cue produces zero training loss and a large generalization gap.

Dataset composition is therefore *part of the learning method*: the examples determine which correlations the model can exploit. Similar shortcuts occur when a network segment or capture time predicts attacks, a hospital or scanner type identifies a diagnosis, or a customer identifier predicts an outcome in historical records.

Experiment: The shortcut probe

Setup. A classifier trained on the collection.

What we hold fixed. The fitted model. We do not retrain it for the main measurement.

What we change. Only the nuisance factor: we show the model a cow standing on sand, or feed it flows from a network segment it has never seen, or from a new week.

What we measure. Accuracy on these probe cases, next to accuracy on the original split.

What the plot shows. A short table. Original split high; probe cases lower if the shortcut was used.

How to read it. A drop on the probe is a shortcut. Run two controls alongside. First, shuffle the labels and retrain: performance must collapse to chance, and if it does not, something in the pipeline leaks the label. Second, delete the metadata columns and retrain: performance must *not* collapse, and if it does, the model was reading identifiers rather than behaviour.

Technique: Weakening shortcuts

Vary nuisance factors at *collection* time: several cameras, sites, seasons, devices, so that no nuisance factor lines up with the label. Use *data augmentation* only with transformations that are realistic and preserve the label (flip a cat photo, yes; flip a digit "6", no: it becomes a "9"). Evaluate under deployment-like conditions (Section 3.6); a random split hides shortcuts that are present in every part of the collection.

Telling them apart: shortcut versus leakage

Students mix these up because both produce a number that is too good. A shortcut is a cue that is genuinely there in the collection but will not hold at deployment, like the grass. Leakage is a feature that will not even exist at decision time, like a field filled in after the investigation. Different causes,

same inflated estimate, and both are caught by the same question: what will I actually know about the next case at the moment I decide?

Inductive bias

Definition 3.1.4 (Inductive bias)**.** A finite sample is compatible with many functions, including functions that agree on the sample and differ everywhere else. Learning therefore requires an *inductive bias*: assumptions that make some solutions more plausible than others beyond the observed examples. It enters through the *data and representation*, which determine which cues are available; the *hypothesis space*, which determines which relationships can be expressed; *regularisation*, which favours some fitting solutions over others; and the *optimisation algorithm*, which may prefer particular solutions when many parameters minimise the training objective. Here "bias" means a preference among solutions; it is broader than the statistical bias of Section 3.3.

Key insight

Generalization is never a property of the data alone. It succeeds when the assumptions encoded by the learning procedure remain true in the deployment environment. Every technique in this chapter is a way of choosing, strengthening or checking those assumptions.

Telling them apart: inductive bias versus statistical bias

Same word, two different objects. Inductive bias is a *preference among functions*, something the designer chooses through the four doors above. Statistical bias, $\mathrm{Bias}(x) = \bar{f}(x) - f^*(x)$, is a *number at each input*, the systematic error of a procedure, which we trade against variance. A bias that happens to be false at deployment, like "grass means cow," fails silently, with a perfect training score; you catch it only by probing.

How the professor asks it

Q. *Prove that the expected generalization gap of an empirical risk minimiser is nonnegative, and say exactly where the assumption "$\widehat{f}_{\mathcal{D}}$ minimises the empirical risk" is used.*
A. The claim is $\mathbb{E}[\widehat{R}_{\mathcal{D}}(\widehat{f}_{\mathcal{D}})] \le \mathbb{E}[R(\widehat{f}_{\mathcal{D}})]$. Let $f^*_{\mathcal{H}}$ be the population minimiser in the family. Minimisation is used exactly once: because $\widehat{f}_{\mathcal{D}}$ minimises the empirical risk, $\widehat{R}_{\mathcal{D}}(\widehat{f}_{\mathcal{D}}) \le \widehat{R}_{\mathcal{D}}(f^*_{\mathcal{H}})$ on every sample. Taking expectations turns the right side into $R(f^*_{\mathcal{H}})$, because $f^*_{\mathcal{H}}$ is fixed and an average of i.i.d. losses is unbiased for a fixed function. Finally $R(f^*_{\mathcal{H}}) \le R(\widehat{f}_{\mathcal{D}})$ pointwise, because $f^*_{\mathcal{H}}$ is the best in the family. Chaining the three gives the sandwich. What it means: a model's own training score is not evidence about new cases, and that is the whole reason validation sets exist.

Q. *A model scores 100% on a collection where all cows stand on grass and all camels on sand. Explain why this number is uninformative and design the experiment that would reveal the problem.*
A. Two cues, shape and background, predict every training label, and training performance cannot separate them: both give zero loss. Only inputs where

the two cues disagree can. So hold the animal fixed and manipulate the background: collect or construct cows on sand and camels on grass and measure accuracy on them; a collapse shows the model used the grass. Add a label-shuffle control, which must fall to chance, and a grouped or site-wise split. The fix is at collection time, by varying the background so that it no longer lines up with the label.

3.2 Model Capacity

What are we trying to get?

We want a vocabulary for "how flexible is this family of models," a proof that flexibility always looks good on training data, and a concrete demonstration that it does not always look good on new data.

Definition 3.2.1 (Capacity, nested families)**.** The *capacity* of a hypothesis space $\mathcal{H} = \{f_\theta : \theta \in \Theta\}$ describes how varied the functions in it can be. It is not determined by parameter count alone: structure, parameter constraints, regularisation, optimisation, and the amount and geometry of the data all matter. Typical capacity-controlling hyperparameters are polynomial degree, tree depth, the number of units in a network, and the neighbourhood size k in kNN. Families are *nested* when $\mathcal{H}_1 \subseteq \mathcal{H}_2 \subseteq \cdots \subseteq \mathcal{H}_M$: every function of a smaller family remains available in a larger one.

Proposition 3.2.2 (Training error favours capacity)**.** *For nested families* $\mathcal{H}_m \subseteq \mathcal{H}_{m+1}$, $\min_{f\in\mathcal{H}_{m+1}} \widehat{R}_{\text{train}}(f) \leq \min_{f\in\mathcal{H}_m} \widehat{R}_{\text{train}}(f)$ *[on your sheet]. Increasing capacity cannot make the best achievable training error worse.*

Proof. Let f_m minimise $\widehat{R}_{\text{train}}$ over $\mathcal{H}_m$. Since f_m also belongs to $\mathcal{H}_{m+1}$, it is one of the candidates there, so the minimum over the larger family cannot exceed its value. The same argument applies whether the knob is degree, depth, $1/k$, $1/\lambda$ or epochs. □

Reading the formula

This is why training loss *alone* systematically favours richer families: it can only go down as you add flexibility. It does not follow that the population risk goes down. Additional functions can fit stable structure, but they can also fit accidental variation. The population risk has no such monotonicity, and that is why the validation curve, never the training curve, chooses the capacity.

Underfitting and overfitting

Definition 3.2.3 (Underfitting, overfitting)**.** An *underfitting* model is too restrictive to capture important structure: it usually has high training *and* high validation error. An *overfitting* model is too sensitive to the particular training sample: it may have very low training error and substantially higher validation error. The distinction is not whether the model is complicated in isolation, but whether its effective flexibility matches the available evidence: a family that is unstable with a small dataset may be well constrained by a

much larger one.

Example 3.2.4 (Three polynomials on four points, judged on a fifth)**.** Chapter 2 fitted the points $(0,1),(1,3),(2,2),(3,4)$ with polynomials of degree 0, 1 and 3, with training risks 1.25, 0.45 and 0. Now a *validation* point arrives: $(4,5)$. Degree 0: the constant 2.5 predicts 2.5; squared error $(5-2.5)^2 = 6.25$. Degree 1: the line $0.8x + 1.3$ predicts 4.5; squared error 0.25. Degree 3: the interpolating cubic is $f(x) = 1 + 5.5x - 4.5x^2 + x^3$ (check: $f(1) = 3$, $f(2) = 2$, $f(3) = 4$); it predicts $f(4) = 1 + 22 - 72 + 64 = 15$; squared error 100.

degree	0	1	3
training risk	1.25	0.45	0.00
validation error at $x = 4$	6.25	0.25	100

The constant underfits (bad everywhere). The cubic overfits: perfect on the four points it was shaped by, and wildly wrong one step outside them, because it bent itself around the noise in the sample. The line, which fits the training data worst of the last two, is the only one worth deploying.

Experiment: The validation curve: sweep capacity

Setup. Training rows, validation rows (or folds), and a nested family with one knob.

What we hold fixed. The rows, the preprocessing (fitted on the training rows), the metric.

What we change. One knob over a grid: the degree $0, 1, 2, \ldots$, or the depth, or k (plot against $1/k$ so that capacity increases to the right).

What we measure. At each value we fit and record two numbers, training risk and validation risk.

What the plot shows. The training curve falls monotonically, by Proposition 3.2.2. The validation curve falls at first, because the extra flexibility captures real structure, and then rises, because the extra flexibility starts capturing noise: the U.

How to read it. Left of the minimum both errors are high, underfitting. Right of it the training error is low and the validation error rises, overfitting. The minimum of the U is the capacity your data can support, and if you collect more data the minimum moves to the right. For kNN remember the direction: small k means small, highly adaptive neighbourhoods (low training error, high sensitivity), large k averages broadly.

What people do wrong: Comparing families by training error, and celebrating zero training error

Comparing by training error always picks the largest family, by Proposition 3.2.2. And zero training error is not an achievement: with enough capacity it is guaranteed (enough flexibility fits anything, Chapter 2). Both habits inflate your opinion of the flexible model. You catch them by reading the validation curve and nothing else. A second trap: for kNN, *decreasing* k increases capacity; students routinely read that axis backwards.

Telling them apart: underfitting versus overfitting

Underfitting: training error high, validation error high and close to it; the fix is more capacity, better features or less regularisation. Overfitting: training error low, validation error much higher; the fix is regularisation, less capacity, more data or ensembles. Well matched: training error low and validation error slightly above it; stop there. The diagnosis is always made from the pair of numbers, never from one.

How the professor asks it

Q. *Why does training error never increase when you go from polynomial degree 2 to degree 3, and why does this not imply that degree 3 generalises better?*
A. Degree-2 polynomials are degree-3 polynomials with the cubic coefficient set to zero, so the degree-2 minimiser is a candidate in the degree-3 family and the degree-3 minimum is at most its value: nested families, training error nonincreasing. The population risk has no such monotonicity: the extra coefficient can fit stable structure or accidental variation, and on four points the cubic fits all the noise (training risk 0, validation error 100 in Example 3.2.4). Which happened is decided by validation data, never by the training score.

3.3 Bias and Variance

What are we trying to get?

"Sensitive to the sample" is a vague phrase. We want to make it a number, separate it from a second kind of error (being systematically wrong), and prove that squared prediction error splits exactly into noise plus these two quantities. That decomposition is the theoretical spine of the whole chapter.

Imagine rerunning the entire training procedure on many different samples from the same world, then looking at the cloud of predictions you would get at one input.

Imagine repeatedly drawing training sets $\mathcal{D}_1, \mathcal{D}_2, \dots$ from the same population and applying exactly the same learning procedure. The fitted functions $\widehat{f}_{\mathcal{D}_1}, \widehat{f}_{\mathcal{D}_2}, \dots$ need not coincide.

Definition 3.3.1 (Bias and variance of a learning procedure)**.** For squared-error regression let $f^*(x) = \mathbb{E}[Y \mid X = x]$ be the optimal regression function and $\bar{f}(x) = \mathbb{E}_{\mathcal{D}}[\widehat{f}_{\mathcal{D}}(x)]$ the average prediction of the procedure at x. The *pointwise bias* and *pointwise variance* are

$$\mathrm{Bias}(x) = \bar{f}(x) - f^*(x), \qquad \mathrm{Var}(x) = \mathbb{E}_{\mathcal{D}}\Big[\big(\widehat{f}_{\mathcal{D}}(x) - \bar{f}(x)\big)^2\Big]. \quad \text{[on your sheet]}$$

Reading the formula

Fix one input x, say a particular house. Train a thousand times on a thousand different training sets and record the thousand predicted prices. $\bar{f}(x)$ is their average. *Bias* asks: is that average off from the true conditional mean price, in a consistent direction? *Variance* asks: how widely do the thousand predictions scatter around their own average? A model that always predicts "1 million" has zero variance and possibly large bias; a very flexible model may have tiny bias and huge variance. Please notice that these are properties of the *procedure*, the family together with the fitting rule and the sample size. One

fitted model is one draw; it has no variance. Restrictive models and inadequate features create bias; merely collecting more examples may not remove it. High capacity, small datasets, irrelevant variables and unstable feature selection increase variance; more representative data reduce it.

Theorem 3.3.2 (Bias–variance decomposition)**.** *Assume* $Y = f^*(X) + \varepsilon$ *with* $\mathbb{E}[\varepsilon \mid X] = 0$ *and* $\mathrm{Var}(\varepsilon \mid X = x) = \sigma^2(x)$*, and assume the training sample* $\mathcal{D}$ *is independent of the test pair* (x, Y)*. Then at every fixed input* x*,*

$$\mathbb{E}_{\mathcal{D},\,Y|x}\Big[\big(Y - \widehat{f}_{\mathcal{D}}(x)\big)^2\Big] = \sigma^2(x) + \mathrm{Bias}(x)^2 + \mathrm{Var}(x). \quad \textit{[on your sheet]}$$

Proof. **Step 1 (split off the noise).** Write $Y - \widehat{f}_{\mathcal{D}}(x) = \varepsilon + (f^*(x) - \widehat{f}_{\mathcal{D}}(x))$ and square: $\varepsilon^2 + 2\varepsilon(f^* - \widehat{f}_{\mathcal{D}}) + (f^* - \widehat{f}_{\mathcal{D}})^2$. Take expectations. $\mathbb{E}[\varepsilon^2 \mid x] = \sigma^2(x)$. In the cross term, ε is independent of $\mathcal{D}$ (the test noise has nothing to do with the training sample), so the expectation factorises into $\mathbb{E}[\varepsilon \mid x] \cdot \mathbb{E}_{\mathcal{D}}[f^*(x) - \widehat{f}_{\mathcal{D}}(x)]$, and the first factor is 0.

Step 2 (split around the average prediction). Write $f^*(x) - \widehat{f}_{\mathcal{D}}(x) = (f^*(x) - \bar{f}(x)) + (\bar{f}(x) - \widehat{f}_{\mathcal{D}}(x))$. The first bracket is a constant, $-\mathrm{Bias}(x)$, whose square is $\mathrm{Bias}(x)^2$; the second has $\mathbb{E}_{\mathcal{D}}$-mean zero by definition of $\bar{f}$, so the cross term $2(f^* - \bar{f})\,\mathbb{E}_{\mathcal{D}}[\bar{f} - \widehat{f}_{\mathcal{D}}]$ vanishes; the square of the second bracket averages to $\mathrm{Var}(x)$. Adding the pieces gives the result. □

Key insight

Twice the same move: add and subtract a mean, square, and watch the cross term die because a mean-zero quantity multiplies a constant. First around the true regression function (killing the noise cross term), then around the average prediction (killing the sampling cross term).

Reading the formula

Three sources of error at one input. $\sigma^2(x)$ is *irreducible noise*: even the perfect predictor f^* makes this error, because Y has randomness that x does not explain. $\mathrm{Bias}(x)^2$ is the *systematic error* of the procedure, the part that repeating the experiment never averages away. $\mathrm{Var}(x)$ is *sample sensitivity*, the part that a different training set would change. Increasing capacity often reduces bias while increasing variance, producing the U-shaped validation curve. This is a useful diagnostic principle, not a universal law.

Experiment: Measuring bias and variance by repeated sampling

Setup. A simulated world where we know f^* and the noise σ, a procedure (family plus fitting rule), and a grid of test inputs.

What we hold fixed. f^*, σ, the sample size N and the test grid.

What we change. The training set: we draw 500 of them and refit the procedure each time. Then, in a second sweep, the capacity (degree, or k); in a third, N.

What we measure. At each test input we now have a cloud of 500 predictions. The centre of the cloud minus $f^*(x)$ is the bias; the spread of the cloud

is the variance. We average both over the grid and compare with the test error measured directly.

What the plot shows. Against capacity: the squared bias falls, the variance rises, and their sum plus σ^2 traces exactly the U you saw on the validation curve. Against N: the variance falls and the bias stays flat.

How to read it. That flat line is the most important thing in the picture. More data buys variance reduction, not bias reduction. And as N grows, the minimum of the U moves to higher capacity, which is the whole story of "more data lets you afford a bigger model."

Example 3.3.3 (Bias and variance computed exactly)**.** The simplest regression problem: no input, $Y = \mu + \varepsilon$ with $\mathrm{Var}(\varepsilon) = \sigma^2$, so $f^* = \mu$. The training set is N draws. *Sample mean*, $\widehat{f} = \frac{1}{N}\sum_i y^{(i)}$: $\mathbb{E}_{\mathcal{D}}[\widehat{f}] = \mu$, bias 0, variance σ^2/N. *Ignore the data*, $\widehat{f} = 0$: bias $-\mu$, variance 0. *Shrunk mean*, half the sample mean: bias $-\mu/2$, variance $\sigma^2/(4N)$. With $\sigma^2 = 1$ and $N = 4$:

bias^2 + variance	sample mean	ignore data	shrunk mean
$\mu = 1$	$0 + 0.25 = 0.25$	$1 + 0 = 1$	$0.25 + 0.0625 = 0.3125$
$\mu = 0.2$	$0 + 0.25 = 0.25$	$0.04 + 0 = 0.04$	$0.01 + 0.0625 = 0.0725$

When the truth is far from zero the unbiased procedure wins. When the truth is near zero both biased procedures beat it: they trade a little systematic error for a large reduction in variance. Regularisation (Section 3.4) is exactly this trade, made tunable.

What people do wrong: Two sentences I hear every year

"Doubling the data halves the bias." No: it halves the variance in Example 3.3.3; the bias comes from the family and the features, and N does not touch it. "Moderate capacity gives low bias and low variance." At the bottom of the U both are positive, just small; zero bias and zero variance would mean the procedure returns the truth regardless of the sample, which means it already knew the answer. And a third: speaking of "the variance of this trained model." One fitted model is one draw; the variance belongs to the procedure, and to measure it you must refit.

Telling them apart: three reference points

A 1-nearest-neighbour rule has small bias (it follows the data closely) and large variance (one noisy neighbour decides). A 50-nearest-neighbour rule has higher bias (it averages far from x) and variance of about $\sigma^2/50$; on your sheet, Bias $= \frac{1}{k}\sum_{j \le k} f(x^{(j)}) - f(x)$ and Var $= \sigma^2/k$ [on your sheet]. A constant model has zero variance and large bias. Keep these three in mind whenever you are asked to place a method on the bias–variance map.

Remark 3.3.4 (Three sources of excess error, and double descent). Relative to the Bayes risk, a learned model's extra risk decomposes conceptually into *approximation error* (the hypothesis space cannot express a good enough function), *estimation error* (finite data do not reliably identify the best function in the space) and *optimisation error* (the algorithm fails to reach a good solution of its own objective). Increasing capacity can reduce approximation error while increasing

estimation error. Modern overparameterised models can also display *double descent*: as capacity approaches the smallest level at which the training sample can be interpolated, validation error may increase sharply; beyond this interpolation threshold, further capacity can reduce validation error again [on your sheet]. The second descent depends on regularisation, the optimiser and the data geometry, and does not occur in every problem. The classical U-shape remains a valuable first model, but validation behaviour in modern systems must be examined rather than assumed.

How the professor asks it

Q. *A procedure has* $\mathrm{Bias}(x) = 0.3$, $\mathrm{Var}(x) = 0.05$ *and noise* $\sigma^2 = 0.1$ *at* x. *Compute its expected squared error, state which part more data could reduce, and which part a richer family could reduce.*
A. $0.1 + 0.3^2 + 0.05 = 0.1 + 0.09 + 0.05 = 0.24$. More data acts on the 0.05, because variance shrinks roughly like $1/N$ for a stable procedure. A richer family acts on the 0.09, because it can move the average prediction closer to the truth, but it will raise the variance while doing so. Nothing acts on the 0.1 except new information in the features, because that part is the randomness of Y that x does not explain.

Q. *Can a learning procedure have zero bias and zero variance? Justify.*
A. Only if $\widehat{f}_{\mathcal{D}}(x) = f^*(x)$ for every training set, that is if the procedure returns the truth regardless of the sample, which means it already knew the answer. Any procedure that actually uses the data has either sample dependence (variance) or a restriction that keeps it from f^* (bias), and usually both.

3.4 Controlling Complexity

What are we trying to get?

If flexibility is the problem, we need dials that reduce it *without* throwing away parameters: penalties on parameter size, and stopping training early. We want to see exactly what a penalty does to the hypothesis space, why the L_1 penalty produces exact zeros, and how the strength of the penalty is chosen.

Definition 3.4.1 (Regularised ERM)**.** Given a *penalty* $\Omega : \Theta \to [0, \infty)$ and a strength $\lambda \geq 0$,

$$\widehat{\theta}_\lambda \in \arg\min_{\theta \in \Theta} \Big[\widehat{R}_{\text{train}}(\theta) + \lambda\, \Omega(\theta) \Big], \quad \text{[on your sheet]}$$

$$\Omega_2(\theta) = \|\theta\|_2^2 \text{ (ridge)}, \qquad \Omega_1(\theta) = \|\theta\|_1 \text{ (lasso)}. \quad \text{[on your sheet]}$$

Reading the formula

The objective is now "fit the data *and* keep the parameters small." λ says how much a unit of penalty is worth in units of training loss: $\lambda = 0$ is plain ERM; large λ makes any sizeable coefficient too expensive, so the fit is pulled toward the origin. With two coefficients $(3, -4)$, $\Omega_2 = 9+16 = 25$ and $\Omega_1 = 3+4 = 7$.

Theorem 3.4.2 (A penalty is a soft constraint)**.** *Let $\widehat{\theta}_\lambda$ solve the penalised problem and put $c = \Omega(\widehat{\theta}_\lambda)$. Then $\widehat{\theta}_\lambda$ also solves the* constrained *problem $\min_\theta \widehat{R}_{\text{train}}(\theta)$ subject to $\Omega(\theta) \le c$ [on your sheet]. Consequently regularised ERM fits within the family $\mathcal{H}_c = \{f_\theta : \Omega(\theta) \le c\}$, and these families are nested: $c_1 < c_2 \Rightarrow \mathcal{H}_{c_1} \subseteq \mathcal{H}_{c_2}$.*

Proof. Take any θ with $\Omega(\theta) \le c$. Optimality of $\widehat{\theta}_\lambda$ for the penalised problem gives $\widehat{R}(\widehat{\theta}_\lambda) + \lambda c \le \widehat{R}(\theta) + \lambda\Omega(\theta)$, and since $\Omega(\theta) \le c$ the right side is at most $\widehat{R}(\theta) + \lambda c$. Cancel λc: $\widehat{R}(\widehat{\theta}_\lambda) \le \widehat{R}(\theta)$. So no feasible θ beats $\widehat{\theta}_\lambda$, and $\widehat{\theta}_\lambda$ is itself feasible. Nesting is immediate from $\Omega(\theta) \le c_1 \Rightarrow \Omega(\theta) \le c_2$. □

Key insight

A stronger penalty confines the solution to a smaller *effective* family, even though the number of parameters has not changed. Capacity is about which functions can actually win the fitting contest, not about how many numbers the model stores. And because the families are nested and a larger λ gives a smaller c, the training risk is nondecreasing in λ and smallest at $\lambda = 0$: the training data would always answer "no regularisation," which is why λ is chosen on validation.

Proposition 3.4.3 (Ridge in one variable: shrinkage)**.** *For the model $f_w(x) = wx$ under squared loss with L_2 penalty,*

$$\hat{w}_\lambda = \frac{\sum_i x^{(i)} y^{(i)}}{\sum_i (x^{(i)})^2 + N\lambda} = \hat{w}_{\text{LS}} \cdot \frac{\sum_i (x^{(i)})^2}{\sum_i (x^{(i)})^2 + N\lambda}, \quad \textit{[on your sheet]}$$

the unpenalised least-squares slope times a factor in $(0, 1]$: the coefficient is shrunk *toward 0, smoothly, never exactly to 0 for finite λ.*

Proof. The objective $J(w) = \frac{1}{N}\sum_i (y^{(i)} - wx^{(i)})^2 + \lambda w^2$ is a convex quadratic in w, so setting the derivative to zero gives the minimum: $J'(w) = -\frac{2}{N}\sum_i x^{(i)}(y^{(i)} - wx^{(i)}) + 2\lambda w = 0$. Collecting w gives $w\big(\sum_i (x^{(i)})^2 + N\lambda\big) = \sum_i x^{(i)} y^{(i)}$; the factor form follows by dividing numerator and denominator by $\sum_i (x^{(i)})^2$. □

Reading the formula

With $\sum x^2 = 14$, $\sum xy = 19$ and $N = 4$ (the numbers of the four-point example, ignoring the intercept): $\lambda = 0$ gives $19/14 \approx 1.357$; $\lambda = 1$ gives $19/18 \approx 1.056$; $\lambda = 10$ gives $19/54 \approx 0.352$. The bigger the penalty, the closer to zero, but never zero: the derivative of λw^2 vanishes at $w = 0$, so nothing holds the coefficient there. As $\lambda \to \infty$ all penalised coefficients vanish and the model is reduced to its intercept. When two features are identical, ridge splits the weight equally between them, because $a^2 + b^2$ is smallest at $a = b$ for a fixed sum.

Proposition 3.4.4 (Lasso in one variable: soft thresholding)**.** *For $z \in \mathbb{R}$ and $\lambda > 0$, the minimiser of $\frac{1}{2}(w - z)^2 + \lambda|w|$ over $w \in \mathbb{R}$ is*

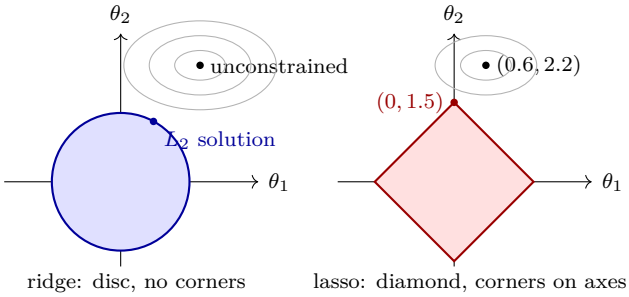
$$\hat{w} = \operatorname{sign}(z)\, \max\big(|z| - \lambda,\ 0\big). \quad \textit{[on your sheet]}$$

In particular $\hat{w} = 0$ exactly whenever $|z| \le \lambda$.

Proof. Suppose first $z > 0$. For $w \geq 0$ the objective is $\frac{1}{2}(w-z)^2 + \lambda w$, a parabola with unconstrained minimiser $w = z - \lambda$; if $z > \lambda$ this is positive and gives the minimum on $[0, \infty)$; if $z \leq \lambda$ the parabola is increasing on $[0, \infty)$ and the minimum on that side is at $w = 0$. For $w < 0$ the objective is $\frac{1}{2}(w-z)^2 - \lambda w$, whose unconstrained minimiser $z+\lambda$ is positive, so the objective decreases toward 0 and no negative w beats $w = 0$. Combining: $\hat{w} = \max(z - \lambda, 0)$ for $z > 0$. The case $z < 0$ is symmetric, and $z = 0$ gives $\hat{w} = 0$. □

Reading the formula

z is where the unpenalised fit would put the coefficient. The L_1 penalty moves it toward zero by exactly λ, and if it would cross zero, it stops *at* zero, because the kink of $|w|$ at 0 absorbs any pull of size up to λ. With $\lambda = 0.5$: $z = 2.2 \to 1.7$; $z = 0.4 \to 0$; $z = -3 \to -2.5$. With $\lambda = 1.5$ and $z = 1.2$: 0. Geometrically, the constraint region $\|\theta\|_1 \leq c$ is a diamond whose corners lie on the coordinate axes, and an optimum often lands on a corner where some coordinates are exactly zero; the slides' two-dimensional example has unconstrained optimum $(0.6, 2.2)$ and lasso solution $(0, 1.5)$, dropping the first feature altogether. L_2's region is a disc with no corners, so it shrinks everything and keeps everything. L_1 therefore performs *embedded feature selection*, although correlated features can make the selected subset unstable, and in more than one dimension the lasso solution is not obtained by coordinate-wise thresholding, which is why the result is stated in one variable.



Experiment: The regularisation path: sweep λ

Setup. Standardised features (standardised on the training rows), ridge and lasso, training and validation rows.

What we hold fixed. The rows, the features, the metric; the intercept is left unpenalised.

What we change. λ on a logarithmic grid, $10^{-3}, 10^{-2}, \ldots, 10^{2}$, because the useful range spans orders of magnitude.

What we measure. Every coefficient at every λ (the *path*), plus the training and validation risk.

What the plot shows. The coefficient paths against $\log \lambda$: ridge paths shrink smoothly toward zero and never reach it; lasso paths hit zero one after another, so the number of nonzero coefficients steps down. Below them, the training risk climbs from its minimum at $\lambda = 0$ and the validation risk makes its U.

How to read it. The minimum of the U gives $\hat{\lambda}$. On the lasso path, the order in which coefficients die is a ranking of the features. With two near-identical features ridge splits the weight while the lasso keeps one and may switch which

one between folds, which is the instability you must report honestly. Without standardisation a feature in millimetres is penalised a thousand times more than the same feature in metres, and the path is meaningless.

Choosing the strength, and early stopping

The regularisation strength is a hyperparameter: for each candidate λ, training estimates $\widehat{\theta}_\lambda$; validation selects $\hat{\lambda} \in \arg\min_{\lambda \in \mathcal{C}} \widehat{R}_{\text{val}}(\widehat{f}_\lambda)$ [on your sheet]; the test data stay outside this selection.

Definition 3.4.5 (Early stopping)**.** Iterative optimisation produces a sequence $\widehat{\theta}_0, \widehat{\theta}_1, \ldots$. Early iterations often learn strong recurring patterns; later ones may fit exceptional observations or noise. *Early stopping* retains the checkpoint with smallest validation risk, $t^* = \arg\min_t \widehat{R}_{\text{val}}(\widehat{\theta}_t)$ [on your sheet]. Training time thereby controls effective capacity while the parameter count stays fixed. A *patience* p lets training continue p epochs without improvement before stopping, keeping the best checkpoint seen.

Experiment: The training curve over epochs

Setup. An iterative learner, training and validation rows.

What we hold fixed. Everything except the number of epochs.

What we change. The epoch t: we simply keep training, saving a checkpoint after every epoch.

What we measure. Training and validation loss after each epoch.

What the plot shows. The training loss falls throughout. The validation loss falls, reaches its bottom at t^*, and then rises.

How to read it. When you finally stop, you roll back to the checkpoint of t^*, never the last one. Patience protects against stopping on a temporary wobble of a noisy curve; it can still miss a later decrease under epoch-wise double descent. This is the same U as the capacity sweep, with epochs on the horizontal axis instead of degree; it is not a different phenomenon.

Technique: Regularisation in practice

Standardise features on the training rows before penalising. Do not penalise the intercept. Search λ on a logarithmic grid. Use L_1 when you believe few features matter and want to know which; L_2 when many features contribute a little. Early stopping is regularisation for free in any iterative method; always keep the best checkpoint, not the last. If two configurations are within one standard error of each other on validation, treat them as tied and prefer the simpler.

What people do wrong: Four things people do with λ

They choose λ on the training data, which always answers zero (Theorem 3.4.2 and the nested-family argument). They penalise unstandardised features, so the penalty depends on the units. They penalise the intercept, which is not a flexibility knob at all. They search λ on a linear grid, when the useful range spans orders of magnitude. And with early stopping: they keep the last

epoch, or they stop on the test loss, which quietly turns the test set into a validation set.

Telling them apart: three dials for one quantity

Students treat penalties, family restrictions and early stopping as unrelated. All three shrink the set of functions that can win the fitting contest. A penalty is a soft constraint: it trades fit against size at the rate λ. Restricting the family (degree, depth) is a hard constraint. Early stopping limits how far the optimiser is allowed to travel. Ridge multiplies every coefficient by a factor and never reaches zero, keeping everything; the lasso subtracts λ and stops at zero, producing exact zeros, but it is unstable under correlation. Remember the direction too: capacity grows with degree, depth and epochs, but with *smaller* λ and *smaller* k, so on those axes the U is mirrored.

How the professor asks it

Q. *Derive the one-variable ridge solution and explain why it can never set the coefficient exactly to zero, while the lasso can.*
A. Minimise $\frac{1}{N}\sum(y-wx)^2+\lambda w^2$: the derivative $-\frac{2}{N}\sum x(y-wx)+2\lambda w=0$ gives $\hat{w}_\lambda=\sum xy/(\sum x^2+N\lambda)$, the least-squares slope times $\sum x^2/(\sum x^2+N\lambda)\in(0,1]$. The factor is strictly positive for finite λ, so the coefficient keeps its sign and is only scaled; the deeper reason is that the derivative of λw^2 at $w=0$ is zero, so there is no force holding the coefficient at zero. The L_1 penalty has a kink at zero whose subgradient is the whole interval $[-\lambda,\lambda]$, so whenever the pull of the data $|z|$ is at most λ the kink absorbs it and the solution is exactly zero: the soft threshold $\operatorname{sign}(z)\max(|z|-\lambda,0)$.

Q. *Two identical features are included in a ridge model and in a lasso model. What does each do with them, and why does this matter for interpreting a lasso's selected subset?*
A. Ridge splits the weight equally, because for a fixed sum $a+b$ the penalty a^2+b^2 is smallest at $a=b$. The lasso is indifferent among all same-sign splits, because $|a|+|b|$ is then constant, so it may keep either feature and drop the other, and which one survives can change between folds or samples. A lasso's selected subset is therefore an unstable ranking under correlation; report groups of correlated features, or the path across folds, not a single survivor.

3.5 Feature and Model Selection

What are we trying to get?

Choosing *which inputs* to use is a model choice like any other, and it can go wrong in the same ways. We want to see why exhaustive search is impossible, what the practical searches do, why they must live inside the training pipeline, and why the validation set itself can be worn out by overuse.

Feature selection changes the representation supplied to the learner. It can remove irrelevant variables that increase estimation difficulty, reduce redundancy, improve interpretability, and exclude variables unavailable at decision time. It is a data-dependent model choice and must be evaluated accordingly.

Proposition 3.5.1 (Subset selection is combinatorial)**.** *With d features there are 2^d subsets [on your sheet], since each feature is independently in or out. The idealised search over all subsets requires 2^d fits; for $d = 30$, $2^{30} =$* 1,073,741,824*.*

Definition 3.5.2 (Filters and greedy searches)**.** A *correlation filter* scores each variable separately, e.g. $s_j = |\widehat{\mathrm{Corr}}(X_j, Y)|$, and keeps the top-scoring ones. It is fast and model-independent, but it misses nonlinear relationships and interactions, and it ignores redundancy: two highly ranked features may carry the same information. *Forward selection* starts from $S = \emptyset$ and repeatedly adds the feature giving the largest validation improvement. *Backward elimination* starts from all features and repeatedly removes the least useful; both cost about $d(d+1)/2$ fits [on your sheet] and both are greedy: an early choice can block a better combination later, and backward elimination requires fitting the full model, which is impossible when $d > N$. The *embedded* route is the lasso: one regularisation path does the selection.

Example 3.5.3 (Two features that are useless alone and perfect together)**.** Let X_1, X_2 be independent fair coin flips (± 1) and $Y = X_1 X_2$. Each feature has correlation 0 with Y (knowing X_1 alone says nothing about the sign of the product), so a correlation filter discards both. Together they determine Y exactly. Filters see one variable at a time; interactions are invisible to them. Weak marginal correlation never proves that a feature is useless jointly with others.

Experiment: Feature selection inside versus outside the folds

Setup. 300 rows, 5,000 features, and labels generated independently of the features, so there is nothing to learn.

What we hold fixed. The data and 10-fold cross-validation of logistic regression on 20 features.

What we change. Only where the selection step runs: (a) rank the features by correlation on all 300 rows, keep the top 20, then cross-validate; (b) inside each fold, rank on that fold's 270 training rows only.

What we measure. The cross-validation accuracy in both arrangements.

What the plot shows. Arrangement (a) reports above 90%; arrangement (b) reports about 50%, which is the truth.

How to read it. In (a) the validation rows helped choose the representation they are then scored on: with 5,000 features and 300 rows, some features correlate strongly with the labels by pure chance, and ranking on all rows picks out exactly those lucky ones. All data-dependent scoring and subset search belong *inside* the training pipeline and are repeated within every fold.

Gotcha

All data-dependent feature scoring and subset search belong *inside* the training pipeline. In cross-validation they must be repeated separately within every training fold. Otherwise information from a validation fold shapes the representation that is then evaluated on that fold: *leakage*, and an optimistic

estimate.

The roles of the three subsets, and their overuse

Let h denote an entire hyperparameter configuration (features, model family, λ, k, thresholds), not only one number. The development loop fits $\widehat{f}_h$ on training data, measures validation performance, and updates the configuration: $\hat{h} \in \arg\min_{h \in \mathcal{C}} \widehat{R}_{\text{val}}(\widehat{f}_h)$ [on your sheet]. The test set remains outside this loop.

Proposition 3.5.4 (The best of many validation scores is optimistic)**.** *Let $\widehat{R}_{\text{val}}(\widehat{f}_h)$, $h \in \mathcal{C}$, be validation risks of models fitted without the validation data, so that each is an unbiased estimate of $R(\widehat{f}_h)$. Then*

$$\mathbb{E}\Big[\min_{h \in \mathcal{C}} \widehat{R}_{\text{val}}(\widehat{f}_h)\Big] \;\le\; \min_{h \in \mathcal{C}} R(\widehat{f}_h), \qquad \textit{[on your sheet]}$$

with strict inequality as soon as the estimates are noisy and no single h is best in every sample.

Proof. For every $h_0 \in \mathcal{C}$, $\min_h \widehat{R}_{\text{val}}(\widehat{f}_h) \le \widehat{R}_{\text{val}}(\widehat{f}_{h_0})$ pointwise; take expectations, which turns the right side into $R(\widehat{f}_{h_0})$ because each candidate's score is honest; this holds for every h_0, so it holds for the best one. □

Reading the formula

Try a hundred configurations and report the lowest validation score: that number is the minimum of a hundred noisy estimates, and minima of noisy things are pulled downward, because picking the best partly picks the luckiest noise. Every reported validation result also influences the next design choice, so the development process gradually adapts to the validation set. The size of the effect is about $\sigma/\sqrt{\pi}$ for two equal candidates and $\sigma\sqrt{2\ln n}$ for n [on your sheet]: linear in the noise σ of the estimate and only logarithmic in n, so a small validation set hurts more than a long search. The final test set protects against this accumulated adaptation; it does *not* repair it if its results are then used to continue development, because at that point it has become another validation set.

Experiment: Adaptive overfitting of a validation set

Setup. Labels that are pure coin flips, 1,000 rows, split 500/500.

What we hold fixed. The data; every "model" is a random rule that ignores x.

What we change. The number n of random rules tried: $1, 10, 100, 1{,}000, 10{,}000$; for each n we keep the rule with the best validation accuracy.

What we measure. The best validation accuracy, and the accuracy of that same rule on the 500 test rows.

What the plot shows. The best validation accuracy rises with n: about 50%, 53%, 56%, 57%, 59%. The test accuracy of the winner stays at 50% for every n.

How to read it. Nothing was learned, yet the validation number improved steadily. Any real development loop does a weaker version of this. The flat

test line is what an untouched test set buys; start choosing among test scores and the test line starts climbing the same way.

Definition 3.5.5 (K-fold cross-validation, restated for configurations)**.** Partition the development data into K folds. For configuration h and fold j, fit $\widehat{f}_h^{(-j)}$ without fold j and evaluate it on fold j; then

$$\widehat{R}_{\mathrm{CV}}(h) = \frac{1}{K}\sum_{j=1}^{K} \widehat{R}_{\mathrm{val}}^{(j)}(\widehat{f}_h^{(-j)}), \qquad \hat{h} \in \arg\min_{h\in\mathcal{C}} \widehat{R}_{\mathrm{CV}}(h). \quad \text{[on your sheet]}$$

Each observation validates once and trains $K-1$ times.

Cross-validation uses limited development data efficiently, at roughly K fits per configuration; what it estimates is the risk of the *procedure* at training size $N(K-1)/K$, not of any one fold model, so after choosing $\hat{h}$ you refit on all development rows. Its estimate still varies with the available sample; standard random folds assume exchangeable observations, so grouped or temporal dependence needs adapted folds; and it does not replace final evaluation on an untouched test set. When you cannot afford a test set, *nested* cross-validation, with an outer loop for estimation and an inner loop for selection, is honest for the whole procedure "select, then fit."

What people do wrong: Wearing out the validation set

Reporting the winner's validation or cross-validation score as the final number; continuing to develop after looking at the test score, at which point the test set has become one more validation set; running the test set "just to see" after each change and keeping the change when the number improved; choosing the threshold or the metric after seeing test results; using random folds on grouped or temporal rows; deploying one of the fold models instead of refitting on all development rows.

Telling them apart: validation set versus cross-validation versus test set

A single validation set is one slice, noisy if small: the standard error of an accuracy near 0.9 on 100 rows is about 0.03. Cross-validation holds every row out once, so it is less noisy, but it is still optimistic for the winner. The test set chose nothing, which is why it is honest, and only once. Leave-one-out looks ideal but its N models differ by one row, so their scores are strongly correlated and their average is unstable, at N fits; five or ten folds are the usual compromise.

How the professor asks it

Q. *With 300 rows and 5,000 features, a student ranks the features by correlation with the label on all 300 rows, keeps the best 20, then runs 10-fold cross-validation of logistic regression on those 20 and obtains 97%. The labels are in fact unrelated to the features. Explain the 97%.*
A. With 5,000 features and only 300 rows, some features correlate strongly with the labels by pure chance, and ranking on all rows picks out exactly

those lucky ones. Every cross-validation fold then scores rows that already took part in choosing the features, so the validation rows are not unseen by the selection step: the fold's labels leaked into the representation it is scored on. If the selection were repeated inside each fold on the 270 training rows, different lucky features would be chosen each time and the validation folds would score about 50%, which is the truth.

Q. *Explain why the minimum of 100 honest validation estimates is not an honest estimate of the winner's risk. Give the one-line inequality.*
A. Each estimate scatters around its own true risk; picking the minimum picks, in part, the luckiest scatter. Formally $\min_h \widehat{R}_{\text{val}}(\widehat{f}_h) \le \widehat{R}_{\text{val}}(\widehat{f}_{h_0})$ for any fixed h_0, so $\mathbb{E}[\min_h \widehat{R}_{\text{val}}(\widehat{f}_h)] \le \min_{h_0} R(\widehat{f}_{h_0})$, with strict inequality whenever the estimates are noisy and no single configuration always wins. The winner's true risk is at least $\min_h R(\widehat{f}_h)$, so the reported score underestimates it; only rows that took no part in the selection can estimate it honestly.

3.6 Reliable Evaluation and Data Splitting

What are we trying to get?

An evaluation is only as honest as its split. We want a rule for choosing the split from the deployment question, metrics that do not hide minority-class failure, and a checklist of the ways information leaks from the future into the model.

Ask first: what kind of new *case will the model face? Then hold out exactly that kind of case.*

If the model must predict future events, the evaluation should preserve time order. If it must generalise to new users, devices, patients or organisations, entire groups should be held out. A random split is credible only when new deployment cases are reasonably approximated by independent draws from the same population.

Definition 3.6.1 (Random, stratified, grouped and temporal splits)**.** *Random*: each observation is assigned to a subset approximately independently. *Stratified*: random, but preserving approximate class proportions in every subset, which stabilises rare-class counts; stratification must not override dependence, so records from one device should not be scattered across subsets merely to match proportions. *Grouped*: every group (user, device, patient, network) lies entirely in one subset, so the learner cannot use identity-specific patterns and is evaluated on unseen groups. *Temporal*: train on the past, validate and test on later observations; this preserves the direction of prediction and reveals deterioration as behaviour, sensors or attack patterns change. [on your sheet]

Experiment: Three protocols, one model

Setup. Flows recorded from 50 devices over 6 months, and one model.

What we hold fixed. The model, the features, the metric.

What we change. Only the split rule: random; grouped by device, holding out 10 devices; temporal, training on months 1 to 4 and testing on months 5 and 6, during which a new attack tool appeared.

What we measure. Test accuracy under each.

What the plot shows. Random 99.5%: the model learned that device 17 is the compromised laptop. Grouped 96%: it must recognise attack behaviour. Temporal 90%: the new tool.

How to read it. Only the third number describes next month. The first is not wrong arithmetic; it answers a question nobody asked. "Will this customer churn next quarter" is temporal (features from before the quarter, outcome after); "does this model work on a hospital it has never seen" is grouped by hospital; when deployment is both, combine them.

Class imbalance and metrics

Proposition 3.6.2 (Accuracy hides minority failure)**.** *If a fraction π of the data is positive, the classifier that always predicts "negative" has accuracy $1-\pi$ and recall 0 on the positive class [on your sheet].*

Proof. It is correct on every negative and wrong on every positive: accuracy $= 1 - \pi$; it detects no positive, so TP $= 0$ and recall $= 0/(0 + \text{FN}) = 0$. Its specificity is 1, so its balanced accuracy is $(0 + 1)/2 = 0.5$, which is the number that exposes it. □

With $\pi = 0.01$ this is 99% accuracy and no attack ever detected.

Definition 3.6.3 (Metrics from the confusion matrix)**.** With attacks as the positive class, count TP, FN, FP and TN on the evaluation set. Then [on your sheet]

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \qquad \text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \qquad \text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}},$$

$$\text{BalancedAcc} = \frac{\text{Recall} + \text{Specificity}}{2}, \qquad F_1 = 2\,\frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Reading the formula

Each metric answers one question. *Recall*: of the real attacks, what fraction did we catch? *Precision*: of the alarms we raised, what fraction were real? *Specificity*: of the benign flows, what fraction did we leave alone? *Balanced accuracy*: the average of the two per-class rates, immune to imbalance. F_1: the harmonic mean of precision and recall, high only when both are high; it ignores true negatives. A detector on 10,000 flows with 50 attacks, TP $= 40$, FN $= 10$, FP $= 90$, TN $= 9860$: accuracy 0.990, recall 0.80, precision $40/130 = 0.31$, specificity 0.991, balanced accuracy 0.895, $F_1 = 0.44$. Accuracy says 99%; precision says two alarms in three are false. Which number matters depends on whether an analyst has to investigate each alarm. Another case: TP $= 30$, FN $= 70$, FP $= 0$ gives precision 1.0, recall 0.3, $F_1 \approx 0.46$.

Proposition 3.6.4 (F_1 lies between the smaller and the average of precision and recall)**.** *For $P, R \in (0, 1]$: $\min(P, R) \le F_1 \le \frac{P+R}{2}$, with equality on the right iff $P = R$.*

Proof. Lower bound: if $P \le R$ then $F_1 = P \cdot \frac{2R}{P+R} \ge P$ because $2R \ge P + R$. Upper bound: $\frac{P+R}{2} - \frac{2PR}{P+R} = \frac{(P+R)^2 - 4PR}{2(P+R)} = \frac{(P-R)^2}{2(P+R)} \ge 0$, zero iff $P = R$. □

Interpretation & discussion

F_1 is dragged toward the *worse* of the two rates: precision 0.31 and recall 0.80 give $F_1 = 0.44$, far below their average 0.56. That is the point of a harmonic mean: it will not let a great recall hide a poor precision. No single metric is universally correct; metric and threshold choice should reflect the costs of missed attacks, false alarms and alert volume.

Experiment: The threshold sweep

Setup. A probabilistic classifier, validation rows, and costs C_{FP}, C_{FN}.

What we hold fixed. The fitted model and the validation rows.

What we change. The decision threshold t from 0 to 1.

What we measure. Recall, precision, false alarms per 100,000 benign flows, and the expected cost at each t.

What the plot shows. Recall falls and precision rises as t increases; the expected cost is U-shaped with its minimum near the cost threshold $t = C_{\mathrm{FP}}/(C_{\mathrm{FP}} + C_{\mathrm{FN}})$ when the probabilities are calibrated.

How to read it. The threshold is a hyperparameter: validation chooses it, the test set reports it. Lab 1 does exactly this and reports test recall, false-positive rate and false alarms per 100,000 benign flows.

Data leakage

Definition 3.6.5 (Leakage)**.** *Leakage* occurs when development uses information that would not be available when a new deployment decision is made [on your sheet]. Common forms: duplicate or near-duplicate events in several subsets; records from the same device, session or patient in training and test data; predictors created after the outcome or after a human investigation (a "ticket closed as intrusion" field, "number of analyst comments"); preprocessing fitted using validation or test observations; hyperparameters repeatedly chosen using test results; a time series shuffled before splitting. Leakage creates an unrealistically favourable estimate of generalization.

Technique: Preprocessing belongs inside the training pipeline

For standardisation, the training set estimates $\hat{\mu}_j = \frac{1}{N_{\mathrm{tr}}} \sum_{i \in \mathrm{tr}} x_j^{(i)}$ and $\hat{\sigma}_j^2 = \frac{1}{N_{\mathrm{tr}}} \sum_{i \in \mathrm{tr}} (x_j^{(i)} - \hat{\mu}_j)^2$, and the *same* $\hat{\mu}_j, \hat{\sigma}_j$ are reused unchanged on validation, test and deployment data [on your sheet]. The same rule applies to imputation, feature selection, dimensionality reduction and resampling. In cross-validation each fold fits these operations on its own training portion only. Leaking pipeline: all data → estimate μ_j, σ_j → split. Valid pipeline: split → estimate on training → reuse on validation, test, deployment.

What people do wrong: How information from the future gets in

Standardising all 10,000 rows before splitting is a small leak, because two sample means differ little; it feels harmless, and for a scaler it nearly is. Selecting features or fitting an imputer on all rows is a large leak. Shuffling a

time series before splitting is a total leak, because the model sees the future of the series it is tested on. A feature such as "number of analyst comments" predicts intrusions perfectly in the historical table and is zero for every new event at decision time. You catch all of these with one question per feature and per preprocessing step: at what moment does this value become known, relative to the moment of the decision?

How the professor asks it

Q. *A detector reports 99% accuracy on a dataset with 1% attacks. Explain what this number can and cannot mean, compute the balanced accuracy of the trivial classifier, and name the two metrics you would ask for.*
A. With $\pi = 0.01$, the classifier that always says "benign" has accuracy $1 - \pi = 0.99$ and recall 0: it detects nothing. So 99% accuracy is consistent with a detector that never fires. Its specificity is 1 and its recall 0, so its balanced accuracy is $(0 + 1)/2 = 0.5$, which exposes it. Ask for recall (the fraction of attacks caught) and for precision or false alarms per 100,000 benign flows (what each alarm costs an analyst); F_1 summarises the two but ignores true negatives, so say which one matters for the deployment.

Q. *You standardise all 10,000 rows, split 80/20, train logistic regression and report 94% on the 20%. Is the number trustworthy?*
A. Not fully. The scaler's means and variances were computed with the test rows included, so the test rows helped shape the representation on which they are then graded; that is leakage, even if mild. For a scaler on 10,000 rows the effect is small, because two sample means differ little. The same mistake with feature selection on all rows, or with imputation by a fitted model, can be several points, and with few rows it can be enormous. The fix costs nothing: split first, fit the scaler on the 8,000, apply it unchanged to the 2,000.

3.7 Diagnosis and Lab Preparation

Definition 3.7.1 (Learning curves)**.** *Learning curves* vary the amount of training data and plot training and validation performance against it [on your sheet].

Experiment: The learning curve

Setup. Development rows and a fixed configuration h.

What we hold fixed. The configuration, the validation rows, the metric, the preprocessing rule.

What we change. The number of training rows N, by nested subsamples $10\%, 20\%, \ldots, 100\%$ of the training set, several random draws at each size, averaged.

What we measure. At each N the training risk of the fitted model on its own rows and the validation risk on the fixed validation rows.

What the plot shows. The training risk *rises* with N, because more rows are harder to fit perfectly; the validation risk *falls*; the two approach each other. The sandwich of Proposition 3.1.2 holds at every N, and as N grows both outer terms approach $R(f^*_{\mathcal{H}})$, so the curves converge to one limit: ap-

proximation error plus noise, which N cannot change. The gap at finite N is estimation error.

How to read it. Both curves high and already flat, close together: bias; more data will not help; change the representation or increase capacity. Training low, validation high, a wide gap that narrows slowly: variance; regularise, reduce capacity, or collect more representative data, and the gap will close. Both still falling and converging: data-limited; more rows are likely to help. Test performance degrading over calendar time is a fourth case, distribution shift, which a learning curve does not show.

Observed pattern	Likely issue	Possible response
Training and validation errors both high	underfitting	improve the representation or increase capacity
Training error low, validation error high	high variance	regularise, reduce capacity, or collect representative data
Errors close and both decrease as N grows	data-limited	additional representative examples are likely to help
Test performance degrades over time	distribution shift	update the data and use a time-aware evaluation protocol

What people do wrong: Reading the wrong axis, and the gap that will not close

Students confuse the three horizontal axes: capacity for the validation curve, epochs for the training curve, data size for the learning curve. Each has its own shape and its own diagnosis. And here is a subtle one: a gap that stays constant across a fiftyfold increase in data is not variance, because variance shrinks with N. It is a systematic difference between training rows and validation rows, a shortcut present in the training collection and absent from a grouped or temporal validation set, duplicates, or label noise concentrated on one side. The response is an audit of the split and the features, not more data.

Example 3.7.2 (Lab 1 at a glance, and a learning-curve decision)**.** The intrusion-detection lab audits a labelled-flow dataset (imbalance, missing or non-finite values, duplicates, conflicting labels, redundancy), pre-declares random-row, grouped and chronological protocols (Section 3.6), fits baselines (majority class, depth-3 tree, logistic regression, kNN) under each protocol, probes metadata shortcuts and shuffled labels (Section 3.1), selects the decision threshold on validation (Section 3.5), and reports test recall, false-positive rate and false alarms per 100,000 benign flows (Definition 3.6.3). Should the team label 20,000 more flows? Validation recall at 1,000 up to 8,000 rows reads $0.71, 0.78, 0.83, 0.86, 0.87, 0.875, 0.878, 0.88$, with training recall near 1: a wide gap closing slowly and a curve that has reached a plateau. Doubling the data would buy perhaps 0.01; larger leaves, fewer features per split and better features are the cheaper levers.

How the professor asks it

Q. *A learning curve shows training error at 2% and validation error at 15% at every size from 1,000 to 50,000 rows, with no narrowing. Which diagnosis*

does the table suggest, and why might the table be wrong here?
A. The table says high variance, because the gap is large. But variance shrinks with N, and a gap that does not narrow across a fiftyfold increase in data is not sampling variance: it is a systematic difference between the training rows and the validation rows. Typical causes are a shortcut present in the training collection and absent from the validation protocol, duplicates in the training rows, or label noise concentrated in one subset. The response is not more data or regularisation but an audit of the split and the features.

Q. *A learning curve shows both curves high, flat and close together from 2,000 to 40,000 rows. What do you conclude and what would you change?*
A. The procedure is limited neither by data (more rows changed nothing) nor by variance (the gap is small): it is limited by bias, meaning the family or the representation cannot express the relationship. Add informative features, use a more flexible family, or reduce regularisation; collecting more of the same rows will not move the plateau.

Chapter takeaways

Summary: From fitting to generalization

- **Low training loss does not establish performance on new data:** the sample chooses and then grades the model; $\mathbb{E}[\widehat{R}_{\mathcal{D}}(\widehat{f}_{\mathcal{D}})] \leq \min_{\mathcal{H}} R \leq \mathbb{E}[R(\widehat{f}_{\mathcal{D}})]$.
- **Inductive bias** enters through data, representation, model family, regularisation and optimisation; it decides what happens off the sample, and shortcuts are caught only by probing.
- **Capacity** trades approximation ability against sensitivity to a finite sample; training error only ever rewards more of it; the validation U chooses it.
- **Bias** is systematic error across repeated samples, **variance** is variation across them; squared error = noise + bias^2 + variance; data lowers variance only.
- **Regularisation**, feature selection and early stopping control effective capacity; validation selects their strength; L_1 zeroes coefficients, L_2 shrinks them.
- **Selection is part of training**: redo it inside every fold; the best of many validation scores is optimistic; the test set is used once.
- **Evaluation splits** must reproduce the novelty expected at deployment; metrics must expose minority-class failure; **leakage** invalidates the estimate.

Chapter 4

Discriminative and Generative Classification

Let me start by telling you what this chapter is really about, because the title hides it. In Chapter 2 you learned that once the posterior $p(y = c \mid x)$ is known, the decision is a one-line rule: declare the class with the smallest expected cost. So the hard part of classification is not deciding; it is *estimating that posterior from labelled examples.* There are exactly two ways to do it. The discriminative way fits the posterior directly: you build a score from the features, squash it into a probability, and train it to predict labels; logistic regression and neural networks live here. The generative way tells a story of how each class produces its inputs, fits that story class by class, and then reverses it with Bayes' rule; Gaussian class models, LDA, QDA and Naive Bayes live here. Every method you will meet is a choice about what you assume about the data, how many numbers you must estimate from it, and what you can and cannot ask the fitted model afterwards: a boundary, a probability, a plausibility, a correction when the prior changes. Keep one question in mind throughout: *what did this method assume, and what did that assumption buy or cost?* Formulas marked [on your sheet] are on your sheet; the few marked [not on your sheet: know it] you must carry in your head.

4.1 Two Routes to the Posterior

What are we trying to get?

Chapter 2 showed that once the posterior $p(y = c \mid x)$ is known, the best decision is a one-line rule. So the real work of classification is *estimating the posterior from labelled examples.* We want to understand that there are two fundamentally different ways to do that, what each one commits us to, and why neither wins in general.

The Bayes rule is easy. The hard part is getting the probabilities it needs.

Let $Y \in \{1, \dots, C\}$ be a class label and $X \in \mathbb{R}^d$ a feature vector. If the posterior were known, the zero–one Bayes decision would be $f^*(x) = \arg\max_c p(y = c \mid x)$, and under costs $C_{\text{FP}}, C_{\text{FN}}$ it would be a threshold on $p(y = 1 \mid x)$. The learning problem arises because $p(y = c \mid x)$ is unknown and must be estimated from labelled examples $\mathcal{D}_N = \{(x^{(i)}, y^{(i)})\}_{i=1}^N$: is this network flow benign or malicious?

Definition 4.1.1 (Discriminative and generative classifiers)**.** • A *discriminative* model directly approximates the posterior with a param-

eterized family $p_\theta(y = c \mid x)$; training produces the fitted posterior $p_{\hat{\theta}}(y = c \mid x)$. It learns *where the label changes* as a function of the features.

- A *generative* classifier models the class prior $p(y = c)$ and the class-conditional distribution $p(x \mid y = c)$, and obtains the posterior through Bayes' rule. It learns *what each class looks like*, then compares how well each class explains the observed input.

Reading the formula

Both routes end with the same object, a posterior over classes. The difference is the *direction of modelling*. Discriminative: "given these features, what is the probability of attack?" Generative: "what do attack flows typically look like, what do benign flows look like, and how common is each? Now, which story fits this flow better?" The second route builds two "worlds" and asks which one this input belongs to. Spam filtering with Naive Bayes is generative (model the token frequencies of spam and of legitimate mail); image classification with a deep network is discriminative (modelling the full density of images per class is far harder than learning where the label changes); fraud and intrusion screening often combine both, because a class-conditional model can also say how *compatible* an event is with normal behaviour.

Telling them apart: discriminative versus generative

Students mix the two routes up because both produce a posterior. The way to keep them apart is to count how many times each route explains an example. The generative route explains it *twice*: its class, through π, and its features, through $p(x \mid c)$. That is more structure, which is why it needs less data, and also more assumptions, which is why a wrong family leaves a bias that no amount of data removes. The discriminative route explains the label given the features and nothing else: fewer assumptions, more data needed, and no opinion about the input itself. The remark that "generative" is about direction, not task, belongs here too: a class-conditional model can be used to *infer* a class (classification) or, in principle, to *draw* a new x for a chosen class (generation); classical generative classifiers and modern generative AI share the modelling direction, not the scale.

What people do wrong: choosing a route by slogan

Students pick a route by the slogan "generative for small data, discriminative for big data" and skip the validation. It feels right because the slogan is often true, but the crossing point depends on the data and on the two families, and sometimes there is no crossing at all. What you should do is ask the three model-choice questions of Section 4.9 (is a class model credible, do you have many labels, is the raw input high-dimensional) and then compare the candidates on the same split, with the same rows, under deployment-like conditions.

How the professor asks it

Q. *What is the difference between a discriminative and a generative classifier, and why does neither win in general?*
A. A discriminative classifier models $p(y \mid x)$ directly and learns where the label changes; a generative classifier models $p(y)$ and $p(x \mid y)$ and obtains $p(y \mid x)$ through Bayes' rule, learning what each class looks like. The generative route explains every example twice, class and features, so it has more structure and needs fewer labels, but a wrong class-conditional family leaves a persistent bias; the discriminative route assumes less and needs more data, and it has no notion of whether the input is plausible. Which is better depends on how credible the assumptions are, how many labels you have, and what the deployment objective is; it is settled by validation, not by argument.

4.2 The Discriminative Route

What are we trying to get?

We want to see how a model that outputs *numbers* (scores) is turned into *probabilities* and then into *actions*, why the cross-entropy loss of Chapter 2 is exactly maximum likelihood here, and what a discriminative model cannot tell us.

Scores, probabilities, actions

A discriminative model often first produces a real-valued *score* (or *logit*) for each class. Scores are converted into posterior estimates and then into an action:

$$\{s_{\widehat{\theta},c}(x)\} \longrightarrow \{p_{\widehat{\theta}}(y = c \mid x)\} \longrightarrow \hat{y}(x).$$

The score model determines how inputs are *ranked*; the probability transformation quantifies *relative uncertainty* among classes; the decision loss determines the *action*. Why a score first? Because a score can be any real number, which is what a linear function or a network naturally produces; the sigmoid below is the bridge from "any number" to "a number in $(0, 1)$".

Definition 4.2.1 (Sigmoid; binary logistic model)**.** The *sigmoid* (logistic) function is $\sigma(s) = \dfrac{1}{1 + e^{-s}}$ [on your sheet]. For binary classification one score suffices, and the posterior estimate is $p_{\widehat{\theta}}(y = 1 \mid x) = \sigma(s_{\widehat{\theta}}(x))$. *Logistic regression* is the special case of a linear score, $s_\theta(x) = w^\top x + b$, $\theta = (w, b)$, so that $\log \frac{p}{1-p} = w^\top x + b$ [on your sheet]: the log-odds are linear in x.

Reading the formula

σ squashes any real number into $(0, 1)$: $\sigma(0) = 0.5$, $\sigma(2) \approx 0.88$, $\sigma(-2) \approx 0.12$, $\sigma(5) \approx 0.993$. A score of 0 means "no idea"; large positive means "confidently class 1"; large negative means "confidently class 0". With $w = (1, -2)$, $b = 0.5$ and $x = (2, 1)$: $s = 2 - 2 + 0.5 = 0.5$ and $p(y = 1 \mid x) = \sigma(0.5) \approx 0.62$.

Proposition 4.2.2 (Properties of the sigmoid)**.** *For all $s \in \mathbb{R}$: (i) $0 < \sigma(s) < 1$; (ii) $\sigma(-s) = 1 - \sigma(s)$; (iii) σ is strictly increasing; (iv) $\sigma(s) > \frac{1}{2} \iff s > 0$;*

(v) for $t \in (0,1)$, $\sigma(s) > t \iff s > \log \dfrac{t}{1-t}$ [on your sheet].

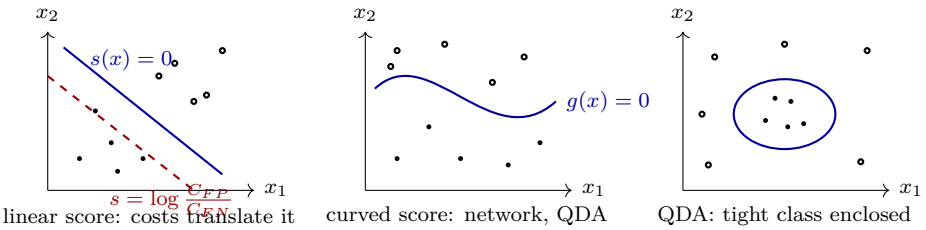
Proof. (i) $e^{-s} > 0$ so the denominator exceeds 1. (ii) $1 - \sigma(s) = \dfrac{e^{-s}}{1+e^{-s}} = \dfrac{1}{e^{s}+1} = \sigma(-s)$ after multiplying numerator and denominator by e^{s}; this is why one score suffices for two classes. (iii) e^{-s} is strictly decreasing, so $1+e^{-s}$ is, so its reciprocal is strictly increasing. (v) $\sigma(s) > t \iff 1+e^{-s} < 1/t \iff e^{-s} < (1-t)/t \iff s > \log \frac{t}{1-t}$; each step is allowed because every quantity is positive and log is increasing. (iv) is (v) with $t = \frac{1}{2}$, since $\log 1 = 0$. □

Theorem 4.2.3 (Decision boundaries of score models)**.** *Under zero–one loss, the binary logistic model decides $\hat{y}(x) = 1 \iff s_{\hat{\theta}}(x) > 0$. For logistic regression the 0.5-posterior contour is the hyperplane $\hat{w}^{\top}x + \hat{b} = 0$. Under costs $C_{\mathrm{FP}}, C_{\mathrm{FN}}$ the decision is $\hat{y}(x) = 1 \iff s_{\hat{\theta}}(x) > \log \dfrac{C_{\mathrm{FP}}}{C_{\mathrm{FN}}}$ [on your sheet]. For a nonlinear score $g_{\hat{\theta}}$, the boundary is the surface $g_{\hat{\theta}}(x) = 0$, shifted by the same constant under unequal costs.*

Proof. The zero–one Bayes decision compares $p(y = 1 \mid x)$ with $\frac{1}{2}$, and Proposition 4.2.2(iv) converts that into $s > 0$; for a linear score, $s = 0$ is a hyperplane. Under costs, Chapter 2 gives the threshold $t = C_{\mathrm{FP}}/(C_{\mathrm{FP}} + C_{\mathrm{FN}})$, so $\frac{t}{1-t} = \frac{C_{\mathrm{FP}}}{C_{\mathrm{FN}}}$: the costs enter only through their ratio, and Proposition 4.2.2(v) gives $s > \log(C_{\mathrm{FP}}/C_{\mathrm{FN}})$. □

Reading the formula

The probability threshold does *not* make the geometry linear; the score family does. A linear score gives a straight boundary; a curved score gives a curved boundary; changing the costs only *slides* the boundary along the score axis. With $C_{\mathrm{FN}} = 10$, $C_{\mathrm{FP}} = 1$ the boundary moves from $s = 0$ to $s = \log(1/10) \approx -2.3$, which is already at probability $\sigma(-2.30) \approx 0.09$: raise the alarm when the score is mildly negative, because an expensive miss lowers the bar for alarming. One probability model supports several policies.



linear score: costs translate it — curved score: network, QDA — QDA: tight class enclosed

Experiment: the threshold sweep

Setup. A fitted logistic regression with weights $\hat{w}, \hat{b}$, a set of validation flows, and the two costs C_{FP} and C_{FN}.

What we hold fixed. The fitted model and the validation rows. Nothing about the model changes during the experiment.

What we change. Only the decision cut: we slide $\tau = \log(C_{\mathrm{FP}}/C_{\mathrm{FN}})$ from -4 to 4.

What we measure. At each value of τ, the recall (fraction of attacks caught) and the false-positive rate (fraction of benign flows alarmed).

What the plot shows. Plotting recall against false-positive rate as τ moves draws this model's ROC curve. Notice what did not move: the *ranking* of the flows by score is the same at every τ; only the place where we cut the ranking moved.

How to read it. If the curve is not good enough, no threshold will save you; you need a better model, because the threshold chooses a point on the curve and the model chooses the curve. The cut itself should come from the costs and be checked on validation rows, never on test rows.

What people do wrong: believing the threshold bends the boundary

The tempting picture is that a different probability threshold gives a different *shape* of boundary. It does not: with a linear score the boundary is a hyperplane and a new threshold is a *parallel* hyperplane; with a curved score it is the same curve shifted in score. The geometry comes from the score family, the costs only translate it. The second temptation is to choose τ by looking at test results, which turns the test set into a validation set and makes the reported number optimistic. Keep three objects apart: the score ranks the inputs, the probability tells you how uncertain the model is between the classes, and the loss, through the costs, picks the action.

Multiclass scores and softmax

Definition 4.2.4 (Softmax)**.** For scores $s = (s_1, \dots, s_C)$, the *softmax* map produces $p_\theta(y = c \mid x) = \mathrm{softmax}(s_\theta(x))_c = \dfrac{\exp[s_{\theta,c}(x)]}{\sum_{k=1}^{C} \exp[s_{\theta,k}(x)]}$ [on your sheet].

Reading the formula

Two moves: exponentiate, so that every entry is positive, then divide by the total, so that the classes compete for probability one. Scores $(2, 0, -1)$: exponentials $(7.39, 1.00, 0.37)$, total 8.76, probabilities $(0.84, 0.11, 0.04)$. The largest score always gets the largest probability, and only differences of scores matter: adding 100 to every score multiplies numerator and denominator by e^{100} and changes nothing. When you compute a softmax by hand, subtract the largest score from all of them first; the answer is identical and you avoid enormous exponentials.

Proposition 4.2.5 (Softmax facts)**.** *(i) The output lies in the simplex and* $\arg\max_c p_\theta(y = c \mid x) = \arg\max_c s_{\theta,c}(x)$*. (ii) Adding the same constant to every score leaves all probabilities unchanged, so only* $C - 1$ *score* contrasts *are identifiable. (iii) With* $C = 2$, $\mathrm{softmax}(s)_1 = \sigma(s_1 - s_2)$ *[on your sheet]: softmax reduces to the sigmoid of the score difference.*

Proof. (i) Exponentials are positive and the entries are divided by their sum, so they are nonnegative and sum to 1; exp is increasing, so the ordering of scores is the ordering of probabilities. (ii) Replacing s_k by $s_k + a$ multiplies every exponential by e^a, which cancels between numerator and denominator. (iii)

$$\frac{e^{s_1}}{e^{s_1}+e^{s_2}} = \frac{1}{1+e^{s_2-s_1}} = \sigma(s_1 - s_2). \qquad \square$$

Interpretation & discussion

Sigmoid and softmax are not two different ideas: the sigmoid is softmax applied to a score difference. Implementations still produce C scores rather than $C-1$: every class then plays the same role, relabelling merely permutes outputs, and one vectorised output layer and loss serve any C.

Cross-entropy is conditional maximum likelihood

For example i, cross-entropy penalises the probability assigned to the true class, $\ell_{\mathrm{CE}}^{(i)}(\theta) = -\sum_c \mathbf{1}(y^{(i)} = c) \log p_\theta(y = c \mid x^{(i)}) = -\log p_\theta(y^{(i)} \mid x^{(i)})$ [on your sheet], so probabilities 0.9, 0.5, 0.1 on the true class cost about 0.11, 0.69, 2.30 (natural logarithms): confident mistakes are punished hardest, which is exactly what you want from a probability model.

Proposition 4.2.6 (Cross-entropy equals negative conditional log-likelihood)**.** *Assuming the labels are conditionally independent given their inputs, the conditional likelihood is* $L_{\mathrm{cond}}(\theta) = \prod_{i=1}^{N} p_\theta(y^{(i)} \mid x^{(i)})$, *and* $-\log L_{\mathrm{cond}}(\theta) = -\sum_{i=1}^{N} \log p_\theta(y^{(i)} \mid x^{(i)}) = N\,\widehat{R}_{\mathrm{CE}}(\theta)$ *[on your sheet]. Minimising empirical cross-entropy is therefore the same as maximising the conditional likelihood.*

Proof. Independence turns the joint probability of the observed labels into a product; the logarithm turns the product into a sum, which is what we can differentiate; with a one-hot label only the true class survives the indicator, so each summand is the per-example cross-entropy; the sum divided by N is the empirical cross-entropy. Since $-\log$ is decreasing, maximising L_{cond} and minimising $\widehat{R}_{\mathrm{CE}}$ have the same solutions. $\square$

Reading the formula

"Conditional" likelihood means we only ask the model to explain the *labels given the inputs*, never the inputs themselves. Three examples with true-class probabilities $0.9, 0.5, 0.1$: likelihood $0.9 \times 0.5 \times 0.1 = 0.045$; negative log $0.105 + 0.693 + 2.303 = 3.10$; empirical cross-entropy $3.10/3 = 1.03$. Same numbers, two names: when you see "maximum likelihood" in a generative chapter and "cross-entropy" in a network chapter, it is the same instruction. Discriminative maximum likelihood maximises $\prod p(y \mid x)$; generative maximum likelihood, in the next section, maximises $\prod p(x, y)$ and so explains each example twice.

What people do wrong: training on accuracy, or reporting cross-entropy

Two temptations. The first is to train directly on accuracy: the zero–one loss is flat almost everywhere, so there is no gradient to follow and nothing to optimise. The second is the reverse: to compare and report models by cross-entropy when the deployment cost is a zero–one count or a cost-weighted count; the two can rank models differently. The rule is to optimise the smooth loss, then select and report with the metric the deployment cares about.

Deep discriminative classifiers, and what the objective does not provide

Definition 4.2.7 (Feed-forward classifier)**.** A neural network computes a sequence of representations $h^{(0)} = x$, $h^{(\ell)} = \varphi(W_\ell h^{(\ell-1)} + b_\ell)$ for $\ell = 1, \dots, L-1$, with φ an elementwise nonlinearity; the final layer produces scores $s_\theta(x) = W_L h^{(L-1)} + b_L$, and softmax produces the posterior [not on your sheet: know it]. Cross-entropy can train all layers end to end.

Reading the formula

Each layer is "multiply by a matrix, add a vector, bend with φ". Logistic regression is the case $L = 1$: no hidden layers, scores linear in x. Adding layers lets the score be a curved function of x, so the boundary of Theorem 4.2.3 can curve; under new costs it is the same curve shifted by $\log(C_{\mathrm{FP}}/C_{\mathrm{FN}})$ in score. The representation the network learns is optimised for predicting the label, not for reconstructing or explaining the input distribution, which is exactly the point of what follows.

A discriminative model ranks the classes it knows. It never asks whether the input makes sense.

Because the objective only asks for $p(y \mid x)$, the model is under no obligation to know whether x is *plausible*; a large $\max_c p_{\widehat{\theta}}(y = c \mid x)$ does not imply that x resembles the training data; and the model provides no direct mechanism for sampling inputs or for marginalising missing features. Softmax probabilities are *relative* comparisons among the modelled classes. If all classes explain an unfamiliar input poorly, one class can still receive a posterior near one.

Experiment: the toaster

Setup. A three-class network trained on cat, dog and horse photographs.

What we hold fixed. The trained network and its softmax output layer.

What we change. The input: we feed photographs from outside the training population, for instance a toaster.

What we measure. The top softmax probability on these inputs.

What the plot shows. The scores come out $(4.0, 0.5, -1.0)$, softmax gives $(0.96, 0.03, 0.01)$, and the network says "cat, 96%".

How to read it. Nothing in the training objective ever asked the network "is this even an animal?", so nothing in the output can answer it. The number 0.96 measures how much better "cat" explains the toaster than "dog" and "horse" do, which is all softmax can measure. Section 4.8 adds the missing check.

What people do wrong: using confidence as an anomaly score

The temptation is to read "low confidence means unfamiliar" and use $\max_c p(c \mid x)$ to detect inputs the model does not know. It feels right because unfamiliar inputs sometimes do get low confidence, but the toaster shows they need not. You catch it by holding out a whole family of inputs and looking at the top probability on them. The fix is a generative compatibility check with a threshold validated on the intended novelty.

Telling them apart: unfamiliar inputs versus class imbalance

Do not confuse this with class imbalance. If training and deployment share the same distribution, an imbalanced training prior is not an error: the fitted posterior *should* reflect that prevalence, and with a 5% positive class the 0.5 contour *should* move toward the positives. The practical difficulty of imbalance is only that few minority examples make the boundary hard to estimate, where weighting or resampling can help estimation at the price of calibration. Imbalance is a question about the prior; familiarity is a question about the input. Section 4.7 separates imbalance from a genuine *change* of prevalence.

Technique: Working with scores and probabilities

- To decide, compare the *score* with $\log(C_{\text{FP}}/C_{\text{FN}})$; you never need to compute σ.
- To convert a probability back to a score use the log-odds $s = \log \frac{p}{1-p}$; $p = 0.9$ gives $s \approx 2.2$.
- Softmax by hand: subtract the largest score first; it changes nothing.
- Never read $\max_c p(y = c \mid x)$ as "probability that the input is normal".

How the professor asks it

Q. *A network trained on cats, dogs and horses outputs "cat, 96%" for a photograph of a toaster. Explain why this is not a bug of the training, what the number measures, and what you would add.*
A. It is not a bug, because the discriminative objective only ever asked for $p(y \mid x)$ among three known classes. The number 0.96 measures how much better "cat" explains the toaster than "dog" and "horse" do; it is a relative comparison produced by the softmax, and nothing in the cross-entropy loss ever rewarded the network for knowing whether x is plausible. What you add is a generative compatibility check: accept the winning class c^* only if $\log p(x \mid c^*) \geq \tau_{c^*}$, with τ validated on held-out novelty, for instance a class excluded from training; otherwise reject the input or send it to review. The softmax ranks, the likelihood checks familiarity.

Q. *Missed attacks cost ten times more than false alarms. Where is the decision boundary of a logistic regression, and does its shape change?*
A. The boundary moves, its shape does not. The cost threshold on the probability is $t = C_{\text{FP}}/(C_{\text{FP}} + C_{\text{FN}}) = 1/11$, and by the sigmoid inversion $\sigma(s) > t \iff s > \log \frac{t}{1-t} = \log(C_{\text{FP}}/C_{\text{FN}}) = \log 0.1 \approx -2.30$. So the alarm fires once the linear score $\hat{w}^\top x + \hat{b}$ exceeds -2.30, which is a hyperplane parallel to the equal-cost boundary $s = 0$, translated toward the benign side; at that point the model's probability of attack is only $\sigma(-2.30) \approx 0.09$. The geometry is fixed by the score family; the costs enter only through their ratio and slide the boundary along the score axis.

4.3 The Generative Route

What are we trying to get?

We want a model that tells a *story* of how a labelled example came to be, to see how Bayes' rule reverses that story into a classifier, and to understand why fitting such a model asks more of the data than the discriminative route does.

Definition 4.3.1 (Generative classifier)**.** A generative classifier imagines that an observation is produced in two steps: first choose a class, then generate features from that class, $Y \sim \text{Categorical}(\pi)$, $X \mid (Y = c) \sim p_\theta(\cdot \mid y = c)$, with parameters $\phi = (\pi, \theta)$ and $\pi_c = p(y = c)$. The joint model factorises as $p_\phi(x, y = c) = \pi_c\, p_\theta(x \mid y = c)$. The *prior* π_c is the class frequency before the input is observed; the *class-conditional* density (or mass function) describes which inputs are plausible within class c.

Theorem 4.3.2 (Bayes' rule for classification)**.** *Under the model of Definition 4.3.1, the posterior is*

$$p_\phi(y = c \mid x) = \frac{\pi_c\, p_\theta(x \mid y = c)}{\sum_{k=1}^{C} \pi_k\, p_\theta(x \mid y = k)}. \quad \textit{[on your sheet]}$$

Classification under zero–one loss therefore chooses the class with the largest prior-weighted likelihood $\pi_c\, p_\theta(x \mid y = c)$. *Equivalently, with the generative score $s_c(x) = \log \pi_c + \log p_\theta(x \mid y = c)$, the posterior is the softmax of the scores [on your sheet], and for two classes the odds are $\frac{p(1|x)}{p(0|x)} = \frac{\pi_1}{\pi_0} \cdot \frac{p(x|1)}{p(x|0)}$ [not on your sheet: know it]: prior odds times likelihood ratio.*

Proof. Start from the definition of conditional probability, $p(y = c \mid x) = p(x, y = c)/p(x)$. The numerator is $\pi_c\, p_\theta(x \mid y = c)$ by the factorisation, the story "choose a class, then generate features". The denominator is the marginal $p(x) = \sum_k p(x, y = k) = \sum_k \pi_k\, p_\theta(x \mid y = k)$, a sum over the mutually exclusive, exhaustive classes. The denominator does not depend on c, so the arg max over c is decided by the numerator, and taking logs of the numerator gives the score form. For the odds, divide the two numerators: the common denominator cancels. □

Reading the formula

Numerator: "how common is class c" times "how well does class c explain this x". Denominator: the same quantity summed over all classes, which just rescales so that the posteriors add to 1. A rare class can still win if it explains the input much better. Network security: benign flows have prior 0.99 but give this flow likelihood 0.001, product 0.00099; attacks have prior 0.01 but likelihood 0.20, product 0.00200. So $p(y = \text{attack} \mid x) = 0.002/(0.002 + 0.00099) \approx 0.67$; in odds form, $0.01 \cdot 0.20/(0.99 \cdot 0.001) = 2.02$ and $2.02/3.02 = 0.67$. Attacks remain rare in the population, yet this particular flow is more compatible with the attack model, and the posterior says so. The prior is a weight, not a veto.

Proposition 4.3.3 (The generative likelihood splits)**.** *For independent labelled observations the negative log-likelihood is*

$$-\log L(\phi) = \underbrace{-\sum_{c=1}^{C} N_c \log \pi_c}_{\text{prior part}} \underbrace{-\sum_{c=1}^{C} \sum_{i:\, y^{(i)}=c} \log p_\theta(x^{(i)} \mid y = c)}_{\text{one term per class}}.$$

Hence the prior and each class-conditional model can be fitted separately*: the prior from the class counts alone, and the model of class c from the examples of class c alone.*

Proof. Take logs of the product $\prod_i p_\phi(x^{(i)}, y^{(i)})$ and substitute the factorisation $p_\phi(x, y) = \pi_y\, p_\theta(x \mid y)$. Group the prior terms by class: the label c occurs N_c times, giving $N_c \log \pi_c$. Group the class-conditional terms by class likewise. The first group depends only on π, the second only on θ, and within the second each class's parameters appear only in that class's sum. □

Proposition 4.3.4 (Maximum-likelihood prior)**.** *The prior part is maximised, over the simplex, by the empirical class frequencies $\hat{\pi}_c = N_c/N$ [on your sheet].*

Proof. Write $q_c = N_c/N$, a probability vector. Then

$$\sum_c N_c \log \pi_c = N \sum_c q_c \log \pi_c = -N\, H(q, \pi),$$

the negative cross-entropy between the empirical class distribution and the model prior. By Gibbs' inequality, $H(q, \pi) = H(q) + \mathrm{D}_{\mathrm{KL}}(q \| \pi) \geq H(q)$ with equality iff $\pi = q$ [on your sheet]. So the prior part is largest exactly at $\pi = q$. □

Key insight

Fitting the prior is Gibbs' inequality again. Generative training asks each example to be explained *twice*: its class (through π) and its features (through $p_\theta(x \mid y)$). That is a stronger requirement than the discriminative route's "explain the class given the features", and it is both the strength (more structure, less data needed) and the weakness (more assumptions that can be wrong) of the generative route. Fit class c's model from its own rows, then add a thousand rows of another class: nothing in class c's model changes. Discriminative fitting, by contrast, couples all parameters through every example, because each example pulls on the whole boundary.

Interpretation & discussion

Two approximations should be kept separate. *Shape approximation*: the family $p_\theta(x \mid y)$ restricts the class-conditional distributions and may not contain the truth; this error does not vanish with more data. *Finite-sample estimation*: even within the family, $\hat{\phi}$ differs from the best population parameter because it is estimated from finite data; this error shrinks with more data. On a learning curve you can see both: the first is the plateau of the generative curve, the second is its spread at small N. This is Chapter 3's approximation error versus estimation error, applied to density estimation.

What people do wrong: forgetting the prior, or counting it twice

Two errors I see every year: forgetting the prior altogether and comparing raw likelihoods, or, in Naive Bayes, counting the prior once per feature. There is exactly one $\log \pi_c$ in each class score. A subtler version: estimating one class's parameters from all rows, or "fitting" the prior from a resampled training set, in which case $\hat{\pi}$ equals the resampled prevalence and not the real one. Always know where your prior came from.

How the professor asks it

Q. *Derive the maximum-likelihood estimate of the class prior and explain why the class-conditional models can be fitted class by class.*
A. Claim: $\hat{\pi}_c = N_c/N$, and each class model is fitted from its own rows. Taking the negative log of $\prod_i \pi_{y^{(i)}} p_\theta(x^{(i)} \mid y^{(i)})$ and grouping by class gives $-\sum_c N_c \log \pi_c - \sum_c \sum_{i:y^{(i)}=c} \log p_\theta(x^{(i)} \mid c)$; the first part involves only π and the second only θ, with each class's parameters appearing only in its own sum, so the problems decouple. For the prior, with $q_c = N_c/N$ the prior part equals $N\,H(q,\pi)$, and Gibbs' inequality $H(q,\pi) \geq H(q)$ with equality iff $\pi = q$ gives the class frequencies. The lesson is that generative fitting is cheap and stable because it decouples by class, and that the prior is an explicit component you can later replace.

4.4 Gaussian Class-Conditional Models

What are we trying to get?

For continuous features, the simplest class "world" is a bell-shaped cloud. We want the fitted parameters in closed form, one worked posterior computed by hand, and, most importantly, to see how an *assumption about covariance* becomes a *shape of the decision boundary*: shared covariance gives straight boundaries (LDA), separate covariances give curved ones (QDA).

Definition 4.4.1 (Multivariate Gaussian class model)**.** For $x \in \mathbb{R}^d$, the Gaussian class-conditional model is $p_\theta(x \mid y = c) = \mathcal{N}(x; \mu_c, \Sigma_c) = \frac{1}{(2\pi)^{d/2}|\Sigma_c|^{1/2}} \exp\left[-\frac{1}{2}(x-\mu_c)^\top \Sigma_c^{-1}(x-\mu_c)\right]$ [on your sheet], with mean $\mu_c \in \mathbb{R}^d$ and symmetric positive-definite covariance Σ_c. In one dimension this is $\mathcal{N}(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp[-(x-\mu)^2/(2\sigma^2)]$.

Reading the formula

The mean μ_c is the centre of the cloud of class c. The covariance Σ_c is its shape: a multiple of the identity gives circular contours; unequal diagonal entries stretch the circle into an axis-aligned ellipse; off-diagonal entries rotate it (correlated features). The quadratic form $(x-\mu_c)^\top \Sigma_c^{-1}(x-\mu_c)$ is the *squared Mahalanobis distance*: "how many standard deviations away from the centre, in the direction of x". The prefactor with $|\Sigma_c|$ makes the density integrate to one; a diffuse class (large determinant) spreads its probability thin.

Theorem 4.4.2 (Gaussian MLE)**.** *Let N_c be the number of training examples of class c. The maximum-likelihood estimates are*

$$\hat{\pi}_c = \frac{N_c}{N}, \qquad \hat{\mu}_c = \frac{1}{N_c} \sum_{i:\, y^{(i)}=c} x^{(i)},$$

$$\widehat{\Sigma}_c = \frac{1}{N_c} \sum_{i:\, y^{(i)}=c} (x^{(i)} - \hat{\mu}_c)(x^{(i)} - \hat{\mu}_c)^\top. \quad \textit{[on your sheet]}$$

No iterative optimisation is needed.

Proof. The prior is Proposition 4.3.4. By Proposition 4.3.3 each class is fitted from its own examples, so fix a class and write its $n = N_c$ examples as $x^{(1)}, \dots, x^{(n)}$. In one dimension the negative log-likelihood is $J(\mu, \sigma^2) = \frac{n}{2}\log(2\pi\sigma^2) + \frac{1}{2\sigma^2}\sum_i (x^{(i)} - \mu)^2$. **Step 1 (mean).** For fixed σ^2, $\partial J/\partial\mu = -\frac{1}{\sigma^2}\sum_i (x^{(i)} - \mu) = 0$ gives $\mu = \frac{1}{n}\sum_i x^{(i)}$, and J is a convex quadratic in μ, so this is the minimum. **Step 2 (variance).** With $\mu = \hat{\mu}$ and $S = \sum_i (x^{(i)} - \hat{\mu})^2$, put $v = \sigma^2$: $\partial J/\partial v = \frac{n}{2v} - \frac{S}{2v^2} = 0$ gives $v = S/n$; the derivative is negative for $v < S/n$ and positive beyond, so it is the minimum. In d dimensions the same two steps with Σ in place of v give the class mean and the average outer product of the centred examples. □

Remark 4.4.3 (N_c versus $N_c - 1$). The denominator N_c is the maximum-likelihood denominator. The familiar *unbiased* sample covariance uses $N_c - 1$. For classification the difference is immaterial when N_c is large, but the two are different estimators and you should say which one you use.

Example 4.4.4 (One dimension, by hand)**.** Class 0: $\{0, 1, 2, 0, 1, 2\}$; class 1: $\{3, 4, 5\}$. Then $\hat{\pi}_0 = 6/9 = 2/3$, $\hat{\pi}_1 = 1/3$; $\hat{\mu}_0 = 1$, $\hat{\mu}_1 = 4$; $\hat{\sigma}_0^2 = (1 + 0 + 1 + 1 + 0 + 1)/6 = 2/3$, $\hat{\sigma}_1^2 = (1 + 0 + 1)/3 = 2/3$. At $x = 2$ the two densities share the prefactor $1/\sqrt{2\pi \cdot 2/3}$, so compare exponentials: class 0 gives $\exp[-(2-1)^2/(2 \cdot \frac{2}{3})] = \exp(-0.75) \approx 0.472$, class 1 gives $\exp(-3) \approx 0.050$. Prior-weighted: $\frac{2}{3}(0.472) = 0.315$ and $\frac{1}{3}(0.050) = 0.017$. Normalising, $p(y = 0 \mid x = 2) = 0.315/0.332 \approx 0.950$. The point $x = 2$ is one unit from the centre of class 0 and two units from the centre of class 1; both the distance and the larger prior push the posterior toward class 0.
Where is the boundary? With equal variances the log prior-weighted likelihoods are $\log \hat{\pi}_c - (x - \hat{\mu}_c)^2/(2 \cdot \frac{2}{3})$. Setting them equal: $\log \frac{2}{3} - \frac{3}{4}(x-1)^2 = \log \frac{1}{3} - \frac{3}{4}(x-4)^2$, i.e. $\frac{3}{4}[(x-4)^2 - (x-1)^2] = -\log 2$, i.e. $\frac{3}{4}(15 - 6x) = -0.693$, giving $x \approx 2.65$. With equal priors the boundary would sit at the midpoint 2.5; the larger prior of class 0 pushes it toward class 1 by 0.15. This is LDA in one dimension: a single threshold.

Scores, LDA and QDA

Theorem 4.4.5 (LDA: shared covariance gives linear scores)**.** *Assume every class has the same covariance, $\widehat{\Sigma}_c = \widehat{\Sigma}$. Then, up to a term common to all classes, $s_{\hat{\phi},c}(x) = x^\top \widehat{\Sigma}^{-1}\hat{\mu}_c - \frac{1}{2}\hat{\mu}_c^\top \widehat{\Sigma}^{-1}\hat{\mu}_c + \log \hat{\pi}_c$ [on your sheet], which is* affine *in x. Consequently the boundary between any two classes is a hyperplane; in the binary case its normal is $w = \widehat{\Sigma}^{-1}(\hat{\mu}_1 - \hat{\mu}_0)$ [on your sheet]. This is* Linear Discriminant Analysis.

Proof. Take the log of the Gaussian density: $\log p(x \mid y = c) = -\frac{d}{2}\log(2\pi) - \frac{1}{2}\log|\widehat{\Sigma}| - \frac{1}{2}(x-\hat{\mu}_c)^\top \widehat{\Sigma}^{-1}(x-\hat{\mu}_c)$. Expand the quadratic form, using the symmetry of $\widehat{\Sigma}^{-1}$ to merge the two cross terms: $x^\top\widehat{\Sigma}^{-1}x - 2x^\top\widehat{\Sigma}^{-1}\hat{\mu}_c + \hat{\mu}_c^\top\widehat{\Sigma}^{-1}\hat{\mu}_c$. This is the key step: with a *shared* covariance, the terms $-\frac{d}{2}\log(2\pi)$, $-\frac{1}{2}\log|\widehat{\Sigma}|$ and $-\frac{1}{2}x^\top\widehat{\Sigma}^{-1}x$ are identical for every class, so they drop out of any comparison between classes. What remains, times $-\frac{1}{2}$, plus $\log\hat{\pi}_c$, is the displayed score. Two affine functions of x are equal on a hyperplane. □

Key insight

The only x-quadratic term, $x^\top\Sigma^{-1}x$, is the same for every class when Σ is shared, so it cancels in every comparison. The word *linear* in LDA describes this induced boundary, not the Gaussian density itself, which is anything but linear.

Proposition 4.4.6 (QDA: class-specific covariance gives quadratic scores)**.** *With separate covariances, up to a term common to all classes,* $s_{\hat{\phi},c}(x) = -\frac{1}{2}(x-\hat{\mu}_c)^\top\widehat{\Sigma}_c^{-1}(x-\hat{\mu}_c) - \frac{1}{2}\log|\widehat{\Sigma}_c| + \log\hat{\pi}_c$ *[on your sheet]. The first term is a squared Mahalanobis distance to the class centre; the determinant term penalises diffuse classes. The class-dependent quadratic terms make the boundaries curved (conic sections in two dimensions). This is* Quadratic Discriminant Analysis.

Proof. Same log density as above; now $\log|\widehat{\Sigma}_c|$ and the quadratic form both depend on c, so the key step of Theorem 4.4.5 fails: only $-\frac{d}{2}\log(2\pi)$ is common. Equality of two such scores is a quadratic equation in x. □

Reading the formula

LDA versus QDA is a decision about *how many numbers to estimate.* A symmetric $d \times d$ covariance has $d(d+1)/2$ distinct entries [on your sheet](d on the diagonal plus $d(d-1)/2$ above it), which is 5050 for $d = 100$. LDA estimates one shared covariance plus C means; QDA estimates C covariances: with $d = 10$ and $C = 3$ that is 165 covariance numbers against 55. In two dimensions LDA can only draw straight lines between classes; QDA can draw ellipses, parabolas and hyperbolas, and with unequal variances in one dimension there can be two cut points, the narrow class enclosed by the wide one. More flexibility, more data required. The geometry of the boundary is the visible fingerprint of the assumption you made about the class clouds.

Experiment: LDA against QDA as the class shrinks

Setup. Two classes of flows with two numerical features, the same rows and the same priors for both models.

What we hold fixed. The rows, the features, the priors. We change only the covariance assumption, and later the class size.

What we change. First the assumption: shared $\widehat{\Sigma}$ (LDA) or separate $\widehat{\Sigma}_c$ (QDA). Then the number of examples per class, from 500 down to 5, refitting on fresh samples each time.

What we measure. The drawn boundary, the validation error, and whether $\widehat{\Sigma}_c$ can be inverted at all.

What the plot shows. LDA draws a line, tilted by $\widehat{\Sigma}^{-1}$ rather than perpendicular to the segment between the means; QDA draws an ellipse or a hyperbola, and the tighter class ends up enclosed by the diffuse one. As the class shrinks, QDA's boundary swings wildly from sample to sample, and once $N_c < d + 1$ the covariance has rank at most $N_c - 1$, it is singular, and the score cannot even be computed.

How to read it. The assumption becomes a shape, and flexibility must be estimable: an extra $d(d+1)/2$ numbers per class are a liability when N_c is small. If LDA and QDA tie on validation, the extra parameters bought nothing, so take LDA.

Definition 4.4.7 (Covariance shrinkage)**.** $\widehat{\Sigma}_\lambda = (1-\lambda)\widehat{\Sigma} + \lambda\alpha I$ for $0 \le \lambda \le 1$ and a reference variance $\alpha > 0$ (e.g. the average diagonal entry) [not on your sheet: know it]. $\lambda = 0$ keeps the empirical covariance; $\lambda = 1$ removes all correlations and gives every direction the same variance. Validation selects λ.

Reading the formula

Shrinkage blends the noisy, data-specific matrix with a stable, featureless one. For any $\lambda > 0$ the result is positive definite (a positive-definite matrix plus a positive multiple of the identity), so it can always be inverted. Its validation curve over λ is Chapter 3's capacity U in another costume. Taking $\lambda \to 1$ and then dropping the remaining correlations altogether leads to Naive Bayes.

What people do wrong: choosing QDA because it is more flexible

With 120 defaults in a credit dataset and $d = 40$ features, QDA estimates $40{\cdot}41/2 = 820$ covariance numbers for the default class from 120 rows: training error goes down, validation error goes up, the classic overfitting signature, and with fewer rows the matrix is singular. The remedy is LDA, or shrinkage with λ chosen on validation. A second slip, when variances differ: keeping the constants that differ between classes out of the comparison; with unequal variances the $-\frac{1}{2}\log\sigma_c^2$ term does not cancel and must stay.

Technique: Gaussian classifiers by hand

- Compute class counts, means, and variances (divide by N_c).
- Drop constants common to all classes before comparing: with equal variances compare $\log\hat{\pi}_c - (x-\hat{\mu}_c)^2/(2\sigma^2)$ only.
- To get a posterior, exponentiate the scores and normalise (softmax of the generative scores).
- Equal priors and equal variances in one dimension: the boundary is the midpoint of the means. A larger prior moves it away from that class; a larger variance in QDA bends the region.

How the professor asks it

Q. *Why is LDA's boundary linear although the Gaussian density is not? What breaks when each class gets its own covariance?*
A. The boundary is linear because of a cancellation, not because of the density. The class score is $\log \pi_c + \log \mathcal{N}(x; \mu_c, \Sigma)$, and the only term quadratic in x, $-\frac{1}{2} x^\top \Sigma^{-1} x$, together with the normaliser $-\frac{1}{2} \log |\Sigma|$, is the same for every class when Σ is shared; both drop out of every comparison, leaving the affine score $x^\top \Sigma^{-1} \mu_c - \frac{1}{2} \mu_c^\top \Sigma^{-1} \mu_c + \log \pi_c$, and two affine functions are equal on a hyperplane. With a Σ_c per class both terms depend on c, nothing cancels, the comparison becomes a quadratic equation and the boundary a conic. It also costs $d(d+1)/2$ extra numbers per class, which is why QDA needs far more data and becomes unstable or singular when N_c is small.

Q. *Two classes, equal priors, $\mu_0 = 0$, $\mu_1 = 4$, shared $\sigma^2 = 1$: where is the boundary? And with $\pi_0 = 0.9$, $\pi_1 = 0.1$?*
A. With equal priors and equal variances the boundary is the midpoint, $x = 2$, because the two scores $-\frac{1}{2}(x-\mu_c)^2$ are equal exactly there. With priors 0.9/0.1 set $-\frac{1}{2}x^2 + \log 0.9 = -\frac{1}{2}(x-4)^2 + \log 0.1$; the quadratic terms cancel (shared variance), leaving $4x - 8 = \log 9 \approx 2.197$, so $x \approx 2.55$: the boundary moves away from the common class, because the rare class must now explain the input much better before it is declared.

Q. *A class has 5 examples in $d = 10$ dimensions. What goes wrong with QDA, why, and what are the two remedies?*
A. The class covariance is an average of 5 centred outer products, so its rank is at most $4 < 10$: it is singular, cannot be inverted, and the Mahalanobis score cannot be computed at all. With a few more examples it becomes invertible but remains unstable, meaning a different sample would give a very different ellipse, which is variance. The two remedies are to shrink, $\widehat{\Sigma}_\lambda = (1-\lambda)\widehat{\Sigma} + \lambda \alpha I$, positive definite for any $\lambda > 0$ with λ chosen on validation, or to impose structure: a shared covariance (LDA, 55 numbers estimated from all classes together) or a diagonal one (Gaussian Naive Bayes, 10 variances).

4.5 Naive Bayes

What are we trying to get?

When estimating a full joint distribution of the features is hopeless, we can pretend the features are independent *within each class*. We want to see exactly what this buys (one small model per feature, additive evidence, stability on little data), what it costs (double-counted evidence, overconfident probabilities), and how to fix the two failure modes that appear in practice: zero variances and zero counts.

Definition 4.5.1 (Naive Bayes)**.** Naive Bayes assumes the features are *conditionally independent given the class*: $p_\theta(x \mid y = c) = \prod_{j=1}^{d} p_\theta(x_j \mid y = c)$ [on your sheet]. Taking logarithms, the generative score becomes *additive*: $s_{\hat{\phi},c}(x) = \log \hat{\pi}_c + \sum_{j=1}^{d} \log p_{\widehat{\theta}}(x_j \mid y = c)$ [on your sheet].

Reading the formula

Read the score as a ledger: start with the prior evidence $\log \hat{\pi}_c$, then add one line of evidence per feature, each computed from a *one-dimensional* model of that feature within class c. Flow duration, packet counts and byte statistics may in reality be correlated within an attack class, so the assumption is strong. What it buys is that a difficult d-dimensional density estimation problem becomes d easy one-dimensional ones, each of which can be stable with limited data, and the one-dimensional models need not share a family: a Gaussian here, a Bernoulli there, a Poisson for a count, a histogram for an irregular feature.

Interpretation & discussion

One more fact makes Naive Bayes far more useful than its assumption suggests: a *misspecified density can still rank classes correctly*, because getting the largest score right is a weaker requirement than getting every density right. The main risk is that *correlated features repeat the same evidence*: two nearly identical features count twice, so the classifier may still classify well while its probabilities become too extreme. Gaussian Naive Bayes with class-specific variances per feature is a diagonal QDA, so its boundary is quadratic even though each feature is Gaussian; with shared variances it is a diagonal LDA.

Experiment: the duplicated feature

Setup. Gaussian Naive Bayes on flow data with a "bytes" column among its features, validation rows with labels.

What we hold fixed. Everything except one column.

What we change. We duplicate the "bytes" column k times, for $k = 1, 2, 4, 8$, and refit.

What we measure. Accuracy at threshold 0.5, and the reliability diagram (observed fraction of attacks among flows given probability u, plotted against u).

What the plot shows. Accuracy barely moves, because no information was added or removed. The reliability curve bends into an S: a flow that was given 0.7 is now given 0.9 and then 0.99, while the observed fraction of attacks among such flows has not changed.

How to read it. The log-odds are multiplied by k because the same evidence was counted k times. The ranking survives and the calibration dies, and because accuracy looks fine the problem is missed. Everything that reads the probability as a probability is now broken: cost-sensitive thresholds, the prior-shift correction, soft voting.

What people do wrong: letting correlated features double-count

The temptation is to throw every available feature into Naive Bayes, since each one is "just another line of the ledger". Bytes and packets, which are nearly proportional, then count twice. Either remove redundant features, or use a model that sees correlations, such as LDA or logistic regression. Two practical rules follow: work in logs when there are more than three or four

features, and count the prior exactly once.

Gaussian Naive Bayes and variance smoothing

For numerical features, Gaussian Naive Bayes models $x_j \mid (y = c) \sim \mathcal{N}(\hat{\mu}_{cj}, \hat{\sigma}^2_{cj})$ with the one-dimensional MLEs of Theorem 4.4.2 applied feature by feature. Two problems arise. If a fitted variance is 0 (all class-c values of feature j coincide), the density is undefined. If it is merely tiny, that one feature's narrow density dominates the whole class score.

Definition 4.5.2 (Variance smoothing)**.** Replace each fitted variance by $\tilde{\sigma}^2_{cj} = \hat{\sigma}^2_{cj} + \lambda s^2_j$ with $\lambda > 0$, where s^2_j is a reference scale such as the overall training variance of feature j [not on your sheet: know it]. A small λ prevents numerical instability; if its value materially affects predictions, select it on validation data.

Example 4.5.3 (A tiny variance takes over)**.** Two features, class c: feature 1 has $\hat{\sigma}^2 = 1$, feature 2 has $\hat{\sigma}^2 = 10^{-6}$ (every training example of class c had exactly the same value, say a port number). A test point whose feature 2 differs from that value by 0.01 contributes $-\frac{1}{2}(0.01)^2/10^{-6} = -50$ to the score of class c, dwarfing anything feature 1 can say. With $\lambda s^2_2 = 0.01$ added, the contribution becomes $-\frac{1}{2}(10^{-4})/(0.01) \approx -0.005$: feature 2 still counts, but no longer vetoes. Variance smoothing is the continuous twin of the additive smoothing of counts below: both add a little imaginary spread so that no single observation can produce a zero or an infinity.

The independence assumption is what lets you mix families freely: Gaussian for measurements such as flow statistics, Bernoulli for presence or absence indicators, multinomial for counts such as words or events, and a hybrid where each feature gets the distribution of its type, all multiplied under *one* class prior. A Bernoulli example by hand: features (has a link, from a known sender), spam rates $(0.9, 0.1)$, ham rates $(0.2, 0.8)$, priors 0.5. An email with a link from a known sender scores spam $0.5 \cdot 0.9 \cdot 0.1 = 0.045$ and ham $0.5 \cdot 0.2 \cdot 0.8 = 0.08$: ham, with posterior $0.08/0.125 = 0.64$. Multinomial treats a repeated word as repeated evidence; Bernoulli only records presence, and for it absence is evidence too, through the factor $1 - \theta$.

Multinomial Naive Bayes for text

Definition 4.5.4 (Multinomial Naive Bayes)**.** Fix a vocabulary of V words and represent a document by its count vector $x = (x_1, \ldots, x_V)$. For class c, let $\theta_{cj} = p_\theta(\text{token} = j \mid y = c)$ with $\sum_j \theta_{cj} = 1$. Ignoring the multinomial coefficient (common to all classes), $p_\theta(x \mid y = c) \propto \prod_{j=1}^{V} \theta_{cj}^{x_j}$ [on your sheet], with the convention $0^0 = 1$. Repeated words contribute repeated evidence; absent words contribute the factor 1; word order is ignored.

Reading the formula

Imagine class c as a bag of words with proportions $\theta_{c\cdot}$; a document is produced by drawing its words from the bag. "free free prize" with vocabulary (free, win, prize, meeting, project, report) has $x = (2, 0, 1, 0, 0, 0)$ and likelihood

$\propto \theta_{c,\text{free}}^2\,\theta_{c,\text{prize}}$. The multinomial coefficient (how many orderings produce these counts) is the same for every class and cancels in Bayes' rule.

Example 4.5.5 (Why smoothing is needed)**.** Aggregate training counts, seven tokens per class, over (free, win, prize, meeting, project, report): spam $(3,2,2,0,0,0)$, ham $(0,0,0,3,2,2)$. Without smoothing, $\hat\theta_{cj} = n_{cj}/7$, and the test message "free meeting" gives $p(x \mid \text{spam}) \propto \frac{3}{7}\cdot\frac{0}{7} = 0$ and $p(x \mid \text{ham}) \propto \frac{0}{7}\cdot\frac{3}{7} = 0$. One unseen word zeroes an entire product, and here *both* products vanish, so the posterior 0/0 is undefined. A single word never seen in training in a class would otherwise veto that class no matter how strong the other evidence.

Definition 4.5.6 (Additive smoothing)**.** With n_{cj} the training count of token j in class c and $\alpha > 0$, $\hat\theta_{cj} = \dfrac{n_{cj}+\alpha}{\sum_{\ell=1}^{V} n_{c\ell} + \alpha V}$ [on your sheet]. For a categorical feature x_j with K_j levels, $p_{\widehat{\theta}}(x_j = v \mid y = c) = \dfrac{n_{cjv}+\alpha}{n_{cj}+\alpha K_j}$.

Proposition 4.5.7 (Smoothed estimates are proper and interpolate to uniform)**.** *For every $\alpha > 0$: (i) $\hat\theta_{cj} > 0$ for all j and $\sum_j \hat\theta_{cj} = 1$; (ii) as $\alpha \to 0$, $\hat\theta_{cj} \to n_{cj}/\sum_\ell n_{c\ell}$; (iii) as $\alpha \to \infty$, $\hat\theta_{cj} \to 1/V$. Moreover $\hat\theta_{cj}$ is the estimate one obtains by adding α imaginary occurrences of every word to the class.*

Proof. (i) Numerators are $\geq \alpha > 0$; summing $n_{cj} + \alpha$ over j gives exactly the denominator. (ii) Set $\alpha = 0$ in the limit. (iii) Divide numerator and denominator by α: the counts become negligible and the ratio tends to $1/V$. The last claim is the formula itself: counts $n_{cj} + \alpha$ over total $\sum_\ell n_{c\ell} + V\alpha$. □

Reading the formula

Pretend you saw every word α extra times in every class. With $\alpha = 1$ ("Laplace smoothing") and $V = 6$: spam counts become $(4,3,3,1,1,1)$ over $7+6=13$, ham counts $(1,1,1,4,3,3)$ over 13. Now "free meeting" with equal priors has $\exp[s_{\text{spam}}] = \frac{1}{2}\cdot\frac{4}{13}\cdot\frac{1}{13}$ and $\exp[s_{\text{ham}}] = \frac{1}{2}\cdot\frac{1}{13}\cdot\frac{4}{13}$, so $p(\text{spam} \mid x) = \frac{1}{2}$: both words are now possible under both classes, and in this symmetric message their evidence balances exactly. "free prize" gives $\frac{12}{338}$ against $\frac{1}{338}$, so $p(\text{spam} \mid x) = 12/13 \approx 0.92$; and "free free prize" gives $48/49 \approx 0.98$, the repeated word counting twice, as the multinomial model says it should. Larger α pulls all estimates toward $1/6$; smaller α trusts the counts more. A categorical value that appears at test time but never in training (a new port number) must have a defined representation: create an *other* level among the K_j levels, and smoothing gives it a small positive probability instead of a crash.

Experiment: the α sweep

Setup. Multinomial Naive Bayes on a labelled email corpus, training and validation rows.

What we hold fixed. The vocabulary, the rows, the priors.

What we change. The smoothing constant α on a logarithmic grid from 10^{-3} to 10^{3}.

What we measure. Validation log-loss and accuracy.

What the plot shows. The log-loss is infinite at $\alpha = 0$ (vetoes), has its minimum near 0.1 to 1, and rises toward the uniform model: with $\alpha = 100$ on counts $(5, 0, 3, 2)$ and $V = 4$ the estimates are $(105, 100, 103, 102)/410 \approx (0.256, 0.244, 0.251, 0.249)$, almost no class information left.

How to read it. α is a capacity knob like any other: small trusts the counts (variance, vetoes), large trusts the uniform prior (bias). Choose it on validation, exactly as you choose λ in Chapter 3.

Technique: Naive Bayes by hand

- Work in logs when there are more than three or four features; products of small numbers underflow.
- Apply smoothing *before* normalising: add α to every count, add αV (or αK_j) to the total.
- Compare unnormalised scores first; normalise only if a posterior is asked for.
- Count the prior exactly once, not once per feature; remove near-duplicate features.

How the professor asks it

Q. *Multinomial Naive Bayes classifies "free meeting" as neither spam nor ham. Explain the failure, fix it with $\alpha = 1$, and say what large α does.*
A. The word "meeting" never occurred in spam and "free" never in ham, so each class likelihood contains a factor of zero and the posterior is 0/0: a single unseen word vetoes a class no matter how strong the other evidence is. Additive smoothing adds α imaginary occurrences of every word to every class, $\hat{\theta}_{cj} = (n_{cj} + 1)/(7 + 6)$, so spam becomes $(4, 3, 3, 1, 1, 1)/13$ and ham $(1, 1, 1, 4, 3, 3)/13$; "free meeting" now scores $\frac{1}{2} \cdot \frac{4}{13} \cdot \frac{1}{13}$ under both classes and the posterior is $\frac{1}{2}$. As $\alpha \to \infty$ every $\hat{\theta}_{cj} \to 1/V$ and the classes become indistinguishable, which is bias; as $\alpha \to 0$ the vetoes come back, which is variance. So α is a capacity knob and you choose it on validation.

Q. *Naive Bayes on flow data keeps its accuracy when a duplicate of the bytes feature is added, yet its probabilities become extreme. Explain, and name a consequence.*
A. Conditional independence makes the class score additive, so a duplicated feature adds the same log-likelihood term twice and the log-odds are doubled. The sign of the log-odds, hence the ranking and the accuracy at threshold 0.5, is unchanged, but a flow that deserved 0.7 is now given 0.9 and the reliability curve bends below the diagonal: the model has become overconfident without becoming more wrong. The consequence is that everything which reads the probability as a probability is now broken: cost-sensitive thresholds, the prior-shift correction and soft voting. The fix is to drop redundant features or to use a model that sees correlations, such as LDA or logistic regression.

Q. *Why can Gaussian Naive Bayes have a boundary that is not linear even though each feature is Gaussian?*
A. Because its class scores are quadratic in x. Each feature contributes

$-\frac{1}{2}(x_j - \hat{\mu}_{cj})^2/\hat{\sigma}^2_{cj} - \frac{1}{2}\log\hat{\sigma}^2_{cj}$, and when the variances differ between classes the quadratic terms $x_j^2/\hat{\sigma}^2_{cj}$ do not cancel in the comparison; this is QDA with diagonal covariances. Only with variances shared across classes does the quadratic term cancel and the boundary become linear, a diagonal LDA.

4.6 Comparing the Two Routes

What are we trying to get?

We want a fair statement of when each route is preferable, phrased in terms of assumptions and data, not slogans, and the experiment that settles it on a given dataset.

Telling them apart: the two routes, question by question

Question	Discriminative	Generative
Modelled quantity	posterior $p_\theta(y \mid x)$ or a boundary	prior π_c and class model $p_\theta(x \mid y = c)$
Input assumptions	usually fewer	usually stronger
Primary objective	predict the class	describe class-specific inputs
Missing features	need separate treatment	can sometimes be marginalised
Sampling inputs	usually unavailable	possible in principle
Raw high-dimensional input	often practical	full density estimation is hard

Interpretation & discussion

Generative structure can *reduce estimation error*: a restricted description of each class has few parameters, so with little labelled data it can be fitted reliably, provided the assumptions are approximately right. A wrong class-conditional family creates *approximation bias* that does not vanish with more data. A sufficiently flexible discriminative model may eventually reach lower classification error because it spends no capacity modelling aspects of X that are irrelevant to Y. Schematically, error versus labelled data often shows the generative curve lower at small N and the discriminative curve lower at large N, but this is not a universal law: the crossing point, and whether it exists, depends on the data distribution, the two families, regularisation and optimisation. The right question is not "which route is better" but "which assumptions are credible, and which deployment objective matters".

Example 4.6.1 (Same data, two verdicts)**.** Spam data with 200 labelled emails and a 10,000-word vocabulary. Multinomial Naive Bayes needs to estimate $2 \times 10{,}000$ smoothed word probabilities, each from simple counts: it works reasonably from day one. Logistic regression on the same counts must locate a 10,000-dimensional hyperplane from 200 points: badly underdetermined, prone to overfitting. With 2 million labelled emails the picture flips:

logistic regression (or a network) learns interactions such as "*free* is spammy unless *invoice* is nearby", which the independence assumption forbids Naive Bayes from ever representing.

Experiment: the data-size experiment

Setup. One labelled corpus, a fixed feature representation, Naive Bayes with its smoothing α and logistic regression with its regularisation λ, and an untouched test set.

What we hold fixed. The features, α, λ, the test set.

What we change. The number of training rows, $N \in \{50, 200, 10^3, 10^4, 10^5, 2 \cdot 10^6\}$, with ten random subsets at each size.

What we measure. Test error of each model at each N, with its spread across the ten subsets.

What the plot shows. Naive Bayes starts low and flattens at its bias floor. Logistic regression starts high with a wide spread, falls, and crosses below somewhere between 10^3 and 10^5.

How to read it. The crossing point is the result you report; its existence and location depend on the data and on both families. Do not read the flat generative curve as "it needs more data": the floor is approximation error, so the fix is a better class model, not more rows. And do not read the wide spread of the discriminative curve at small N as bias: it is variance.

How the professor asks it

Q. *With 200 labelled emails and a 10,000-word vocabulary, which route would you start with and why? What changes with 2 million emails? Describe the experiment that settles it.*
A. Start with the generative route. Multinomial Naive Bayes estimates $2 \times 10{,}000$ smoothed word probabilities from simple counts and is stable from day one, whereas logistic regression must locate a 10,000-dimensional hyperplane from 200 points, which is badly underdetermined and therefore high-variance. With millions of labels the picture flips: the independence assumption is now a persistent approximation bias, since it cannot represent interactions such as "free unless invoice", while logistic regression's variance has vanished. The experiment: fix the features, α, λ and an untouched test set; vary N on a log scale with repeated random subsets; plot both test-error curves; report where they cross and the height of the generative floor.

4.7 Class Imbalance and Prior Shift

What are we trying to get?

Two situations are constantly confused: a class that is rare, and a class whose frequency *changes* between training and deployment. We want to separate them, and for the second, derive the exact correction for both routes.

Definition 4.7.1 (Calibration)**.** A probabilistic classifier is *calibrated* if, among cases assigned probability u, approximately a fraction u are positive.

Calibration concerns the interpretation of probabilities; the choice of a decision threshold from costs concerns the action taken from them. These are separate questions, and both must be right for a cost-sensitive decision to be right.

Telling them apart: calibration versus threshold

Draw the reliability diagram on validation rows: bin the predicted probabilities, plot the observed positive fraction in each bin against the bin's probability; points below the diagonal mean the model is overconfident. The temptation is to repair a calibration problem by moving the threshold. But the ranking is unchanged by any monotone distortion of the probabilities, so accuracy may look fine while the expected number of alarms and the precision at the cost threshold are still wrong. Calibrate first, then set the threshold from the costs. Resampling and duplicated features both break calibration while leaving accuracy alone, which is exactly why the problem goes unnoticed until the counts come in wrong.

If training and deployment share the same joint distribution, an imbalanced training prior is not an error: the fitted posterior should reflect that prevalence. The difficulty is finite minority data: with few positives, the minority distribution or the boundary is poorly estimated, and weighting or resampling may help estimation. Otherwise their benefit is limited and the resulting probabilities may need recalibration.

Definition 4.7.2 (Class-prior shift)**.** Between training (t) and deployment (d), the class-conditional distributions are unchanged but the priors differ: $p_d(x \mid y = c) = p_t(x \mid y = c)$, $\pi_{d,c} \neq \pi_{t,c}$ [not on your sheet: know it]. Example: attacks form 20% of the training data but 5% of deployment traffic.

Theorem 4.7.3 (Generative correction)**.** *Under prior shift, the deployment posterior is obtained by keeping the fitted class-conditional models and replacing the training priors by the deployment priors:* $p_d(y = c \mid x) = \dfrac{\pi_{d,c}\, p_{\widehat{\theta}}(x \mid y = c)}{\sum_k \pi_{d,k}\, p_{\widehat{\theta}}(x \mid y = k)}$ *[not on your sheet: know it]. No class-conditional model needs to be refitted.*

Proof. Bayes' rule at deployment uses π_d and $p_d(x \mid y)$; by Definition 4.7.2 the latter equals $p_t(x \mid y)$, which is what the fitted $p_{\widehat{\theta}}(x \mid y)$ estimates. Substituting gives the formula. The prior is an explicit component of the model, so it can simply be swapped. □

Theorem 4.7.4 (Discriminative correction, binary case)**.** *Let* $r_t(x) = p_t(y = 1 \mid x)$, $r_d(x) = p_d(y = 1 \mid x)$, $\pi_t = p_t(y = 1)$, $\pi_d = p_d(y = 1)$. *Under prior shift,*

$$r_d(x) = \frac{(\pi_d/\pi_t)\, r_t(x)}{(\pi_d/\pi_t)\, r_t(x) + \dfrac{1-\pi_d}{1-\pi_t}\,[1 - r_t(x)]}. \quad \text{[not on your sheet: know it]}$$

Proof. **Step 1 (odds form).** At training time, $\dfrac{r_t(x)}{1 - r_t(x)} = \dfrac{\pi_t\, p(x \mid y = 1)}{(1 - \pi_t)\, p(x \mid y = 0)}$

by the odds form of Theorem 4.3.2; likewise at deployment with π_d. **Step 2 (the likelihood ratio is shared).** Under prior shift $p(x \mid y = 1)/p(x \mid y = 0)$ is the same at both times, because the class-conditionals did not change; this is the assumption that makes the correction possible. Solving the training odds for it and substituting into the deployment odds, $\dfrac{r_d}{1-r_d} = \dfrac{\pi_d/(1-\pi_d)}{\pi_t/(1-\pi_t)} \cdot \dfrac{r_t}{1-r_t}$. **Step 3 (solve for r_d).** If odds $= A/B$ then probability $= A/(A+B)$. With $A = \frac{\pi_d}{\pi_t} r_t$ and $B = \frac{1-\pi_d}{1-\pi_t}(1-r_t)$ the odds above equal A/B, hence $r_d = A/(A+B)$. Step 1 read r_t as a posterior, so the formula silently requires r_t to be calibrated. □

Reading the formula

The correction *reweights the two hypotheses*: multiply the training probability of class 1 by "how much rarer class 1 became" (π_d/π_t), multiply the probability of class 0 by "how much commoner class 0 became" $((1-\pi_d)/(1-\pi_t))$, then renormalise. Attacks 20% → 5%: $\pi_d/\pi_t = 0.25$ and $(1-\pi_d)/(1-\pi_t) = 0.95/0.80 = 1.1875$. A flow with training posterior $r_t = 0.5$ gets $A = 0.125$, $B = 0.594$, so $r_d = 0.125/0.719 \approx 0.17$. What looked like a coin flip in training is a 17% attack probability in deployment, because attacks are four times rarer there. The generative route makes the prior an explicit component and swaps it exactly; the discriminative route can only correct after the fact and depends on the quality of its probabilities.

Experiment: prior shift from 20% to 5%

Setup. One pool of labelled flows, from which we build a training set at 20% attacks and a test set at 5% attacks, so that the class-conditionals are identical by construction.

What we hold fixed. The fitted model, the test rows, the costs.

What we change. Only the correction applied to the probabilities: none, swap the priors (generative), reweight the odds (discriminative).

What we measure. Log-loss, the reliability diagram, and the number of alarms at the cost threshold.

What the plot shows. Uncorrected, flows predicted at 0.5 are observed at about 0.17 and you raise four times too many alarms. Corrected, the reliability diagram is back on the diagonal and the alarms are what the costs intended; recall drops slightly, and accuracy is almost unchanged.

How to read it. Accuracy hides the shift because the correction is monotone and does not reorder the flows; probabilities and alarm counts do not hide it. Apply the correction before the cost threshold, never after.

What people do wrong: resampling and then trusting the probabilities

The temptation is to oversample to a balanced training set and then read the fitted probabilities as deployment probabilities. The effective prior is now 0.5, so the rare-class probabilities are inflated; the fitted posterior corresponds to the resampled prior and must be corrected back with Theorem 4.7.4, using π_t equal to the resampled prior, if calibrated probabilities are wanted. And remember the order: imbalance ≠ shift; imbalance with enough minority data

needs no correction at all, shift needs one even with abundant data; cost-sensitive thresholds and prior correction are different operations, so apply the correction first, then the threshold.

How the professor asks it

Q. *Attacks are 20% of the training set and 5% in deployment; the class-conditional distributions are unchanged. A flow gets training posterior 0.5. What is its deployment posterior, how do you get it for a generative and for a discriminative model, and which assumption does the discriminative correction add?*
A. About 0.17. For a generative model keep the fitted $p(x \mid c)$ and replace $\pi_t = (0.8, 0.2)$ by $\pi_d = (0.95, 0.05)$ in Bayes' rule; that is exact and needs no refit, because the prior is an explicit component of the model. For a discriminative model use the fact that the likelihood ratio is shared between the two times and reweight the odds: multiply the attack side by $\pi_d/\pi_t = 0.25$ and the benign side by $(1-\pi_d)/(1-\pi_t) = 1.1875$, giving $A = 0.125$, $B = 0.594$ and $r_d = 0.125/0.719 = 0.17$. The extra assumption is that r_t is a calibrated probability; a model trained on a resampled set, or an overconfident one, must be calibrated first. And apply the cost threshold after the correction, not before.

Q. *Attacks are 5% of both the training data and the deployment traffic. A colleague says the classifier is "biased" because its* 0.5 *contour sits toward the attacks and wants to rebalance. Is a correction needed?*
A. No. When the joint distribution is the same at training and deployment, the fitted posterior should reflect the 5% prevalence, and the boundary sitting toward the positives is Bayes' rule doing its job, not an error. Imbalance only becomes a problem of estimation, when the minority is too small to locate the boundary reliably; then weighting or resampling can help, but it changes the effective prior, and the probabilities must be corrected back before any cost threshold is applied. A correction is needed when the prior *changes* between training and deployment, and that is a different situation with a different name.

4.8 Unfamiliar Inputs and Combined Systems

What are we trying to get?

Security systems meet inputs that belong to no known class. We want a mechanism that can say "I do not recognise this", and to see how modern systems combine the two routes, including generative language models used for classification.

A high value of $\max_c p_{\hat{\theta}}(y = c \mid x)$ only means that one modelled class outranks the others. It does not establish that x resembles the training data. A new attack mechanism may lie far from every known class while still receiving a confident known-class prediction.

Definition 4.8.1 (Reject option via a compatibility check)**.** Choose the most probable known class $c^* = \arg\max_c p_{\hat{\phi}}(y = c \mid x)$, then compare the fitted class

likelihood with a class-specific threshold: $\hat{y} = c^*$ if $\log p_{\widehat{\theta}}(x \mid y = c^*) \geq \tau_{c^*}$, and *reject* otherwise [not on your sheet: know it].

Reading the formula

Two questions instead of one. *Which known class fits best?* (the posterior). *Does even the best class fit well enough?* (the likelihood against a threshold). Only a generative model can answer the second, because only it has a notion of what inputs of class c look like. The threshold τ_c should be validated on data that reproduce the intended novelty, e.g. attack mechanisms excluded from training. A low likelihood is evidence of incompatibility, not a guarantee of detection: high-dimensional likelihoods behave counterintuitively, and a misspecified class model may give low likelihood to perfectly valid cases.

Experiment: the held-out attack family

Setup. Flow data with six attack families; a discriminative ranker and a generative class model per known class.

What we hold fixed. Training on five families only; the sixth is never seen in training.

What we change. The reject threshold τ.

What we measure. False rejects of known flows against true rejects of the novel family, as τ moves.

What the plot shows. A trade-off curve; the softmax-only system gives no handle at all, because its top probability on the novel family is high.

How to read it. The curve is the result, and the threshold is chosen on validation data that contain the kind of novelty you expect. Tuning τ on data with no novelty, or treating a low likelihood as proof of attack, are the two ways to misuse it.

Example 4.8.2 (Combining both routes, and language models)**.** A discriminative model ranks the known classes for an observed flow, producing $p_{\widehat{\theta}_d}(y = c \mid x)$. A generative or anomaly model computes $\log p_{\widehat{\theta}_g}(x \mid y = c)$ for the predicted class. The final policy accepts the known class, requests human review, or rejects the input as unfamiliar: the discriminative model is good at the ranking, the generative model is the plausibility check it lacks. Language models are the same marriage at scale. An autoregressive model is trained *generatively* by factorising a token sequence, $p_\theta(w_{1:T}) = \prod_{t=1}^{T} p_\theta(w_t \mid w_{<t})$, with next-token prediction as its objective; a *sequence-classification head* then models $p_\psi(y \mid w_{1:T})$ and minimises the discriminative objective $-\sum_i \log p_\psi(y^{(i)} \mid w^{(i)}_{1:T_i})$, one label per sequence; alternatively a prompt asks the model to generate or score label tokens directly.

Telling them apart: softmax confidence versus the likelihood check

Softmax confidence compares the known classes with one another; the likelihood check asks whether the input resembles the training data of the winning class. Different questions, different tools, and only the second can say "I have not seen this before".

How the professor asks it

Q. *Why can a classifier say "cat, 96%" for a toaster, and what would you add?*
A. The softmax was trained only to compare the known classes, so 96% means cat outranks dog and horse under that model; it does not mean the input resembles a cat or any training input. Add a compatibility or likelihood check for the winning class, validate its rejection threshold on deployment-like novelty, and reject or review the input when even the best known class fits poorly.

4.9 Model Choice and Lab 2

Three questions organise model choice. *Can a credible class-conditional model* $p_\theta(x \mid y)$ *be stated?* If so, its structure may reduce the amount of labelled data required. *Is the main objective accurate prediction with many labelled examples?* A flexible discriminative model avoids unnecessary assumptions about the input distribution. *Is the raw input complex and high-dimensional?* Learn a representation first, then compare simple classification heads; that is more credible than modelling the raw density. Deployment-like validation decides between credible alternatives; the labels "generative" and "discriminative" do not replace empirical comparison under the intended use conditions. Lab 2 is this chapter as an experiment: Gaussian, multinomial and hybrid Naive Bayes against logistic regression and a decision tree, on one fixed stratified split and the same rows for every model, so that differences reflect the models rather than the data; then limited training data (20% of each class); then a deployment prior $\pi_d = 0.05$ corrected by swapping priors for the generative models and by Theorem 4.7.4 for the discriminative ones; then multiclass classification with and without destination-port information.

Telling them apart: the five classifiers of the chapter

Model	Modelled quantity	Parameters	Boundary	Data needed	Main failure
Logistic regression	$p(y \mid x)$, log-odds linear	$(C-1)(d+1)$	linear	most	underdetermined when $d \gg N$; no input plausibility
LDA	Gaussian $p(x \mid c)$, shared Σ	$(C-1) + Cd + \frac{d(d+1)}{2}$	linear	little	wrong shape if clouds differ
QDA	Gaussian, own Σ_c	$(C-1) + Cd + C\frac{d(d+1)}{2}$	conic	much per class	singular or unstable Σ_c when $N_c \lesssim d$
Gaussian NB	independent Gaussians per feature	$(C-1) + 2Cd$	quadratic (diagonal QDA)	very little	double counting, tiny variances
Multi. NB	word-count model per class	$C(V-1)$, smoothed	linear in counts	very little	zero counts without α; no interactions

How the professor asks it

Q. *With very little labelled text and a large vocabulary, which route is the sensible baseline, and how do you test whether that choice should change?*

A. Start with smoothed multinomial Naive Bayes: its class-conditional counts impose useful structure and are stable with few labels. With much more data, a discriminative model may overtake it because the independence assumption becomes persistent approximation bias. Hold the representation and test set fixed, vary the training size with repeated subsets, and compare the two learning curves; the crossing point answers the question empirically.

Summary: Discriminative and generative classification

- **Decisions** depend on the posterior and the loss; the posterior must be *learned*, by one of two routes.
- **Discriminative** models estimate $p(y \mid x)$: scores, sigmoid/softmax, cross-entropy as conditional maximum likelihood; the score family fixes the boundary geometry and the costs only slide it; softmax confidence is relative, not familiarity.
- **Generative** classifiers model $p(x \mid y)$ and $p(y)$, then apply Bayes' rule; the likelihood splits by class and the MLE prior is the class frequency; a wrong family leaves a persistent bias.
- **Gaussian** class models: closed-form MLE with denominator N_c; shared covariance gives linear boundaries (LDA), separate covariances quadratic ones (QDA); a covariance costs $d(d+1)/2$ parameters, so shrink or simplify when data are scarce.
- **Naive Bayes** trades a strong independence assumption for stability; smooth variances and counts; correlated features double-count and ruin calibration while accuracy survives.
- **Trade-off:** generative structure helps with little data if credible; flexible discriminative models win with much data; the data-size experiment decides.
- **Imbalance** is an estimation issue; **prior shift** needs a posterior correction: swap priors, or reweight the odds, before thresholding.
- **Confidence is not familiarity**: add a likelihood-based reject option; combine routes; language models pair generative pretraining with discriminative heads.

Chapter 5

Learning by Combining Models

Chapter 3 showed you that a flexible model trained on one sample has high variance: hand the same algorithm a second sample from the same world and it returns a noticeably different fit. The idea of this chapter is almost embarrassingly simple. If one fit is unstable, train several and combine them, because the parts of their errors that differ will cancel in the average while the signal they share survives. That sentence contains a condition, and the condition is everything: the errors must differ. So the chapter is really a catalogue of ways to manufacture models whose errors differ, by using different algorithms, different data or different targets, together with the rules for combining them: a vote, an average, a sum of corrections, or a learned combination. The base learner throughout is the decision tree, because it is flexible, fast and, conveniently for us, unstable. Every formula below is on your sheet unless tagged otherwise; what the professor tests is whether you know what each knob manipulates, what the experiment looks like, and what goes wrong.

5.1 Why Combine Models?

What are we trying to get?

We want to see why *averaging several fits* attacks exactly the variance problem of Chapter 3, under what condition it works (the errors must differ), and which three ways of producing different fits are in common use.

The wisdom of the crowd

Many rough guesses, averaged, can beat each rough guess.

At a livestock fair in 1906, 787 visitors guessed the dressed weight of an ox. Individual guesses varied widely, yet their mean, 1197 lb, was within one pound of the true weight, 1198 lb (Galton, *Vox Populi*, 1907). The same idea is used to label data: when three annotators look at a network event and two say *malicious* while one says *benign*, the majority label is kept and strong disagreements go to an expert. Replace "annotators" by "fitted models" and you have an *ensemble*: several models produce several predictions, and a vote or an average produces one.

Two conditions make a crowd wise. The members must be *competent*, because averaging random noise gives noise. And their errors must *differ*, because a mistake that everybody makes survives the average untouched. Three copies of the same model are not an ensemble; they have the same strengths and exactly the same mistakes.

One fitted model can be unstable

Same algorithm, different sample, different boundary: that is variance.

Train the same learning algorithm on two different samples from the same population and the two fitted decision boundaries differ. An unstable learner fits the signal *and* the sample-specific noise, and the noise part changes from sample to sample. In the language of Chapter 3, the prediction at a fixed x has a large $\mathrm{Var}(x)$. Where positive and negative errors occur in different places for different fits, averaging cancels part of the fluctuation and leaves the shared signal.

Experiment: Two samples, two boundaries

Setup. One learning algorithm, say a deep tree, and a population from which we can draw samples.

What we hold fixed. The algorithm and all of its settings.

What we change. Only the training sample: we draw two samples of the same size and fit the tree on each.

What we measure. The two fitted decision boundaries, overlaid on one plot, and the places where each fit is wrong.

What the plot shows. The two boundaries do not coincide; the gap between them is $\mathrm{Var}(x)$ made visible. Where the two fits err in different places, their average errs less than either one; where they err in the same place, the average inherits the error.

How to read it. That picture is the whole chapter. Everything that follows is a recipe for making the boundaries differ in the right way, and a rule for averaging them.

Key insight

Combining models helps when the component models are useful *and* their errors differ. Shared errors do not cancel. Every ensemble method is a recipe for producing competent models whose errors are as different as possible, plus a rule for combining them.

Telling them apart: the three families

Students mix up the three families because they all end with "combine". The way to keep them apart is to ask where the diversity comes from. *Voting and averaging* use different algorithms, settings, features or initialisations, usually all fitted to the same data, and combine them by a fixed vote or average. *Bagging and random forests* use the same learner on randomised data: bootstrap samples of the rows and, for a forest, random subsets of candidate features at every split; they too combine by a fixed vote or average. *Boosting* uses a sequence of models, each one aimed at the current errors of the ensemble so far, and combines them by successive additive corrections. Two further architectures replace the fixed combination rule by a learned one: *stacking* learns one global combination of the base predictions, and a *mixture of experts* learns an input-dependent one.

What people do wrong: three copies of the same model

The tempting shortcut is to ensemble three variants of the same model, trained on the same rows with the same missing feature, and to expect an improvement because "it is an ensemble". It feels right because you did train three models. But their errors share the same direction, so the correlation ρ of the next section is close to one and the average changes nothing. You catch this by checking whether the members disagree on validation rows; if they almost never do, the fix is diversity of data or of targets, not more copies.

How the professor asks it

Q. *Give a counterexample to the claim that averaging many models always reduces error.*
A. Take M copies of the same fitted model, or M models that all lack the same essential feature. Their errors at any input are identical, so the correlation between them is one and the variance of the average equals the variance of a single model: nothing is gained. Averaging removes only the part of the error that differs between models; the shared part, which is bias or missing information, survives untouched. This is the formal content of "shared errors do not cancel", and it is why every ensemble method spends its effort manufacturing disagreement.

5.2 Averaging and Voting

What are we trying to get?

We want the two elementary combination rules, to see when hard and soft voting disagree and what each one throws away, and to *prove* how much averaging reduces variance as a function of the correlation between the component errors.

Definition 5.2.1 (Averaging; hard and soft voting)**.** Let $\widehat{f}_1, \dots, \widehat{f}_M$ be fitted models.

- *Regression (averaging):* $\widehat{f}_{\text{ens}}(x) = \dfrac{1}{M} \displaystyle\sum_{m=1}^{M} \widehat{f}_m(x)$ [on your sheet].
- *Hard voting:* each classifier outputs a class $\hat{y}_m(x)$; the ensemble outputs the most frequent class.
- *Soft voting:* each classifier outputs posterior estimates; the ensemble averages them and takes the largest average, $\widehat{p}_{\text{ens}}(y = c \mid x) = \frac{1}{M} \sum_m \widehat{p}_m(y = c \mid x)$, $\hat{y}(x) = \arg\max_c \widehat{p}_{\text{ens}}(y = c \mid x)$ [on your sheet].

Reading the formula

Hard voting counts heads; soft voting pools confidence. A classifier that says 0.49 and one that says 0.05 cast the *same* hard vote (class 0) but contribute very differently to a soft vote. Soft voting therefore uses strictly more information, namely the *margin* by which each model preferred its class. The price is that this information must be trustworthy: the probabilities should

be reasonably calibrated and computed under the same class prior, otherwise one overconfident model dominates the average.

Example 5.2.2 (Hard and soft votes disagree)**.** Three classifiers evaluate the same flow with threshold 0.5. Model A gives class 1 a probability of 0.49, model B gives 0.48, model C gives 0.95. The hard vote reads the three decisions: A and B say class 0, C says class 1, so the tally is two to one for class 0. The soft vote averages the probabilities, $(0.49 + 0.48 + 0.95)/3 = 0.64$, which is above one half, so it says class 1. Why the disagreement? Because A and B were coin flips and C was nearly certain; the soft vote used that information and the hard vote discarded it. Whether the soft vote is right depends entirely on whether C's 0.95 deserves to be believed.

What people do wrong: soft-voting whatever models you have

The temptation is to soft-vote every model you own. If one of them is overconfident, or was calibrated under a different class prior, it dominates every average, because it always contributes a number near 0 or 1. You catch this by looking at the calibration of each member on validation rows. The fixes are to calibrate, to weight the models, or to fall back to hard voting, which caps each model at one vote and so is robust to overconfidence at the cost of discarding the margins.

How much does averaging help? The variance formula

Averaging M independent errors divides the variance by M. Correlated errors leave a floor.

Fix an input x and let $e_m = \widehat{f}_m(x) - f(x)$ be the error of model m at x, viewed as a random variable over training samples. Suppose every error has the same variance and every pair the same correlation.

Theorem 5.2.3 (Variance of an average of correlated errors)**.** *Let $e_1, \dots, e_M$ have $\mathrm{Var}(e_m) = \sigma^2$ for all m and $\mathrm{Corr}(e_m, e_{m'}) = \rho$ for all $m \neq m'$. Then the error of the average, $\bar{e} = \frac{1}{M}\sum_m e_m$, satisfies*

$$\mathrm{Var}(\bar{e}) = \sigma^2\Big(\rho + \frac{1-\rho}{M}\Big). \quad \textit{[on your sheet]}$$

In particular $\rho = 0$ gives σ^2/M; $\rho = 1$ gives σ^2 (averaging changes nothing); and as $M \to \infty$ the variance tends to $\rho\sigma^2$, not to zero.

Proof. Expand the variance of a sum into variances plus covariances, which is allowed for any random variables:

$$\mathrm{Var}\Big(\sum_{m=1}^{M} e_m\Big) = \sum_{m=1}^{M} \mathrm{Var}(e_m) + \sum_{m \neq m'} \mathrm{Cov}(e_m, e_{m'}).$$

Now count the terms, which is where the structure enters: there are M variance terms worth σ^2 each and $M(M-1)$ ordered pairs worth $\rho\sigma^2$ each, giving $M\sigma^2 + M(M-1)\rho\sigma^2$. Divide by M^2, because $\bar{e}$ is the sum divided by M:

$$\mathrm{Var}(\bar{e}) = \frac{\sigma^2}{M} + \frac{(M-1)\rho\sigma^2}{M} = \sigma^2\Big(\rho + \frac{1}{M} - \frac{\rho}{M}\Big) = \sigma^2\Big(\rho + \frac{1-\rho}{M}\Big).$$

The three special cases follow by substitution and by letting $M \to \infty$. □

Key insight

Only the $M(M-1)$ cross terms carry the correlation. The M diagonal terms shrink like $1/M$ after division by M^2; the cross terms do not, because there are M^2 of them. The part of the error that all models *share* is immune to averaging.

Reading the formula

Read ρ as "how much of the error direction is shared". With $\rho = 0.2$ and $M = 25$ trees the variance is $\sigma^2(0.2 + 0.8/25) = 0.232\sigma^2$, not $\sigma^2/25 = 0.04\sigma^2$. Adding more trees moves $0.032\sigma^2$ towards zero but never touches the floor $0.2\sigma^2$; with $M = 1000$ you are still at $0.2008\sigma^2$. Twenty-five intrusion-detection trees with $\sigma^2 = 4$ and $\rho = 0.2$ give an ensemble variance of $4(0.2 + 0.8/25) = 0.928$; had the errors been independent it would have been 0.16, and had ρ been 0.9 it would have been 3.62, barely better than one tree. A useful ensemble therefore needs *accurate* components (σ^2 small) whose errors are *sufficiently different* (ρ small). Every later idea in this chapter, bootstrap samples, random feature subsets, sequential corrections, is a way of lowering ρ without raising σ^2 too much.

Experiment: variance against the number of models

Setup. A base learner with error variance σ^2 at a fixed input, and models whose errors have pairwise correlation ρ.

What we hold fixed. σ^2 and the base learner.

What we change. Only the number of models, $M = 1, 5, 10, 25, 100, 1000$, and we repeat the sweep for several correlations $\rho = 0, 0.2, 0.5, 0.9$.

What we measure. The variance of the averaged error relative to one model, $\mathrm{Var}(\bar{e})/\sigma^2$.

What the plot shows. Every curve falls steeply at first and then flattens, and each one flattens at its own height, which is exactly ρ. Only the $\rho = 0$ curve goes all the way to zero.

How to read it. Read the height of the plateau as "how much error the models share" and the knee of the curve as "how many models are enough". Beyond the knee, extra models cost computation and change nothing, because they attack the $(1-\rho)/M$ term, which is already small. The lever that moves the plateau is ρ.

Interpretation & discussion

Theorem 5.2.3 is about variance only. Averaging does not remove bias: if every model is missing the same important feature, every error has the same sign and ρ is close to 1. Voting and averaging usually fit every model to the *same* training set, so even different algorithms can all learn the same sample-specific pattern and make correlated errors. The next sections manufacture diversity more deliberately, by changing the *data* each model sees.

How the professor asks it

Q. *Explain why adding more trees to a bagged ensemble stops helping after some point, and what a random forest changes about that.*
A. The claim is that the variance of the average has a floor. By $\sigma^2(\rho + (1-\rho)/M)$, the $(1-\rho)/M$ term vanishes as you add trees, but the $\rho\sigma^2$ term, the error that all the trees share, does not move at all. Bagged trees share a lot of their error because a strong feature sits at the root of nearly every one of them, so their shapes and their mistakes are alike. A random forest offers only m_{try} random features at each node, so the trees are forced to differ: ρ falls, each tree gets slightly weaker, and the floor drops. You therefore add trees only until the error plateaus, and you lower the plateau with m_{try}, not with more trees.

Q. *Twenty-five models have error variance* 4 *and pairwise correlation* 0.2. *What is the variance of their average, and what would it be if the errors were independent?*
A. $4\,(0.2 + 0.8/25) = 4 \times 0.232 = 0.928$. With independent errors the formula gives $\sigma^2/M = 4/25 = 0.16$. The difference, almost six times, is entirely the shared part $\rho\sigma^2 = 0.8$, which no number of models removes; with $\rho = 0.9$ the average would sit at 3.62, barely better than one model.

5.3 Decision Trees as Base Learners

What are we trying to get?

Trees are the base learner of almost every ensemble in practice. We want to define a tree precisely as a partition with one constant per region, prove which constant is optimal in a leaf, state the greedy splitting rule with the Gini impurity, work one split by hand, and understand why trees are at once well suited to ensembles (flexible, fast, scale-free) and in need of them (high variance).

Definition 5.3.1 (Decision tree)**.** A tree with J leaves partitions the feature space into disjoint regions $S_1, \dots, S_J$ and assigns one prediction θ_j to each region:

$$T(x) = \sum_{j=1}^{J} \theta_j \, \mathbf{1}[x \in S_j]. \quad \text{[on your sheet]}$$

Each region is reached by a sequence of binary tests of the form $x_j \le t$ (go left) or $x_j > t$ (go right). The *maximum depth* and the *minimum leaf size* are the usual size controls.

Reading the formula

Exactly one indicator is 1 for any x, so $T(x)$ is simply "the constant of the region x falls in". A tree is a piecewise-constant function whose pieces are axis-aligned boxes. For classification θ_j is a class or a vector of class proportions; for regression it is a number.

Ideally one would choose the regions and their constants jointly, minimising $\sum_i \ell(y^{(i)}, \sum_j \theta_j \mathbf{1}[x^{(i)} \in S_j]) + \lambda J$ with a penalty λJ on the number of leaves. The

search over all partitions is infeasible, and two simplifications make trees practical: for *fixed* regions the best constant in each leaf has a closed form, and the regions are chosen *greedily*, one split at a time.

Proposition 5.3.2 (Optimal leaf constant)**.** *For fixed regions the objective separates by leaf, so* $\theta_j^* = \arg\min_\theta \sum_{i:\, x^{(i)} \in S_j} \ell(y^{(i)}, \theta)$*. For squared-error loss* θ_j^* *is the* mean *of the* $y^{(i)}$ *in leaf* j*; for absolute-error loss it is their* median*; for zero–one loss it is the* majority class*.*

Proof. Each training point lies in exactly one leaf, so the sum over i splits into J sums, the j-th of which involves only θ_j; minimise each separately. For squared error, $\frac{d}{d\theta}\sum_i (y^{(i)} - \theta)^2 = -2\sum_i (y^{(i)} - \theta) = 0$ gives $\theta =$ mean. For absolute error the derivative is $\sum_i \operatorname{sign}(\theta - y^{(i)})$, which changes sign at the median. For zero–one loss the leaf should predict the class that is wrong on the fewest points, the majority. □

Definition 5.3.3 (Gini impurity; greedy split)**.** For classification with class proportions $\widehat{p}_c(S)$ in a node S, the *Gini impurity* is $I(S) = 1 - \sum_c \widehat{p}_c(S)^2$ [on your sheet]; for regression the within-node squared error plays the same role. A candidate split is a feature j and threshold t giving $S_L = \{i \in S : x_j^{(i)} \le t\}$ and $S_R = S \setminus S_L$. The *greedy rule* chooses the split with the largest impurity reduction,

$$(j^*, t^*) = \arg\max_{j,t} \Big[I(S) - \frac{|S_L|}{|S|} I(S_L) - \frac{|S_R|}{|S|} I(S_R) \Big],$$

then repeats inside each child until a stopping rule (depth, leaf size, no useful split) is reached.

Reading the formula

$\sum_c \widehat{p}_c^2$ is the probability that two observations drawn from the node have the same class; 1 minus it is the probability they differ. A pure node has $I = 0$; two balanced classes give $I = 1 - \frac{1}{4} - \frac{1}{4} = 0.5$, the maximum for $C = 2$. The reduction formula compares the parent's impurity with the *size-weighted* average of the children's: a child that is pure but tiny counts little. Greedy means "best immediate reduction"; it does not look ahead and need not find the globally best tree.

Example 5.3.4 (Choosing a split by hand)**.** Eight observations, sorted by the candidate feature $x = 1, \dots, 8$, carry labels $y = (0, 0, 0, 1, 0, 1, 1, 1)$. The parent has four of each class, $I(S) = 0.50$. Try $x \le 3.5$: the left child $\{0, 0, 0\}$ has $I = 0$; the right child $\{1, 0, 1, 1, 1\}$ has $I = 1 - 0.8^2 - 0.2^2 = 0.32$; weighting by size, $\frac{3}{8} \cdot 0 + \frac{5}{8} \cdot 0.32 = 0.20$, a reduction of 0.30. Try $x \le 4.5$: both children, $\{0, 0, 0, 1\}$ and $\{0, 1, 1, 1\}$, have $I = 1 - 0.75^2 - 0.25^2 = 0.375$, so the reduction is only 0.125. The greedy rule prefers 3.5, because it produces a pure leaf now, even though 4.5 splits the data in half.

Why trees in ensembles

Trees capture nonlinear effects and feature interactions (a test on x_1 followed by a test on x_2 *is* an interaction), need no scaling of numerical features (only the

Flexible and cheap, but one early split decides everything below it.

order of values matters), fit complex patterns when grown deep, and are fast enough to fit hundreds of times. Their weakness is the one that averaging cures: a small change in the data can alter an early split, and everything below that split is rebuilt. This is *hierarchical lock-in*, and it makes deep trees high-variance learners.

Experiment: hierarchical lock-in

Setup. One dataset, a deep tree, and (for comparison later) a boosted model of shallow trees.

What we hold fixed. The learning algorithms and their settings.

What we change. The data, slightly: drop or relabel a few rows near the root split, and refit.

What we measure. The new decision boundary compared with the old one.

What the plot shows. The deep tree's root test changes and the whole subtree beneath it is rebuilt, so the boundary jumps. The boosted surface, which we meet in Section 5.7, moves only locally, around the rows you changed.

How to read it. A poor early split in a single tree is never revised, and it is easily amplified by noise. The two cures are to average many trees (bagging, forests) or to build the surface incrementally from many shallow corrections (boosting).

Telling them apart: a tree and a logistic regression as base learner

Compared with logistic regression, a tree needs no feature scaling, captures interactions and nonlinearity automatically, but it is greedy and unstable. Logistic regression is stable and linear and cares about scaling. That is precisely why trees, not linear models, are the base learner of ensembles: they have the variance that averaging removes, and averaging stable linear models would gain little.

What people do wrong: growing the single tree deeper

Students try to fix an error by growing the tree deeper. But if the error was made at a split above, no amount of depth below can undo that early wrong test; depth only refines the sub-region the wrong test created. You catch lock-in by refitting on a bootstrap sample and watching the root test change. The remedy is many shallow corrections or many averaged trees, not a deeper single tree.

How the professor asks it

Q. *Compute the Gini reduction of the splits $x \le 3.5$ and $x \le 4.5$ on the labels $(0, 0, 0, 1, 0, 1, 1, 1)$ and explain which one the greedy rule takes and why that is not necessarily the globally best choice.*

A. The parent is balanced, $I = 0.5$. For $x \le 3.5$ the left leaf is pure ($I = 0$) and the right leaf $\{1, 0, 1, 1, 1\}$ has $I = 0.32$; the weighted child impurity is 0.20 and the reduction 0.30. For $x \le 4.5$ both children have $I = 0.375$ and the reduction is 0.125. The greedy rule takes 3.5 because it maximises the *immediate* reduction. It never asks what the children can achieve afterwards;

a split with a smaller immediate reduction might lead to purer grandchildren. Greedy trees are therefore cheap but not optimal, and that is one reason they benefit from being combined.

5.4 Bagging and Out-of-Bag Evaluation

What are we trying to get?

We want a way to give the *same* learner *different* training sets without throwing data away. Bootstrap sampling does this; we prove that each bootstrap sample omits about 36.8% of the observations, and we turn those omitted observations into a free, honest validation set: the out-of-bag estimate.

Splitting $\mathcal{D}$ into M disjoint parts would give each model only N/M observations, too few unless N is huge. Instead each model is trained on a *perturbed copy* of the whole dataset.

Definition 5.4.1 (Bootstrap sample; bagging)**.** A *bootstrap sample* $\mathcal{D}^*$ of a training set $\mathcal{D}$ with N observations consists of N draws from $\mathcal{D}$ *with replacement*. Let $\text{count}(i)$ be the number of times observation i appears in $\mathcal{D}^*$; observations with $\text{count}(i) = 0$ are *out of bag* for that sample. *Bagging* (bootstrap aggregating) draws M bootstrap samples, fits the same base learner to each, and aggregates: $\widehat{f}_{\text{bag}}(x) = \frac{1}{M}\sum_m \widehat{f}_m(x)$, or the average of the probabilities [on your sheet].

Example 5.4.2 (One bootstrap sample)**.** From $\mathcal{D} = \{1, \dots, 8\}$, eight draws with replacement might give $\{2, 5, 2, 8, 1, 5, 7, 2\}$. Counting each row gives $(1, 3, 0, 0, 2, 0, 1, 1)$: observation 2 is seen three times and 5 twice; 3, 4 and 6 are out of bag. Different count patterns give different empirical distributions and so different fitted models.

Theorem 5.4.3 (Out-of-bag fraction)**.** *The probability that a given observation is absent from a bootstrap sample of size N is*

$$\mathbb{P}(\text{count}(i) = 0) = \left(1 - \frac{1}{N}\right)^N \xrightarrow[N\to\infty]{} e^{-1} \approx 0.368. \quad \textit{[on your sheet]}$$

Hence a large bootstrap sample contains about 63.2% *of the* distinct *training observations, and about* 36.8% *are out of bag for each fitted model.*

Proof. Each draw picks observation i with probability $1/N$, so it misses i with probability $1 - 1/N$. The N draws are independent because we draw with replacement, so the probability of missing i every time is $(1 - 1/N)^N$. The limit is $\lim_{N\to\infty}(1 + x/N)^N = e^x$ with $x = -1$. Already for $N = 8$ the value is $(7/8)^8 = 0.344$; for $N = 100$ it is 0.366. □

Reading the formula

Two thirds in, one third out, for *every* model, and a *different* third each time. The "in" part is what makes the models different; the "out" part is what the next definition exploits.

Definition 5.4.4 (Out-of-bag prediction and error)**.** For training observation i, let $\mathcal{M}_i = \{m : i \notin \mathcal{D}_m^*\}$ be the set of models whose bootstrap sample omits i; by Theorem 5.4.3, $|\mathcal{M}_i| \approx 0.368M$. The *out-of-bag (OOB) prediction* uses only those models,

$$\widehat{f}_{\text{OOB}}(x^{(i)}) = \frac{1}{|\mathcal{M}_i|} \sum_{m \in \mathcal{M}_i} \widehat{f}_m(x^{(i)}), \quad \text{[on your sheet]}$$

and the *OOB error* is the loss of the OOB predictions over all training observations.

Reading the formula

For observation i, the OOB models never saw i, so their prediction at $x^{(i)}$ is a genuine unseen-data prediction, even though i is in the training set. Doing this for every i gives a validation estimate *without* holding anything out: the bootstrap already held out a third of the data for each model. It is a slight underestimate of the full ensemble's quality, since only about a third of the trees vote for each point.

Experiment: OOB error against the number of trees

Setup. Bagged trees on one dataset.

What we hold fixed. The data and the tree settings.

What we change. The number of trees, $M = 10, 50, 100, 200, 500$.

What we measure. The OOB error of the ensemble at each M.

What the plot shows. The curve falls and then plateaus. At the left end it is noisy: with small M some rows have very few OOB trees, and with $M = 10$ a row has no OOB tree at all with probability $0.632^{10} \approx 0.010$.

How to read it. The plateau is where you stop adding trees, because more trees stabilise the ensemble but do not deepen the model. An OOB plateau does not mean "the model cannot improve"; it means that M cannot improve it. The tree size, m_{try} and the features still can.

What people do wrong: reporting the OOB error as test performance

The temptation is to report the OOB error of a tuned forest as its final number. But if you used the OOB error to choose m_{try} or the leaf size, repeated OOB-based choices overfit it exactly as repeated validation overfits a validation set: the reported value is the minimum over several candidates and is optimistic by the winner's curse of Chapter 3. OOB is also computed on rows from the training distribution, so it says nothing about deployment shift. OOB replaces the validation set; it does not replace the untouched test set, on which the chosen forest is scored once.

Telling them apart: OOB, out-of-fold, validation set, test set

Four held-out notions that students conflate. *OOB* is the bootstrap by-product inside a bagged ensemble and exists only there. *Out-of-fold* pre-

dictions come from an explicit K-fold split and work for any learner; stacking (Section 5.12) uses them. A *validation set* is one fixed slice used for every choice. The *test set* is used once, chooses nothing, and is the only one honest for the winner.

Technique: Tune the ensemble, not the trees

Treat the tree-size setting ξ (maximum depth, minimum leaf size) as an *ensemble* hyperparameter: the *same* ξ for every tree. For each candidate ξ, fit a complete bagged ensemble and compute its OOB error; pick the best ξ. Bagging usually uses deep trees, since averaging removes their variance; with noisy or limited data the OOB error may favour larger leaves. Tuning each tree separately to its own best single-tree accuracy optimises the wrong target (a single tree's error rather than the average's) and pushes every tree towards the same strong, correlated fit, which raises ρ and defeats the purpose of bagging.

How the professor asks it

Q. *A student reports the OOB error of a tuned random forest as its final test performance. What is wrong?*
A. The OOB error is a validation score, not a test score. It was used to choose m_{try} and the leaf size, so the reported value is the minimum over several candidates and is optimistic by the winner's curse. It is also computed on rows from the training distribution, so it is blind to any shift at deployment. OOB saves you a validation set for free; it does not replace the untouched test set, on which the chosen forest should be scored once.

Q. *Show that a bootstrap sample of size N leaves out about 37% of the rows, and say why this matters for bagging.*
A. One draw misses a given row with probability $1 - 1/N$; the N draws are independent, so the row is missed every time with probability $(1 - 1/N)^N$, which tends to $e^{-1} \approx 0.368$ and is already 0.344 at $N = 8$. It matters twice. The 63% that is in, with repeats, is what makes the M fitted models different from one another, and the 37% that is out, a different third for each model, gives every row about $0.368M$ models that never saw it, which is the free OOB validation estimate.

5.5 Random Forests

What are we trying to get?

Bagged trees can still be too alike: a very strong feature ends up at the root of most of them and their errors stay correlated. We want the one extra randomisation that fixes this, its main hyperparameter m_{try}, and the resulting trade-off between tree strength and tree correlation in the light of Theorem 5.2.3.

If x_1 is the best split everywhere, every bootstrap tree starts with x_1.

If one feature is very strong, almost every bootstrap sample will choose it near the root, and the trees that result have similar shapes and make similar mistakes. In Theorem 5.2.3 that is a large ρ: more trees stop helping early. The remedy is to *randomise the candidate features* at each split so that different trees are forced

to start differently.

Definition 5.5.1 (Random forest)**.** For each tree $m = 1, \ldots, M$: draw a bootstrap sample; at *each node*, select m_{try} of the d features uniformly at random; choose the best split using only those candidates; grow the tree subject to leaf-size or depth constraints. Aggregate the M trees by averaging probabilities or voting [on your sheet]. A common starting value for classification is $m_{\text{try}} \approx \sqrt{d}$ (regression implementations often use $d/3$); a *new* subset is drawn at every node.

Reading the formula

The feature subset is the only difference from bagging. Setting $m_{\text{try}} = d$ recovers bagged trees. Small m_{try} means that at many nodes the strongest feature is simply not on offer, so an alternative is used, the trees take different shapes, and their errors decorrelate. The strong feature is not lost: it is available at a fraction m_{try}/d of the nodes and will be used deeper down.

Example 5.5.2 (Probability that a feature is offered)**.** With $d = 25$ and $m_{\text{try}} = 5$, a given feature is among the candidates at a node with probability $5/25 = 0.2$ (choosing 5 of 25 without replacement, the chance that a particular one is included is $\binom{24}{4}/\binom{25}{5} = 5/25$). It is *absent* from the root of one tree with probability 0.8, and from the roots of all $M = 10$ trees with probability $0.8^{10} \approx 0.107$. So in a ten-tree forest about 2 roots use it, while a bagged forest might use it at all 10.

Experiment: sweeping m_{try}

Setup. One dataset with d features; a random forest.

What we hold fixed. The data, the number of trees, the leaf size.

What we change. m_{try}, from d (bagging) down to a handful.

What we measure. Two numbers at each value: the error of a single tree and the error of the whole forest.

What the plot shows. As m_{try} shrinks, each tree gets a little weaker (its σ^2 rises) but the trees become much less alike (ρ falls), and the forest error goes down although every tree in it is worse. Push m_{try} too small and the nodes are starved of good candidates, the splits are poor, and the forest error rises again: the forest curve is U-shaped with its minimum near $\sqrt{d}$.

How to read it. The target is the average's error, $\sigma^2(\rho + (1-\rho)/M)$. Feature randomisation raises σ^2 a little and lowers ρ a lot; for moderate M the product improves. This is why a forest of individually *weaker* trees can beat a forest of individually *stronger* ones.

Telling them apart: large and small m_{try}

With large m_{try} the strongest features are reused at most nodes, the individual trees are strong (small σ^2) and similar, so their errors are correlated (large ρ). With small m_{try} alternative features get considered, the individual trees are weaker (larger σ^2) but less correlated (small ρ), and if m_{try} is too small a node may have no good candidate at all. The strong feature is still offered at

a fraction m_{try}/d of the nodes and gets used deeper down. On flow data with $d = 16$ and $m_{\text{try}} = 4$, the destination-port feature that would sit at the root of every bagged tree is offered at only a quarter of the nodes, so the other features get their turn and the trees stop looking alike.

What people do wrong: judging the forest by its trees

The mistake is to look at single-tree accuracy and prefer the setting that makes each tree best. The forest's error is the average's error, and a forest of weaker but less correlated trees beats a forest of strong, similar ones. Judge the forest by the forest's OOB or validation error, never by a tree's.

Technique: Forest-level selection

Tune m_{try}, minimum leaf size and, if useful, maximum depth by OOB performance or a separate validation set; each candidate setting requires fitting a complete forest. Increase the number of trees until the OOB error stabilises; beyond that, additional trees cost computation without changing the model's capacity.

How the professor asks it

Q. *Why can a random forest of individually weaker trees beat bagged trees that are individually stronger?*
A. Because the quantity that matters is the error of the *average*, $\sigma^2(\rho + (1-\rho)/M)$, not the error σ^2 of a tree. Offering only m_{try} random features at each node makes every tree slightly worse, so σ^2 rises a little, but it forces the trees to start from different features, so their errors decorrelate and ρ falls a lot. Since ρ is the floor that no number of trees can lower, lowering ρ is worth more than keeping each tree strong. With $d = 25$ and $m_{\text{try}} = 5$ a dominant feature is absent from all ten roots with probability $0.8^{10} \approx 0.107$, so the forest's roots vary where bagging's would not.

5.6 Feature Importance

What are we trying to get?

A forest of hundreds of trees is not readable. We want two numerical summaries of *which features the fitted forest relies on*: one computed from the training process (mean decrease in impurity) and one computed by breaking a feature and watching the error (permutation importance), with the biases of each and the one interpretation both forbid.

Definition 5.6.1 (Split credit; mean decrease in impurity)**.** For a node t containing N_t of the N training observations, split into children of sizes N_L, N_R, the *credit* of the split is its impurity reduction weighted by the fraction of data it acts on,

$$\Delta_t = \frac{N_t}{N}\Big[I(S_t) - \frac{N_L}{N_t}I(S_L) - \frac{N_R}{N_t}I(S_R)\Big], \quad \text{[on your sheet]}$$

and the credit goes to the feature used at t. The *mean decrease in impurity* of feature j adds all credits of splits on x_j, averages across trees, and is usually normalised so that $\sum_j \mathrm{MDI}_j = 1$.

Reading the formula

The bracket is the impurity reduction of Definition 5.3.3. The factor N_t/N says that a reduction achieved on all the data is worth more than the same reduction on 5% of it. The normalised MDI is a share: "this feature produced 75% of all the impurity the forest removed".

Example 5.6.2 (From one credit to a forest-level MDI)**.** A root node holds 10 flows, 5 attacks and 5 benign, Gini 0.50. The test "destination port = 22?" sends 4 benign flows left (Gini 0) and 5 attacks plus 1 benign right (Gini $1-(5/6)^2-(1/6)^2 = 10/36 = 0.278$). The weighted child impurity is $\frac{4}{10}(0)+\frac{6}{10}(0.278) = 0.167$; since $N_t/N = 1$, the credit is $0.50 - 0.167 = 0.333$, booked to destination port. Now two trees: tree 1 credits port 0.18 and duration 0.06; tree 2 credits port 0.12 and packets 0.04. Averaging over the two trees (a feature absent from a tree contributes 0 there) gives port 0.15, duration 0.03, packets 0.02, which sum to 0.20; dividing gives MDI = 0.75, 0.15, 0.10.

Gotcha

MDI is free (it is computed during training) but biased: a feature with many possible split points (a continuous feature, a high-cardinality categorical) has more chances to find *some* reduction, by signal or by noise, and tends to be over-credited. MDI also measures the *training* fit, not generalisation.

Definition 5.6.3 (Permutation importance)**.** For a fitted forest and a feature j: compute the original OOB (or validation) error; shuffle the values of x_j among the OOB observations, keeping every other column and the labels fixed; predict again and measure the error. The importance is the increase, $I_j = \widehat{\mathrm{Err}}_{\mathrm{shuffled}(j)} - \widehat{\mathrm{Err}}_{\mathrm{original}}$ [on your sheet]. No tree is retrained.

Reading the formula

Shuffling destroys the *information* in x_j (its link with the label and the other features) while preserving its marginal distribution, so the trees still receive plausible values and are not thrown off by out-of-range inputs. If the error barely moves, the forest was not relying on x_j; if it jumps, it was. Because this uses OOB or validation data and the fitted model as is, it measures reliance in *prediction*, not in training. On 100 OOB flows a forest makes 10 errors; shuffling destination port gives 18 (increase 0.08), flow duration 13 (0.03), total packets 11 (0.01). Breaking the port adds eight errors: that is what "important for this fitted forest" means.

What people do wrong: reading importance as causality, and ignoring masking

Two misreadings. The first is importance as causality: a high score means the fitted model relies on the feature, which may be a proxy (destination port stands in for the attacked service), a consequence recorded after the fact (leakage, Chapter 3), or an artefact of collection; intervening on it would change nothing. The second is masking: if two features carry the same information, shuffling one alone hurts little because the trees that use the other still have the information, so both look less important than the information they share. Permute them jointly, or drop a column and refit, to see the shared contribution.

Telling them apart: MDI and permutation importance

MDI is training-time, free, and biased toward features with many split points. Permutation importance is OOB- or validation-time, honest about prediction, and masked by correlated features. When the two rankings disagree, the disagreement itself tells you which bias is at work: suspect a feature with many split points if MDI ranks it higher, and correlated features if permutation ranks them lower.

How the professor asks it

Q. *Compare MDI and permutation importance on a dataset with two near-duplicate features.*
A. MDI splits the credit between the twins according to which one the trees happened to pick at each split, and it favours whichever has more split points, so the ranking between them is partly an accident. Permutation importance shuffles one twin while the other still carries the information, so the error barely moves and both twins look unimportant; this is masking. Neither method says the pair is useless, and neither is causal: a high score only means the fitted model relies on the feature. To see the shared contribution, permute the two jointly, or drop one and refit.

5.7 Boosting for Regression

What are we trying to get?

Bagging fits many models *independently* and averages. Boosting asks a different question: *what should the next model learn from the current mistakes?* We want the additive model this leads to, a proof that for squared error "fit the next tree to the residuals" is exactly minimising the loss, the gradient-boosting algorithm, one iteration by hand, and the three hyperparameters that jointly control overfitting.

Tree 1 gets the trend. Tree 2 fixes what tree 1 got wrong. Tree 3 fixes what is still wrong.

The observations stay fixed. A first learner makes a baseline prediction, right in general but wrong in places. Compute the *leftovers*, the residuals $r^{(i)} = y^{(i)} - \hat{y}^{(i)}$. Train a second learner exclusively to predict those residuals; its job is to correct the first. A third learner is trained on what remains after the first two, and so on. Each added learner changes the ensemble's prediction where the current errors are, and nowhere else.

Definition 5.7.1 (Additive boosted model)**.** A boosted ensemble after m rounds is $F_m(x) = F_{m-1}(x) + \eta\, T_m(x)$ [on your sheet], where T_m is the new tree fitted in round m and $\eta \in (0, 1]$ is the *learning rate* (shrinkage). Unrolled, $F_M(x) = F_0(x) + \eta \sum_{m=1}^{M} T_m(x)$: a constant plus a scaled *sum* of trees.

Reading the formula

Compare with bagging's $\frac{1}{M}\sum_m \widehat{f}_m$: there, every model predicts the *whole* target and the average smooths them; here, every tree predicts a *correction* and the corrections are summed. Averaging reduces variance; summing corrections reduces the remaining loss (mostly bias) one step at a time. The learning rate η scales each step down; $\eta = 1$ applies the full correction, $\eta = 0.1$ one tenth of it.

Theorem 5.7.2 (Squared error: boosting fits residuals)**.** *With squared-error loss* $L = \frac{1}{2}\sum_i \left(y^{(i)} - F_m(x^{(i)})\right)^2$ *and the update* $F_m = F_{m-1} + \eta T_m$*,*

$$L = \frac{1}{2}\sum_{i=1}^{N} \Big(\underbrace{y^{(i)} - F_{m-1}(x^{(i)})}_{r_m^{(i)}} - \eta T_m(x^{(i)}) \Big)^2,$$

so minimising L *over* T_m *is the same as fitting* ηT_m *to the current residuals* $r_m^{(i)}$ *by least squares.*

Proof. Substitute $F_m(x^{(i)}) = F_{m-1}(x^{(i)}) + \eta T_m(x^{(i)})$ into L and group $y^{(i)} - F_{m-1}(x^{(i)})$, which is a fixed number at round m because F_{m-1} is already fitted. The expression is then the least-squares objective of a regression whose *targets* are the residuals and whose *predictor* is ηT_m. The gradient view explains the general recipe: $-\partial[\frac{1}{2}(y-F)^2]/\partial F = y - F = r$, so "fit the residual" and "fit the negative gradient" coincide for squared error; for other losses one fits the negative gradient and then sets the leaf outputs by minimising the true loss. □

Definition 5.7.3 (Gradient boosting for regression)**.** Initialise with the best constant, $F_0(x) = \arg\min_w \sum_i \ell(y^{(i)}, w)$, the mean $\bar{y}$ for squared error. For $m = 1, \ldots, M$: compute the pseudo-residuals $r_m^{(i)} = -\partial \ell(y^{(i)}, F)/\partial F\big|_{F=F_{m-1}(x^{(i)})}$, which for squared error is $y^{(i)} - F_{m-1}(x^{(i)})$; fit a regression tree T_m to the targets $r_m^{(i)}$; set each leaf output to the mean residual in the leaf; update $F_m = F_{m-1} + \eta T_m$.

Example 5.7.4 (One boosting round by hand)**.** Three observations with $y = (10, 20, 30)$ at $x = (1.2, 4.5, 7.1)$ and $\eta = 0.1$. Initialise at the mean, $F_0 = 20$, so the residuals are $(-10, 0, +10)$. A split at $x < 2.85$ puts the first row alone in one leaf and the other two together; the leaf outputs are the mean residuals, -10 and $(0+10)/2 = 5$. Update: $F_1 = (20-1,\ 20+0.5,\ 20+0.5) = (19, 20.5, 20.5)$, and the new residuals are $(-9, -0.5, 9.5)$; the next tree is fitted to those.

Why many shallow trees rather than one deep tree

One deep tree fits the whole rule at once and suffers hierarchical lock-in: early splits force rigid boundaries, deep splits only refine the sub-region they sit in, and a

poor early split cannot be revised. Boosted shallow trees have *global adaptability*: tree 1 isolates the loudest pattern, later trees target the leftovers, and each new tree may adjust the boundary anywhere in feature space, with the shrinkage η enforcing gradual convergence. This is the bias–variance trade-off of Chapter 3 in a new guise: each shallow tree has high bias and low variance, the sum of many of them lowers the bias, and η with early stopping keeps the variance in check. In credit scoring, the first tree learns the coarse rule "high debt-to-income means risky"; its residuals are large for a group it scored too harshly, say young applicants with short credit histories; the second tree is fitted to exactly those leftovers and corrects them, and so on.

Overfitting and the interconnected triad

Training loss keeps falling forever; validation loss turns up. Stop there.

Boosting can continue indefinitely and the training loss keeps decreasing; after some number of rounds the validation loss starts rising because new trees fit noise. Early stopping selects the number of rounds at the validation minimum. Three hyperparameters control capacity *jointly*: the number of rounds M (too small underfits, too large overfits); the learning rate η (smaller steps need more rounds, roughly $\eta \propto 1/M$, and usually generalise slightly better at higher cost); and the tree depth or maximum leaves Q, which limits the order of feature interactions a tree can express (depth 1: additive effects only; depth 2: pairwise interactions), with depth 2 to 6 usual.

Experiment: rounds against loss, for two learning rates

Setup. Gradient-boosted trees, training rows and validation rows.

What we hold fixed. The data, the loss, the depth.

What we change. The number of rounds m, by simply continuing to boost, once with $\eta = 0.3$ and once with $\eta = 0.03$.

What we measure. Training loss and validation loss after each round.

What the plot shows. Both training curves fall forever. Each validation curve bottoms out and turns up; the small-rate curve reaches its minimum later, at roughly ten times the rounds, and usually slightly lower.

How to read it. The number of rounds you keep is the round at the minimum *for that rate*. Halving η roughly doubles the best M; deeper trees reach the minimum in fewer rounds. Compare configurations at their own selected round, never at a common fixed M.

Technique: Do not tune in isolation

The three knobs are coupled. Use a grid or randomised search over (η, Q), with M chosen by early stopping on validation or by cross-validation for each candidate, and compare configurations by their validation loss at the selected round.

What people do wrong: changing the learning rate and keeping the rounds

A colleague switches η from 0.1 to 0.01, keeps $M = 400$, and reports a worse validation loss. The model is now underfitted: with a ten times smaller step, 400 rounds apply about one tenth of the total correction, and the validation

curve over rounds has not reached its minimum. The rounds must be re-selected by early stopping for the new rate, which will land near 4,000; the two rates are then compared at their own selected rounds, and the smaller rate may end slightly lower at ten times the cost.

Telling them apart: bagging and boosting as mechanisms

Bagging averages M full predictions, made in parallel from randomised data, and attacks variance; boosting sums M corrections, made sequentially, each aimed at the current residuals, and attacks mostly bias. One deep tree locks in its early splits; boosted shallow trees can adjust the boundary anywhere at every round. Do not confuse the two meanings of depth either: in one tree, depth is how many successive tests a row passes; in boosting, the depth of each tree is the interaction order it can express, while the capacity of the whole model is carried by the number of rounds.

How the professor asks it

Q. *Derive what the next tree fits in squared-error boosting, and say what replaces it for cross-entropy.*
A. With $F_m = F_{m-1} + \eta T_m$ the loss is $\frac{1}{2}\sum(r_m - \eta T_m)^2$, where $r_m = y - F_{m-1}$ is frozen at round m, so the new tree is a least-squares fit of the current residuals. Those residuals are also the negative gradient $-\partial \ell / \partial F$ of the squared loss, which is why the two descriptions coincide. For cross-entropy the negative gradient is $y - \widehat{p}$: the tree structure is fitted to that, and each leaf output is then set by a Newton step, residual over curvature (Section 5.8). XGBoost goes one step further and grows the tree directly on the Newton target $-g/h$ with weights h.

Q. *Perform one round of gradient boosting on $y = (10, 20, 30)$ with $\eta = 0.1$, a split that isolates the first row, and report the new residuals.*
A. Initialise at the mean, $F_0 = 20$; residuals $(-10, 0, 10)$. The leaf with the first row outputs -10, the leaf with the other two outputs the mean residual 5. Scaled by η, the updates are -1 and $+0.5$, so $F_1 = (19, 20.5, 20.5)$ and the new residuals are $(-9, -0.5, 9.5)$. Each row moved one tenth of the way toward its target; the next tree sees a plain regression problem whose targets are these leftovers.

5.8 Boosting for Binary Classification

What are we trying to get?

Probabilities live in $(0, 1)$ and cannot be summed with tree outputs. We want the fix (boost the *logit*), the gradient and Hessian of the cross-entropy loss with respect to the logit (proved), why the leaf output no longer has a closed form, and the second-order (Newton) shortcut that restores one, first without and then with regularisation.

Definition 5.8.1 (Boosting the logit; binary cross-entropy)**.** The ensemble produces an unconstrained score $F^{(i)}$ for observation i and maps it to

a probability through the sigmoid, $\widehat{p}^{(i)} = \sigma(F^{(i)}) = 1/(1+e^{-F^{(i)}})$. The loss is the binary cross-entropy written as a function of the logit, $\ell^{(i)} = -\big[y^{(i)} \ln \widehat{p}^{(i)} + (1-y^{(i)}) \ln(1-\widehat{p}^{(i)})\big]$ [on your sheet]. Trees are added to F: $F_m = F_{m-1} + \eta T_m$, and $\widehat{p}$ is recomputed from F_m.

Reading the formula

Adding a tree output to a probability could give 1.3. Adding it to a logit is always legal, and the sigmoid brings the result back inside $(0,1)$. This is exactly the discriminative route of Chapter 4, with the score now a sum of trees instead of $w^\top x + b$.

Theorem 5.8.2 (Gradient and Hessian of the cross-entropy in the logit)**.** *With $\widehat{p} = \sigma(F)$,*

$$g = \frac{\partial \ell}{\partial F} = \widehat{p} - y, \qquad h = \frac{\partial^2 \ell}{\partial F^2} = \widehat{p}(1-\widehat{p}). \quad \textit{[on your sheet]}$$

Proof. First the sigmoid's derivative: writing $\sigma = (1+e^{-F})^{-1}$, $\sigma' = e^{-F}(1+e^{-F})^{-2} = \sigma \cdot \frac{e^{-F}}{1+e^{-F}} = \sigma(1-\sigma)$. Then by the chain rule

$$\frac{\partial \ell}{\partial F} = \Big(-\frac{y}{\widehat{p}} + \frac{1-y}{1-\widehat{p}}\Big)\widehat{p}(1-\widehat{p}) = -y(1-\widehat{p}) + (1-y)\widehat{p} = -y + y\widehat{p} + \widehat{p} - y\widehat{p} = \widehat{p} - y.$$

Differentiating once more, $\partial(\widehat{p}-y)/\partial F = d\widehat{p}/dF = \widehat{p}(1-\widehat{p})$. □

Reading the formula

The *negative* gradient $-g = y - \widehat{p}$ is the classification analogue of the residual: how far the current probability is from the label. The Hessian $h = \widehat{p}(1-\widehat{p})$ is the variance of a Bernoulli with parameter $\widehat{p}$: it peaks at 0.25 when $\widehat{p} = 0.5$ (the model is unsure, the loss curves most) and vanishes when $\widehat{p} \to 0$ or 1 (confident, loss flat in F). For $y = 1$ and a very negative F (confidently wrong), $-g \to 1$ is at its largest while $h \to 0$: a large residual with almost no curvature. Keep this picture; it explains both the Newton step and why it needs regularisation.

The line-search bottleneck and the Newton step

At round m the pseudo-residuals are $r_m^{(i)} = y^{(i)} - \widehat{p}_{m-1}^{(i)}$; a regression tree is grown on them by squared-error splits (*split search determines the regions*); then, with the structure fixed, each leaf output should be chosen by a *line search* on the true loss, $w_{q,m} = \arg\min_w \sum_{i \in S_{q,m}} \ell(y^{(i)}, F_{m-1}^{(i)} + w)$. For squared error this had the closed form "mean residual". For log loss it has none: the sum of log-sigmoids must be minimised numerically, leaf by leaf, with repeated loss evaluations. The remedy is to replace the loss by a local quadratic.

Theorem 5.8.3 (Newton leaf value)**.** *Expand the loss of observation i to second order around the current logit, $\ell^{(i)}(F_{m-1}^{(i)} + w) \approx \ell^{(i)}(F_{m-1}^{(i)}) + g^{(i)} w +$*

$\frac{1}{2}h^{(i)}w^2$. *Minimising the sum of these approximations over a leaf gives*

$$w_{q,m} = \frac{\sum_{i\in S_{q,m}}(-g^{(i)})}{\sum_{i\in S_{q,m}} h^{(i)}} = \frac{\sum_i (y^{(i)} - \widehat{p}^{(i)}_{m-1})}{\sum_i \widehat{p}^{(i)}_{m-1}(1-\widehat{p}^{(i)}_{m-1})},$$

and adding an L_2 penalty $\frac{\lambda}{2}w^2$ to the leaf objective gives the regularised *value* $w_{q,m} = \sum_i(-g^{(i)})/(\sum_i h^{(i)} + \lambda)$ *[not on your sheet: know it].*

Proof. The sum over the leaf, $\sum_i \left[\ell^{(i)} + g^{(i)}w + \frac{1}{2}h^{(i)}w^2\right] + \frac{\lambda}{2}w^2$, is a quadratic in the single number w with positive leading coefficient $\frac{1}{2}(\sum_i h^{(i)} + \lambda)$, since $h^{(i)} > 0$; the expansion is legitimate for a small step. Its derivative $\sum_i g^{(i)} + (\sum_i h^{(i)} + \lambda)w$ vanishes at $w = -\sum_i g^{(i)}/(\sum_i h^{(i)} + \lambda)$. Setting $\lambda = 0$ gives the unregularised step; substituting Theorem 5.8.2 gives the explicit form. □

Reading the formula

Numerator: the total residual in the leaf, how far and in which direction the probabilities are off. Denominator: the total curvature, how fast the loss bends. A Newton step is "residual divided by curvature": in a leaf full of unsure observations ($h \approx 0.25$ each) the step is moderate; in a leaf of confidently-wrong observations ($h \approx 0$) the denominator collapses and the step *explodes*. The λ in the denominator is a floor that caps the step; it matters most exactly where $\sum h$ is smallest.

Example 5.8.4 (Newton leaf value by hand)**.** A leaf holds $(y, \widehat{p}) = (1, 0.2), (1, 0.4), (0, 0.8)$. Gradients $g = \widehat{p} - y$: $-0.8, -0.6, +0.8$, so $G = -0.6$. Hessians $h = \widehat{p}(1-\widehat{p})$: $0.16, 0.24, 0.16$, so $H = 0.56$. Unregularised leaf value $-G/H = 0.6/0.56 = 1.071$; with $\lambda = 1$, $0.6/1.56 = 0.385$. With $\eta = 0.1$ the logit actually moves by 0.0385 (regularised). The positive sign raises all three probabilities, helping the two positives more than it hurts the negative.

What people do wrong: the exploding step

Picture a leaf full of confidently-wrong rows: $y = 1$ with a very negative F. Then $-g \to 1$, the largest possible residual, while $h \to 0$, almost no curvature. The Newton step G/H divides a large number by a tiny one and explodes, which is extreme overfitting on exactly the rows you can trust least. The λ in the denominator caps the step, and a minimum child weight (Section 5.9) refuses such leaves altogether.

Telling them apart: regression and classification boosting

In regression boosting the residual is $y - F$ and the leaf output is the mean residual, a closed form. In classification the residual is $y - \widehat{p}$ and the leaf output is a Newton step, residual divided by curvature. Same skeleton, different leaf arithmetic. And a sign convention to watch: the slides use both $r = y - \widehat{p}$ (the negative gradient) and $g = \widehat{p} - y$ (the gradient); they are consistent, $r = -g$, so always ask "gradient or negative gradient?" before reading a formula.

How the professor asks it

Q. *Why does gradient boosting for classification need a second-order (Newton) step, and why must it be regularised?*
A. Because you cannot add tree outputs to probabilities, boosting works on the logit, and there the leaf output that minimises a sum of log-sigmoids has no closed form, which would force a numerical line search in every leaf. A second-order expansion of the loss turns each leaf into a quadratic and gives the closed form $w = \sum(-g)/\sum h$ with $g = \widehat{p} - y$ and $h = \widehat{p}(1-\widehat{p})$. The danger appears in a leaf of confidently-wrong rows: there $-g \to 1$ while $h \to 0$, so the step divides a large residual by almost no curvature and explodes. Putting λ in the denominator caps the step, and a minimum child weight refuses such leaves outright.

Q. *Compute the Newton leaf value for the rows* $(y, \widehat{p}) = (1, 0.2), (1, 0.4), (0, 0.8)$ *with and without* $\lambda = 1$.
A. $G = \sum(\widehat{p} - y) = -0.8 - 0.6 + 0.8 = -0.6$ and $H = \sum \widehat{p}(1-\widehat{p}) = 0.16 + 0.24 + 0.16 = 0.56$. Unregularised, $w = -G/H = 1.071$; with $\lambda = 1$, $w = 0.6/1.56 = 0.385$; after the learning rate $\eta = 0.1$ the logit moves by 0.0385. The step is positive because the two positives dominate the residual, and the penalty roughly cut it by two thirds because H is smaller than λ.

5.9 XGBoost: One Objective for Structure and Leaf Values

What are we trying to get?

We want the single regularised second-order objective that XGBoost uses to evaluate a *whole tree*, to show (by completing the square) that it is a weighted least-squares problem with a new target and a new weight, to derive the optimal leaf value and the *gain* of a split, and to see precisely what the two penalties λ and γ do.

Definition 5.9.1 (XGBoost tree objective)**.** A proposed tree with leaves $S_{1,m}, \dots, S_{Q_m,m}$ and outputs $w_{q,m}$ is scored by

$$\mathcal{L}_m = \sum_{q=1}^{Q_m} \sum_{i:\, x^{(i)} \in S_{q,m}} \left[g^{(i)} w_{q,m} + \tfrac{1}{2} h^{(i)} w_{q,m}^2 \right] + \gamma Q_m + \frac{\lambda}{2} \sum_{q=1}^{Q_m} w_{q,m}^2, \qquad \text{[on your sheet]}$$

with $g^{(i)}, h^{(i)}$ the first and second derivatives of the loss at the current prediction $F_{m-1}(x^{(i)})$.

Reading the formula

Three parts. The double sum is the second-order Taylor approximation of the total loss change produced by the tree (the constant $\sum_i \ell^{(i)}$ is dropped since it does not depend on the tree). γQ_m charges γ per leaf. $\frac{\lambda}{2}\sum_q w_q^2$ charges for large outputs. The *same* number scores the structure (through which observations share a leaf) and the outputs; XGBoost therefore uses gradients, Hessians and regularisation *while growing* the tree, not after, whereas the

GBM variants of the previous section grow the structure by plain squared-error splits on the raw residual and are blind to curvature.

Theorem 5.9.2 (The objective is a weighted regression)**.** *For observation i in leaf q,*

$$g^{(i)}w_q + \tfrac{1}{2}h^{(i)}w_q^2 = \tfrac{1}{2}h^{(i)}\big(r_{\mathrm{XGB}}^{(i)} - w_q\big)^2 + \mathit{const},$$
$$r_{\mathrm{XGB}}^{(i)} = -\frac{g^{(i)}}{h^{(i)}} = \frac{y^{(i)} - \widehat{p}_{m-1}^{(i)}}{\widehat{p}_{m-1}^{(i)}\big(1 - \widehat{p}_{m-1}^{(i)}\big)}.$$

so $\mathcal{L}_m$ is a weighted least-squares objective with target r_{XGB} and weight h, plus the two penalties.

Proof. Complete the square: $\frac{1}{2}h(w+g/h)^2 = \frac{1}{2}hw^2+gw+g^2/(2h)$, so $gw+\frac{1}{2}hw^2 = \frac{1}{2}h(w-(-g/h))^2 - g^2/(2h)$, and $-g^2/(2h)$ does not involve w. Substituting $g = \widehat{p} - y$ and $h = \widehat{p}(1-\widehat{p})$ gives the explicit target. □

Reading the formula

Each observation asks the tree to output $r_{\mathrm{XGB}} = -g/h$, the one-step Newton correction *for that observation alone*, and its request is weighted by h. An unsure observation (h large) has a modest target and a loud voice; a confidently-wrong one has a huge target ($\approx 1/\widehat{p}$ when $y = 1$, $\widehat{p} \to 0$) but a whisper of a weight. A row with $y = 1$ and $\widehat{p} = 0.01$ asks to move the logit by $0.99/0.0099 = 100$, but with weight 0.0099, so it pulls with a force $h\,r_{\mathrm{XGB}} = 0.99$; a row with $y = 1$ and $\widehat{p} = 0.5$ asks for $0.5/0.25 = 2$ with weight 0.25, force 0.5. Target and weight must always be read together: their product, $h \cdot r_{\mathrm{XGB}} = -g = y - \widehat{p}$, is bounded by 1, which is why the leaf value stays finite. The tree is grown by *weighted* squared-error splits on these targets, which is what "XGBoost uses the Hessian in the split search" means.

Theorem 5.9.3 (Optimal leaf value and split gain)**.** *For a leaf q let $G_q = \sum_{i\in S_q} g^{(i)}$ and $H_q = \sum_{i\in S_q} h^{(i)}$. The leaf's contribution to $\mathcal{L}_m$ is $\phi_q(w) = G_q w + \frac{1}{2}(H_q+\lambda)w^2$, minimised at*

$$w_q^* = -\frac{G_q}{H_q+\lambda}, \qquad \phi_q(w_q^*) = -\frac{1}{2}\,\frac{G_q^2}{H_q+\lambda}. \quad \textit{[on your sheet]}$$

Splitting a parent with (G,H) into children (G_L,H_L) and (G_R,H_R) changes the objective by $-$gain, with

$$\mathrm{gain} = \frac{1}{2}\Big[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{G^2}{H+\lambda}\Big] - \gamma, \quad \textit{[on your sheet]}$$

and the split is accepted only when gain > 0.

Proof. Summing the per-observation terms over a leaf gives $\sum_i g^{(i)}w + \frac{1}{2}\sum_i h^{(i)}w^2 + \frac{\lambda}{2}w^2 = G_q w + \frac{1}{2}(H_q+\lambda)w^2$. Its derivative $G_q + (H_q+\lambda)w$ vanishes at w_q^*; substituting, $\phi_q(w_q^*) = -G_q^2/(H_q+\lambda) + \frac{1}{2}G_q^2/(H_q+\lambda) = -\frac{1}{2}G_q^2/(H_q+\lambda)$. Replacing one leaf by two changes the data-fit part from $\phi_{\mathrm{parent}}(w^*)$ to

$\phi_L(w_L^*) + \phi_R(w_R^*)$, a decrease of $\frac{1}{2}[G_L^2/(H_L+\lambda) + G_R^2/(H_R+\lambda) - G^2/(H+\lambda)]$, and raises the leaf count by one, so the penalty rises by γ. □

Reading the formula

$-\frac{1}{2}G^2/(H+\lambda)$ is "how much loss this leaf can remove with its best output"; more negative is better. A split is worth taking when the two children can jointly remove more than the parent could, by more than the price γ of the extra leaf. The two penalties act differently: λ sits inside every denominator and *shrinks* all leaf outputs, most strongly where H is small; γ is a flat per-split threshold that *discourages* splits whose improvement is marginal. In one line: λ shrinks leaf outputs; γ discourages unnecessary splits.

Example 5.9.4 (Does the split pay for its leaf?)**.** Parent $(G, H) = (-4, 3)$, children $(G_L, H_L) = (-3, 1)$ and $(G_R, H_R) = (-1, 2)$, $\lambda = 1$, $\gamma = 0.2$. Parent: $w^* = 4/4 = 1$, $\phi = -\frac{1}{2} \cdot 16/4 = -2$. Left: $w^* = 3/2$, $\phi = -\frac{1}{2} \cdot 9/2 = -2.25$. Right: $w^* = 1/3$, $\phi = -\frac{1}{2} \cdot 1/3 = -0.167$. Children together -2.417; the data fit improves by 0.417; gain $= 0.417 - 0.2 = 0.217 > 0$, accept. The split would be rejected for any $\gamma \geq 5/12 \approx 0.417$.

Experiment: the fingerprints of λ and γ

Setup. XGBoost on one dataset.

What we hold fixed. The depth, the learning rate, the number of rounds.

What we change. First λ alone over a grid, then γ alone over a grid.

What we measure. The distribution of leaf values and the number of leaves per tree, together with training and validation loss.

What the plot shows. As λ grows the leaf values shrink and the extreme outputs disappear, most visibly in leaves with small H; the number of leaves barely changes. As γ grows each tree has fewer leaves, because marginal splits no longer pay for themselves; the surviving leaf values are not shrunk.

How to read it. Each penalty leaves its own fingerprint: λ is continuous shrinkage inside every denominator, γ is a discrete threshold on the gain. If you see small trees, suspect γ; if you see timid leaf values, suspect λ.

Telling them apart: the four leaf-value mechanisms

Basic boosting grows the structure on the raw error $y - \hat{p}$ and finds outputs by an exact line search. Unregularised GBM: same structure, Newton outputs $\sum(-g)/\sum h$. Regularised GBM: same structure, $\sum(-g)/(\sum h + \lambda)$. XGBoost grows the structure on the Newton target $-g/h$ weighted by h, uses the same regularised leaf found in the same weighted pass that chose the splits, and includes γ in the gain. Unifying structure selection and weight optimisation into a single pass over one objective is both faster and better regularised.

What people do wrong: reading the target without its weight

Students see the target for a confidently-wrong row, $(y - \hat{p})/(\hat{p}(1-\hat{p})) \approx 1/\hat{p}$ when $y = 1$ and $\hat{p} \to 0$, and conclude that the step will be huge. Read the target together with its weight: the weight h is tiny, and the product

$h\, r_{\mathrm{XGB}} = y - \widehat{p}$ is bounded by one. The confident row shouts a huge number in a whisper; the unsure row asks for a moderate correction in a clear voice. That is why the leaf values stay finite.

Beyond λ and γ, XGBoost offers *row subsampling* (a random fraction of observations per tree), *feature subsampling* (the random-forest idea inside boosting), *minimum child weight* (reject a split if either child's H is too small, which directly prevents the exploding step), and *tree-size limits*. Its speed comes from engineering: histogram-based split search, sparse and missing-value handling, parallel split evaluation across features, and GPU support. Boosting rounds themselves remain sequential.

How the professor asks it

Q. *Decide whether the split with parent* $(G, H) = (-4, 3)$ *and children* $(-3, 1)$, $(-1, 2)$ *is accepted at* $\lambda = 1$, $\gamma = 0.2$, *and find the* γ *at which it would be rejected.*
A. The parent's best contribution is $-\frac{1}{2} \cdot 16/4 = -2$; the children give $-\frac{1}{2} \cdot 9/2 = -2.25$ and $-\frac{1}{2} \cdot 1/3 = -0.167$, together -2.417. The data fit improves by 0.417, the extra leaf costs $\gamma = 0.2$, so the gain is $0.217 > 0$ and the split is accepted. The gain is $0.417 - \gamma$, so the split is rejected exactly when $\gamma \geq 5/12 \approx 0.417$. Note how λ entered: it sat in every denominator and shrank every term, most strongly for the left child, whose $H_L = 1$ is smallest.

Q. *In one sentence each: what does* λ *do, what does* γ *do, and why are they not interchangeable?*
A. λ sits inside every denominator and shrinks every leaf output continuously, most where the curvature H is small. γ is a flat price per leaf that rejects any split whose improvement is below it. They are not interchangeable because one acts on the *values* of the leaves and the other on the *number* of leaves: a large λ gives a big tree with timid outputs, a large γ gives a small tree with full-size outputs.

5.10 Multi-Class Boosting and Other Libraries

For K classes the ensemble maintains a vector of logits $F_1, \ldots, F_K$ and maps it to probabilities with the softmax, $\widehat{p}_k = e^{F_k} / \sum_j e^{F_j}$; the loss is the multi-class cross-entropy with one-hot label y_k, $\ell = -\sum_k y_k \ln \widehat{p}_k$. Instead of one tree per round, the algorithm builds K trees per round, one per logit.

Proposition 5.10.1 (Class-specific derivatives)**.** *For one observation,* $g_k = \partial \ell / \partial F_k = \widehat{p}_k - y_k$, *and* $\partial^2 \ell / \partial F_k \partial F_r$ *equals* $\widehat{p}_k(1 - \widehat{p}_k)$ *for* $k = r$ *and* $-\widehat{p}_k \widehat{p}_r$ *for* $k \neq r$ *[on your sheet]. Using only the diagonal,* $h_k = \widehat{p}_k(1 - \widehat{p}_k)$, *the XGBoost target for class* k *is* $r_{\mathrm{XGB},k} = (y_k - \widehat{p}_k)/(\widehat{p}_k(1 - \widehat{p}_k))$, *formally identical to the binary case.*

Proof. Write $\ell = -\sum_k y_k F_k + \ln \sum_j e^{F_j}$ (since $\ln \widehat{p}_k = F_k - \ln \sum_j e^{F_j}$ and $\sum_k y_k = 1$). Then $\partial \ell / \partial F_k = -y_k + e^{F_k} / \sum_j e^{F_j} = \widehat{p}_k - y_k$. Differentiating $\widehat{p}_k = e^{F_k} / \sum_j e^{F_j}$ with respect to F_r by the quotient rule gives $\widehat{p}_k(\mathbf{1}[k = r] - \widehat{p}_r)$, which is $\widehat{p}_k(1 - \widehat{p}_k)$ on the diagonal and $-\widehat{p}_k \widehat{p}_r$ off it. □

Reading the formula

The softmax *couples* the classes: raising F_1 lowers every other $\widehat{p}_k$, which is what the off-diagonal terms record. Keeping the full Hessian would make the K leaf values of one round depend on each other. Dropping the off-diagonal terms makes the approximate objective a *sum* of K independent binary-style objectives, so one tree can be fitted per class, and the K fits can run in parallel; the coupling is restored at the end of the round when the softmax is recomputed jointly from all updated logits. Rounds stay sequential, and multi-class boosting does K times the work per round.

Gotcha

AdaBoost is a different boosting *algorithm*: it increases the weights of misclassified observations so that the next weak classifier focuses on them, and combines the classifiers by a weighted vote. XGBoost, LightGBM (histogram-based splits, leaf-wise growth for large datasets) and CatBoost (ordered treatment of categorical features to limit target leakage) are *libraries* implementing gradient-boosted trees with different engineering choices. Do not list them as four algorithms of the same kind; and do not call the K trees of multi-class boosting "independent" across rounds, when they are independent only within a round and only under the diagonal approximation.

How the professor asks it

Q. *Why does multi-class boosting fit K trees per round, and in what sense are they only approximately independent?*
A. Each class has its own logit and therefore its own gradient component, so the diagonal-Hessian approximation gives one binary-style weighted regression problem per class. Those K fits can run in parallel within the round. They are not truly independent: the softmax couples every class through off-diagonal Hessian terms, which the approximation drops, and after the updates the probabilities are recomputed jointly before the next sequential round.

5.11 Bagging versus Boosting, and When Ensembles Do Not Help

Telling them apart: bagging and boosting, side by side

Question	Bagging / random forest	Boosting
training	parallel randomised fits	sequential corrections
main	average unstable predictions	reduce the current loss
mechanism	(variance)	(mostly bias)
typical trees	deep or moderately deep	shallow
diversity	bootstrap rows, feature	changing residuals or
source	subsets	gradients
main controls	m_{try}, leaf size, M to the plateau	η, depth, rounds by early stopping; λ, γ
noise sensitivity	generally robust	can focus on difficult or noisy cases
parallel work	natural across trees	limited by sequential dependence

These are tendencies: either method can affect both bias and variance.

Technique: Start from the source of error

Is one strong learner *unstable* across training samples? Try bagging or a random forest. Can many small corrections improve a *structured residual pattern*? Try gradient-boosted trees. Do credible but different model families make *complementary* errors? Try validated voting or averaging. Is the dominant error caused by *missing information* or *deployment shift*? An ensemble alone is unlikely to solve it.

Experiment: bagging and boosting on flipped labels

Setup. A forest and a boosted model on the same data.

What we hold fixed. Everything except the labels.

What we change. Flip 10% of the training labels at random, refit both, and continue adding trees or rounds.

What we measure. Validation loss against the number of trees (forest) or rounds (boosting).

What the plot shows. The forest degrades gently, because averaging dilutes the flipped rows. The boosted model's validation loss turns up early, because later trees are fitting the flipped rows as if they were signal.

How to read it. Boosting treats every residual as signal, so noisy or mislabelled rows attract its later trees; early stopping and a smaller η limit the damage, and the forest's robustness is the reason it remains the default baseline on messy data.

Gotcha

Ensembles do not help when all models are missing the same important information ($\rho \approx 1$); when the models are too similar and make the same mistakes; when boosting starts fitting noise, outliers or incorrect labels; when validation data are reused so often that the choices overfit them; and when test or deployment data differ substantially from the training data. Additional models also cost fit time, prediction time and memory, which Lab 3 measures explicitly.

How the professor asks it

Q. *When should you try bagging, and when should you try boosting?*
A. Bagging is the first choice when a strong learner is unstable across training samples: independent perturbations plus averaging reduce variance. Boosting is appropriate when shallow learners leave a structured residual pattern that sequential corrections can remove. Neither repairs information missing from every model or a deployment shift, and boosting needs validation-based early stopping when difficult rows may be noise.

5.12 Stacking

What are we trying to get?

Voting fixes the combination rule in advance. We want to *learn* it: a meta-model that takes the base predictions as inputs, and the one procedural rule that keeps this honest, out-of-fold predictions.

Definition 5.12.1 (Stacking)**.** Base models $m = 1, \dots, M$ (e.g. logistic regression, a random forest, gradient boosting) produce predictions z_m, usually class probabilities. A *meta-model* h_γ maps them to the final prediction. For binary classification a common meta-model is logistic regression, $\widehat{p}_{\text{stack}} = \sigma\big(\beta_0 + \sum_m \beta_m z_m\big)$ [on your sheet], fitted by cross-entropy on the true labels and on *out-of-fold* base predictions.

Reading the formula

Soft voting is the special case $\beta_m = 1/M$ (up to the sigmoid). Stacking lets the data decide the weights: a large positive β_m gives model m more influence; a small or negative one down-weights a redundant model. The meta-model is kept simple, a regularised logistic regression on only M inputs, because it must not overfit.

Technique: Out-of-fold predictions

Training the meta-model. Split the training set into K folds. For each fold, fit every base model on the other folds and predict the held-out fold. Join the held-out predictions from all folds: every training observation now has a prediction from a model that never saw it. Fit h_γ on these. *Predicting a new observation.* Refit each base model on *all* the training data, compute $(z_1, \dots, z_M)$ for x_{new}, pass them through the fitted meta-model.

Experiment: in-sample versus out-of-fold base predictions

Setup. Three base models (a logistic regression, a deep forest, a boosted model) and a logistic meta-model.

What we hold fixed. The base models, the data, the validation rows.

What we change. How the meta-model's training inputs are produced: once from in-sample base predictions, once from out-of-fold predictions.

What we measure. The meta-model's weights and its validation loss in both arrangements.

What the plot shows. In-sample, the deep forest predicts its own training rows almost perfectly, so the meta-model gives it nearly all the weight and its own training fit looks unrealistically good, while the validation loss is poor. Out-of-fold, the weights reflect genuine complementarity between the base models and the validation loss is the better of the two.

How to read it. Training the meta-model on in-sample predictions leaks training information into the combination rule: the meta-model learns to trust whichever base model overfits most. Out-of-fold predictions are the cure.

Gotcha

Out-of-fold predictions are not the same as OOB predictions: OOB uses the bootstrap by-product inside a bagged ensemble; out-of-fold uses an explicit fold split and works for any base model. And keep the three combination rules apart: voting fixes the weights in advance, stacking learns one global set of weights, a mixture of experts (next) learns weights that depend on the input.

How the professor asks it

Q. *Why must a stacking meta-model be trained on out-of-fold predictions, and how is that different from OOB?*
A. If you fit the meta-model on in-sample predictions, a deep forest predicts its own training rows almost perfectly, so the meta-model learns to trust the most overfitted base model and its own fit becomes unrealistically good: training information has leaked into the combination rule. Out-of-fold predictions avoid this because they come from base models that never saw the row, through an explicit K-fold split that works for any learner. OOB is a different thing: it is the bootstrap by-product inside a bagged ensemble, and it exists only there.

5.13 Mixture of Experts

Definition 5.13.1 (Mixture of experts)**.** Experts $p_m(y \mid x)$, $m = 1, \dots, M$, are combined by a *gating model* $g_m(x) \geq 0$ with $\sum_m g_m(x) = 1$: $p(y \mid x) = \sum_m g_m(x)\, p_m(y \mid x)$ [on your sheet]. A linear gate scores expert m by $a_m(x) = \beta_m^\top x + b_m$ (a small network for nonlinear routing) and normalises with a softmax. *Soft routing* uses all experts with their weights; *top-k routing* evaluates only the k experts with the largest scores.

Reading the formula

Compare with stacking's $\hat{y} = h(z_1, \dots, z_M)$: there the weights β_m are the same for every x; here $g_m(x)$ changes with the input, so one expert can own one region of feature space and another a different one. Top-k routing is also a computational device: with M large experts and $k = 2$, only two are evaluated per input (*conditional computation*), which is how very large

language and vision models use mixtures of experts.

Technique: Joint training

Minimise the negative log-likelihood of the mixture, $-\sum_i \log\left[\sum_m g_m(x^{(i)})\, p_m(y^{(i)} \mid x^{(i)})\right]$, jointly over the gate parameters and the expert parameters. Experts improve on the observations that receive high gate weight; the gate learns which expert does well where; repeated joint updates make the experts specialise. The routing regions are not specified in advance.

Experiment: specialisation and expert collapse

Setup. A mixture of experts trained jointly.

What we hold fixed. The experts' architecture and the data.

What we change. Training time, with and without a load-balancing penalty on the gate.

What we measure. Each expert's share of the total gate weight over training.

What the plot shows. With the penalty, the shares separate by region and stay comparable: that is specialisation. Without it, an expert that received more weight early learns faster, becomes better, attracts still more inputs, and ends up handling almost everything while the others sit idle.

How to read it. That runaway is *expert collapse*; load-balancing penalties that encourage the gate to spread its weight are used to counter it.

How the professor asks it

Q. *Distinguish voting, stacking and a mixture of experts by how their weights are obtained.*
A. Voting fixes the combination rule in advance, usually equal weights. Stacking learns one global combination from out-of-fold base predictions, so its weights are the same for every input. A mixture of experts learns a gate $g_m(x)$ jointly with the experts, so the weights depend on the input and can route different regions to different specialists; load balancing is needed to prevent one expert taking everything.

What people do wrong: the five that cost the most points

Expecting M to drive the variance to zero, when the floor is $\rho\sigma^2$ and averaging never removes bias or shared missing information. Soft-voting uncalibrated models, tuning each tree instead of the ensemble, and reading the OOB error as a test score. Confusing the gradient $g = \widehat{p} - y$ with the residual $y - \widehat{p}$, and forgetting that classification boosting acts on the logit. Tuning η, depth and rounds separately, not early-stopping on validation, and letting boosting chase noisy labels. Reading importance as causality, ignoring masking by correlated features, fitting the meta-model on in-sample predictions, and listing AdaBoost as a library.

Chapter takeaways

Summary: Learning by combining models

- **Why:** one flexible fit has high variance; averaging competent models with *different* errors cancels part of it. Var $= \sigma^2(\rho + (1-\rho)/M)$: correlation leaves a floor that no number of models removes, and averaging never removes bias.
- **Voting:** hard counts classes, soft averages posteriors and needs calibration.
- **Trees:** partitions with one constant per leaf (mean, median, majority); greedy Gini splits; flexible but high-variance because of hierarchical lock-in.
- **Bagging:** bootstrap samples (63.2% in, 36.8% out); OOB error is free internal validation, not a test set; tune the tree size at ensemble level.
- **Random forests:** m_{try} random candidates per split decorrelate trees; weaker trees, better forest; add trees until the OOB plateau.
- **Importance:** MDI (training, fast, biased to many split points) and permutation (prediction, honest, masked by correlation); never causality.
- **Boosting:** $F_m = F_{m-1} + \eta T_m$; fit the negative gradient; squared error gives residuals; M, η, depth tuned jointly with early stopping; chases noisy labels.
- **Classification:** boost the logit; $g = \widehat{p} - y$, $h = \widehat{p}(1-\widehat{p})$; Newton leaf $\sum(-g)/(\sum h + \lambda)$; the step explodes where $h \to 0$.
- **XGBoost:** one regularised second-order objective for splits and outputs; target $-g/h$ with weight h; $w^* = -G/(H+\lambda)$; gain minus γ per split; λ shrinks values, γ removes leaves.
- **Stacking** learns a global combination from out-of-fold predictions; **mixture of experts** learns an input-dependent one jointly with the experts, at the risk of collapse.

Chapter 1

What Does It Mean to Learn?

This chapter asks the question on which the whole course rests: how can a machine acquire an ability from experience rather than from a hand-written rule? We begin by separating learning from ordinary programming, then classify the feedback a learner can receive. We follow a supervised example from its raw representation to a prediction, a decision and an action, and use four ingredients to describe the whole system without hiding behind an algorithm name. The central difficulty is generalization: finitely many observations never determine what should happen everywhere else. Shortcuts, memorisation and unrepresentative experience all exploit that gap. The final section introduces entropy, cross-entropy and KL divergence, the information-theoretic language that later explains why classification probabilities can be learned. Formulas marked [on your sheet] appear on the exam sheet; those marked [not on your sheet: know it] are worth understanding rather than memorising.

1.1 Learning from Experience

What are we trying to get?

We want machines that can *do things we cannot fully describe*: recognise a handwritten digit, flag a fraudulent payment, translate a sentence. This section pins down what "learning" means so that we can later ask, precisely, what a machine needs in order to do it.

Motivation

Learning is what happens between "could not do it" and "can do it", and experience is the fuel.

A child learns to understand language and to recognise faces. A musician becomes fluent through practice. A student becomes able to solve problems that were unfamiliar a month earlier. In each case something changed in the learner, and the change was driven by *experience*: observation, practice, interaction, instruction, feedback. But experience alone is not enough. A stone exposed to a thousand piano lessons learns nothing. The learner also brings a *capacity to change* in response to experience. For humans and animals this capacity comes from a biological system shaped by evolution and development: the brain and nervous system provide structures that support perception, memory and adaptation.

A first picture, which we will refine throughout the course, is

$$\text{Capacity to Learn} + \text{Experience} \xrightarrow{\text{Learning}} \text{Acquired Abilities}.$$

Definition 1.1.1 (Learning, informal)**.** *Learning* is the process through which experience changes a learner and develops what it can do. The capacity to learn is itself shaped by development and previous experience, so the picture above is a starting point rather than a complete description of biological learning.

What kinds of abilities do we learn?

Many acquired abilities involve four broad activities. They overlap, and they are not an exhaustive classification of intelligence, but they give us a vocabulary that also applies to artificial systems.

Recognise	**Predict**	**Reason**	**Act**
What is it?	What may happen?	What follows?	What should I do?

- *Recognition* identifies objects, people, sounds, words and patterns despite variations in how they appear. You recognise your friend from behind, in poor light, wearing a hat.
- *Prediction* uses previous experience and current information to anticipate what may happen: dark clouds suggest rain, a tilting glass suggests a spill.
- *Reasoning* works through a new problem rather than recalling a stored answer: solving an equation you have never seen, planning a route around a road closure.
- *Acting* includes physical and practical skills acquired through repeated interaction: riding a bicycle, pouring water without spilling.

Language draws on several of these at once: recognising words, predicting what comes next, reasoning about meaning, and acting by producing speech.

Example 1.1.2 (One task, four abilities)**.** A radiologist reading a chest X-ray *recognises* anatomical structures and abnormal shadows, *predicts* how a lesion may evolve, *reasons* from symptoms and image to a differential diagnosis, and *acts* by ordering a test or a treatment. A machine-learning system for radiology may take on any subset of these: a classifier that flags suspicious regions addresses recognition; a model of tumour growth addresses prediction; a treatment-recommendation system addresses decision and action. Keeping the four activities separate helps state precisely what such a system is supposed to do.

This raises the question that has driven much of artificial intelligence:

How can we give machines similar abilities?

Summary: Learning from experience

- Learning = capacity to learn + experience → acquired ability.
- Abilities we care about: recognise, predict, reason, act. Language combines them.
- The question of the course: what does a *machine* need in order to acquire abilities from experience?

Experiment: one ability, four observable tests

Setup. Give the same radiology system four tasks: locate a suspicious region, forecast how it changes, justify a differential diagnosis, and choose whether to order another test.

What we hold fixed. The patient record and image.

What we change. Only the ability being tested.

What we measure. A task-appropriate held-out score for each ability.

How to read it. A system can be strong at recognition and weak at reasoning or action. Calling all four results "intelligence" hides the actual learning problem; the experiment forces the desired ability to be stated before the model is chosen.

How the professor asks it

Q. *What does it mean for a machine to learn rather than merely execute?*
A. Learning means that experience changes the system so that it acquires an ability; execution only applies a fixed procedure. The capacity to change is supplied by the model and algorithm, the evidence by the experience, and success by the evaluation criterion. A stored lookup table may execute perfectly on its entries without having acquired behaviour that extends to a new case.

1.2 Two Routes to Machine Ability

What are we trying to get?

There are two ways to get a machine to do something: tell it *how* (write the rules), or give it *experience* and let it work the rules out. We want to understand when each route is the right one, and why the second route exists at all.

The first idea: encode human knowledge

Route 1: humans learn first, then transfer the result to the machine as explicit instructions.

One approach is to let humans acquire the knowledge and then transfer it to a machine. The idea predates modern computers: Pascal's seventeenth-century *Pascaline* embodied the procedures of arithmetic in a physical mechanism. Programmable computers later allowed far richer procedures to be expressed as algorithms. The principle is

$$\text{Human Knowledge} \longrightarrow \text{Explicit Rules or Procedures} \longrightarrow \text{Computer}.$$

This remains the right approach for many problems. When we know a reliable procedure for sorting numbers, calculating a tax, checking a password, or solving a well-defined numerical problem, we implement it directly. Learning it again from examples would only add uncertainty.

Example 1.2.1 (Programs that should stay programs)**.**

- *Income tax.* The rule is written in law: brackets, rates, deductions. A program applies it exactly. A learned model would approximate the law from past tax returns and be wrong on rare cases. Nobody wants an

approximately correct tax bill.

- *Sorting.* Merge sort is proved correct for every input. There is nothing to learn.
- *Password check.* The procedure is "hash the input and compare." A learned model would have to be right *every* time, which learning never guarantees.

Early artificial intelligence extended the first idea to domains requiring expertise. In an *expert system*, knowledge obtained from specialists was represented as explicit facts and rules. A medical system could contain rules relating symptoms and test results to diagnoses. Systems such as MYCIN showed that carefully constructed rule bases could support sophisticated reasoning in restricted domains. As such systems became more ambitious, extracting, expressing and maintaining the required knowledge became increasingly difficult. This became known as the *knowledge acquisition bottleneck*.

Example 1.2.2 (The knowledge engineer's year)**.** Imagine building, in the 1980s, an expert system for diagnosing faults in industrial machines, with two hours per week of access to an experienced engineer. After months the system contains hundreds of rules such as

```
IF motor temperature is high AND vibration is strong THEN
                    inspect the bearings.
```

Three kinds of difficulty appear, and it is worth separating them.

- *Obtaining the knowledge.* The engineer "just knows" that a certain rattle means a loose coupling, and cannot say what "that rattle" is in terms a program can test. Two hours a week is far too little to elicit thousands of such judgements.
- *Representing it as rules.* Real judgements are graded ("fairly high temperature"), interacting ("unless the machine has just started"), and full of exceptions. Encoding them produces long chains of rules that contradict each other in corners nobody anticipated.
- *Unfamiliar situations.* When the factory installs a new generation of machines, faults appear that the engineer has never seen. The rules say nothing, and there is no mechanism for the system to improve from the new failures it observes.

A learning-based approach attacks all three: it extracts patterns from logged sensor readings and repair records rather than from interviews, it represents graded and interacting effects numerically, and it can be retrained on the new machines' data. Expert knowledge remains valuable nonetheless: it tells us *which* measurements to record, it can check whether a learned model's behaviour is physically plausible, and it covers rare but catastrophic faults for which no data exist.

Can we always write the rules?

The gap: we can do *things we cannot* describe.

The difficulty is sharpest for abilities that humans perform easily without being able to describe the procedure they use. Consider handwritten-digit recognition. People recognise the same digit across large changes in handwriting, position, ori-

entation, size and stroke thickness. Writing a program that covers these variations through explicit rules is much harder.

Example 1.2.3 (Trying to write the rules for a "3")**.** You are given a 20×20 binary image and must decide whether it shows a 3. Some natural rules:

R1. There are two "bumps" opening to the left: the rightmost black pixel of the top half and the rightmost black pixel of the bottom half both lie in the right third of the image.

R2. The middle row contains black pixels only in the right half (the "waist" of the 3).

R3. The number of black pixels lies between 40 and 90.

R4. No black pixels touch the left border (a 3 has no left vertical stroke, unlike an 8 or a 0).

Now shift the digit four pixels to the right: R1 still holds, R2 may fail if the waist crosses the centre column, R4 holds. Write it narrowly: R1 may fail because the bumps no longer reach the right third. Rotate by 15°: the waist is no longer on the middle row, so R2 fails. Draw it with a thick stroke: R3 fails because the pixel count doubles.

Every fix creates new problems. "Right third" becomes "right third of the bounding box," which requires computing the bounding box, which is corrupted by a stray dot. "Middle row" becomes "the row of minimum horizontal extent between the two bumps," which is ambiguous for a flattened 3 and also matches some 5s and 8s. The pixel-count range widens until it excludes nothing. Rules pile up, each with parameters nobody knows how to set, and the interactions between rules become impossible to reason about. At this point, showing the computer a few thousand labelled 3s and non-3s, and letting it find its own regularities, is the sensible move. The thought experiment reveals the essential difference: explicit programming requires us to *know and state* the procedure; learning requires only that we can *recognise* correct behaviour on examples.

Human vision is the broader version of the same point. We know a great deal about the visual system, yet we do not possess a complete explicit description of the computations that allow people to recognise the enormous variety of objects in natural scenes. This illustrates a distinction that the whole field rests on:

Having an ability $\neq$ having an explicit procedure that describes the ability.

The second idea: make the machine learn

When useful experience is available while a complete procedure is hard to specify, another approach becomes attractive. Instead of transferring the *result* of human learning to a computer, we ask whether the machine can acquire the ability from experience itself. The question changes from "what rules should humans write?" to

How can we give a machine the capacity to learn from experience?

The picture of Section 1.1 now applies to machines:

$$\text{Capacity to Learn} + \text{Experience} \xrightarrow{\text{Learning}} \text{Acquired Ability}.$$

Designing computational methods that make this possible is the subject of *machine learning*. The capacity to learn is provided by design choices: what the system can represent, what can change through experience, what information guides those changes, and what counts as good performance. Machine learning still involves programming. Human designers construct the learning system and its learning mechanisms; part of the task-specific behaviour is then acquired from experience rather than written as rules.

Technique: Deciding between the two routes

Ask three questions.

1. *Can a competent person write down the complete procedure?* If yes (tax, sorting, physics simulation), program it.
2. *Can we recognise correct behaviour on examples, and do we have (or can we collect) many examples?* If yes and question 1 was no (digits, speech, fraud), learn it.
3. *What is the cost of a mistake?* Learning gives approximately correct behaviour. When "approximately" is unacceptable (safety interlocks, cryptography), keep the explicit program or wrap the learned component in explicit safeguards.

Most real systems mix both: a learned model produces a prediction and an explicit program decides what to do with it under fixed business rules.

Gotcha

- "Machine learning replaces programming" is false. Learning replaces the *task-specific rules*; the learner itself, the data pipeline and the decision logic are still programmed.
- "Expert knowledge is obsolete" is also false. It decides what to measure, sanity-checks the model, and covers cases with no data.

Summary: Two routes

- Route 1: humans learn, then write rules. Right when the procedure is known and exactness matters.
- Route 2: give the machine experience and a capacity to learn. Right when we can recognise correct behaviour but cannot describe the procedure.
- Expert systems hit the knowledge acquisition bottleneck; digit and object recognition show that ability does not imply a describable procedure.

What people do wrong: "machine learning replaces programming"

The slogan is tempting because the final task-specific rule is no longer typed line by line. The learner still needs a data pipeline, a model, an objective, an optimiser, an evaluation procedure and explicit decision logic. Learning replaces only the rule that nobody can reliably write down. If a complete, exact procedure is known and errors are unacceptable, ordinary programming

remains the right route.

Telling them apart: written rules versus learned rules

Written rules are explicit, inspectable and exact on the cases their author anticipated; they need no examples but are brittle outside those cases. Learned rules are implicit in fitted parameters, adapt when their experience changes and degrade statistically, but require representative data and independent evaluation. Real systems often combine them: a model predicts and a program applies safeguards and business rules.

How the professor asks it

Q. *When should a problem be programmed, and when should it be learned?*
A. Program it when a competent person can state the complete procedure and approximate correctness is unacceptable. Learn it when correct behaviour can be recognised on many examples but the procedure cannot be written, as with handwriting or speech. The choice is not ideological: it follows from whether the rule is known, whether usable experience exists, and what a failure costs.

1.3 What Counts as Experience?

What are we trying to get?

Before choosing any algorithm we must know *what information the learner will receive*. Different forms of experience carry different information, and that difference decides which kind of learning is even possible.

Three forms of experience

Experience can reach a machine in three broad forms.

Examples	**Observations**	**Interactions**
input + desired output	observed data only	actions + consequences

- *Examples* pair an input with a desired output: an image with the digit it shows, a house description with its sale price. Examples give *direct guidance* about what the output should be.
- *Observations* are data without a supplied answer: images, text, audio, sensor logs. They contain *regularities* that can be exploited without anyone labelling them.
- *Interactions* arise when an agent acts in an environment and observes what follows. The learner partly *shapes its own future experience*, because its actions change what happens next.

Definition 1.3.1 (The three learning settings)**.** The three forms of experience lead to three broad learning settings.

- *Supervised learning*: experience consists of examples with desired outputs.

- *Unsupervised learning*: the learner searches for useful structure in observations without desired outputs. *Self-supervised learning* also learns from observations, but constructs a learning signal from the observations themselves (Section 1.5).
- *Reinforcement learning*: experience comes through interaction; consequences and rewards provide information about the quality of behaviour.

These are broad settings, not mutually exclusive kinds of system. A single system can use more than one.

Example 1.3.2 (Same hospital, four projects)**.** A hospital holds a large collection of chest X-rays; for some patients a diagnosis was recorded.

1. *Predict whether a new X-ray shows pneumonia.* Experience: images paired with diagnoses. Feedback: whether the prediction matches the recorded diagnosis. **Supervised.**
2. *Organise thousands of unlabelled X-rays into groups with similar visual characteristics.* Experience: images only. Feedback: none from outside; the learner judges structure by internal criteria such as within-group similarity. **Unsupervised.**
3. *Remove part of each X-ray and predict the missing region.* Experience: images only, but each image supplies its own target (the hidden region). Feedback: how well the reconstruction matches the hidden pixels. **Self-supervised.**
4. *Teach a robotic device to position an imaging sensor, rewarding good positioning and penalising unsafe movements.* Experience: the device's own actions and their consequences. Feedback: delayed and evaluative (a reward), not a correct answer. **Reinforcement.**

The paradigms differ not by the data type (all four use X-rays) but by *what information about "doing well" reaches the learner*: a correct answer per example, nothing external, an answer manufactured from the data itself, or a reward earned through action.

Example 1.3.3 (Same records, three questions)**.** A university has ten years of student records: previous grades, attendance, course choices, and eventual outcome.

- *Will each student graduate?* Input: the record at entry; output: graduated or not. Supervised, with a categorical output: *classification.*
- *What will the final graduation grade be?* Same input; output: a number. Supervised: *regression.*
- *Which students have similar academic trajectories?* Input: the records; no desired output at all. Unsupervised.

The same collection of observations supports several learning problems because the *problem* is defined by the question asked, not by the data alone.

Gotcha

The word "unsupervised" does not mean "no objective." Grouping X-rays still needs a criterion for what a good grouping is; the difference is that no human supplies the desired answer for each example.

Summary: What counts as experience

- Examples (input + output), observations (data only), interactions (actions + consequences).
- They lead to supervised, unsupervised or self-supervised, and reinforcement learning.
- The paradigm is decided by what feedback about "doing well" reaches the learner, not by the data type.

Experiment: one hospital collection, four learning settings

Setup. Use one collection of chest X-rays, some with diagnoses, and a sensor-positioning task.

What we hold fixed. The application domain.

What we change. The feedback: recorded diagnoses, no labels, a masked image region used as its own target, or reward after an action.

What we measure. Prediction error, clustering quality, reconstruction error, or accumulated reward respectively.

How to read it. The setting is determined by the feedback that tells the learner what "better" means: supervised, unsupervised, self-supervised or reinforcement learning. The file type alone does not decide it.

How the professor asks it

Q. *Why is self-supervised learning not supervised learning with free labels?*
A. Its targets are manufactured from the observations themselves, for example by hiding a word and asking the model to recover it; no external desired answer was attached to each example. It nevertheless has a training objective, which is why "unsupervised" never means "without a criterion". The learned representation can later support a supervised task with far fewer human labels.

1.4 Supervised Learning: Learning from Examples

What are we trying to get?

Supervised learning is the setting this course develops mathematically in Chapter 2. Here we want a clear picture of its ingredients: what an example is, where the desired outputs come from, how the machine uses them, and what kinds of outputs it may be asked to produce.

Definition 1.4.1 (Supervised example, dataset)**.** A *supervised example* is a pair (x, y) of an *input* x and a *desired output* y. A *training dataset* is a finite

collection of such pairs, $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$. The desired output is called the *target*, the *label*, or the *ground truth*, depending on the application. The learner's goal is to acquire a model that produces useful outputs for *new* inputs.

Where do labels come from?

"Ground truth" is a strong phrase for something that was measured with error, judged by a person, or observed only later.

The desired output must itself come from somewhere, and this has consequences for what can be learned.

- *Measurement.* Temperature, electricity consumption, the recorded sale price of a house. Measurements carry instrument error and recording mistakes.
- *Annotation.* Identifying an object in an image, interpreting a medical scan. Experts disagree, and labels depend on the conventions chosen when the task was defined (is a "dog" label correct for a wolf-dog hybrid?).
- *Delayed outcome.* Whether a customer repays a loan or a machine fails within a month is only known after time has passed. Such labels are available only for old inputs, and the world may have changed since.

The quality of the supervision limits what can be learned. A model trained on labels that are wrong 10% of the time cannot be expected to agree with the (noisy) labels more than about 90% of the time, and its errors will partly mirror the annotators' errors.

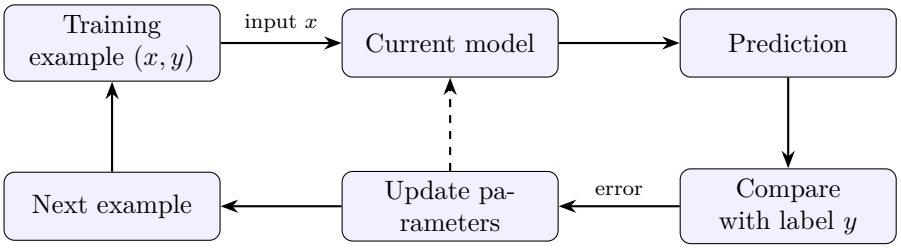
Example 1.4.2 (Phishing detection: designing the experience)**.** Asked to build a phishing-website detector, the first job is to decide what experience to collect, before any algorithm is chosen.

- *Where do examples come from?* Reported phishing pages (from security feeds and user reports) and legitimate pages (from popular sites and from the organisation's own traffic).
- *How varied should they be?* Many languages, many impersonated brands, pages built with different tools, both crude and sophisticated attacks; otherwise the model learns "phishing = badly spelled English page imitating one bank."
- *How are correct outcomes established?* A page is phishing if a security analyst confirmed it, or if the impersonated organisation confirmed it. Both are imperfect; some legitimate pages are mislabelled by over-eager reporters.
- *How fast does the world change?* Phishing strategies change monthly. Labels from three years ago describe attacks nobody uses any more, and today's attacks are absent from the data.

A very large dataset can still give a poor system if the experience is badly chosen: a million pages all from one feed, one language and one year teaches a narrow, outdated notion of phishing. Volume does not substitute for coverage.

How does the machine learn from examples?

During training, the model receives an input and produces a prediction. The prediction is compared with the desired output, and the comparison produces feedback that the learning algorithm uses to change the model. The process continues across the available training experience.



The computational process through which experience changes the model is called *training*. Different learning methods perform the changes differently; Chapter 2 turns this informal cycle into a mathematical problem.

Regression and classification

Definition 1.4.3 (Regression, classification)**.** A supervised problem is a *regression* problem when the desired output is numerical (a real number or a vector of real numbers), and a *classification* problem when the desired output is a category from a finite set of classes. The type of problem is decided by the *output*, not by the input.

Example 1.4.4 (Ten tasks, sorted by their output)**.**

Task	Input	Output	Type
House price	description	price in MAD	regression
Digit recognition	image	one of $\{0, \dots, 9\}$	classification
Spam filter	email	spam / legitimate	classification
Tomorrow's rainfall	weather data	millimetres	regression
Will it rain?	weather data	yes / no	classification
Tumour size	medical image	millimetres	regression
Tumour type	medical image	benign / malignant	classification
Battery lifetime	usage logs	hours	regression
Species	photograph	one of 10 species	classification
Energy demand at 18:00	calendar, weather	megawatts	regression

The same medical image appears twice: predicting its size is regression, predicting its category is classification. The same weather data appears twice as well. The *desired output* decides.

Some supervised problems have richer outputs. *Object detection* returns several objects with their locations (a list of boxes with class labels); *semantic segmentation* assigns a class to every pixel; *machine translation* outputs a sentence. These are still supervised (examples come with desired outputs), but the output is structured rather than a single number or class.

Prediction, decision, action

Definition 1.4.5 (Prediction, decision, action)**.** A *prediction* expresses what the model says about an unknown output. A *decision* uses the available information together with a criterion to select a response. An *action* is what is carried out when the decision calls for something to be done.

Example 1.4.6 (A screening system)**.** A model predicts that a patient has a 12% probability of a disease (prediction). A protocol says: recommend further testing when the probability exceeds 10% (decision). The test is scheduled and performed (action). Change the protocol threshold to 20% and the same prediction leads to a different decision. In simple applications prediction and decision coincide; they separate as soon as different errors have different consequences, a theme developed precisely in Section 2.2.

Gotcha

- A label is not truth. Ask how it was produced before trusting an accuracy figure.
- "Regression" does not mean "linear regression" and "classification" does not mean "two classes." The words describe output types.
- A model that predicts probabilities has not yet made a decision. Do not confuse "the model says 0.6" with "the model says yes."

Summary: Supervised learning

- Examples are (input, desired output) pairs; labels come from measurement, annotation or delayed outcomes, each imperfect.
- Training cycle: predict, compare with the label, update, repeat.
- Regression: numerical output. Classification: categorical output. The output decides the type.
- Prediction, decision and action are distinct steps; they separate when errors have unequal costs.

What people do wrong: treating a label as unquestionable truth

A label may be an instrument reading, an annotator's judgement or a delayed outcome. Each can be noisy, biased or unavailable at decision time. The mistake inflates the apparent ceiling of the task and makes disagreements with the annotator look like model errors. Record how labels were produced, measure disagreement where possible, and interpret the attainable score against that process.

Telling them apart: prediction, decision and action

A prediction describes an unknown quantity, such as a disease probability of 0.12. A decision combines that prediction with costs or a protocol, for example "test above 0.10". The action is the test actually ordered. Changing the threshold changes the decision without changing the prediction, so these three stages must be evaluated separately.

How the professor asks it

Q. *Is a task regression or classification when the input is an image?*
A. The input does not decide. Predicting tumour size from the image is

regression because the output is numerical; predicting benign or malignant is classification because the output is categorical. A probabilistic classifier may output numbers internally, but those numbers describe probabilities of categories and are converted to an action by a decision rule.

1.5 Representations: What Is Presented to the Learner?

What are we trying to get?

A learner never sees a house or a photograph; it sees numbers. We want to understand what those numbers should be, why the choice matters enormously, and how modern systems learn the numbers themselves, sometimes without any labels at all.

Feature engineering

The form of experience says what information exists*; the representation says what information the model* gets.

For some problems, useful input features can be chosen with ordinary knowledge of the application. A house-price model might receive floor area, location, age, and numbers of bedrooms and bathrooms. Human designers decide these are worth presenting; the model discovers how they relate to price. Some supplied features may turn out to have little influence.

Perceptual problems make the choice much harder. An image contains millions of pixel values; an audio recording is a long waveform; useful information in text depends on relationships among many words. For many years, successful systems relied on representations designed by specialists: edges, corners, textures and shapes for vision; spectral features such as mel-frequency cepstral coefficients for speech. This *feature engineering* let domain knowledge determine what information was presented, and the model learned how to use it. The choice of representation can decide whether learning succeeds at all.

Example 1.5.1 (Two representations of the same email)**.** A spam filter could receive (a) the raw sequence of characters, or (b) a vector of counts: how many times each of 10,000 common words appears, whether the sender is in the address book, the number of links, the number of exclamation marks. Representation (b) makes "many links, no known sender, the word *prize*" directly visible as a few coordinates; representation (a) hides it behind thousands of characters whose positions vary from message to message. With a few thousand examples, (b) is learnable and (a) is hopeless. With billions of examples and a model able to build its own representation, (a) becomes viable and can surpass (b), because the designer of (b) forgot features that matter.

Representation learning

Feature engineering raises a further possibility: instead of asking humans to decide all useful features in advance, can the machine discover a useful representation from experience? This is the aim of *representation learning*. The idea is broader than deep learning and predates its success; representations can be learned by many methods.

Definition 1.5.2 (Representation)**.** A *representation* is a transformation of the original input into a form that makes useful structure easier for a learning model to exploit. The two pipelines are

$$\text{Raw Input} \longrightarrow \text{Human-Designed Features} \longrightarrow \text{Learned Predictor},$$
$$\text{Raw Input} \longrightarrow \text{Learned Representation} \longrightarrow \text{Learned Predictor}.$$

Deep learning suits the second pipeline because several layers can construct successive representations, with higher-level features (object parts, shapes) built from lower-level ones (edges, local patterns). The second pipeline does not remove humans from the design: they still choose the architecture, the learning method, the training experience and the objective. What changes is that more of the representation is acquired from data.

Learning representations from observations: self-supervision

Representation learning does not require a label for every observation. In *self-supervised learning* a learning problem is constructed from the observations themselves: part of an observation is used to predict another part.

Example 1.5.3 (Three self-supervised tasks)**.**

- *Text.* Hide a word in "The train to _____ leaves at nine" and ask the model to fill it in. The sentence supplies both the context and the answer.
- *Images.* Mask a square region of a photograph and ask the model to reconstruct it from the rest. Success requires knowing what objects and textures look like.
- *Audio.* Remove half a second of speech and predict it from the surrounding sound.

Nobody labelled anything; the data are their own supervision. The task gives the model a reason to discover regularities, and the representation it builds can later be adapted, with a small labelled dataset, to classification, detection or other tasks. Learning from unlabelled observations and transferring the result predates the current generation of large models and remains one of the main uses of representation learning.

From self-supervision to generation

Self-supervised learning can also produce a model whose predictive ability is useful *directly*. Every piece of text already tells us which token followed the previous ones, so a long document provides many training examples for free. An *autoregressive language model* learns to predict the next token from the tokens that came before. Once it can predict, it can *generate*: predict a next token, append it, predict again.

Example 1.5.4 (A next-token model you can compute by hand)**.** Take the tiny corpus

```
the cat sat on the mat .  the cat ate the rat .
```

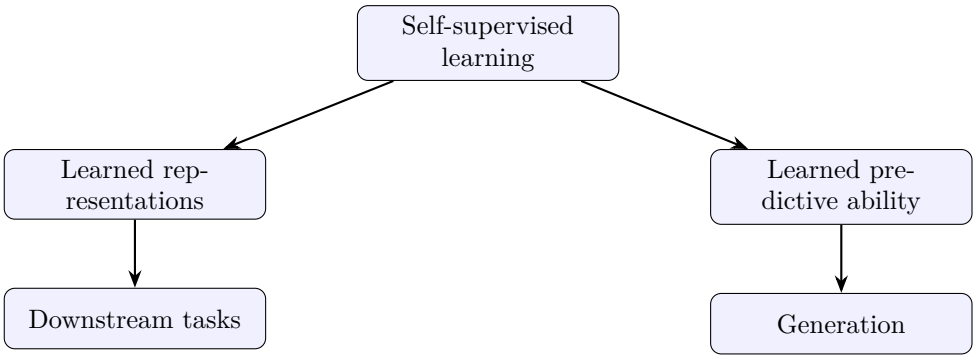
The simplest next-token model looks only at the previous word and estimates probabilities by counting. After `the`, the next word is `cat` twice, `mat` once,

`rat` once:

$$\mathbb{P}(\texttt{cat} \mid \texttt{the}) = \tfrac{2}{4}, \qquad \mathbb{P}(\texttt{mat} \mid \texttt{the}) = \tfrac{1}{4}, \qquad \mathbb{P}(\texttt{rat} \mid \texttt{the}) = \tfrac{1}{4}.$$

After `cat`: `sat` and `ate`, each with probability $\frac{1}{2}$. After `sat`: `on` with probability 1. After `on`: `the`. After `mat` or `rat`: the full stop. Nobody wrote a label: the corpus itself says what followed what.

Generation. Start from `the`. Sample the next word from $(\frac{1}{2}, \frac{1}{4}, \frac{1}{4})$, say `cat`; then from $(\frac{1}{2}, \frac{1}{2})$, say `ate`; after `ate` the only continuation is `the`; and so on. One possible output is `the cat ate the mat .`, a sentence that never appears in the corpus. The model has recombined regularities it found. Large language models do the same thing with a context of thousands of tokens and a model with billions of parameters instead of a count table, but the principle, self-supervised next-token prediction followed by repeated sampling, is exactly this.



The two branches are two *uses* of what was learned, not two kinds of model. A generative language model also learns internal representations, and a representation learned by self-supervision can become part of a generative system.

Summary: Representations

- The representation decides which information the model actually receives; a good one can make an impossible problem easy.
- Feature engineering: humans design the features. Representation learning: the machine learns them.
- Self-supervision manufactures targets from the data (fill in the missing word, pixel, sound). Next-token prediction is self-supervised, and repeated sampling from a next-token model is generation.

Experiment: designed features versus a learned representation

Setup. Train the same downstream classifier first on human-designed email features and then on a representation learned from raw characters.

What we hold fixed. The train/validation split, downstream classifier and evaluation metric.

What we change. The representation and the number of labelled emails.

What we measure. Validation error.

What the plot shows. With few labels the designed representation may lead because it already encodes useful prior knowledge; with enough data the

learned representation may cross below it.

How to read it. The representation is part of the model's evidence, not neutral preprocessing. Information omitted at this stage cannot be recovered downstream.

How the professor asks it

Q. *How can next-token prediction be both self-supervised and generative?*
A. It is self-supervised during training because the text supplies its own target: the next token already present in the sequence. It becomes generative at use time by sampling a predicted token, appending it, and repeating. The same learned conditional distribution therefore supplies an internal representation and a procedure for producing a new sequence.

1.6 Four Ingredients of a Learning System

What are we trying to get?

We want a checklist that applies to *any* learning system, from a count table to a language model, so that we can describe a new application without naming an algorithm and can see what each design choice is responsible for.

Definition 1.6.1 (The four ingredients)**.** Almost any learning system can be examined through four questions, corresponding to four ingredients.

Experience	What information is available?
Model	In what form can acquired knowledge be represented?
Learning algorithm	How does experience change the model?
Evaluation criterion	How is success judged?

This framework begins to answer the question of Section 1.2: what gives a machine the capacity to learn? The model determines what can be represented; the learning algorithm provides the mechanism through which experience changes it; the evaluation criterion indicates which changes are useful; the experience determines what evidence exists in the first place. Representation learning adds one possibility: the way the input is internally represented can itself be shaped by experience.

Example 1.6.2 (Four systems, four ingredients)**.**

	Experience	Model	Learning algorithm	Evaluation
Bigram text model (Ex. 1.5.4)	a corpus	a table of next-word probabilities	count and normalise	probability assigned to held-out text
Digit classifier	labelled images	a function from pixels to 10 probabilities	adjust parameters to reduce error on examples	accuracy on new images
Fraud detector	past transactions with confirmed outcomes	a function from transaction features to fraud probability	same, but with rare positives weighted up	cost of missed fraud plus cost of blocked customers
Warehouse robot	its own actions in simulation	a policy from sensor readings to motor commands	reinforcement from rewards	task completion and safety in the real warehouse

Each row can be discussed, criticised and improved without naming a single algorithm. That is the point of the checklist.

Example 1.6.3 (Designing an application from scratch)**.** *Task: predict, from a farm's sensor data, whether an irrigation pump will fail in the next week.*

1. *Experience.* Two years of hourly readings (current draw, vibration, temperature, flow) from 300 pumps, with the maintenance log recording each failure and its date. Each week of readings becomes an input; the label is whether a failure occurred in the following week.
2. *Model output.* A probability of failure within seven days, so that the maintenance team can rank pumps.
3. *Information for improvement.* Disagreement between predicted probability and what actually happened, aggregated over many pump-weeks. Failures are rare, so the learner must be told that missing one is much worse than a false alarm.
4. *Judging success.* On pumps and months not used during development: how many failures were flagged in advance, and how many unnecessary inspections were triggered, converted into money.

Ingredient by ingredient: (1) is experience, (2) is the model (its output form), (3) is the learning algorithm's signal, (4) is the evaluation criterion.

Technique: Writing the four ingredients for a new problem

- Start with the *evaluation criterion*: what would make the client say "it works"? Everything else must serve it.
- Then *experience*: what is logged already, what can be labelled, what changes over time.
- Then the *model output*: a number, a class, a probability, a ranking, a sentence?

- Only then the *learning algorithm*. It is the ingredient with the most textbooks and the least influence on whether the project makes sense.

Summary: Four ingredients

- Experience, model, learning algorithm, evaluation criterion.
- The capacity to learn lives in the last three; the evidence lives in the first.
- Describe any application through the four questions before choosing methods.

Telling them apart: model versus learning algorithm

The model is the family of behaviours the system can express; the learning algorithm is the procedure that chooses one member of that family from data. A family that cannot express the required relationship causes approximation error even with perfect optimisation. An algorithm that fails to find a good member causes optimisation error even when the family is adequate.

How the professor asks it

Q. *State the four ingredients for a new learning application without naming an algorithm.*
A. Specify the experience available, the model output and family, the learning algorithm and objective that select a fitted model, and the evaluation criterion used on appropriate held-out situations. Starting from the criterion exposes what errors cost; starting from an algorithm name often hides missing labels or a mismatch between the training objective and the real task.

1.7 The Challenge of Generalization

What are we trying to get?

Experience is always finite, and the situations a deployed system meets are new. We want a precise name for "being useful on new situations," a clear reason why performance on the training examples does not settle it, and a way to test for it.

Finite experience, infinite situations

A digit recogniser cannot see every handwritten digit that will ever be produced; a medical system cannot observe every future patient; a robot cannot encounter every situation before deployment. A useful learner must acquire something from finite experience that remains useful when a new input appears.

Doing well on what you have seen is not the goal. It is a side effect, and a dangerous one to optimise for.

Definition 1.7.1 (Generalization)**.** *Generalization* is the ability of a learned system to perform well in new situations that were not encountered during learning. The central challenge of machine learning is

$$\text{Finite Experience} \xrightarrow{\text{Learning}} \text{Ability Useful in New Situations.}$$

The following small proposition is the mathematical root of the difficulty. It says that the training data never pin down the rule.

Proposition 1.7.2 (Finite data are consistent with infinitely many rules)**.** *Let $\mathcal{X}$ be an infinite input space, let $\mathcal{Y}$ contain at least two elements, and let $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$ be any finite dataset. Then there are infinitely many functions $f : \mathcal{X} \to \mathcal{Y}$ that agree with the data, i.e. $f(x^{(i)}) = y^{(i)}$ for all i, and any two of them may disagree on every input not in the data.*

Proof. Let $S = \{x^{(1)}, \ldots, x^{(N)}\}$, a finite set, so $\mathcal{X} \setminus S$ is infinite. Fix two distinct outputs $y_0 \neq y_1$ in $\mathcal{Y}$. For each subset $T \subseteq \mathcal{X} \setminus S$ define $f_T(x) = y^{(i)}$ if $x = x^{(i)} \in S$, $f_T(x) = y_1$ if $x \in T$, and $f_T(x) = y_0$ otherwise. Every f_T agrees with the data. Distinct subsets $T \neq T'$ give distinct functions (they differ on any $x \in T \triangle T'$), and an infinite set has infinitely many subsets. Taking $T = \mathcal{X} \setminus S$ and $T' = \emptyset$ gives two data-consistent functions that disagree on every input outside the data. □

Key insight

Off the training inputs, the data say nothing. Whatever a learned system does there comes from its *model* and its *learning algorithm*, that is, from assumptions built in by the designer. Generalization is therefore never a property of the data alone; it is a property of the match between the assumptions and the world. Chapter 2 introduces the formal objects (hypothesis spaces, risks) needed to study this match.

Example 1.7.3 (The perfect memoriser)**.** The function "look up x in the training set and return its label; otherwise return class 0" has zero error on the training examples. It has learned nothing. A learner evaluated only on its training data cannot distinguish this lookup table from a genuinely useful model. This is why performance on previously encountered examples gives only part of the evidence about what has been learned.

Shortcuts: right answers for the wrong reasons

Example 1.7.4 (Wolves and snow)**.** A system learns to distinguish photographs of wolves from dogs and performs extremely well on the developers' examples. After deployment its performance drops. Investigation shows that most wolf photographs used in learning contained snow, whereas most dog photographs did not.

1. *Initial success.* "White background ⇒ wolf" is a rule consistent with almost all of the training examples (Proposition 1.7.2 guarantees such alternative rules exist). It is far simpler to detect than wolf anatomy, so the learner adopted it.
2. *Failure on new photographs.* Deployed users photograph dogs in snow and wolves in summer. The shortcut now gives systematically wrong answers, because the correlation it exploited was a property of the collection, not of wolves.
3. *Diagnosis.* Two test photographs: a husky standing in snow, and a wolf on green grass. A system that learned about wolves classifies them correctly; a system that learned about snow gets both wrong.

Nothing here assumes anything about the model's internals. The reasoning uses only the definition of generalization: the system performed well on the situations it had seen and poorly on new ones, and the explanation is a regularity that held in the experience but not in the world.

Example 1.7.5 (Quantity versus diversity)**.** A company must choose one image collection for recognising ten bird species: Dataset A, one million photographs from one camera in one park; or Dataset B, fifty thousand photographs by many people in many countries, seasons, weathers, backgrounds and angles. For a system to be used by the public, B is the better choice: A's million images share lighting, background, altitude and season, so many of the regularities a learner will find in A (this park's foliage, this camera's colour balance) will not hold elsewhere, exactly as in Example 1.7.4. B covers more of the situations the deployed system will meet. A would be preferable if the system is to be deployed *in that park with that camera*, or if the species are so similar that the sheer number of examples is needed to tell them apart and the deployment conditions match. The lesson: the amount of experience and its *representativeness* of future situations are different quantities, and the second is the one generalization depends on.

Example 1.7.6 (The robot that learned in the wrong world)**.** A warehouse robot trained entirely in a simulator with perfectly flat floors, identical box friction and constant lighting performs well in simulation and poorly in the real warehouse. If only the simulator can be changed, vary the things that vary in reality: floor slope and bumps, box weights and surface friction, lighting, sensor noise, worker behaviour, wheel slippage. A robot exposed to a range of conditions must acquire behaviour that works across them, rather than behaviour tuned to one idealised world. Making the simulator *visually* more realistic would not by itself help: if the floor is still perfectly flat, the robot still learns that wheels never slip. What matters is covering the variations that affect the decisions, not photorealism.

Example 1.7.7 (A claim worth questioning)**.** An advertisement says: "Our AI has learned every possible situation from millions of examples." The claim confuses situations that were *observed* with situations that *may occur*. Millions of examples are a finite sample; the situations a deployed system will face are not a fixed list and include ones nobody has recorded, and the world changes after the data were collected. What a system can honestly claim is to have learned regularities from the examples it saw that *are expected* to carry over to similar new situations. How similar, and with what guarantee, is exactly the study of generalization, and no amount of data makes the question disappear.

Technique: Testing for generalization without opening the model

- *Hold out.* Evaluate on examples the learner never saw. (Chapter 2 makes this precise with validation and test sets.)
- *Break the shortcuts.* Construct test inputs where suspected spurious cues are decoupled from the label (dog in snow, wolf on grass).
- *Shift the conditions.* Evaluate on data from a different camera, hospital,

country or year than the training data. A large drop reveals reliance on collection-specific regularities.

- *Ask what the data cannot know.* List the situations absent from the experience; the model's behaviour there is an assumption, not a learned fact.

Gotcha

- High accuracy on the training examples is compatible with having learned nothing (the memoriser) or the wrong thing (the snow detector).
- More data helps only if it covers more of the situations that will occur. A billion photographs of one park are one situation.

Summary: Generalization

- Generalization: performing well in situations not encountered during learning.
- Finite data are consistent with infinitely many rules (Proposition 1.7.2); what a learner does off the data comes from its built-in assumptions.
- Shortcuts (snow for wolves) give excellent training performance and fail on new data. Test on held-out, shifted and shortcut-breaking inputs.

Experiment: wolves and snow: break the cue

Setup. Use a wolf-versus-dog classifier trained where wolves usually appear in snow.

What we hold fixed. The fitted model.

What we change. Only the test conditions, adding huskies in snow, wolves on grass and images from another season.

What we measure. Accuracy overall and on each cue-breaking slice.

What the plot shows. A snow detector can score 90% on an ordinary random split and collapse toward 50% once snow is decoupled from the label.

How to read it. The collapse diagnoses a shortcut without opening the model. More near-duplicate images would not repair it; the experience must vary the nuisance cue independently of the target.

Telling them apart: shortcut versus target leakage

A shortcut is available at decision time but predicts only because of the collection process, such as snow or camera type. Leakage is unavailable at decision time or caused by the label, such as a treatment code entered after diagnosis. Remove leaked fields; expose shortcuts by testing under changed conditions and repair them by diversifying the collection.

How the professor asks it

Q. *Why can a model with zero training error still be useless?*
A. A finite sample constrains a function only on finitely many inputs. Infinitely many rules agree there and disagree everywhere else, including a memoriser that returns stored labels and an arbitrary default otherwise. Performance on held-out, shifted and shortcut-breaking inputs is therefore evidence of generalization; performance on the fitted rows is not.

1.8 A Connection to Information Theory

What are we trying to get?

We want a numerical way to say how good a set of predicted probabilities is, and a way to talk about how much information a representation keeps. Both come from information theory, and the single inequality proved here is the reason cross-entropy is the workhorse loss of classification in Chapter 2.

Entropy, cross-entropy, divergence

Entropy: how uncertain the truth is. Cross-entropy: how surprised you are, on average, when you believed q and the truth was drawn from p.

Recall that *entropy* describes uncertainty, and *information* tells us how much uncertainty is removed by observing something. From a learning perspective, an input is useful for prediction when it contains information about the output.

Definition 1.8.1 (Entropy, cross-entropy, Kullback–Leibler divergence)**.** Let $p = (p_1, \dots, p_C)$ and $q = (q_1, \dots, q_C)$ be probability distributions on the same finite set of C outcomes, with $q_c > 0$ whenever $p_c > 0$. Define

$$H(p) = -\sum_c p_c \log p_c, \qquad H(p, q) = -\sum_c p_c \log q_c,$$

$$\mathrm{D}_{\mathrm{KL}}(p \,\|\, q) = \sum_c p_c \log \frac{p_c}{q_c},$$

with the convention $0 \log 0 = 0$. Base-2 logarithms measure in bits; natural logarithms, customary in machine learning, measure in nats.

Expanding the divergence, $\mathrm{D}_{\mathrm{KL}}(p\|q) = \sum_c p_c \log p_c - \sum_c p_c \log q_c = -H(p) + H(p, q)$, so

$$H(p, q) = H(p) + \mathrm{D}_{\mathrm{KL}}(p \,\|\, q). \tag{1.1}$$

Theorem 1.8.2 (Gibbs' inequality)**.** $\mathrm{D}_{\mathrm{KL}}(p \,\|\, q) \geq 0$*, with equality if and only if $p = q$. Equivalently, $H(p, q) \geq H(p)$ with equality iff $q = p$: among all distributions q, the expected surprise $H(p, q)$ under truth p is smallest when q is the truth.*

Proof. **Step 1 (a one-line inequality about logarithms).** For every $t > 0$, $\ln t \leq t - 1$, with equality iff $t = 1$. Indeed $g(t) = t - 1 - \ln t$ has $g'(t) = 1 - 1/t$, negative on $(0, 1)$ and positive on $(1, \infty)$, so g has its unique minimum at $t = 1$ where $g(1) = 0$.

Step 2 (apply it term by term). Let $S = \{c : p_c > 0\}$. Using natural

logarithms,

$$-\,\mathrm{D_{KL}}(p\|q) = \sum_{c\in S} p_c \ln \frac{q_c}{p_c} \;\le\; \sum_{c\in S} p_c\Big(\frac{q_c}{p_c} - 1\Big) = \sum_{c\in S} q_c - \sum_{c\in S} p_c = \sum_{c\in S} q_c - 1 \le 0,$$

since $\sum_{c\in S} q_c \le 1$. Hence $\mathrm{D_{KL}}(p\|q) \ge 0$. Changing the base of the logarithm multiplies everything by a positive constant, so the sign is unaffected.

Step 3 (equality). Equality requires equality in Step 1 for every $c \in S$, i.e. $q_c = p_c$ for all c with $p_c > 0$, and also $\sum_{c\in S} q_c = 1$, which forces $q_c = 0$ whenever $p_c = 0$. So $q = p$. Conversely $p = q$ gives every term $\log 1 = 0$. □

Key insight

The whole theorem is the tangent-line inequality $\ln t \le t - 1$, applied to the ratios q_c/p_c and averaged under p. The averaging turns a family of pointwise inequalities into one statement about the whole distribution. This trick, "bound the log by its tangent, then average," recurs throughout machine learning.

Example 1.8.3 (Numbers)**.** Let the truth be $p = (0.7, 0.3)$ over two classes. In bits, $H(p) = -0.7\log_2 0.7 - 0.3\log_2 0.3 \approx 0.881$.

Prediction q	$H(p,q)$	$\mathrm{D_{KL}}(p\|q)$	Comment
$(0.7, 0.3)$	0.881	0	the truth: minimum possible
$(0.5, 0.5)$	1.000	0.119	hedging costs 0.119 bits
$(0.9, 0.1)$	1.103	0.222	overconfident in the right direction: worse than hedging
$(0.3, 0.7)$	1.370	0.489	wrong direction
$(0.99, 0.01)$	2.003	1.122	very overconfident: the 0.3 mass pays $\log_2 100$ each

Two lessons. The floor 0.881 is a property of the *problem*: no prediction beats it, because the outcome is genuinely uncertain. And overconfidence is punished sharply: $(0.9, 0.1)$ is further from the truth in cross-entropy than the uninformative $(0.5, 0.5)$, even though it favours the right class.

Why this matters for learning

Two connections deserve to be kept in mind for the rest of the course.

- *Cross-entropy as a criterion.* A classifier may assign probabilities to several classes. Cross-entropy compares those probabilities with what actually occurs: giving high probability to the correct class incurs a small penalty, giving it little probability a large one. By Theorem 1.8.2, the expected penalty is minimised exactly when the predicted probabilities equal the true conditional probabilities. A familiar information-theoretic quantity thus becomes one of the most widely used learning criteria (Section 2.5).

- *Representations and information.* Raw observations contain many kinds of information, only some of it useful for a given purpose. Representation learning asks how data can be transformed so that useful structure becomes easier to exploit. Information theory supplies the vocabulary: an informative representation retains what is relevant about the output and discards the rest.

Technique: Computing with entropies

- $H(p, q) = H(p) + \mathrm{D}_{\mathrm{KL}}(p\|q)$: compute the entropy once, then only the divergence changes with the prediction.
- Cross-entropy of a *one-hot* truth ($p_y = 1$) is simply $-\log q_y$: the penalty for the probability given to the observed class. This is the form used for training in Chapter 2.
- D_{KL} is not symmetric: $\mathrm{D}_{\mathrm{KL}}(p\|q) \neq \mathrm{D}_{\mathrm{KL}}(q\|p)$ in general. Check the order before quoting it.
- $q_c = 0$ for an outcome with $p_c > 0$ gives $H(p, q) = +\infty$: never predict probability exactly zero for something that can happen.

Classic problems

- *Which q minimises $H(p, q)$?* $q = p$, by Gibbs' inequality; the minimum value is $H(p)$.
- *p uniform on C outcomes: $H(p)$?* $\log C$. It is the maximum entropy on C outcomes (apply Gibbs with q uniform: $\log C - H(p) = \mathrm{D}_{\mathrm{KL}}(p\|\mathrm{unif}) \geq 0$).
- *Truth $(0.5, 0.5)$, prediction $(0.99, 0.01)$: cross-entropy in bits?* $-0.5 \log_2 0.99 - 0.5 \log_2 0.01 \approx 3.33$, versus 1 bit for the honest prediction.

Summary: Information theory

- $H(p, q) = H(p) + \mathrm{D}_{\mathrm{KL}}(p\|q)$ and $\mathrm{D}_{\mathrm{KL}} \geq 0$ with equality iff $q = p$ (Gibbs, Theorem 1.8.2).
- Cross-entropy therefore rewards *calibrated* probabilities, punishes overconfidence, and has an irreducible floor $H(p)$ set by the problem.
- Representation learning can be phrased as keeping the information about the output and discarding the rest.

Experiment: calibration at one input

Setup. Suppose the true class distribution at an input is $p = (0.7, 0.3)$.

What we hold fixed. The truth p.

What we change. The reported probabilities q, including $(0.7, 0.3)$, $(0.5, 0.5)$ and $(0.9, 0.1)$.

What we measure. Cross-entropy $H(p, q)$.

What the plot shows. The truthful report reaches the floor $H(p)$; hedging and overconfidence add exactly $\mathrm{D}_{\mathrm{KL}}(p\|q)$.

How to read it. A report can have the correct most-probable class and still pay extra for wrong confidence. Gibbs' inequality is therefore a probability-calibration statement, not merely an accuracy statement.

How the professor asks it

Q. *Why does minimising cross-entropy recover probabilities rather than only the most likely class?*
A. At each input, the expected cross-entropy is $H(p, q) = H(p) + \mathrm{D}_{\mathrm{KL}}(p\|q)$. The first term is fixed by the problem and Gibbs' inequality makes the second non-negative, with equality only at $q = p$. A prediction with the correct argmax but wrong confidence still has positive KL divergence and therefore a larger loss.

Chapter takeaways

Summary: What it means to learn

- **Learning** turns experience into ability. Machines can be given abilities by writing rules or by giving them experience plus a capacity to learn; the second route exists because having an ability does not imply having a describable procedure.
- **Experience** comes as examples, observations or interactions, leading to supervised, unsupervised/self-supervised and reinforcement learning. The paradigm is decided by the feedback the learner receives.
- **Supervised learning** pairs inputs with desired outputs, whose quality limits what can be learned; regression and classification are distinguished by the output; prediction, decision and action are distinct.
- **Representations** decide what the model sees. They can be designed or learned, and self-supervision lets them be learned from unlabelled data; next-token prediction turns this into generation.
- **Four ingredients** describe any learning system: experience, model, learning algorithm, evaluation criterion.
- **Generalization** is the real goal. Finite data never determine the rule; training performance is not evidence of it; shortcuts are the typical failure.
- **Cross-entropy** is minimised by the true probabilities (Gibbs' inequality), which is why it will be our training criterion for classification.

Chapter 2

From Representations to Decisions

This chapter turns the conceptual picture of learning into one mathematical story. Objects are first represented as vectors, and whatever that representation omits is unavailable to every later method. If the conditional distribution of the output were known, a decision loss would tell us the best action at each input: a mean, a median, a quantile, the most probable class or a cost-sensitive threshold. In reality the distribution is unknown, so finite data must stand in for it. We study the most literal local estimate, k-nearest neighbours, then the shared structure of parameterised models and empirical risk minimisation. Cross-entropy connects fitting a probabilistic classifier to maximum likelihood and to calibrated probabilities. Finally, training, validation and test data are given separate jobs so that fitting and choosing do not corrupt the performance number. Formulas marked [on your sheet] appear on the exam sheet.

How to read the formulas in this chapter

This chapter is where the mathematics starts, and the slides can look intimidating. Two things make it manageable. First, there are only a handful of symbols, listed below; every formula in the chapter is built from them. Second, every formula is followed by a *Reading the formula* box that says the same thing in words and then plugs in small numbers. If a formula looks frightening, skip to its box, then come back.

Symbol	Meaning in words
$x \in \mathcal{X} \subseteq \mathbb{R}^d$	one input, written as a list of d numbers (the *features*)
$y \in \mathcal{Y}$	the output we want to predict (a number, or a class label)
X, Y	the input and output regarded as *random*: "a house drawn at random and its price"
$\mathbb{P}(X, Y)$	the joint distribution: how likely each (input, output) pair is in the world
$\mathbb{P}(Y \mid X = x)$	the distribution of the output *once we know* the input equals x
$\mathbb{E}[\cdot \mid X = x]$	an average over that conditional distribution
$a \in \mathcal{A}$	an *action* we might take (a predicted number, a chosen class, "raise the alarm")
$\ell(y, a)$	a *loss*: how much it costs to take action a when the truth is y
$f^*(x)$	the best possible decision rule when the distribution is known (the *Bayes rule*)
$\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^N$	the training data: N examples; superscript (i) = example number
$x_j^{(i)}$	feature number j of example number i
f_θ, $\theta \in \Theta$	a model with adjustable numbers θ (the *parameters*)
$\mathcal{H} = \{f_\theta : \theta \in \Theta\}$	the *hypothesis space*: all the functions the model can represent
$R(\theta)$	the *population risk*: average loss over the whole world (unknowable)
$\widehat{R}(\theta)$	the *empirical risk*: average loss over the examples we have
λ	a *hyperparameter*: a setting chosen by us, not learned (e.g. k in kNN)
$\mathbf{1}$(condition)	the *indicator*: equals 1 if the condition holds, 0 otherwise
$\arg\min_a g(a)$	the value of a that makes $g(a)$ smallest ("the best a")
Δ^{C-1}	the set of probability vectors over C classes (nonnegative, sum to 1)

2.1 From Objects to Numerical Representations

What are we trying to get?

A learning algorithm cannot receive a house, only numbers. We want to know how an object becomes a vector x, which choices are hidden in that step, and why one careless choice (coding categories as integers) can silently break everything that follows.

Describe, then encode

Two steps, both made by humans: choose what to describe, then choose how to turn the description into numbers.

Chapter 1 described learning as acquiring, through experience, the ability to make appropriate decisions in new situations. A mathematical learning problem begins by specifying the information available for those decisions. Properties of the object are first *selected* to describe it and then *encoded* mathematically:

$$\text{object} \longrightarrow \text{features describing it} \longrightarrow \text{representation } x \in \mathbb{R}^d.$$

Definition 2.1.1 (Input space, feature vector)**.** Once an object is described by d numerical components, we write

$$x = [x_1, x_2, \ldots, x_d]^\top \in \mathcal{X} \subseteq \mathbb{R}^d,$$

where $\mathcal{X}$ is the *input space*. Components may be of different kinds: a nor-

malised measurement in $[0, 1]$, a binary feature in $\{0, 1\}$, or a categorical feature spread over several binary components (*one-hot encoding*: a category among K possibilities becomes a vector of length K with a single 1 in the position of the category).

Reading the formula

$x = [x_1, \ldots, x_d]^\top$ is just "a list of d numbers, written as a column." For a house described by area 145 m^2, 4 rooms, age 12 years and neighbourhood A out of $\{A, B, C\}$:

$$x = [\,145,\ 4,\ 12,\ \underbrace{1,\ 0,\ 0}_{\text{neighbourhood } A}\,]^\top \in \mathbb{R}^6.$$

Here $d = 6$: three numerical features and three binary components for one categorical feature. A house in neighbourhood C would end with $0, 0, 1$.

Example 2.1.2 (Why not code neighbourhoods as $1, 2, 3$?)**.** Suppose neighbourhoods A, B, C are coded as a single number $1, 2, 3$. Then, in any method that measures similarity by distance (Section 2.3), a house in A is at distance 1 from a house in B but at distance 2 from one in C: the coding has invented an ordering and a notion of "B is halfway between A and C" that has no meaning. With one-hot coding the three neighbourhoods are pairwise at the same distance $\sqrt{2}$: $\|(1,0,0) - (0,1,0)\| = \sqrt{1+1}$. The encoding step is not a formality; it decides what "similar" means.

The representation controls the available information

A house described only by its area offers less information for predicting price than one described by area, location, age, condition and number of rooms. Additional components help when they expose information relevant to the decision; irrelevant, redundant or unreliable features increase d without making the representation more informative. Images, audio and text make this visible: pixel values provide numbers, but shape, object identity and meaning are not explicitly encoded in them. Traditional machine learning relied on human-designed features; modern methods can learn representations from data (Section 1.5).

Example 2.1.3 (Three descriptions of the same email)**.** Description 1: the length in characters ($d = 1$). Description 2: counts of 10,000 words ($d = 10{,}000$). Description 3: the same counts plus 500 random numbers ($d = 10{,}500$). Description 2 contains everything needed to detect most spam; description 1 does not; description 3 contains the same information as 2 but forces the learner to discover that 500 coordinates are noise, which costs data. More features are not automatically better.

Gotcha

- Integer codes for categories create fake distances (Example 2.1.2). Use one-hot coding unless the categories are genuinely ordered ("small, medium, large").
- Units matter. A feature in metres and the same feature in millimetres are

the same information but behave very differently in any distance-based method (Section 2.3).

Summary: Numerical representations

- Object → chosen features → vector $x \in \mathbb{R}^d$. Both arrows are human choices.
- Categorical features become one-hot blocks; integer codes invent spurious order.
- The representation bounds what any later stage can learn.

Experiment: the same objects under three representations

Setup. Describe the same emails by length alone, by a bag of words, and by words plus sender metadata.

What we hold fixed. The learning algorithm, split and metric.

What we change. Only the representation.

What we measure. Held-out error.

What the plot shows. Error falls when the representation exposes information that predicts the label; irrelevant or unreliable coordinates may instead hurt a distance-based method.

How to read it. The residual error under the richest useful description is a floor for that representation. No downstream model can reconstruct information that was never encoded.

What people do wrong: integer-coding unordered categories

Coding neighbourhoods A, B, C as $1, 2, 3$ feels compact, but it invents an order and fake distances: B becomes halfway between A and C. A distance method and a linear model will use that arithmetic. One-hot encode genuinely unordered categories, and ask of every numeric code whether its differences and ratios have a real meaning.

How the professor asks it

Q. *Why is representation part of the learning problem rather than harmless preprocessing?*
A. It determines the information and geometry presented to the learner. Omitting a relevant property raises the lowest risk any method can attain; inventing a distance through bad encoding changes which observations appear similar. The same model can therefore succeed or fail on the same objects solely because the representation changed.

2.2 Decision Making When the Distribution Is Known

What are we trying to get?

Before worrying about data, we ask an easier question: *if we knew everything about the world*, what would the best decision be? The answer, the Bayes decision rule, is the target that every learning method tries to approximate, and it already tells us that "the best prediction" depends on what mistakes cost.

Uncertainty that remains after seeing the input

Let the random vector X represent the input and the random variable Y the output of interest. Their joint distribution $\mathbb{P}(X, Y)$ describes which pairs may occur and how likely they are. Once the input takes the value x, the conditional distribution $\mathbb{P}(Y \mid X = x)$ summarises the remaining uncertainty about the output. Houses with the same recorded description may still sell at different prices because relevant information is missing, measurements are imperfect, or market conditions vary. The conditional distribution accounts for this instead of forcing one output onto every input.

Even with a perfect description, two houses can sell at different prices. The conditional distribution is the honest way to say "we know a lot but not everything."

Reading the formula

$\mathbb{P}(Y \mid X = x)$ reads "the probabilities of the possible outputs, *given that* the input is x." Concretely, for a 145 m^2 house in neighbourhood A, it might say: price 1.0 million with probability 0.5, 1.2 million with probability 0.3, 1.6 million with probability 0.2. All three prices are possible for that same description. Nothing is being predicted yet; this is a description of the world.

Actions and their consequences

Knowing the possible outputs and their probabilities does not by itself say what to do. That also depends on the consequences of each action.

Definition 2.2.1 (Decision loss)**.** Let $\mathcal{A}$ be the *action space*, the set of choices available to the decision maker. If the true output is y and the action $a \in \mathcal{A}$ is chosen, the *decision loss*

$$\ell_{\text{dec}}(y, a)$$

is the cost assigned to that outcome. In regression, $\mathcal{A}$ may be the set of numerical predictions; in classification, the set of class labels. Keeping a distinct from y is useful because decisions and outcomes need not be the same kind of object.

Reading the formula

$\ell_{\text{dec}}(y, a)$ is a table (or formula) with two inputs: what actually happened (y) and what you did (a). It returns a number: the damage. Examples:

- Squared error, $\ell_{\text{dec}}(y, a) = (y - a)^2$: predicting $a = 200$ when $y = 230$ costs $30^2 = 900$.
- Absolute error, $|y - a|$: the same mistake costs 30.
- Zero–one loss, $\mathbf{1}(y \neq a)$: any wrong class costs 1, a right one costs 0.

- A cost table for a medical decision: rows are the true condition, columns the action, entries the cost. Deciding "discharge" when the truth is "severe" might cost 20; "treat" when "healthy" might cost 3.

The Bayes decision rule

For a given input, different outputs may still occur. We therefore choose the action with the smallest *expected* loss under the conditional distribution.

Definition 2.2.2 (Bayes decision rule)**.** For each input x, the *Bayes decision rule* chooses

$$f^*(x) \in \arg\min_{a \in \mathcal{A}} \ \mathbb{E}\big[\ell_{\text{dec}}(Y, a) \,\big|\, X = x\big].$$

The quantity minimised, $\mathbb{E}[\ell_{\text{dec}}(Y, a) \mid X = x]$, is the *conditional risk* of action a at input x. The word "Bayes" refers to optimal decision making under a probability distribution; it does not require the parameters of any model to have prior distributions.

Reading the formula

Read it in three steps, for one fixed x.

1. List the candidate actions a.
2. For each a, compute the *average damage*: go through every possible output y, multiply its loss $\ell_{\text{dec}}(y, a)$ by its probability $\mathbb{P}(Y = y \mid X = x)$, and add up. That is $\mathbb{E}[\ell_{\text{dec}}(Y, a) \mid X = x]$.
3. Pick the a with the smallest average damage. That is $\arg\min_a$.

With the house above (1.0, 1.2, 1.6 million with probabilities $0.5, 0.3, 0.2$) and squared loss, the average damage of predicting $a = 1.2$ is $0.5(0.2)^2 + 0.3(0)^2 + 0.2(0.4)^2 = 0.02 + 0 + 0.032 = 0.052$. Try $a = 1.16$: $0.5(0.16)^2 + 0.3(0.04)^2 + 0.2(0.44)^2 = 0.0128 + 0.00048 + 0.03872 = 0.052$; the two are equal to three decimals, and Theorem 2.2.4 will tell us the exact best value is $a = 1.18$.

The rule minimises *pointwise*, one x at a time. The next theorem confirms that doing the best thing at every x is the best thing overall.

Theorem 2.2.3 (Pointwise optimality is global optimality)**.** *Let $f : \mathcal{X} \to \mathcal{A}$ be any decision rule and f^* the Bayes rule of Definition 2.2.2 (assume the minimum exists at every x and all expectations are finite). Then*

$$\mathbb{E}\big[\ell_{\text{dec}}(Y, f^*(X))\big] \ \le \ \mathbb{E}\big[\ell_{\text{dec}}(Y, f(X))\big].$$

The left side is called the Bayes risk*; no rule has smaller expected loss.*

Proof. By the tower property of conditional expectation, $\mathbb{E}[\ell_{\text{dec}}(Y, f(X))] = \mathbb{E}\big[\,\mathbb{E}[\ell_{\text{dec}}(Y, f(X)) \mid X]\,\big]$. For every value x, $f(x)$ is one particular action, so by the definition of $f^*(x)$ as a minimiser,

$$\begin{aligned}\mathbb{E}[\ell_{\text{dec}}(Y, f(x)) \mid X = x] \ &\ge \ \min_a \mathbb{E}[\ell_{\text{dec}}(Y, a) \mid X = x] \\ &= \mathbb{E}[\ell_{\text{dec}}(Y, f^*(x)) \mid X = x].\end{aligned}$$

Averaging this inequality over x (taking $\mathbb{E}$ over X) preserves it, giving the claim. □

Key insight

An expectation over pairs (X, Y) can be computed as "first average over Y for each fixed x, then average over x." Because the inner averages are minimised separately at each x, the outer average is minimised too. This is why we may think about one input at a time.

Regression: which number to report

Theorem 2.2.4 (Squared loss selects the conditional mean)**.** *Let $\ell_{\text{dec}}(y, a) = (y - a)^2$ and suppose $\mathbb{E}[Y^2 \mid X = x] < \infty$. Write $m(x) = \mathbb{E}[Y \mid X = x]$. Then for every action a,*

$$\mathbb{E}\big[(Y - a)^2 \,\big|\, X = x\big] = \operatorname{Var}(Y \mid X = x) + \big(m(x) - a\big)^2,$$

so the conditional risk is minimised uniquely at $a = m(x)$: $f^*(x) = \mathbb{E}[Y \mid X = x]$.

Proof. Write $Y - a = (Y - m(x)) + (m(x) - a)$ and square:

$$(Y - a)^2 = (Y - m(x))^2 + 2\,(Y - m(x))(m(x) - a) + (m(x) - a)^2.$$

Take conditional expectations given $X = x$. The first term gives $\operatorname{Var}(Y \mid X = x)$. In the middle term, $(m(x) - a)$ is a constant (it does not involve Y), and $\mathbb{E}[Y - m(x) \mid X = x] = m(x) - m(x) = 0$, so the middle term vanishes. The last term is constant. Hence the formula. The variance does not depend on a, and $(m(x) - a)^2 \geq 0$ with equality only at $a = m(x)$. □

Reading the formula

The formula says: *average squared error = unavoidable spread + your own offset squared.* The spread $\operatorname{Var}(Y \mid X = x)$ is there whatever you do: it is the variability of prices among houses with this description. The only part you control is how far your number a is from the conditional mean $m(x)$.
House example: outcomes $1.0, 1.2, 1.6$ with probabilities $0.5, 0.3, 0.2$. Conditional mean $m = 0.5 + 0.36 + 0.32 = 1.18$. Variance $= 0.5(0.18)^2 + 0.3(0.02)^2 + 0.2(0.42)^2 = 0.0162 + 0.00012 + 0.03528 = 0.0516$. Predicting $a = 1.2$: risk $= 0.0516 + (0.02)^2 = 0.0520$, which is what we computed by hand above. Predicting $a = 1.18$: risk $= 0.0516$, the floor.

Key insight

Add and subtract the mean, then square. The cross term dies because the mean is exactly the number that makes $\mathbb{E}[Y - m] = 0$. This "complete the square around the mean" move is the single most reused computation in statistics.

The loss determines *which summary* of the conditional distribution should be reported.

Proposition 2.2.5 (Absolute loss selects a conditional median)**.** *Let $\ell_{\text{dec}}(y, a) = |y - a|$ and suppose $\mathbb{E}[|Y| \mid X = x] < \infty$. Any conditional median m, i.e. any number with $\mathbb{P}(Y \leq m \mid X = x) \geq \frac{1}{2}$ and $\mathbb{P}(Y \geq m \mid X = x) \geq \frac{1}{2}$, minimises $\mathbb{E}[|Y - a| \mid X = x]$ over a.*

Proof. All probabilities and expectations below are conditional on $X = x$; we drop the conditioning to lighten notation. Let m be a median and $a > m$. For each possible value y, compare the two losses:

- if $y \le m$: $|y-a| - |y-m| = (a-y) - (m-y) = a - m$;
- if $y \ge a$: $|y-a| - |y-m| = (y-a) - (y-m) = -(a-m)$;
- if $m < y < a$: $|y-a| - |y-m| = (a-y) - (y-m) = a + m - 2y > a + m - 2a = -(a-m)$.

In all cases $|y-a| - |y-m| \ge (a-m)\big[\mathbf{1}(y \le m) - \mathbf{1}(y > m)\big]$. Taking expectations,

$$\begin{aligned}\mathbb{E}|Y-a| - \mathbb{E}|Y-m| \;&\ge\; (a-m)\big[\mathbb{P}(Y \le m) - \mathbb{P}(Y > m)\big] \\ &= (a-m)\big[2\mathbb{P}(Y \le m) - 1\big] \;\ge\; 0,\end{aligned}$$

because $\mathbb{P}(Y \le m) \ge \frac{1}{2}$. The case $a < m$ is symmetric, using $\mathbb{P}(Y \ge m) \ge \frac{1}{2}$. So no a beats m. □

Proposition 2.2.6 (Asymmetric absolute loss selects a conditional quantile)**.** *Fix $\tau \in (0,1)$ and let $\ell_\tau(y,a) = \tau(y-a)$ if $y \ge a$ and $(1-\tau)(a-y)$ if $y < a$ (under-prediction costs τ per unit, over-prediction $1-\tau$ per unit). Any conditional τ-quantile m, i.e. any m with $\mathbb{P}(Y \le m \mid x) \ge \tau$ and $\mathbb{P}(Y \ge m \mid x) \ge 1-\tau$, minimises $\mathbb{E}[\ell_\tau(Y,a) \mid x]$.*

Proof. Same three cases as in Proposition 2.2.5, for $a > m$: if $y \le m$ the difference $\ell_\tau(y,a) - \ell_\tau(y,m)$ equals $(1-\tau)(a-m)$; if $y \ge a$ it equals $-\tau(a-m)$; if $m < y < a$ it equals $(1-\tau)(a-y) - \tau(y-m) \ge -\tau(y-m) \ge -\tau(a-m)$. Hence the difference is at least $(a-m)\big[(1-\tau)\mathbf{1}(y \le m) - \tau\mathbf{1}(y > m)\big]$, and taking expectations gives $(a-m)\big[(1-\tau)\mathbb{P}(Y \le m) - \tau(1 - \mathbb{P}(Y \le m))\big] = (a-m)\big[\mathbb{P}(Y \le m) - \tau\big] \ge 0$. The case $a < m$ is symmetric with $\mathbb{P}(Y \ge m) \ge 1-\tau$. Taking $\tau = \frac{1}{2}$ recovers Proposition 2.2.5 up to a factor $\frac{1}{2}$. □

Example 2.2.7 (Three losses, three answers, one distribution)**.** A delivery company must promise a delivery time a (minutes) for a route whose actual time Y is 100 with probability 0.5, 200 with probability 0.3, and 600 with probability 0.2 (occasional road closures).

- *Squared loss.* Report the mean: $0.5(100) + 0.3(200) + 0.2(600) = 230$. Check against $a = 200$: risk at 230 is $0.5(130)^2 + 0.3(30)^2 + 0.2(370)^2 = 8450 + 270 + 27380 = 36100$; risk at 200 is $0.5(100)^2 + 0 + 0.2(400)^2 = 37000$. The mean wins.
- *Absolute loss.* $\mathbb{P}(Y \le 100) = 0.5$ and $\mathbb{P}(Y \ge 100) = 1$; also $\mathbb{P}(Y \le 200) = 0.8$ and $\mathbb{P}(Y \ge 200) = 0.5$. Every $m \in [100, 200]$ is a median, and indeed the expected absolute loss is 130 for every a in that interval ($a = 100$: $0.3(100) + 0.2(500) = 130$; $a = 200$: $0.5(100) + 0.2(400) = 130$), and larger outside (at $a = 230$: $0.5(130) + 0.3(30) + 0.2(370) = 148$). Note that the mean 230 is *not* optimal here: the rare 600 pulls the mean but barely moves the median.
- *Asymmetric loss.* If being late costs 9 times more than being early, take $\tau = 0.9$: we need $\mathbb{P}(Y \le m) \ge 0.9$ and $\mathbb{P}(Y \ge m) \ge 0.1$, which holds exactly for $m = 600$. Promise 600 minutes: the company prefers to disappoint nobody.

"The best prediction" has three different values. Nothing about the world changed; only the consequences did.

Decision loss	Optimal prediction	Typical use
$(y-a)^2$	conditional mean	symmetric errors
$\lvert y-a\rvert$	conditional median	robust point prediction
asymmetric absolute	conditional quantile	unequal under/over cost

Classification: which class to declare

In classification the output belongs to one of C categories, and the conditional distribution gives the probability of each class, $\mathbb{P}(Y=c \mid X=x)$ for $c=1,\dots,C$. These probabilities describe what can be predicted about the unknown class. A class *decision* requires an additional step, so in classification the distinction between prediction and decision is particularly clear:

$$\underbrace{\{\mathbb{P}(Y=c \mid X=x)\}_{c=1}^{C}}_{\text{predicted class probabilities}} \xrightarrow{\text{decision criterion}} \underbrace{f^*(x)}_{\text{class decision}} .$$

Theorem 2.2.8 (Zero–one loss selects the most probable class)**.** *Let $\ell_{\mathrm{dec}}(y,a)=\mathbf{1}(y\neq a)$ (every mistake costs 1, correct decisions cost 0). Then*

$$f^*(x)=\arg\max_{c}\mathbb{P}(Y=c \mid X=x),$$

the Bayes classifier*, and its conditional risk is $1-\max_c \mathbb{P}(Y=c \mid X=x)$.*

Proof. For action a, the conditional risk is $\mathbb{E}[\mathbf{1}(Y\neq a) \mid X=x]=\mathbb{P}(Y\neq a \mid X=x)=1-\mathbb{P}(Y=a \mid X=x)$. Minimising $1-\mathbb{P}(Y=a \mid X=x)$ over a is the same as maximising $\mathbb{P}(Y=a \mid X=x)$. □

Reading the formula

$\mathbb{E}[\mathbf{1}(Y\neq a) \mid X=x]$ is "the probability of being wrong if I say a." With class probabilities $(0.5, 0.3, 0.2)$, saying class 1 is wrong with probability 0.5, class 2 with probability 0.7, class 3 with probability 0.8. Say class 1. The expected error rate is 0.5, and no decision does better; that residual is the problem's own uncertainty.

Choosing the most probable class is optimal *only when all mistakes cost the same*. When they do not, Bayes decision theory still applies: choose the action with the smallest conditional expected cost, which need not be the most probable class.

Theorem 2.2.9 (Cost-sensitive threshold for a binary decision)**.** *Consider a binary problem in which class 1 is an event to detect, $p(x)=\mathbb{P}(Y=1 \mid X=x)$, correct decisions cost 0, a false positive (declaring 1 when the truth is 0) costs $C_{\mathrm{FP}}>0$ and a false negative (declaring 0 when the truth is 1) costs $C_{\mathrm{FN}}>0$. The Bayes rule declares positive if and only if*

$$p(x) > \frac{C_{\mathrm{FP}}}{C_{\mathrm{FP}}+C_{\mathrm{FN}}}$$

(with either decision optimal at equality). Equal costs give the threshold $\frac{1}{2}$.

Proof. Deciding "positive" is wrong exactly when $Y = 0$, which has probability $1 - p(x)$, so its conditional expected cost is $C_{\mathrm{FP}}\,[1 - p(x)]$. Deciding "negative" is wrong exactly when $Y = 1$, so its cost is $C_{\mathrm{FN}}\,p(x)$. Positive is better when

$$\begin{aligned} C_{\mathrm{FP}}(1 - p(x)) < C_{\mathrm{FN}}\,p(x) \quad &\Longleftrightarrow \quad C_{\mathrm{FP}} < (C_{\mathrm{FP}} + C_{\mathrm{FN}})\,p(x) \\ &\Longleftrightarrow \quad p(x) > \frac{C_{\mathrm{FP}}}{C_{\mathrm{FP}} + C_{\mathrm{FN}}}. \end{aligned}$$

With $C_{\mathrm{FP}} = C_{\mathrm{FN}}$ the right side is $\frac{1}{2}$. □

Reading the formula

Two numbers to compare, nothing more.

- Cost of raising the alarm $= C_{\mathrm{FP}} \times \mathbb{P}(\text{no event}) = C_{\mathrm{FP}}\,(1 - p)$.
- Cost of staying quiet $= C_{\mathrm{FN}} \times \mathbb{P}(\text{event}) = C_{\mathrm{FN}}\,p$.

Cyberattack detection with $C_{\mathrm{FN}} = 10$ (missed intrusion) and $C_{\mathrm{FP}} = 1$ (an hour of investigation): threshold $1/(1 + 10) \approx 0.09$. At $p(x) = 0.2$: alarm costs $1 \times 0.8 = 0.8$, silence costs $10 \times 0.2 = 2$. Raise the alarm, even though an attack is *unlikely*. Expensive false negatives push the threshold down; expensive false positives push it up.

Example 2.2.10 (Where the same probability gives different decisions)**.**

- *Cyberattack detection.* Missed attack: prolonged intrusion. False alarm: investigation time. Threshold well below 0.5; accept many false alarms.
- *Card fraud with amount-dependent cost.* Blocking a legitimate payment costs a fixed $C_{\mathrm{FP}} = 5$ (customer annoyance). Missing fraud costs the amount A of the transaction: $C_{\mathrm{FN}} = A$. The threshold is $5/(5 + A)$. For a 50 MAD purchase, block if $p > 0.09$; for 1000 MAD, block if $p > 0.005$. The threshold depends on x through the amount: the decision criterion is more elaborate than one fixed number.
- *Court trial.* Wrongful conviction (false positive) is usually treated as far worse than wrongful acquittal. Threshold far above 0.5: "beyond reasonable doubt."
- *Medical screening.* A missed disease is costly, an unnecessary follow-up test is cheap. Threshold below 0.5.

In each case the probability $p(x)$ is the same kind of object; only the loss differs. Probability estimation and decision policy are separate design choices.

Example 2.2.11 (Three classes and a cost table: the most probable class loses)**.** A triage system estimates that a patient is healthy, mildly ill or severely ill with probabilities $(0.5, 0.3, 0.2)$. Three actions: discharge, monitor, treat. Costs (rows: truth; columns: action):

	discharge	monitor	treat
healthy	0	1	3
mild	4	0	2
severe	20	6	0

Conditional expected costs: discharge $= 0.5(0) + 0.3(4) + 0.2(20) = 5.2$; monitor $= 0.5(1) + 0.3(0) + 0.2(6) = 1.7$; treat $= 0.5(3) + 0.3(2) + 0.2(0) = 2.1$. The

Bayes decision is *monitor*, although "healthy" is the most probable state and would be the zero–one answer. The 20 in the corner (discharging a severe case) dominates the calculation.

Technique: Solving a Bayes decision problem by hand

1. Write the conditional probabilities of the outcomes at the given x as a row.
2. Write the loss table: one row per outcome, one column per action.
3. For each column, take the dot product with the probability row. That is the conditional risk of the action.
4. Pick the smallest column. For regression with a formula loss, use the shortcuts: squared $\to$ mean, absolute $\to$ median, τ-asymmetric $\to$ τ-quantile.
5. For binary detection, skip the table: compare $C_{\mathrm{FP}}(1-p)$ with $C_{\mathrm{FN}}\,p$, or use the threshold $C_{\mathrm{FP}}/(C_{\mathrm{FP}}+C_{\mathrm{FN}})$.

Classic problems

- $Y \mid x$ *is* 0 *with probability* 0.6 *and* 10 *with probability* 0.4. *Best prediction under squared loss? Under absolute loss?* Mean 4; median 0 (since $\mathbb{P}(Y \le 0) = 0.6 \ge \frac{1}{2}$ and $\mathbb{P}(Y \ge 0) = 1$).
- $C_{\mathrm{FN}} = 4\,C_{\mathrm{FP}}$. *Threshold?* $1/(1+4) = 0.2$.
- *Bayes error rate with class probabilities* $(0.45, 0.35, 0.20)$*?* $1 - 0.45 = 0.55$: no classifier can do better at this x.
- *When is the zero–one Bayes classifier also optimal under unequal costs?* Whenever the probabilities are so lopsided that the ranking of conditional risks does not change, e.g. $p(x) = 0.999$ for any moderate cost ratio. In general it is not.

Gotcha

- "Predict the most likely value" is the Bayes rule for zero–one loss only. Under squared loss the mean may be a value that never occurs (230 minutes in Example 2.2.7).
- The threshold formula has C_{FP} on top. Students often invert it: remember that expensive false negatives (C_{FN} large) must give a *small* threshold.
- The Bayes rule needs the true $\mathbb{P}(Y \mid X = x)$. In practice we only have estimates; Section 2.3 onward is about getting them.

Summary: Decisions with a known distribution

- Probabilities describe uncertainty; the loss describes consequences; together they determine the optimal action via $f^*(x) = \arg\min_a \mathbb{E}[\ell_{\mathrm{dec}}(Y, a) \mid X = x]$.

- Squared loss → conditional mean (Theorem 2.2.4); absolute loss → median; asymmetric → quantile.
- Zero–one loss → most probable class (Theorem 2.2.8); unequal costs → threshold $C_{\mathrm{FP}}/(C_{\mathrm{FP}}+C_{\mathrm{FN}})$ (Theorem 2.2.9), or a cost-table comparison in general.
- "Best prediction" has no meaning until the loss is specified.

Telling them apart: decision loss versus training loss

The decision loss is what the application pays when action a meets truth y; it determines which action is optimal once probabilities are available. The training loss is the computable objective used to fit parameters from data. Zero–one error may be the real decision loss and still be a poor training loss because it is flat in the parameters. One probability model can serve several policies by changing only the decision loss or threshold.

Experiment: the threshold-versus-cost curve

Setup. Use calibrated probabilities on 10,000 validation examples and fix $C_{\mathrm{FP}} = 1$.

What we hold fixed. The fitted model and validation rows.

What we change. $C_{\mathrm{FN}} \in \{1, 3, 10, 30, 100\}$ and sweep the threshold.

What we measure. Total decision cost.

What the plot shows. Each run is U-shaped and its minimum moves to $0.5, 0.25, 0.09, 0.03, 0.01$, the values of $C_{\mathrm{FP}}/(C_{\mathrm{FP}} + C_{\mathrm{FN}})$ [on your sheet].

How to read it. Expensive misses lower the threshold and create more alarms. If the empirical minimum is far from the formula, the reported probabilities are not calibrated or the cost model is wrong.

What people do wrong: always reporting the mean or the most probable class

Those answers feel universal because they are familiar. They are optimal only for particular losses: squared loss selects the conditional mean, absolute loss a median, asymmetric absolute loss a quantile, and zero–one loss the most probable class. The same distribution can therefore have several honest optimal reports. Start from the loss, not from a favourite summary.

How the professor asks it

Q. *Why does minimising conditional risk separately at every x minimise total risk?*

A. Total risk is the expectation over X of the conditional risk at X. The Bayes rule chooses, at each input, an action whose conditional risk is no larger than that of any competing rule at that same input. Averaging these pointwise inequalities over the input distribution preserves the inequality, so pointwise optimality is global optimality.

Q. *A missed attack costs ten times a false alarm. Where is the binary thresh-*

old?
A. With $C_{\mathrm{FP}} = 1$ and $C_{\mathrm{FN}} = 10$, declare positive when $p > 1/(1+10) = 1/11 \approx 0.09$. Making misses expensive must lower the threshold, not raise it. At $p = 0.2$, alarming costs 0.8 in expectation and staying silent costs 2, so the apparently unlikely attack is still acted on.

2.3 From Known Distributions to Finite Data

What are we trying to get?

We never know $\mathbb{P}(X, Y)$. What we have is a finite pile of examples. We want the most direct possible way to imitate the Bayes rule from examples (look at nearby ones), to see exactly what "nearby" depends on, and to understand why this direct approach breaks down when inputs have many features.

The training sample

Definition 2.3.1 (Training dataset, i.i.d. assumption)**.** A *training dataset* is a finite collection

$$\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$$

of N realised input–output pairs. Superscripts index examples, subscripts index components: $x_j^{(i)}$ is feature j of example i. In the standard supervised setting, the pairs are modelled as *independent and identically distributed* (i.i.d.) draws from $\mathbb{P}(X, Y)$: one draw does not alter another, and all follow the same distribution.

Reading the formula

$\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$ is a table with N rows. Row i holds the d features of example i and its label. With $N = 3$ houses and $d = 2$ features (area, rooms):

i	$x_1^{(i)}$ (area)	$x_2^{(i)}$ (rooms)	$y^{(i)}$ (price)
1	145	4	1.2
2	80	2	0.7
3	200	5	1.9

$x_1^{(2)} = 80$: the area of the second house. The i.i.d. assumption says the three rows are like three independent tickets drawn from the same urn. It can fail: consecutive days of a time series are dependent; houses in one street are dependent; and if the market changes, future draws come from a different urn (*distribution shift*).

The distinction is fundamental:

True distribution $\mathbb{P}(X, Y)$	Training data $\mathcal{D}$
unknown	observed finite sample

If the distribution were known, Definition 2.2.2 would give the optimal rule. Since we have only the sample, the task of learning is to use it to construct a rule that approximates what we would want under the unknown distribution. This move from a known distribution to finite observations is *the* central statistical step in supervised learning.

Estimating from nearby examples

To estimate the average price of houses "like this one," average the prices of the training houses most like this one.

The Bayes rules suggest a direct strategy. For regression we need the conditional mean at x. If many training examples had exactly the input x, we would average their outputs. Exact repetitions are rare when inputs contain continuous features, so we use examples whose representations are *close* to x.

Definition 2.3.2 (k-nearest neighbours)**.** Let $N_k(x)$ be the set of indices of the k training inputs nearest to x (for a chosen distance). The *kNN regression estimate* and the *kNN class-probability estimate* are

$$\widehat{f}_k(x) = \frac{1}{k} \sum_{i \in N_k(x)} y^{(i)}, \qquad \widehat{p}_k(Y = c \mid X = x) = \frac{1}{k} \sum_{i \in N_k(x)} \mathbf{1}(y^{(i)} = c).$$

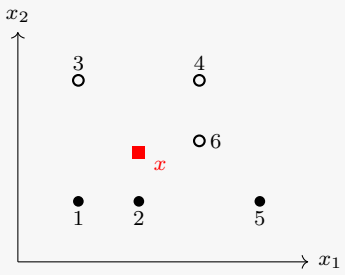
Under zero–one loss, the kNN classifier chooses $\arg\max_c \widehat{p}_k(Y = c \mid X = x)$: the *majority vote* among the neighbours.

Reading the formula

$\sum_{i \in N_k(x)}$ means "add up over the k neighbours." $\frac{1}{k}\sum y^{(i)}$ is their average label. $\frac{1}{k}\sum \mathbf{1}(y^{(i)} = c)$ counts how many neighbours have class c and divides by k: the *fraction* of neighbours in class c. If $k = 5$ and three neighbours are class B, then $\widehat{p}_5(Y = \text{B} \mid x) = 3/5$, and majority vote says B.

Example 2.3.3 (kNN by hand)**.** Six training points in the plane, each with a class and a numerical value:

i	$x_1^{(i)}$	$x_2^{(i)}$	class	$y^{(i)}$
1	1	1	A	10
2	2	1	A	12
3	1	3	B	20
4	3	3	B	22
5	4	1	A	14
6	3	2	B	18



(filled: class A; open: class B.) Query $x = (2, 1.8)$. Squared Euclidean distances: to point 1: $1 + 0.64 = 1.64$; point 2: $0 + 0.64 = 0.64$; point 3: $1 + 1.44 = 2.44$; point 4: $1 + 1.44 = 2.44$; point 5: $4 + 0.64 = 4.64$; point 6: $1 + 0.04 = 1.04$. Order of closeness: $2, 6, 1, \{3, 4\}, 5$.

- $k = 1$: neighbour $\{2\}$. Class A; regression 12.
- $k = 3$: neighbours $\{2, 6, 1\}$, classes A, B, A. Estimated $\widehat{p}(\text{A}) = 2/3$: class A; regression $(12 + 18 + 10)/3 = 13.3$.
- $k = 5$: neighbours $\{2, 6, 1, 3, 4\}$, classes A, B, A, B, B. $\widehat{p}(\text{B}) = 3/5$: class B; regression $(12 + 18 + 10 + 20 + 22)/5 = 16.4$.

The decision flips from A to B as k grows: small k listens to the single closest point, large k averages over a wider region.

The meaning of "nearest" depends on the representation and on the distance used. The neighbourhood size k controls how local the estimate is: with small k the prediction follows individual observations and is sensitive to noise; with large k it averages over a broad region and can erase genuine local variation. k

is a *hyperparameter*: it is chosen as part of model development (Section 2.6), not estimated by the averaging rule itself. Geometrically, for $k = 1$ the input space is partitioned into *Voronoi cells*: the cell of training point $x^{(i)}$ is the set of points closer to $x^{(i)}$ than to any other training point, and every query in that cell receives the label $y^{(i)}$.

Distance defines the neighbourhood

Definition 2.3.4 (Minkowski, Euclidean, Manhattan, Hamming distances)**.** For $x, z \in \mathbb{R}^d$ and $p \geq 1$, the *Minkowski distance* is

$$d_p(x, z) = \Big(\sum_{j=1}^{d} |x_j - z_j|^p \Big)^{1/p}.$$

$p = 1$ gives the *Manhattan* distance (sum of coordinate gaps), $p = 2$ the *Euclidean* distance (straight-line length), and $p \to \infty$ the largest coordinate gap. For binary vectors, the *Hamming* distance counts the positions where they differ. A *scaled, weighted* Euclidean distance is

$$d_w(x, z) = \sqrt{\sum_{j=1}^{d} w_j \Big(\frac{x_j - z_j}{s_j} \Big)^2},$$

where s_j is a scale (units, spread) and $w_j \geq 0$ a weight ($w_j = 0$ ignores feature j).

Reading the formula

For $x = (1, 2)$ and $z = (3, 5)$: coordinate gaps 2 and 3. Manhattan $d_1 = 2 + 3 = 5$ (walk along the streets). Euclidean $d_2 = \sqrt{2^2 + 3^2} = \sqrt{13} \approx 3.61$ (as the crow flies). Largest gap $d_\infty = 3$. Hamming distance between 10110 and 11100: positions 2 and 4 differ, so 2. In d_w, dividing by s_j converts each feature to "how many typical spreads apart," and w_j says how much that feature should count.

Example 2.3.5 (Scaling decides who the neighbours are)**.** Two customers, described by (income in USD, age in years): $P = (30{,}000, 25)$ and $Q = (31{,}000, 60)$. Query: $(30{,}800, 25)$, a 25-year-old earning 30,800. Raw Euclidean distances: to P: $\sqrt{800^2 + 0^2} = 800$; to Q: $\sqrt{200^2 + 35^2} = \sqrt{41{,}225} \approx 203$. The nearest neighbour is Q, a 60-year-old, because income differences of a few hundred dollars dwarf age differences of decades *in these units*. Standardise with typical spreads $s_{\text{income}} = 1000$, $s_{\text{age}} = 10$: $P \to (30.0, 2.5)$, $Q \to (31.0, 6.0)$, query $\to (30.8, 2.5)$. Now to P: 0.8; to Q: $\sqrt{0.04 + 12.25} \approx 3.5$. The nearest neighbour is P, the customer of the same age. Nothing about the customers changed; the choice of units silently chose the answer. Standardising each feature, $z_j = (x_j - \mu_j)/\sigma_j$ with μ_j, σ_j its mean and standard deviation on the training data, is the usual remedy.

Gotcha

Compute μ_j and σ_j on the *training* data only and reuse them for new inputs. Recomputing them on the test data leaks information and makes evaluation optimistic (Section 2.6).

The curse of dimensionality

In high dimension, "nearby" points do not exist: everything is far from everything.

kNN needs enough observations near the query. Two ways to see that high-dimensional inputs defeat this.

Proposition 2.3.6 (Grid count)**.** *Covering $[0,1]^d$ with a grid of 10 positions per coordinate requires 10^d locations. Going from $d = 2$ to $d = 10$ raises this from 10^2 to 10^{10}.*

Proof. Each of the d coordinates is chosen independently among 10 positions, giving $10 \times \cdots \times 10 = 10^d$ combinations. □

Proposition 2.3.7 (Probability of finding a neighbour)**.** *Let n points be uniformly distributed in the unit ball of $\mathbb{R}^d$ and let the query sit at its centre. Call a point a "true neighbour" if its distance to the query is below 0.1. The probability that a given point is a true neighbour is $p_d = (0.1)^d = 10^{-d}$, and the probability that at least one of the n points is a true neighbour is*

$$\mathbb{P}(\textit{at least one neighbour}) = 1 - \left(1 - 10^{-d}\right)^n.$$

Proof. A uniform point lands in a region with probability equal to the region's volume divided by the ball's volume. The ball of radius r in $\mathbb{R}^d$ has volume $V_d(r) = r^d V_d(1)$ (scaling every coordinate by r scales d-dimensional volume by r^d), so $p_d = V_d(0.1)/V_d(1) = 0.1^d$. The n points are independent, so the probability that *none* is a neighbour is $(1 - p_d)^n$, and the complement gives the claim. □

Reading the formula

With $n = 1000$ points:

d	1	2	3	4	5	6	10
$\mathbb{P}(\geq 1$ neighbour$)$	$\approx 100\%$	99.996%	63.2%	9.5%	1.0%	0.1%	$10^{-5}\,\%$

For $d = 3$: $1-(1-0.001)^{1000} \approx 1-e^{-1} \approx 0.632$. For $d = 4$: $1-(1-10^{-4})^{1000} \approx 1 - e^{-0.1} \approx 0.095$. With a thousand data points and ten features, the chance that *any* training point is genuinely close to the query is one in ten million. The "neighbours" kNN uses are then far away, and the local averaging idea loses its justification.

This does not mean every high-dimensional problem is hopeless. It shows that *local* estimation cannot exploit high-dimensional inputs efficiently without additional structure, and motivates a different strategy: describe how predictions vary across the input space through a *shared* model and learn its adjustable quantities from all training examples at once (Section 2.4).

Computation

kNN is a *lazy* method: nothing is computed at training time; all the work happens at prediction time. The basic implementation stores all N inputs, memory $O(Nd)$, and computes N distances per query, each costing $O(d)$, for $O(Nd)$ per prediction. Tree structures help in low dimension; approximate nearest-neighbour search and batched distance computations help at scale.

Technique: kNN in practice

- Standardise features first (Example 2.3.5); one-hot categorical features (Example 2.1.2).
- Choose k on validation data (Section 2.6); odd k avoids ties in binary classification.
- For a hand computation, use *squared* Euclidean distances: same ordering, no square roots.
- Distrust kNN when d is large relative to N (Proposition 2.3.7).

Classic problems

- *In Example 2.3.3, what does $k = 6$ predict?* All points: $\widehat{p}(\mathrm{A}) = 3/6$: a tie; regression $(10 + 12 + 20 + 22 + 14 + 18)/6 = 16$. With $k = N$ every query gets the same answer: the global average.
- $n = 1000$, $d = 3$, *radius* 0.1*: expected number of true neighbours?* $np_d = 1000 \times 10^{-3} = 1$.
- *What is the* 1*-NN training error?* Zero (each training point is its own nearest neighbour), which says nothing about generalization; recall the memoriser of Chapter 1.

Summary: From distribution to data

- Data $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$, modelled as i.i.d. draws from the unknown $\mathbb{P}(X, Y)$.
- kNN imitates the Bayes rule locally: average (regression) or vote (classification) over the k nearest training points.
- "Nearest" depends on the distance and on feature scaling; standardise.
- Curse of dimensionality: $\mathbb{P}(\text{a neighbour within } 0.1) = 1 - (1 - 10^{-d})^n$ collapses as d grows; local methods need shared structure to survive in high dimension.

Experiment: the k validation curve

Setup. Fix a split, representation and distance for kNN.

What we hold fixed. Everything except the neighbourhood size.

What we change. k from 1 to N.

What we measure. Training and validation error.

What the plot shows. Training error begins at zero for $k = 1$ and generally rises; validation error falls from a high-variance left side, reaches a minimum, then rises toward the global-majority rule at $k = N$.

How to read it. Training error would always choose the most local, highest-capacity rule. Validation identifies the useful compromise, and with more data the optimum can move to a larger k because more neighbours remain genuinely local.

Experiment: units choose the nearest neighbour

Setup. Compare $P = (30{,}000, 25)$ and $Q = (31{,}000, 60)$, income and age, to query $(30{,}800, 25)$.

What we hold fixed. The three people and Euclidean distance.

What we change. Raw units versus scales 1000 for income and 10 for age.

What we measure. The nearest point.

What the plot shows. Raw distances choose Q because the income coordinate dominates; scaled distances choose P because the age difference is revealed.

How to read it. The distance is the inductive bias of kNN. Fit scaling constants on training rows only; using all rows leaks test information.

What people do wrong: calling a distant point a neighbour in high dimension

For $n = 1000$ uniform points, the expected number within radius 0.1 is $n10^{-d}$: one at $d = 3$, 0.01 at $d = 5$, and 10^{-7} at $d = 10$. In high dimension a neighbourhood must expand toward the edge of the space merely to contain data, so the estimate is no longer local. Select or learn structure before applying a local method.

How the professor asks it

Q. *Why is $k = 1$ training error usually zero, and why does that not justify $k = 1$?*

A. Each training point is its own nearest neighbour, so its stored label is returned. That is interpolation, not evidence about a new point. Small k gives low bias and high variance; the valid choice comes from held-out validation or cross-validation after scaling and all other preprocessing have been fitted inside the training part.

2.4 Parameterized Models

What are we trying to get?

Since finite data cannot tell us what to do at every input, we restrict ourselves in advance to a *family* of candidate rules and let the data pick one member. We want to see what such a family looks like, what its "knobs" are, and which knobs are learned versus chosen by us.

Definition 2.4.1 (Parameterized model, hypothesis space)**.** A *parameterized model* is a function $f_\theta(x)$ whose behaviour is controlled by a vector of adjustable numbers $\theta \in \Theta$, the *parameters*. The set of functions the model can represent,

$$\mathcal{H} = \{f_\theta : \theta \in \Theta\},$$

is the *hypothesis space*. The model *form* determines how parameters are shared across inputs; θ selects one member of the family. Choosing $\mathcal{H}$ introduces structure before any data are seen: an *inductive bias*.

Reading the formula

Think of f_θ as a machine with dials. θ is the dial settings; $\mathcal{H}$ is everything the machine can do across all settings. *Linear regression*: $f_\theta(x) = w^\top x + b$ with $\theta = (w, b)$. For $w = (2, -1)$, $b = 3$, and input $x = (1, 4)$: $f_\theta(x) = 2 \cdot 1 + (-1) \cdot 4 + 3 = 1$. Turning the dials to $w = (0, 1)$, $b = 0$ gives a different linear function, $f(x) = x_2$. Every function in $\mathcal{H}$ is a flat surface (a line in one dimension, a plane in two); no dial setting can produce a curve. That restriction is the inductive bias of linear regression.

Example 2.4.2 (Parameters versus model form)**.** A polynomial model in one variable, $f_\theta(x) = \theta_0 + \theta_1 x + \cdots + \theta_p x^p$, makes the distinction clear. Changing the coefficients $\theta_0, \ldots, \theta_p$ selects another polynomial *of the same degree*: same form, different parameters. Changing the degree p changes the *family* being considered: with $p = 1$ only lines are available, with $p = 2$ parabolas as well. The coefficients are learned; the degree is a decision made before learning.

Definition 2.4.3 (Nested hypothesis spaces)**.** Hypothesis spaces $\mathcal{H}_1 \subseteq \mathcal{H}_2$ are *nested* when every function of the first is also available in the second. Polynomials of degree $\leq p$ are nested in polynomials of degree $\leq p + 1$ (set $\theta_{p+1} = 0$).

Classification models output probabilities

Definition 2.4.4 (Probability simplex, probabilistic classifier)**.** For C-class classification, a model estimates the vector of conditional class probabilities

$$\widehat{p}_\theta(x) = \left[\widehat{p}_\theta(Y = 1 \mid x),\ \ldots,\ \widehat{p}_\theta(Y = C \mid x)\right]^\top \in \Delta^{C-1},$$

where the *probability simplex* is

$$\Delta^{C-1} = \Big\{p \in \mathbb{R}^C : p_c \geq 0,\ \sum_{c=1}^{C} p_c = 1\Big\}.$$

A decision rule (Section 2.2) then converts this vector into an action.

Reading the formula

Δ^{C-1} is just "all lists of C nonnegative numbers adding to 1," i.e. all legitimate probability assignments over C classes. For $C = 3$: $(0.5, 0.3, 0.2)$ is in it; $(0.5, 0.5, 0.5)$ is not (sums to 1.5); $(1.2, -0.2, 0)$ is not (negative entry). A

common way to build such a model is to compute C real-valued *scores* $s_{\theta,c}(x)$ and pass them through the *softmax* map $p_c = e^{s_c}/\sum_{c'} e^{s_{c'}}$: scores $(2, 0, -1)$ become $(7.39, 1, 0.37)/8.76 \approx (0.84, 0.11, 0.04)$, automatically nonnegative and summing to one.

Parameters and hyperparameters

Definition 2.4.5 (Parameters, hyperparameters)**.** *Parameters* are quantities learned from the training examples. *Hyperparameters* are choices made around that estimation: some define the hypothesis space (polynomial degree, number of layers), others control the learning procedure (number of neighbours k, step sizes).

$$\text{model form} + \text{hyperparameters} \;\to\; \mathcal{H} \;\to\; \theta \;\to\; \text{particular model.}$$

Model	Parameters (learned)	Hyperparameters (chosen)
kNN	none	k, the distance, feature scales
Linear regression	w, b	which features to include
Polynomial regression	coefficients $\theta_0, \ldots, \theta_p$	degree p
Neural network	all weights	layers, width, learning rate

Gotcha

kNN has hyperparameters but *no* parameters: nothing is fitted, the data themselves are the model. That is why it is called lazy, and why k cannot be "learned" by the averaging rule: with $k = 1$ the training error is always zero, so training data alone would always choose $k = 1$.

Summary: Parameterized models

- f_θ: a function with adjustable parameters; $\mathcal{H}$: all functions it can produce. Choosing $\mathcal{H}$ is choosing an inductive bias.
- Linear and polynomial regression; probabilistic classifiers output a vector in the simplex Δ^{C-1}.
- Parameters are learned; hyperparameters are chosen and define the family or the procedure.

Telling them apart: kNN versus a parameterised model

kNN performs almost no fitting and stores the sample; prediction depends on finding nearby stored examples and costs roughly $O(Nd)$ per query. A parameterised model compresses the rows into θ and shares that structure across the input space. That sharing can survive high dimension, but it introduces assumptions and approximation error. Parameters are fitted inside a fixed family; hyperparameters choose the family or fitting regime.

How the professor asks it

Q. *Distinguish a parameter from a hyperparameter using kNN and a polynomial model.*

A. Polynomial coefficients are parameters because the training algorithm estimates them within the chosen degree. The degree is a hyperparameter because it selects the hypothesis family and must be compared on validation evidence. In kNN there are no fitted coefficients, but k, the distance and feature scales are hyperparameters. Training error cannot choose them honestly because increased capacity can only improve the fit to the fitted rows.

2.5 Learning the Parameters from Data

What are we trying to get?

We have a family of candidate rules and a pile of examples. We want a principle that says which parameter values the data prefer, and we want to understand exactly what that principle can and cannot promise: it finds the best fit to the examples, which is not the same as the best rule for the world.

Two losses, two jobs

Learning a parameterized model requires a criterion for comparing parameter values. It is useful to distinguish two losses:

ℓ_{train} : guides the learning of the model from data,
ℓ_{dec} : guides the choice of an action from inferred information.

They can coincide, as when squared loss is used both to fit and to assess a regression. They can differ: a classifier may be *trained* with cross-entropy to estimate probabilities and then *decide* with zero–one loss or application costs.

Population risk and empirical risk

Definition 2.5.1 (Population risk, empirical risk, ERM)**.** For parameters θ, the *population risk* is the average training loss over the whole distribution,

$$R(\theta) = \mathbb{E}_{(X,Y)\sim\mathbb{P}}\big[\ell_{\text{train}}\big(Y, f_\theta(X)\big)\big],$$

and the *empirical (training) risk* is the average over the observed examples,

$$\widehat{R}_{\text{train}}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \ell_{\text{train}}\big(y^{(i)}, f_\theta(x^{(i)})\big).$$

Empirical risk minimisation (ERM) chooses parameters that make the training risk as small as possible:

$$\widehat{\theta} \in \arg\min_{\theta\in\Theta} \widehat{R}_{\text{train}}(\theta).$$

The best parameters *in the population sense* are $\theta^*_{\mathcal{H}} \in \arg\min_\theta R(\theta)$; their population risk may exceed the Bayes risk when $\mathcal{H}$ does not contain the Bayes rule.

Reading the formula

Three quantities, in increasing order of how much we can compute them.

- $R(\theta)$: "if I used dial setting θ on *every* house that will ever exist, what would my average loss be?" This is what we care about and can never compute, because it needs $\mathbb{P}$.
- $\widehat{R}_{\text{train}}(\theta)$: "on the N houses in my table, what *was* my average loss?" Computable. With three examples whose squared errors under θ are $1, 4, 0$: $\widehat{R} = (1 + 4 + 0)/3 = 5/3$.
- $\widehat{\theta}$: the dial setting with the smallest $\widehat{R}_{\text{train}}$. The hat says "computed from this particular sample"; another sample would give another $\widehat{\theta}$.

Key insight

ERM replaces an average over the unknown world by an average over the sample. Everything that can go wrong in learning is contained in the gap between R and $\widehat{R}$, and the next two propositions say when that gap is harmless and when it is not.

Proposition 2.5.2 (For a fixed model, training risk is an honest estimate)**.** *Fix θ (chosen without looking at the data). If the examples are i.i.d. from $\mathbb{P}$, then $\mathbb{E}[\widehat{R}_{\text{train}}(\theta)] = R(\theta)$, and by the law of large numbers $\widehat{R}_{\text{train}}(\theta) \to R(\theta)$ as $N \to \infty$.*

Proof. $\widehat{R}_{\text{train}}(\theta)$ is the average of N terms $\ell_{\text{train}}(y^{(i)}, f_\theta(x^{(i)}))$, each an independent copy of the random variable $\ell_{\text{train}}(Y, f_\theta(X))$, whose mean is $R(\theta)$ by definition. The expectation of an average of terms with mean $R(\theta)$ is $R(\theta)$. The convergence statement is the law of large numbers for i.i.d. terms with finite mean. □

Proposition 2.5.3 (After minimisation, training risk is optimistic)**.** *Let $\widehat{\theta}$ be the ERM solution. Then*

$$\mathbb{E}\big[\widehat{R}_{\text{train}}(\widehat{\theta})\big] \;\le\; \min_{\theta} R(\theta) \;\le\; \mathbb{E}\big[R(\widehat{\theta})\big].$$

In words: the training risk of the fitted model underestimates, on average, both the best achievable population risk and the population risk of the fitted model itself.

Proof. Let θ^* minimise R. Since $\widehat{\theta}$ minimises $\widehat{R}_{\text{train}}$, $\widehat{R}_{\text{train}}(\widehat{\theta}) \le \widehat{R}_{\text{train}}(\theta^*)$ for every sample. Taking expectations and using Proposition 2.5.2 for the fixed θ^*: $\mathbb{E}[\widehat{R}_{\text{train}}(\widehat{\theta})] \le \mathbb{E}[\widehat{R}_{\text{train}}(\theta^*)] = R(\theta^*) = \min_\theta R(\theta)$. The second inequality holds because $R(\widehat{\theta}) \ge \min_\theta R(\theta)$ for every sample. □

Reading the formula

The fitted model has been *chosen because* it looks good on these particular examples. Part of its good training score is real, part is luck that will not recur. The proposition quantifies that "the training score is at least as flattering as the truth." A small training risk confirms a good fit to the observed examples; it is only an (optimistic) estimate of performance on unseen ones.

Proposition 2.5.4 (Bigger families fit the training data at least as well)**.** *If $\mathcal{H}_1 \subseteq \mathcal{H}_2$, then $\min_{f\in\mathcal{H}_2} \widehat{R}_{\text{train}}(f) \le \min_{f\in\mathcal{H}_1} \widehat{R}_{\text{train}}(f)$. In particular, increasing the polynomial degree never increases the minimum training risk.*

Proof. The minimum over a larger set is at most the minimum over a subset, because every candidate of $\mathcal{H}_1$ is also a candidate in $\mathcal{H}_2$. □

Proposition 2.5.5 (Enough flexibility fits anything)**.** *Given N examples with distinct inputs $x^{(1)}, \dots, x^{(N)} \in \mathbb{R}$, there is a polynomial of degree at most $N-1$ passing through all N points; its training risk under any loss with $\ell_{\text{train}}(y, y) = 0$ is zero.*

Proof. For each i, the polynomial $L_i(x) = \prod_{j\neq i} \frac{x - x^{(j)}}{x^{(i)} - x^{(j)}}$ has degree $N-1$, equals 1 at $x^{(i)}$ and 0 at every other $x^{(j)}$ (the denominators are nonzero because the inputs are distinct). Then $f(x) = \sum_{i=1}^{N} y^{(i)} L_i(x)$ has degree at most $N-1$ and $f(x^{(i)}) = y^{(i)}$ for all i, so every training loss term is $\ell_{\text{train}}(y^{(i)}, y^{(i)}) = 0$. □

Interpretation & discussion

Propositions 2.5.4 and 2.5.5 together explain why the training risk is a *poor guide for choosing model complexity*: making the family more flexible (higher degree, smaller k in kNN) can only lower the training risk, all the way to zero, even after performance on new examples has started to deteriorate. Parameter estimation and hyperparameter selection therefore need different evidence: training data estimate parameters *inside* each fixed configuration; separate validation data compare the fitted configurations (Section 2.6).

Example 2.5.6 (ERM solved by hand: least squares in one variable)**.** Some objectives have an analytical solution. Fit $f_\theta(x) = wx + b$ to the four points $(0,1), (1,3), (2,2), (3,4)$ under squared loss. The training risk is $\widehat{R}(w,b) = \frac{1}{4}\sum_i (y^{(i)} - wx^{(i)} - b)^2$. Setting both partial derivatives to zero,

$$\frac{\partial \widehat{R}}{\partial b} = -\frac{2}{N}\sum_i (y^{(i)} - wx^{(i)} - b) = 0, \qquad \frac{\partial \widehat{R}}{\partial w} = -\frac{2}{N}\sum_i x^{(i)}(y^{(i)} - wx^{(i)} - b) = 0,$$

gives the *normal equations* $\sum y = w\sum x + Nb$ and $\sum xy = w\sum x^2 + b\sum x$, whose solution is

$$w = \frac{N\sum xy - \sum x \sum y}{N\sum x^2 - (\sum x)^2}, \qquad b = \frac{\sum y - w\sum x}{N}.$$

Here $N = 4$, $\sum x = 6$, $\sum y = 10$, $\sum x^2 = 14$, $\sum xy = 0 + 3 + 4 + 12 = 19$. So $w = (76 - 60)/(56 - 36) = 0.8$ and $b = (10 - 4.8)/4 = 1.3$. Predictions $1.3, 2.1, 2.9, 3.7$; residuals $-0.3, 0.9, -0.9, 0.3$; squared $0.09, 0.81, 0.81, 0.09$; training risk $1.8/4 = 0.45$.

Nested families in action. Degree 0 (a constant): best constant is the mean 2.5, residuals $-1.5, 0.5, -0.5, 1.5$, risk $5/4 = 1.25$. Degree 1: 0.45. Degree 3: by Proposition 2.5.5 a cubic passes through all four points, risk 0. The sequence $1.25, 0.45, \dots, 0$ is non-increasing, exactly as Proposition 2.5.4 says, and it says nothing about which degree predicts a fifth point best.

Learning class probabilities with cross-entropy

Zero–one loss only notices which *class won. Cross-entropy notices* how much *probability the true class got, so it can guide small improvements.*

For classification, the natural decision loss is zero–one, $\ell_{01}(y, a) = \mathbf{1}(y \neq a)$, with decision $a(x) = \arg\max_c \widehat{p}_{\theta,c}(x)$. As a *training* loss it is unsuitable. Direct minimisation would use

$$\widehat{R}_{01}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}\Big(y^{(i)} \neq \arg\max_c s_{\theta,c}(x^{(i)})\Big),$$

which (i) discards changes in confidence, (ii) is piecewise constant in θ, and (iii) has zero gradient wherever it is defined and no gradient where the decision flips.

Example 2.5.7 (Why the zero–one training risk cannot be optimised smoothly)**.** One feature, a threshold classifier $f_\theta(x) = 1$ if $x > \theta$, else 0. Data: $x = 1, 2, 3, 4$ with labels $0, 0, 1, 1$. Then $\widehat{R}_{01}(\theta)$ is $2/4$ for $\theta < 1$, $1/4$ for $1 \le \theta < 2$, 0 for $2 \le \theta < 3$, $1/4$ for $3 \le \theta < 4$, $2/4$ for $\theta \ge 4$: a staircase. Between the steps the slope is exactly 0, so "move θ in the direction that decreases the loss" gets no information; at the steps the function jumps. Any method that improves by small steps is blind. A useful training loss must vary smoothly with the model output and reward better probability estimates.

Definition 2.5.8 (Cross-entropy training loss)**.** Let $\widehat{p}_{\theta,c}(x) = \widehat{p}_\theta(Y = c \mid X = x)$ be the model's probability for class c. For an example $(x^{(i)}, y^{(i)})$, the *cross-entropy loss* is

$$\ell_{\mathrm{CE}}(y^{(i)}, \widehat{p}_\theta(x^{(i)})) = -\log \widehat{p}_{\theta,y^{(i)}}(x^{(i)}),$$

the negative log of the probability assigned to the class that actually occurred. The *empirical cross-entropy* is

$$\widehat{R}_{\mathrm{CE}}(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \log \widehat{p}_{\theta,y^{(i)}}(x^{(i)}), \qquad \widehat{\theta}_{\mathrm{CE}} \in \arg\min_\theta \widehat{R}_{\mathrm{CE}}(\theta).$$

For binary classification, writing $\widehat{p} = \widehat{p}_\theta(Y = 1 \mid x)$ and $y \in \{0, 1\}$, this is $\ell_{\mathrm{BCE}}(y, \widehat{p}) = -y \log \widehat{p} - (1 - y)\log(1 - \widehat{p})$.

Reading the formula

$\widehat{p}_{\theta,y^{(i)}}(x^{(i)})$ is a mouthful for a simple thing: "look up the probability the model gave to the *correct* class of example i." Then take minus its log. Natural logs:

probability given to the true class	0.9	0.6	0.1	0.01
loss $-\ln(\cdot)$	0.105	0.511	2.303	4.605

Three examples with true-class probabilities $0.9, 0.6, 0.1$ give $\widehat{R}_{\mathrm{CE}} = (0.105 + 0.511 + 2.303)/3 \approx 0.973$. The confident mistake (0.1) dominates. The binary formula is the same idea written for $y \in \{0, 1\}$: if $y = 1$ only the term $-\log \widehat{p}$ survives; if $y = 0$ only $-\log(1 - \widehat{p})$, the log-probability of the class that occurred.

Where does this formula come from? Encode the observed label of example i as a *one-hot* vector $t_c^{(i)} = \mathbf{1}(y^{(i)} = c)$: a probability vector that puts all its mass on the observed class. Its cross-entropy with the model's vector, in the sense

of Definition 1.8.1, is $-\sum_c t_c^{(i)} \log \widehat{p}_{\theta,c}(x^{(i)}) = -\log \widehat{p}_{\theta,y^{(i)}}(x^{(i)})$, because only the term $c = y^{(i)}$ survives. The one-hot vector represents the *observed* label, not the unknown true distribution; averaging over many examples is what estimates the population quantity, as the next theorem shows.

Theorem 2.5.9 (Cross-entropy is minimised by the true probabilities)**.** *Let $p_c(x) = \mathbb{P}(Y = c \mid X = x)$ be the true conditional probabilities. At a fixed x, the conditional expected cross-entropy of the model's vector $\widehat{p}_\theta(x)$ is*

$$\mathbb{E}\big[-\log \widehat{p}_{\theta,Y}(x) \,\big|\, X = x\big] = H\big(p(x)\big) + \mathrm{D}_{\mathrm{KL}}\left(p(x) \,\|\, \widehat{p}_\theta(x)\right) \;\geq\; H\big(p(x)\big),$$

with equality iff $\widehat{p}_\theta(x) = p(x)$. Consequently the population cross-entropy risk decomposes as

$$R_{\mathrm{CE}}(\theta) = H(Y \mid X) + \mathbb{E}_X\left[\mathrm{D}_{\mathrm{KL}}\left(p(X) \,\|\, \widehat{p}_\theta(X)\right)\right],$$

where $H(Y \mid X) = \mathbb{E}_X[H(p(X))]$ is the conditional entropy, and R_{CE} is minimised, over all probability models, exactly by models that output the true conditional probabilities.

Proof. At fixed x, Y takes value c with probability $p_c(x)$, so $\mathbb{E}[-\log \widehat{p}_{\theta,Y}(x) \mid X = x] = -\sum_c p_c(x) \log \widehat{p}_{\theta,c}(x) = H(p(x), \widehat{p}_\theta(x))$, the cross-entropy of Definition 1.8.1. By equation (1.1) this equals $H(p(x)) + \mathrm{D}_{\mathrm{KL}}(p(x) \| \widehat{p}_\theta(x))$, and by Gibbs' inequality (Theorem 1.8.2) the divergence is ≥ 0 with equality iff the two vectors coincide. Averaging over X gives the decomposition of R_{CE}; the first term does not depend on θ and the second is ≥ 0 with equality iff $\widehat{p}_\theta(x) = p(x)$ for (almost) every x. □

Reading the formula

Two terms. $H(Y \mid X)$ is the uncertainty about the class that *remains even if you know x*: the problem's own noise, which no model can remove. The divergence term is *your* error: how far your probabilities sit from the true ones, averaged over inputs. Training with cross-entropy pushes only the second term down. Recall the numbers of Example 1.8.3: with truth $(0.7, 0.3)$ the floor is 0.881 bits, a hedged $(0.5, 0.5)$ pays 0.119 extra, an overconfident $(0.9, 0.1)$ pays 0.222 extra.

Key insight

Cross-entropy is the unique combination of two facts: (i) the one-hot cross-entropy of an example is just $-\log$(probability of the observed class), which is computable from data; (ii) its expectation is $H + \mathrm{D}_{\mathrm{KL}}$, which is minimised at the truth (Gibbs). So minimising something we *can* compute drives the model toward something we *want*: calibrated probabilities, usable afterwards by any decision rule of Section 2.2.

Proposition 2.5.10 (Cross-entropy minimisation is maximum likelihood)**.** *Minimising $\widehat{R}_{\mathrm{CE}}(\theta)$ is equivalent to maximising the conditional likelihood of the observed labels, $L(\theta) = \prod_{i=1}^{N} \widehat{p}_{\theta,y^{(i)}}(x^{(i)})$.*

Proof. $\log L(\theta) = \sum_i \log \widehat{p}_{\theta,y^{(i)}}(x^{(i)}) = -N\, \widehat{R}_{\mathrm{CE}}(\theta)$. Since log is increasing and $-N < 0$, maximising L is the same as minimising $\widehat{R}_{\mathrm{CE}}$. □

Interpretation & discussion

Cross-entropy gives a graded assessment: predicting the observed class with probability 0.9 costs less than with 0.6, and a confident wrong prediction costs a lot. It is differentiable for positive probabilities, so with a smooth probability model gradient methods can optimise it (later in the course). Zero–one loss stays in the picture as the *decision* loss: train with cross-entropy, then decide with arg max or with a cost-sensitive threshold. The probabilities are learned once and can serve several decision rules.

Learning objective and learning algorithm

Definition 2.5.1 specifies a *learning objective*: which parameter values are preferred. A *learning algorithm* specifies how suitable values are found: some objectives have closed-form solutions (Example 2.5.6); most are solved approximately by iterative optimisation, producing a sequence $\theta_0, \theta_1, \ldots$. The complete sequence separates learning, inference and decision:

$$\mathcal{D} \xrightarrow{\ell_{\text{train}}} \text{learned model } f_{\widehat{\theta}} \xrightarrow{\text{new } x} \text{inferred quantity} \xrightarrow{\ell_{\text{dec}}} \text{action.}$$

Technique: Working with risks

- To compute an empirical risk by hand: one loss per example, then average. Keep a column of losses; the mistakes are usually in the arithmetic, not the concept.
- Squared-loss ERM for a line: use the normal-equation formulas of Example 2.5.6; check that residuals sum to zero (they must, from $\partial \widehat{R}/\partial b = 0$).
- Cross-entropy by hand: write the probability of the *true* class for each example, take $-\ln$, average. Never take the log of the largest probability by mistake.
- Training risk after fitting is optimistic (Proposition 2.5.3); never report it as the model's performance.

Classic problems

- *A model gives the true class probability* 0.5 *on every example. Empirical cross-entropy?* $-\ln 0.5 = 0.693$ nats, whatever N.
- *Degree-*5 *polynomial fitted to* 6 *points with distinct* x*: training risk?* Zero (Proposition 2.5.5).
- *Can a larger hypothesis space have larger minimum training risk?* No (Proposition 2.5.4). Can it have larger population risk for the fitted model? Yes, and that is the whole problem of overfitting.
- *Binary example,* $y = 0$*, model says* $\widehat{p} = 0.8$*. BCE loss?* $-\ln(1 - 0.8) = 1.609$.

Summary: Learning from data

- Population risk $R(\theta)$ (what we want, unknowable) versus empirical risk $\widehat{R}_{\text{train}}(\theta)$ (what we can compute). ERM minimises the latter.

- Training risk is honest for a fixed model (Proposition 2.5.2) and optimistic after fitting (Proposition 2.5.3); bigger families only lower it (Propositions 2.5.4, 2.5.5).
- Cross-entropy $-\log \widehat{p}_{\theta,y}(x)$ is a smooth training loss whose expectation is $H(Y \mid X) + \mathbb{E}[\mathrm{D_{KL}}]$, minimised by the true probabilities (Theorem 2.5.9); it equals maximum likelihood.
- Objective (which θ is preferred) and algorithm (how to find it) are separate.

Experiment: honest before fitting, optimistic after fitting

Setup. Simulate a known world, choose one line θ_0 in advance, and repeatedly draw samples of size 20.

What we hold fixed. The data-generating distribution and sample size.

What we change. Only the random sample.

What we measure. The training risk of fixed θ_0, the minimum training risk after fitting a line, and the true risk of that fitted line.

What the plot shows. The fixed line's empirical risk is centred on its population risk; the fitted minimum lies systematically below the best population risk, while the fitted model's population risk lies above it.

How to read it. Reusing the same noise to choose and score the model creates optimism. The inequality is about selection, not dishonesty in the arithmetic.

Telling them apart: objective versus algorithm

The learning objective says which parameter values are preferred, such as minimising empirical cross-entropy. The learning algorithm says how a preferred value is found, for example a closed form or gradient descent. A wrong objective can be optimised perfectly and still solve the wrong problem; a weak optimiser can fail to reach a good solution to the right objective.

What people do wrong: training on zero–one loss or expecting cross-entropy to reach zero

Zero–one loss matches classification error but is piecewise constant in the parameters and gives no useful gradient. Cross-entropy is smooth and its expectation is $H(Y \mid X) + \mathbb{E}[\mathrm{D_{KL}}]$ [on your sheet], so its irreducible floor is the uncertainty of the problem, not necessarily zero. A confident wrong probability is punished strongly because it is evidence of miscalibration.

How the professor asks it

Q. *Why is training risk unbiased for a fixed model and optimistic for a fitted one?*
A. For fixed θ, every empirical-loss term has expectation $R(\theta)$, so their average is unbiased. After fitting, $\widehat{\theta}$ was chosen because its empirical risk happened to be small on those same rows. Taking the minimum over noisy estimates

selects favourable noise, giving $\mathbb{E}[\widehat{R}(\widehat{\theta})] \leq \min_\theta R(\theta) \leq \mathbb{E}[R(\widehat{\theta})]$. Independence is restored by evaluation on rows not used for fitting or selection.

Q. *Why does cross-entropy learn probabilities?*
A. At fixed x, its expectation is $H(p(x), \widehat{p}_\theta(x)) = H(p(x)) + \mathrm{D}_{\mathrm{KL}}(p(x) \| \widehat{p}_\theta(x))$. Gibbs' inequality makes the second term non-negative and zero only at the true probability vector. The one-hot sample loss is computable, while its expectation is minimised by the calibrated quantity we want; maximum likelihood gives the same objective.

2.6 Evaluating a Learned Model

What are we trying to get?

Since training risk flatters the fitted model, we need evidence from examples that did *not* shape it. We want a disciplined way to split the data so that parameters, hyperparameters and the final performance estimate each rest on information not already used, and a way to do this when data are scarce.

Three subsets, three roles

Definition 2.6.1 (Training, validation and test sets)**.** The available data are partitioned into three subsets with different roles.

Subset	Main role
Training set $\mathcal{D}_{\text{train}}$	learn model parameters
Validation set $\mathcal{D}_{\text{val}}$	select models and hyperparameters
Test set $\mathcal{D}_{\text{test}}$	evaluate the final model, after all choices are fixed

Their sizes need not be equal; the defining difference is how information from each subset is allowed to influence the learning procedure.

Every subset is informative about new data only until you use it to make a choice. Then it becomes part of "training" in disguise.

The training set enters parameter learning through ℓ_{train}. The validation set compares the fitted models and may select k in kNN, the degree of a polynomial, a representation, or a model family. Reusing validation results for many design decisions gradually adapts the development process to that set, so a third sample is reserved for the final assessment. The test set is used once, after parameters, hyperparameters and evaluation rules are fixed; consulting it repeatedly during development would turn it into another validation set and make its estimate optimistic, by exactly the mechanism of Proposition 2.5.3.

Proposition 2.6.2 (Held-out risk of a fixed model is honest)**.** *If the model f was chosen without using the examples of a held-out set of size M drawn i.i.d. from $\mathbb{P}$, then the held-out empirical risk $\frac{1}{M}\sum_i \ell(y^{(i)}, f(x^{(i)}))$ has expectation equal to the population risk of f.*

Proof. Identical to Proposition 2.5.2: f is fixed relative to the held-out examples, so each term has mean equal to the population risk. □

The catch is the word *fixed*. Once we select, among several candidates, the one with the lowest validation risk, that minimum is optimistic (Proposition 2.5.3

with the validation set playing the role of the training set and the candidates playing the role of Θ). The test set restores honesty because nothing was selected on it.

Selecting hyperparameters

Definition 2.6.3 (Hyperparameter selection by validation)**.** Let λ denote a hyperparameter. For each candidate value, learn parameters on the training set,

$$\widehat{\theta}_\lambda = \arg\min_\theta \widehat{R}_{\text{train}}(\theta; \lambda),$$

then compare the fitted models on the validation set and keep the best,

$$\hat{\lambda} = \arg\min_\lambda \widehat{R}_{\text{val}}(\widehat{\theta}_\lambda; \lambda).$$

Reading the formula

Two nested "pick the best" operations, on two different datasets. *Inner:* for degree $\lambda = 2$, find the best parabola on the training points; for $\lambda = 3$, the best cubic; and so on. *Outer:* evaluate each of those fitted curves on the validation points and keep the degree whose fitted curve did best *there*. Concretely, suppose validation risks come out as

degree λ	0	1	2	3	5
training risk of $\widehat{\theta}_\lambda$	1.25	0.45	0.40	0.10	0.00
validation risk of $\widehat{\theta}_\lambda$	1.30	0.50	0.62	1.90	8.40

Training risk keeps falling (Proposition 2.5.4); validation risk is smallest at $\lambda = 1$. Choose the line, $\hat{\lambda} = 1$. The validation criterion may differ from ℓ_{train} when the application values something else (e.g. cost-weighted errors).

When data are limited: cross-validation

A fixed validation set leaves fewer observations for parameter learning and makes the selection depend on one particular split.

Definition 2.6.4 (K-fold cross-validation)**.** Divide the development data (training plus validation) into K folds of roughly equal size. For each candidate λ and each fold $j = 1, \dots, K$, train on the other $K - 1$ folds and compute the validation risk $\widehat{R}^{(j)}_{\text{val}}(\lambda)$ on fold j. The *cross-validation estimate* is the average

$$\widehat{R}_{\text{CV}}(\lambda) = \frac{1}{K} \sum_{j=1}^{K} \widehat{R}^{(j)}_{\text{val}}(\lambda),$$

and $\hat{\lambda}$ minimises it. Every development example serves as validation data exactly once. After selecting $\hat{\lambda}$, parameters are typically re-learned on all development data. The test set remains untouched until final evaluation. $K = N$ is *leave-one-out* cross-validation.

Reading the formula

Cut the deck into K piles. Round 1: hide pile 1, train on piles $2, \ldots, K$, score on pile 1. Round 2: hide pile 2, and so on. Average the K scores. With $K = 3$ and candidates $k \in \{1, 3\}$ for kNN, suppose the fold scores are

	fold 1	fold 2	fold 3	$\widehat{R}_{\mathrm{CV}}$
$k = 1$	0.40	0.20	0.60	0.40
$k = 3$	0.30	0.30	0.30	0.30

Choose $k = 3$. Notice that a single split using fold 2 alone would have chosen $k = 1$; averaging over folds protects against such accidents.

Technique: Splitting data without fooling yourself

- Decide the split *before* looking at any result. Typical: 60/20/20 or, with cross-validation, 80/20 development/test.
- Anything computed from data (feature scaling constants, vocabulary, imputed values) is computed on the training portion and *applied* to the others. Otherwise information leaks.
- Time series: split by time (train on the past, validate on the future); random splits leak the future into the past.
- Grouped data (several images of the same patient): keep a group entirely in one subset; otherwise the model is tested on patients it has seen.
- Touch the test set once. If you go back and change something afterwards, you need a new test set.

Gotcha

- "I chose k by minimising the validation error, so the validation error is my performance estimate." No: that minimum is optimistic (Proposition 2.5.3). Report the test error.
- Cross-validation selects hyperparameters; it does not by itself produce a final model. Refit on all development data with $\hat{\lambda}$.
- A validation set with 10 examples cannot distinguish a 12% error rate from an 18% one. Small held-out sets give noisy verdicts.

Summary: Evaluation

- Training data learn θ; validation data choose λ; test data estimate final performance once.
- Held-out risk is honest for a fixed model (Proposition 2.6.2) and optimistic after selecting among candidates; hence the third set.
- K-fold cross-validation averages K validation risks so that every example validates once; refit afterwards.

Experiment: training and validation tell opposite stories

Setup. Fit polynomial degrees $0, 1, 2, 3, 5$ on the same development problem.

What we hold fixed. The split and fitting procedure.

What we change. Only degree.

What we measure. Training and validation risk.

What the plot shows. Training risk decreases monotonically, for example $1.25, 0.45, 0.40, 0.10, 0.00$, while validation risk forms a U, for example $1.30, 0.50, 0.62, 1.90, 8.40$.

How to read it. The line wins despite not fitting best. The selected validation minimum is itself optimistic, so a test set is still needed for the final number.

What people do wrong: preprocessing before the split

Scaling, feature selection and imputation learn quantities from data. Fitting them on all rows before cross-validation lets each validation fold influence the model being scored, so the error is biased downward. Fit the whole pipeline inside each training fold, apply it to that fold's validation rows, choose the configuration, and only then refit on all development data.

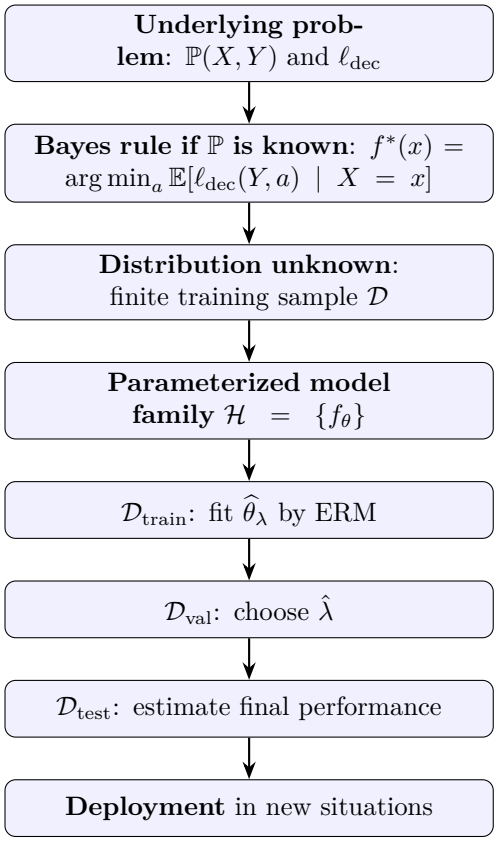
Telling them apart: training, validation and test

Training rows fit parameters. Validation rows or cross-validation choose hyperparameters and decision rules. Test rows are used once after the whole procedure is fixed. A validation score becomes optimistic when it wins a search; a test set restores an honest estimate only if it did not participate in that search.

How the professor asks it

Q. *Why can the winning validation score not be reported as final performance?*
A. It is the minimum of several noisy estimates. Selection favours candidates whose validation noise happened to be favourable, so the minimum is optimistic by the same mechanism as fitted training risk. Freeze the complete configuration, including preprocessing and threshold, then evaluate once on an untouched test set drawn according to the intended deployment protocol.

2.7 The Learning Problem in One Picture



The picture also clarifies the relationship with Chapter 1. Experience is represented by finite data; the hypothesis space specifies which rules can be considered; the loss measures the quality of their outputs. Training risk guides parameter estimation, validation performance guides model complexity, and test performance estimates how the completed procedure may behave on new observations. The four ingredients of Definition 1.6.1 appear as $\mathcal{D}$ (experience), $\mathcal{H}$ (model), ERM plus an optimiser (learning algorithm), and ℓ_{dec} evaluated on held-out data (evaluation criterion). Why finite samples provide reliable evidence about future performance is the subject of generalization, developed in Lecture 3.

Example 2.7.1 (The whole pipeline on one problem)**.** *Fraud detection.* Underlying problem: transactions X and fraud indicator Y; decision loss: $C_{\text{FP}} = 5$ per blocked legitimate payment, $C_{\text{FN}} =$ amount. Bayes rule: block iff $p(x) > 5/(5 + \text{amount})$; unknown, since p is unknown. Data: two years of labelled transactions, split by time into development (first 20 months) and test (last 4 months). Model family: a probabilistic classifier producing $\widehat{p}_\theta(x)$, with hyperparameters λ (representation, model size). Training: minimise empirical cross-entropy on $\mathcal{D}_{\text{train}}$ for each λ. Validation: choose λ by cost-weighted error on $\mathcal{D}_{\text{val}}$ (not by cross-entropy, because the application cares about money). Test: apply the Bayes threshold to $\widehat{p}_{\widehat{\theta}}$ on the final four months and report the total cost. Deployment: the threshold stays adjustable, because the decision policy is separate from the learned probabilities.

How the professor asks it

Q. *Walk from an object to an action and name where each kind of data is used.*
A. The representation maps the object to x; a hypothesis f_θ turns x into a score or probability; a decision rule combines that output with the decision loss to choose an action. Training rows fit θ, validation evidence selects the representation, family, hyperparameters and threshold, and the test set estimates the risk of the frozen pipeline. The deployment distribution determines whether that estimate remains relevant.

Chapter takeaways

Summary: From representations to decisions

- **Representation.** Objects become vectors $x \in \mathbb{R}^d$ by human choices; one-hot code categories; the representation bounds what can be learned.
- **Known distribution.** The Bayes rule $f^*(x) = \arg\min_a \mathbb{E}[\ell_{\text{dec}}(Y, a) \mid X = x]$ is optimal (Theorem 2.2.3); squared loss gives the conditional mean, absolute the median, zero–one the most probable class, unequal costs the threshold $C_{\text{FP}}/(C_{\text{FP}} + C_{\text{FN}})$.
- **Finite data.** kNN imitates the Bayes rule locally; distance and scaling define "near"; the curse of dimensionality $(1 - (1 - 10^{-d})^n)$ kills locality in high dimension.
- **Models.** A hypothesis space $\mathcal{H} = \{f_\theta\}$ shares structure across inputs; parameters are learned, hyperparameters chosen.
- **ERM.** Minimise empirical risk; it is honest for fixed models and optimistic after fitting; cross-entropy trains calibrated probabilities because its expectation is $H(Y \mid X) + \mathbb{E}[\mathrm{D}_{\text{KL}}]$.
- **Evaluation.** Train / validate / test, or cross-validate; touch the test set once.